User Guide

AT32F403A/407 Firmware BSP&Pack

Introduction

This application note is written to give a short description of how to use AT32F403A/407 BSP (Board Support Package) and install AT32 pack.

Contents

1 Overview 36

2 How to install Pack 37

2.1 IAR Pack installation 37

2.2 Keil_v5 Pack installation 39

2.3 Keil_v4 Pack installation 40

2.4 Segger Pack installation 43

3 Flash algorithm file 47

3.1 How to use Keil algorithm file 47

3.2 How to use IAR algorithm files 49

4 BSP introduction 53

4.1 Quick start 53

4.1.1 Template project 53

4.1.2 BSP macro definitions 54

4.2 BSP specifications 56

4.2.1 List of abbreviations for peripherals 56

4.2.2 Naming rules 57

4.2.3 Encoding rules 58

4.3 BSP structure 61

4.3.1 BSP folder structure 61

4.3.2 BSP function library structure 62

4.3.3 Initialization and configuration for peripherals 63

4.3.4 Peripheral functions description format 63

5 AT32F403A/407 peripheral library functions 64

5.1 HICK automatic clock calibration (ACC) 64

5.1.1 acc_calibration_mode_enable function 65

5.1.2 acc_step_set function 66

5.1.3 acc_interrupt_enable function 67

5.1.4 acc_hicktrim_get function 67

5.1.5 acc_hickcal_get function 68
5.1.6 acc_write_c1 function 68
5.1.7 acc_write_c2 function 69
5.1.8 acc_write_c3 function 69
5.1.9 acc_read_c1 function 70
5.1.10 acc_read_c2 function 70
5.1.11 acc_read_c3 function 71
5.1.12 acc_flag_get function 71
5.1.13 acc_interrupt_flag_get function 72
5.1.14 acc_flag_clear function 73
5.2 Analog-to-digital converter (ADC) 74
5.2.1 adc_reset function 76
5.2.2 adc_enable function 76
5.2.3 adc_combine_mode_select function 77
5.2.4 adc_base_default_para_init function 78
5.2.5 adc_base_config function 78
5.2.6 adc_dma_mode_enable function 79
5.2.7 adc_interrupt_enable function 80
5.2.8 adc_calibration_init function 80
5.2.9 adc_calibration_init_status_get function 81
5.2.10 adc_calibration_start function 81
5.2.11 adc_calibration_status_get function 82
5.2.12 adc_voltage_monitor_enable function 82
5.2.13 adc_voltage_monitor_threshold_value_set function 83
5.2.14 adc_voltage_monitor_single_channel_select function 84
5.2.15 adc_ordinary_channel_set function 84
5.2.16 adc_preempt_channel_length_set function 85
5.2.17 adc_preempt_channel_set function 86
5.2.18 adc_ordinary_conversion_trigger_set function 86
5.2.19 adc_preempt_conversion_trigger_set function 88
5.2.20 adc_preempt_offset_value_set function 89
5.2.21 adc_ordinary_part_count_set function 90
5.2.22 adc_ordinary_part_mode_enable function 90
5.2.23 adc_preempt_part_mode_enable function 91
5.2.24 adc_preempt_auto_mode_enable function 91

5.2.25 adc_tempersensor_vintrv_enable function 92
5.2.26 adc_ordinary_software_trigger_enable function 92
5.2.27 adc_ordinary_software_trigger_status_get function 93
5.2.28 adc_preempt_software_trigger_enable function 93
5.2.29 adc_preempt_software_trigger_status_get function 94
5.2.30 adc_ordinary_conversion_data_get function 94
5.2.31 adc_combine_ordinary_conversion_data_get function 95
5.2.32 adc_preempt_conversion_data_get function 95
5.2.33 adc_flag_get function 96
5.2.34 adc_interrupt_flag_get function 96
5.2.35 adc_flag_clear function 97
5.3 Battery powered domain (BPR) 98
5.3.1 bpr_reset function 100
5.3.2 bpr_flag_get function 100
5.3.3 bpr_interrupt_flag_get function 101
5.3.4 bpr_flag_clear function 101
5.3.5 bpr_interrupt_enable function 102
5.3.6 bpr_data_read function 102
5.3.7 bpr_data_write function 103
5.3.8 bpr_rtc_output_select function 103
5.3.9 bpr_rtc_clock_calibration_value_set function 104
5.3.10 bpr_tamper_pin_enable function 104
5.3.11 bpr_tamper_pin_active_level_set function 105
5.4 Controller area network (CAN) 106
5.4.1 can_reset function 108
5.4.2 can_baudrate_default_para_init function 108
5.4.3 can_baudrate_set function 109
5.4.4 can_default_para_init function 110
5.4.5 can_base_init function 110
5.4.6 can_filter_default_para_init function 112
5.4.7 can_filter_init function 112
5.4.8 can_debug_transmission_prohibit function 114
5.4.9 can_ttc_mode_enable function 114
5.4.10 can_message_transmit function 115
5.4.11 can_transmit_status_get function 116

5.4.12 can_transmit_cancel function 117
5.4.13 can_message_receive function 118
5.4.14 can_receive_fifo_release function 119
5.4.15 can_receive_message_pending_get function 120
5.4.16 can_operating_mode_set function 120
5.4.17 can_doze_mode_enter function 121
5.4.18 can_doze_mode_exit function 122
5.4.19 can_error_type_record_get function 122
5.4.20 can_receive_error_counter_get function 123
5.4.21 can_transmit_error_counter_get function 123
5.4.22 can_interrupt_enable function 124
5.4.23 can_flag_get function 125
5.4.24 can_interrupt_flag_get function 126
5.4.25 can_flag_clear function 127
5.5 CRC calculation unit (CRC) 128
5.5.1 crc_data_reset function 129
5.5.2 crc_one_word_calculate function 129
5.5.3 crc_block_calculate function 130
5.5.4 crc_data_get function 130
5.5.5 crc_common_data_set function 131
5.5.6 crc_common_date_get function 131
5.5.7 crc_init_data_set function 132
5.5.8 crc_reverse_input_data_set function 132
5.5.9 crc_reverse_output_data_set function 133
5.5.10 crc_poly_value_set function 133
5.5.11 crc_poly_value_get function 134
5.5.12 crc_poly_size_set function 134
5.5.13 crc_poly_size_get function 135
5.6 Clock and reset management (CRM) 136
5.6.1 crm_reset function 138
5.6.2 crm_lext_bypass function 138
5.6.3 crm_hext_bypass function 139
5.6.4 crm_flag_get function 139
5.6.5 crm_interrupt_flag_get function 140
5.6.6 crm_hext_stable_wait function 141

5.6.7 crm_hick_clock_trimming_set function 141
5.6.8 crm_hick_clock_calibration_set function 142
5.6.9 crm_periph_clock_enable function 142
5.6.10 crm_periph_reset function 143
5.6.11 crm_periph_sleep_mode_clock_enable function 144
5.6.12 crm_clock_source_enable function 145
5.6.13 crm_flag_clear function 145
5.6.14 crm_rtc_clock_select function 146
5.6.15 crm_rtc_clock_enable function 147
5.6.16 crm_ahb_div_set function 147
5.6.17 crm_apb1_div_set function 148
5.6.18 crm_apb2_div_set function 148
5.6.19 crm_adc_clock_div_set function 149
5.6.20 crm_usb_clock_div_set function 149
5.6.21 crm_clock_failure_detection_enable function 150
5.6.22 crm_battery_powered_domain_reset function 150
5.6.23 crm_pll_config function 151
5.6.24 crm_sysclk_switch function 152
5.6.25 crm_sysclk_switch_status_get function 152
5.6.26 crm_clocks_freq_get function 153
5.6.27 crm_clock_out_set function 154
5.6.28 crm_interrupt_enable function 154
5.6.29 crm_auto_step_mode_enable function 155
5.6.30 crm_usb_interrupt_remapping_set function 155
5.6.31 crm_hick_sclk_frequency_select function 156
5.6.32 crm_usb_clock_source_select function 156
5.6.33 crm_clkout_to_tmr10_enable function 157
5.6.34 crm_hext_clock_div_set function 157
5.6.35 crm_clkout_div_set function 158
5.6.36 crm_emac_output_pulse_set function 159
5.7 Digital-to-analog converter (DAC) 160
5.7.1 dac_reset function 161
5.7.2 dac_enable function 161
5.7.3 dac_output_buffer_enable function 162
5.7.4 dac_trigger_enable function 162

5.7.5 dac_trigger_select function 163
5.7.6 dac_software_trigger_generate function 164
5.7.7 dac_dual_software_trigger_generate function 164
5.7.8 dac_wave_generate function 165
5.7.9 dac_mask_amplitude_select function 165
5.7.10 dac_dma_enable function 166
5.7.11 dac_data_output_get function 167
5.7.12 dac_1_data_set function 167
5.7.13 dac_2_data_set function 168
5.7.14 dac_dual_data_set function 169
5.8 DMA controller 170
5.8.1 dma_default_para_init function 172
5.8.2 dma_init function 173
5.8.3 dma_reset function 175
5.8.4 dma_data_number_set function 175
5.8.5 dma_data_number_get function 176
5.8.6 dma_interrupt_enable function 176
5.8.7 dma_channel_enable function 177
5.8.8 dma_ flexible_config function 177
5.8.9 dma_flag_get function 179
5.8.10 dma_flag_clear function 181
5.9 Debug 183
5.9.1 debug_device_id_get function 183
5.9.2 debug_periph_mode_set function 184
5.10 External interrupt/event controller (EXINT) 185
5.10.1 exint_reset function 186
5.10.2 exint_default_para_init function 186
5.10.3 exint_init function 187
5.10.4 exint_flag_clear function 188
5.10.5 exint_flag_get function 188
5.10.6 exint_interrupt_flag_get function 189
5.10.7 exint_software_interrupt_event_generate function 189
5.10.8 exint_interrupt_enable function 190
5.10.9 exint_event_enable function 190
5.11 Flash memory controller (FLASH) 191

5.11.1 flash_flag_get function 193
5.11.2 flash_flag_clear function 194
5.11.3 flash_operation_status_get function 195
5.11.4 flash_bank1_operation_status_get function 195
5.11.5 flash_bank2_operation_status_get function 196
5.11.6 flash_spim_operation_status_get function 196
5.11.7 flash_operation_wait_for function 197
5.11.8 flash_bank1_operation_wait_for function 198
5.11.9 flash_bank2_operation_wait_for function 198
5.11.10 flash_spim_operation_wait_for function 199
5.11.11 flash_unlock function 199
5.11.12 flash_bank1_unlock function 200
5.11.13 flash_bank2_unlock function 200
5.11.14 flash_spim_unlock function 201
5.11.15 flash_lock function 201
5.11.16 flash_bank1_lock function 201
5.11.17 flash_bank2_lock function 202
5.11.18 flash_spim_lock function 202
5.11.19 flash_sector_erase function 203
5.11.20 flash_internal_all_erase function 203
5.11.21 flash_bank1_erase function 204
5.11.22 flash_bank2_erase function 204
5.11.23 flash_spim_all_erase function 205
5.11.24 flash_user_system_data_erase function 205
5.11.25 flash_word_program function 206
5.11.26 flash_halfword_program function 206
5.11.27 flash_byte_program function 207
5.11.28 flash_user_system_data_program function 208
5.11.29 flash_epp_set function 208
5.11.30 flash_epp_status_get function 209
5.11.31 flash_fap_enable function 210
5.11.32 flash_fap_status_get function 210
5.11.33 flash_ssb_set function 211
5.11.34 flash_ssb_status_get function 212
5.11.35 flash_interrupt_enable function 212

5.11.36 flash_spim_model_select function 213
5.11.37 flash_spim_encryption_range_set function 213
5.11.38 flash_slib_enable function 214
5.11.39 flash_slib_disable function 214
5.11.40 flash_slib_remaining_count_get function 215
5.11.41 flash_slib_state_get function 215
5.11.42 flash_slib_start_sector_get function 216
5.11.43 flash_slib_datastart_sector_get function 216
5.11.44 flash_slib_end_sector_get function 217
5.11.45 flash_crc_calibrate function 217
5.12 Geneal-purpose I/Os (GPIO) 218
5.12.1 gpio_reset function 220
5.12.2 gpio_iomux_reset function 220
5.12.3 gpio_init function 221
5.12.4 gpio_default_para_init function 222
5.12.5 gpio_input_data_bit_read function 223
5.12.6 gpio_input_data_read function 223
5.12.7 gpio_output_data_bit_read function 224
5.12.8 gpio_output_data_read function 224
5.12.9 gpio_bits_set function 225
5.12.10 gpio_bits_reset function 225
5.12.11 gpio_bits_toggle function 226
5.12.12 gpio_bits_write function 226
5.12.13 gpio_port_write function 227
5.12.14 gpio_pin_wp_config function 227
5.12.15 gpio_pins_huge_driven_config function 228
5.12.16 gpio_event_output_config function 228
5.12.17 gpio_event_output_enable function 229
5.12.18 gpio_pin_remap_config function 229
5.12.19 gpio_exint_line_config function 230
5.13 I2C interfaces 231
5.13.1 i2c_reset function 233
5.13.2 i2c_software_reset function 233
5.13.3 i2c_init function 234
5.13.4 i2c_own_address1_set function 234

5.13.5 i2c_own_address2_set function....235
5.13.6 i2c_own_address2_enable function....235
5.13.7 i2c_smbus_enable function....236
5.13.8 i2c_enable function....236
5.13.9 i2c_fast_mode_duty_set function....237
5.13.10 i2c_clock_stretch_enable function....237
5.13.11 i2c_ack_enable function....238
5.13.12 i2c_master_receive_ack_set function....238
5.13.13 i2c_pec_position_set function....239
5.13.14 i2c_general_call_enable function....240
5.13.15 i2c_arp_mode_enable function....240
5.13.16 i2c_smbus_mode_set function....241
5.13.17 i2c_smbus_alert_set function....241
5.13.18 i2c_pec_transmit_enable function....242
5.13.19 i2c_pec_calculate_enable function....242
5.13.20 i2c_pec_value_get function....243
5.13.21 i2c_dma_end_transfer_set function....243
5.13.22 i2c_dma_enable function....244
5.13.23 i2c_interrupt_enable function....244
5.13.24 i2c_start_generate function....245
5.13.25 i2c_stop_generate function....245
5.13.26 i2c_7bit_address_send function....246
5.13.27 i2c_data_send function....246
5.13.28 i2c_data_receive function....247
5.13.29 i2c_flag_get function....247
5.13.30 i2c_interrupt_flag_get function....248
5.13.31 i2c_flag_clear function....249
5.13.32 i2c_config function....250
5.13.33 i2c_lowlevel_init function....252
5.13.34 i2c_wait_end function....253
5.13.35 i2c_wait_flag function....254
5.13.36 i2c_master_transmit function....255
5.13.37 i2c_master_receive function....256
5.13.38 i2c_slave_transmit function....256
5.13.39 i2c_slave_receive function....257

5.13.40 i2c_master_transmit_int function 257
5.13.41 i2c_master_receive_int function 258
5.13.42 i2c_slave_transmit_int function 259
5.13.43 i2c_slave_receive_int function 259
5.13.44 i2c_master_transmit_dma function 260
5.13.45 i2c_master_receive_dma function 261
5.13.46 i2c_slave_transmit_dma function 261
5.13.47 i2c_slave_receive_dma function 262
5.13.48 i2c_memory_write function 263
5.13.49 i2c_memory_write_int function 264
5.13.50 i2c_memory_write_dma function 265
5.13.51 i2c_memory_read function 266
5.13.52 i2c_memory_read_int function 267
5.13.53 i2c_memory_read_dma function 268
5.13.54 i2c_evt_irq_handler function 269
5.13.55 i2c_err_irq_handler function 269
5.13.56 i2c_dma_tx_irq_handler function 270
5.13.57 i2c_dma_rx_irq_handler function 270
5.14 Nested vectored interrupt controller (NVIC) 271
5.14.1 nvic_system_reset function 272
5.14.2 nvic_irq_enable function 272
5.14.3 nvic_irq_disable function 273
5.14.4 nvic_priority_group_config function 273
5.14.5 nvic_vector_table_set function 274
5.14.6 nvic_lowpower_mode_config function 275
5.15 Power controller (PWC) 276
5.15.1 pwc_reset function 277
5.15.2 pwc_battery_powered_domain_access function 277
5.15.3 pwc_pvm_level_select function 278
5.15.4 pwc_power_voltage_monitor_enable function 278
5.15.5 pwc_wakeup_pin_enable function 279
5.15.6 pwc_flag_clear function 279
5.15.7 pwc_flag_get function 280
5.15.8 pwc_sleep_mode_enter function 280
5.15.9 pwc_deep_sleep_mode_enter function 281

5.15.10 pwc_voltage_regulate_set function....281
5.15.11 pwc_standby_mode_enter function....282
5.16 Real-time clock (RTC)....283
5.16.1 rtc_counter_set function....284
5.16.2 rtc_counter_get function....284
5.16.3 rtc_divider_set function....285
5.16.4 rtc_divider_get function....285
5.16.5 rtc_alarm_set function....286
5.16.6 rtc_interrupt_enable function....286
5.16.7 rtc_flag_get function....287
5.16.8 rtc_interrupt_flag_get....287
5.16.9 rtc_flag_clear function....288
5.16.10 rtc_wait_config_finish function....288
5.16.11 rtc_wait_update_finish function....289
5.17 SDIO interfaces....290
5.17.1 sdio_reset function....292
5.17.2 sdio_power_set function....292
5.17.3 sdio_power_status_get function....293
5.17.4 sdio_clock_config function....293
5.17.5 sdio_bus_width_config function....294
5.17.6 sdio_clock_bypass function....294
5.17.7 sdio_power_saving_mode_enable function....295
5.17.8 sdio_flow_control_enable function....295
5.17.9 sdio_clock_enable function....296
5.17.10 sdio_dma_enable function....296
5.17.11 sdio_interrupt_enable function....297
5.17.12 sdio_flag_get function....298
5.17.13 sdio_interrupt_flag_get function....299
5.17.14 sdio_flag_clear function....300
5.17.15 sdio_command_config function....301
5.17.16 sdio_command_state_machine_enable function....302
5.17.17 sdio_command_response_get function....302
5.17.18 sdio_response_get function....303
5.17.19 sdio_data_config function....303
5.17.20 sdio_data_state_machine_enable function....305

5.17.21 sdio_data_counter_get function 305
5.17.22 sdio_data_read function 306
5.17.23 sdio_buffer_counter_get function 306
5.17.24 sdio_data_write function 307
5.17.25 sdio_read_wait_mode_set function 307
5.17.26 sdio_read_wait_start function 308
5.17.27 sdio_read_wait_stop function 308
5.17.28 sdio_io_function_enable function 309
5.17.29 sdio_io_suspend_command_set function 309
5.18 Serial peripheral interface (SPI)/ I²S 310
5.18.1 spi_i2s_reset function 311
5.18.2 spi_default_para_init function 311
5.18.3 spi_init function 312
5.18.4 spi_crc_next_transmit function 314
5.18.5 spi_crc_polynomial_set function 314
5.18.6 spi_crc_polynomial_get function 315
5.18.7 spi_crc_enable function 315
5.18.8 spi_crc_value_get function 316
5.18.9 spi_hardware_cs_output_enable function 316
5.18.10 spi_software_cs_internal_level_set function 317
5.18.11 spi_frame_bit_num_set function 318
5.18.12 spi_half_duplex_direction_set function 318
5.18.13 spi_enable function 319
5.18.14 i2s_default_para_init function 319
5.18.15 i2s_init function 320
5.18.16 i2s_enable function 321
5.18.17 spi_i2s_interrupt_enable function 322
5.18.18 spi_i2s_dma_transmitter_enable function 323
5.18.19 spi_i2s_dma_receiver_enable function 323
5.18.20 spi_i2s_data_transmit function 324
5.18.21 spi_i2s_data_receive function 324
5.18.22 spi_i2s_flag_get function 325
5.18.23 spi_i2s_interrupt_flag_get function 326
5.18.24 spi_i2s_flag_clear function 327
5.19 SysTick 328

5.19.1 systick_clock_source_config function 328
5.19.2 SysTick_Config function 329
5.20 TMR 330
5.20.1 tmr_reset function 332
5.20.2 tmr_counter_enable function 332
5.20.3 tmr_output_default_para_init function 333
5.20.4 tmr_input_default_para_init function 333
5.20.5 tmr_brkdt_default_para_init function 334
5.20.6 tmr_base_init function 335
5.20.7 tmr_clock_source_div_set function 335
5.20.8 tmr_cnt_dir_set function 336
5.20.9 tmr_repetition_counter_set function 336
5.20.10 tmr_counter_value_set function 337
5.20.11 tmr_counter_value_get function 337
5.20.12 tmr_div_value_set function 338
5.20.13 tmr_div_value_get function 338
5.20.14 tmr_output_channel_config function 339
5.20.15 tmr_output_channel_mode_select function 341
5.20.16 tmr_period_value_set function 341
5.20.17 tmr_period_value_get function 342
5.20.18 tmr_channel_value_set function 342
5.20.19 tmr_channel_value_get function 343
5.20.20 tmr_period_buffer_enable function 343
5.20.21 tmr_output_channel_buffer_enable function 344
5.20.22 tmr_output_channel_immediately_set function 344
5.20.23 tmr_output_channel_switch_set function 345
5.20.24 tmr_one_cycle_mode_enable function 345
5.20.25 tmr_32_bit_function_enable function 346
5.20.26 tmr_overflow_request_source_set function 346
5.20.27 tmr_overflow_event_disable function 347
5.20.28 tmr_input_channel_init function 347
5.20.29 tmr_channel_enable function 349
5.20.30 tmr_input_channel_filter_set function 350
5.20.31 tmr_pwm_input_config function 350
5.20.32 tmr_channel1_input_select function 351

5.20.33 tmr_input_channel_divider_set function 352
5.20.34 tmr_primary_mode_select function 352
5.20.35 tmr_sub_mode_select function 353
5.20.36 tmr_channel_dma_select function 354
5.20.37 tmr_hall_select function 354
5.20.38 tmr_channel_buffer_enable function 355
5.20.39 tmr_trigger_input_select function 355
5.20.40 tmr_sub_sync_mode_set function 356
5.20.41 tmr_dma_request_enable function 356
5.20.42 tmr_interrupt_enable function 357
5.20.43 tmr_interrupt_flag_get function 358
5.20.44 tmr_flag_get function 359
5.20.45 tmr_flag_clear function 360
5.20.46 tmr_event_sw_trigger function 360
5.20.47 tmr_output_enable function 361
5.20.48 tmr_internal_clock_set function 361
5.20.49 tmr_output_channel_polarity_set function 362
5.20.50 tmr_external_clock_config function 362
5.20.51 tmr_external_clock_mode1_config function 363
5.20.52 tmr_external_clock_mode2_config function 364
5.20.53 tmr_encoder_mode_config function 365
5.20.54 tmr_force_output_set function 366
5.20.55 tmr_dma_control_config function 366
5.20.56 tmr_brkdt_config function 367
5.21 Universal synchronous/asynchronous receiver/transmitter (USART) 370
5.21.1 usart_reset function 371
5.21.2 usart_init function 372
5.21.3 usart_parity_selection_config function 373
5.21.4 usart_enable function 373
5.21.5 usart_transmitter_enable function 374
5.21.6 usart_receiver_enable function 374
5.21.7 usart_clock_config function 375
5.21.8 usart_clock_enable function 376
5.21.9 usart_interrupt_enable function 376
5.21.10 usart_dma_transmitter_enable function 377

5.21.11 usart_dma_receiver_enable function 377
5.21.12 usart_wakeup_id_set function 378
5.21.13 usart_wakeup_mode_set function 378
5.21.14 usart_receiver_mute_enable function 379
5.21.15 usart_break_bit_num_set function 379
5.21.16 usart_lin_mode_enable function 380
5.21.17 usart_data_transmit function 380
5.21.18 usart_data_receive function 381
5.21.19 usart_break_send function 381
5.21.20 usart_smartcard_guard_time_set function 382
5.21.21 usart_irda_smartcard_division_set function 382
5.21.22 usart_smartcard_mode_enable function 383
5.21.23 usart_smartcard_nack_set function 383
5.21.24 usart_single_line_halfduplex_select function 384
5.21.25 usart_irda_mode_enable function 384
5.21.26 usart_irda_low_power_enable function 385
5.21.27 usart_hardware_flow_control_set function 385
5.21.28 usart_flag_get function 386
5.21.29 usart_interrupt_flag_get function 387
5.21.30 usart_flag_clear function 388
5.22 Watchdog timer (WDT) 389
5.22.1 wdt_enable function 390
5.22.2 wdt_counter_reload function 390
5.22.3 wdt_reload_value_set function 391
5.22.4 wdt_divider_set function 391
5.22.5 wdt_register_write_enable function 392
5.22.6 wdt_flag_get function 392
5.23 Window watchdog timer (WWDT) 393
5.23.1 wwdt_reset function 394
5.23.2 wwdt_divider_set function 394
5.23.3 wwdt_enable function 395
5.23.4 wwdt_interrupt_enable function 395
5.23.5 wwdt_counter_set function 395
5.23.6 wwdt_window_counter_set function 396
5.23.7 wwdt_flag_get function 396

5.23.8 wwdt_interrupt_flag_get function 397
5.23.9 wwdt_flag_clear function 397
5.24 External memory controller (XMC) 398
5.24.1 xmc_nor_sram_reset function 400
5.24.2 xmc_nor_sram_init function 400
5.24.3 xmc_nor_sram_timing_config function 403
5.24.4 xmc_norsram_default_para_init function 405
5.24.5 xmc_norsram_timing_default_para_init function 406
5.24.6 xmc_nor_sram_enable function 407
5.24.7 xmc_ext_timing_config function 408
5.24.8 xmc_nand_reset function 409
5.24.9 xmc_nand_init function 409
5.24.10 xmc_nand_timing_config function 411
5.24.11 xmc_nand_default_para_init function 412
5.24.12 xmc_nand_timing_default_para_init function 413
5.24.13 xmc_nand_enable function 414
5.24.14 xmc_nand_ecc_enable function 414
5.24.15 xmc_ecc_get function 415
5.24.16 xmc_interrupt_enable function 415
5.24.17 xmc_flag_status_get function 416
5.24.18 xmc_interrupt_flag_clear function 417
5.24.19 xmc_flag_clear function 418

6 Precautions 419

6.1 Device model replacement 419
6.1.1 KEIL environment 419
6.1.2 IAR environment 420
6.2 Unable to identify IC by JLink software in Keil 422
6.3 How to change HEXT crystal 424

7 Revision history 425

List of tables

Table 1. Summary of macro definitions 54
Table 2. List of abbreviations for peripherals 56
Table 3. Summary of BSP function library files 62
Table 4. Function format description for peripherals 63
Table 5. Summary of ACC registers 64
Table 6. Summary of ACC library functions 64
Table 7. acc_calibration_mode_enable function 65
Table 8. acc_step_set function 66
Table 9. acc_interrupt_enable function 67
Table 10. acc_hicktrim_get function 67
Table 11. acc_hickcal_get function 68
Table 12. acc_write_c1 function 68
Table 13. acc_write_c2 function 69
Table 14. acc_write_c3 function 69
Table 15. acc_read_c1 function 70
Table 16. acc_read_c2 function 70
Table 17. acc_read_c3 function 71
Table 18. acc_flag_get function 71
Table 19. acc_interrupt_flag_get function 72
Table 20. acc_flag_clear function 73
Table 21. Summary of ADC registers 74
Table 22. Summary of ADC library functions 75
Table 23. adc_reset function 76
Table 24. adc_enable function 76
Table 25. adc_combine_mode_select function 77
Table 26. adc_base_default_para_init function 78
Table 27. adc_base_config function 78
Table 28. adc_dma_mode_enable function 79
Table 29. adc_interrupt_enable function 80
Table 30. adc_calibration_init function 80
Table 31. adc_calibration_init_status_get function 81
Table 32. adc_calibration_start function 81
Table 33. adc_calibration_status_get function 82
Table 34. adc_voltage_monitor_enable function 82

Table 35. adc_voltage_monitor_threshold_value_set function 83
Table 36. adc_voltage_monitor_single_channel_select function 84
Table 37. adc_ordinary_channel_set function 84
Table 38. adc_preempt_channel_length_set function 85
Table 39. adc_preempt_channel_set function 86
Table 40. adc_ordinary_conversion_trigger_set function 86
Table 41. adc_preempt_conversion_trigger_set function 88
Table 42. adc_preempt_offset_value_set function 89
Table 43. adc_ordinary_part_count_set function 90
Table 44. adc_ordinary_part_mode_enable function 90
Table 45. adc_preempt_part_mode_enable 91
Table 46. adc_preempt_auto_mode_enable function 91
Table 47. adc_tempersensor_vintrv_enable function 92
Table 48. adc_ordinary_software_trigger_enable function 92
Table 49. adc_ordinary_software_trigger_status_get function 93
Table 50. adc_preempt_software_trigger_enable function 93
Table 51. adc_preempt_software_trigger_status_get function 94
Table 52. adc_ordinary_conversion_data_get function 94
Table 53. adc_combine_ordinary_conversion_data_get function 95
Table 54. adc_preempt_conversion_data_get function 95
Table 55. adc_flag_get function 96
Table 56. adc_interrupt_flag_get function 96
Table 57. adc_flag_clear function 97
Table 58. Summary of BPR registers 98
Table 59. Summary of BP library functions 99
Table 60. bpr_reset function 100
Table 61. bpr_flag_get function 100
Table 62. bpr_interrupt_flag_get function 101
Table 63. bpr_flag_clear function 101
Table 64. bpr_interrupt_enable function 102
Table 65. bpr_data_read function 102
Table 66. bpr_data_write function 103
Table 67. bpr_rtc_output_select function 103
Table 68. bpr_rtc_clock_calibration_value_set function 104
Table 69. bpr_tamper_pin_enable function 104

Table 70. bpr_tamper_pin_active_level_set function 105
Table 71. Summary of ADC registers 106
Table 72. Summary of CAN library functions 107
Table 73. can_reset function 108
Table 74. can_baudrate_default_para_init function 108
Table 75. can_baudrate_set function 109
Table 76. can_default_para_init function 110
Table 77. can_base_init function 110
Table 78. can_filter_default_para_init function 112
Table 79. can_filter_init function 112
Table 80. can_debug_transmission_prohibit function 114
Table 81. can_ttc_mode_enable function 114
Table 82. can_message_transmit function 115
Table 83. can_transmit_status_get function 116
Table 84. can_transmit_cancel function 117
Table 85. can_message_receive function 118
Table 86. can_receive_fifo_release function 119
Table 87. can_receive_message_pending_get function 120
Table 88. can_operating_mode_set function 120
Table 89. can_doze_mode_enter function 121
Table 90. can_doze_mode_exit function 122
Table 91. can_error_type_record_get function 122
Table 92. can_receive_error_counter_get function 123
Table 93. can_transmit_error_counter_get function 123
Table 94. can_interrupt_enable function 124
Table 95. can_flag_get function 125
Table 96. can_interrupt_flag_get function 126
Table 97. can_flag_clear function 127
Table 98. Summary of CRC registers 128
Table 99. Summary of CRC library functions 128
Table 100. crc_data_reset function 129
Table 101. crc_one_word_calculate function 129
Table 102. crc_block_calculate function 130
Table 103. crc_data_get function 130
Table 104. crc_common_data_set function 131

Table 105. crc_common_date_get function 131
Table 106. crc_init_data_set function 132
Table 107. crc_reverse_input_data_set function 132
Table 108. crc_reverse_output_data_set function 133
Table 109. crc_poly_value_set function 133
Table 110. crc_poly_value_get function 134
Table 111. crc_poly_size_set function 134
Table 112. crc_poly_size_get function 135
Table 113. Summary of CRM registers 136
Table 114. Summary of CRM library functions 136
Table 115. crm_reset function 138
Table 116. crm_lext_bypass function 138
Table 117. crm_hext_bypass function 139
Table 118. crm_flag_get function 139
Table 119. crm_interrupt_flag_get function 140
Table 120. crm_hext_stable_wait function 141
Table 121. crm_hick_clock_trimming_set function 141
Table 122. crm_hick_clock_calibration_set function 142
Table 123. crm_periph_clock_enable function 142
Table 124. crm_periph_reset function 143
Table 125. crm_periph_sleep_mode_clock_enable function 144
Table 126. crm_clock_source_enable function 145
Table 127. crm_flag_clear function 145
Table 128. crm_ertc_clock_select function 146
Table 129. crm_ertc_clock_enable function 147
Table 130. crm_ahb_div_set function 147
Table 131. crm_apb1_div_set function 148
Table 132. crm_apb2_div_set function 148
Table 133. crm_adc_clock_div_set function 149
Table 134. crm_usb_clock_div_set function 149
Table 135. crm_clock_failure_detection_enable function 150
Table 136. crm_battery_powered_domain_reset function 150
Table 137. crm_pll_config function 151
Table 138. crm_sysclk_switch function 152
Table 139. crm_sysclk_switch_status_get function 152

Table 140. crm_clocks_freq_get function 153
Table 141. crm_clock_out_set function 154
Table 142. crm_interrupt_enable function 154
Table 143. crm_auto_step_mode_enable function 155
Table 144. crm_usb_interrupt_remapping_set function 155
Table 145. crm_hick_sclk_frequency_select function 156
Table 146. crm_usb_clock_source_select function 156
Table 147. crm_clkout_to_tmr10_enable function 157
Table 148. crm_hext_clock_div_set function 157
Table 149. crm_clkout_div_set function 158
Table 150. crm_emac_output_pulse_set function 159
Table 151. Summary of DAC registers 160
Table 152. Summary of DAC library functions 160
Table 153. dac_reset function 161
Table 154. dac_enable function 161
Table 155. dac_output_buffer_enable function 162
Table 156. dac_trigger_enable function 162
Table 157. dac_trigger_select function 163
Table 158. dac_software_trigger_generate function 164
Table 159. dac_dual_software_trigger_generate function 164
Table 160. dac_wave_generate function 165
Table 161. dac_mask_amplitude_select function 165
Table 162. dac_dma_enable function 166
Table 163. dac_data_output_get function 167
Table 164. dac_1_data_set function 167
Table 165. dac_2_data_set function 168
Table 166. dac_dual_data_set function 169
Table 167. Summary of DMA registers 170
Table 168. Summary of DMA library functions 171
Table 169. dma_default_para_init function 172
Table 170.dma_init_struct default values 172
Table 171. dma_init function 173
Table 172. dma_reset function 175
Table 173. dma_data_number_set function 175
Table 174. dma_data_number_get function 176

Table 175. dma_interrupt_enable function 176
Table 176. dma_channel_enable function 177
Table 177. dma_interrupt_enable function 177
Table 178. Flexible mapping request source ID 178
Table 179. dma_flag_get function 179
Table 180. dma_flag_clear function 181
Table 181. Summary of debug registers 183
Table 182. Summary of debug library functions 183
Table 183. debug_device_id_get function 183
Table 184. debug_periph_mode_set function 184
Table 185. Summary of EXINT registers 185
Table 186. Summary of EXINT library functions 185
Table 187. exint_reset function 186
Table 188. exint_default_para_init function 186
Table 189. exint_init function 187
Table 190. exint_flag_clear function 188
Table 191. exint_flag_get function 188
Table 192. exint_interrupt_flag_get function 189
Table 193. exint_software_interrupt_event_generate function 189
Table 194. exint_interrupt_enable function 190
Table 195. exint_event_enable function 190
Table 196. Summary of FLASH registers 191
Table 197. Summary of FLASH library functions 192
Table 198. flash_flag_get function 193
Table 199. flash_flag_clear function 194
Table 200. flash_operation_status_get function 195
Table 201. flash_bank1_operation_status_get function 195
Table 202. flash_bank2_operation_status_get function 196
Table 203. flash_spim_operation_status_get function 196
Table 204. flash_operation_wait_for function 197
Table 205. flash_bank1_operation_wait_for function 198
Table 206. flash_bank2_operation_wait_for function 198
Table 207. flash_spim_operation_wait_for function 199
Table 208. flash_unlock function 199
Table 209. flash_bank1_unlock function 200

Table 210. flash_bank2_unlock function 200
Table 211. flash_spim_unlock function 201
Table 212. flash_lock function 201
Table 213. flash_bank1_lock function 201
Table 214. flash_bank2_lock function 202
Table 215. flash_spim_lock function 202
Table 216. flash_sector_erase function 203
Table 217. flash_internal_all_erase function 203
Table 218. flash_bank1_erase function 204
Table 219. flash_bank2_erase function 204
Table 220. flash_spim_all_erase function 205
Table 221. flash_user_system_data_erase function 205
Table 222. flash_word_program function 206
Table 223. flash_halfword_program function 206
Table 224. flash_byte_program function 207
Table 225. flash_user_system_data_program function 208
Table 226. flash_epp_set function 208
Table 227. flash_epp_status_get function 209
Table 228. flash_fap_enable function 210
Table 229. flash_fap_status_get function 210
Table 230. flash_ssb_set function 211
Table 231. flash_ssb_status_get function 212
Table 232. flash_interrupt_enable function 212
Table 233. flash_spim_model_select function 213
Table 234. flash_spim_encryption_range_set function 213
Table 235. flash_slib_enable function 214
Table 236. flash_slib_disable function 214
Table 237. flash_slib_remaining_count_get function 215
Table 238. flash_slib_state_get function 215
Table 239. flash_slib_start_sector_get function 216
Table 240. flash_slib_datastart_sector_get function 216
Table 241. flash_slib_end_sector_get function 217
Table 242. flash_crc_calibrate function 217
Table 243. Summary of GPIO registers 218
Table 244. GPIO and IOMUX library functions 219

Table 245. GPIO and IOMUX library functions 219
Table 246. gpio_reset function 220
Table 247. gpio_iomux_reset function 220
Table 248. gpio_init function 221
Table 249. gpio_default_para_init function 222
Table 250. gpio_init_struct default values 223
Table 251. gpio_input_data_bit_read function 223
Table 252. gpio_input_data_read function 223
Table 253. gpio_output_data_bit_read function 224
Table 254. gpio_output_data_read function 224
Table 255. gpio_bits_set function 225
Table 256. gpio_bits_reset function 225
Table 256. gpio_bits_toggle function 226
Table 257. gpio_bits_write function 226
Table 258. gpio_port_write function 227
Table 259. gpio_pin_wp_config function 227
Table 260. gpio_pins_huge_driven_config function 228
Table 261. gpio_event_output_config function 228
Table 262. gpio_event_output_enable function 229
Table 263. gpio_pin_remap_config function 229
Table 264. gpio_exint_line_config function 230
Table 265. Summary of I2C registera 231
Table 266. Summary of I2C library functions 231
Table 267. I2C application-layer library functions 232
Table 268. i2c_reset function 233
Table 269. i2c_software_reset function 233
Table 270. i2c_init function 234
Table 271. i2c_own_address1_set function 234
Table 272. i2c_own_address2_set function 235
Table 273. i2c_own_address2_enable function 235
Table 274. i2c_smbus_enable function 236
Table 275. i2c_enable function 236
Table 276. i2c_fast_mode_duty_set function 237
Table 277. i2c_clock_stretch_enable function 237
Table 278. i2c_ack_enable function 238

Table 279. i2c_master_receive_ack_set function 238
Table 280. i2c_pec_position_set function 239
Table 281. i2c_general_call_enable function 240
Table 282. i2c_arp_mode_enable function 240
Table 283. i2c_smbus_mode_set function 241
Table 284. i2c_smbus_alert_set function 241
Table 285. i2c_pec_transmit_enable function 242
Table 286. i2c_pec_calculate_enable function 242
Table 287. i2c_pec_value_get function 243
Table 288. i2c_dma_end_transfer_set function 243
Table 289. i2c_dma_enable function 244
Table 290. i2c_interrupt_enable function 244
Table 291. i2c_start_generate function 245
Table 292. i2c_stop_generate function 245
Table 293. i2c_7bit_address_send function 246
Table 294. i2c_data_send function 246
Table 295. i2c_data_receive function 247
Table 296. i2c_flag_get function 247
Table 297. i2c_interrupt_flag_get function 248
Table 298. i2c_flag_clear function 249
Table 299. i2c_config function 250
Table 300. i2c_lowlevel_init function 252
Table 301. i2c_wait_end function 253
Table 302. i2c_wait_flag function 254
Table 303. i2c_master_transmit function 255
Table 304. i2c_master_receive function 256
Table 305. i2c_slave_transmit function 256
Table 306. i2c_slave_receive function 257
Table 307. i2c_master_transmit_int function 257
Table 308. i2c_master_receive_int function 258
Table 309. 2c_slave_transmit_int function 259
Table 310. i2c_slave_receive_int function 259
Table 311. i2c_master_transmit_dma function 260
Table 312. i2c_master_receive_dma function 261
Table 313. i2c_slave_transmit_dma function 261

Table 314. i2c_slave_receive_dma function 262
Table 315. i2c_memory_write function 263
Table 316. i2c_memory_write_int function 264
Table 317. i2c_memory_write_dma function 265
Table 318. i2c_memory_read function 266
Table 319. i2c_memory_read_int function 267
Table 320. i2c_memory_read_dma function 268
Table 321. i2c_evt_irq_handler function 269
Table 322. i2c_err_irq_handler function 269
Table 323. i2c_dma_tx_irq_handler function 270
Table 324. i2c_dma_rx_irq_handler function 270
Table 325. Summary of NVIC registers 271
Table 326. Summary of NVIC library functions 271
Table 327. nvic_system_reset function 272
Table 328. nvic_irq_enable function 272
Table 329. nvic_irq_disable function 273
Table 330. nvic_priority_group_config function 273
Table 331. nvic_vector_table_set function 274
Table 332. nvic_lowpower_mode_config function 275
Table 333. Summary of PWC registers 276
Table 334. Summary of PWC library functions 276
Table 335. pwc_reset function 277
Table 336. pwc_battery_powered_domain_access function 277
Table 337. pwc_pvm_level_select function 278
Table 338. pwc_power_voltage_monitor_enable function 278
Table 339. pwc_wakeup_pin_enable function 279
Table 340. pwc_flag_clear function 279
Table 341. pwc_flag_get function 280
Table 342. pwc_sleep_mode_enter function 280
Table 343. pwc_deep_sleep_mode_enter function 281
Table 344. pwc_voltage_regulate_set function 281
Table 345. pwc_standby_mode_enter function 282
Table 346. Summary of RTC registers 283
Table 347. Summary of RTC library functions 283
Table 348. rtc_counter_set function 284

Table 349. rtc_counter_get function 284
Table 350. rtc_divider_set function 285
Table 351. rtc_divider_get function 285
Table 352. rtc_alarm_set function 286
Table 353. rtc_interrupt_enable function 286
Table 354. rtc_flag_get function 287
Table 355. rtc_interrupt_flag_get function 287
Table 356. rtc_flag_clear function 288
Table 357. rtc_wait_config_finish function 288
Table 358. rtc_wait_update_finish function 289
Table 359. SDIO register description 290
Table 360. SDIO library functions 291
Table 361. sdio_reset function 292
Table 362. sdio_power_set function 292
Table 363. sdio_power_status_get function 293
Table 364. sdio_clock_config function 293
Table 365. sdio_bus_width_config function 294
Table 366. sdio_clock_bypass function 294
Table 367. sdio_power_saving_mode_enable function 295
Table 368. sdio_flow_control_enable function 295
Table 369. sdio_clock_enable function 296
Table 370. sdio_dma_enable function 296
Table 371. sdio_interrupt_enable function 297
Table 372. sdio_flag_get function 298
Table 373. sdio_interupt_flag_get function 299
Table 374. sdio_flag_clear function 300
Table 375. sdio_command_config function 301
Table 376. sdio_command_state_machine_enable function 302
Table 377. sdio_command_response_get function 302
Table 378. sdio_response_get function 303
Table 379. sdio_data_config function 303
Table 380. sdio_data_state_machine_enable function 305
Table 381. sdio_data_counter_get function 305
Table 382. sdio_data_read function 306
Table 383. sdio_buffer_counter_get function 306

Table 384. sdio_data_write function 307
Table 385. sdio_read_wait_mode_set function 307
Table 386. sdio_read_wait_start function 308
Table 387. sdio_read_wait_stop function 308
Table 388. sdio_io_function_enable function 309
Table 389. sdio_io_suspend_command_set function 309
Table 390. Summary of SPI registers 310
Table 391. Summary of SPI library functions 310
Table 392. spi_i2s_reset function 311
Table 393. spi_default_para_init function 311
Table 394. spi_init function 312
Table 395. spi_crc_next_transmit function 314
Table 396. spi_crc_polynomial_set function 314
Table 397. spi_crc_polynomial_get function 315
Table 398. spi_crc_enable function 315
Table 399. spi_crc_value_get function 316
Table 400. spi_hardware_cs_output_enable function 316
Table 401. spi_software_cs_internal_level_set function 317
Table 402. spi_frame_bit_num_set function 318
Table 403. spi_half_duplex_direction_set function 318
Table 404. spi_enable function 319
Table 405. i2s_default_para_init function 319
Table 406. i2s_init function 320
Table 407. i2s_enable function 321
Table 408. spi_i2s_interrupt_enable function 322
Table 409. spi_i2s_dma_transmitter_enable function 323
Table 410. spi_i2s_dma_receiver_enable function 323
Table 411. spi_i2s_data_transmit function 324
Table 412. spi_i2s_data_receive function 324
Table 413. spi_i2s_flag_get function 325
Table 414. spi_i2s_interrupt_flag_get function 326
Table 415. spi_i2s_flag_clear function 327
Table 416. Summary of SysTick registers 328
Table 417. Summary of SysTick library functions 328
Table 418. systick_clock_source_config function 328

Table 419. SysTick_Config function 329
Table 420. Summary of TMR registers 330
Table 421. Summary of TMR library functions 331
Table 422. tmr_reset function 332
Table 423. tmr_counter_enable function 332
Table 424. tmr_output_default_para_init function 333
Table 425. tmr_output_struct default values 333
Table 426. tmr_input_default_para_init function 333
Table 427. tmr_input_struct default values 334
Table 428. tmr_brkdt_default_para_init function 334
Table 429. tmr_brkdt_struct default values 334
Table 430. tmr_base_init function 335
Table 431. tmr_clock_source_div_set function 335
Table 432. tmr_cnt_dir_set function 336
Table 433. tmr_repetition_counter_set function 336
Table 434. tmr_counter_value_set function 337
Table 435. tmr_counter_value_get function 337
Table 436. tmr_div_value_set function 338
Table 437. tmr_div_value_get function 338
Table 438. tmr_output_channel_config function 339
Table 439. tmr_output_channel_mode_select function 341
Table 440. tmr_period_value_set function 341
Table 441. tmr_period_value_get function 342
Table 442. tmr_channel_value_set function 342
Table 443. tmr_channel_value_get function 343
Table 444. tmr_period_buffer_enable function 343
Table 445. tmr_output_channel_buffer_enable function 344
Table 446. tmr_output_channel_immediately_set function 344
Table 447. tmr_output_channel_switch_set function 345
Table 448. tmr_one_cycle_mode_enable function 345
Table 449. tmr_32_bit_function_enable function 346
Table 450. tmr_overflow_request_source_set function 346
Table 451. tmr_overflow_event_disable function 347
Table 452. tmr_input_channel_init function 347
Table 453. tmr_channel_enable function 349

Table 454. tmr_input_channel_filter_set function 350
Table 455. tmr_pwm_input_config function 350
Table 456. tmr_channel1_input_select function 351
Table 457. tmr_input_channel_divider_set function 352
Table 458. tmr_primary_mode_select function 352
Table 459. tmr_sub_mode_select function 353
Table 460. tmr_channel_dma_select function 354
Table 461. tmr_hall_select function 354
Table 462. tmr_channel_buffer_enable function 355
Table 463. tmr_trigger_input_select function 355
Table 464. tmr_sub_sync_mode_set function 356
Table 465. tmr_dma_request_enable function 356
Table 466. tmr_interrupt_enable function 357
Table 467. tmr_interrupt_flag_get function 358
Table 468. tmr_flag_get function 359
Table 469. tmr_flag_clear function 360
Table 470. tmr_event_sw_trigger function 360
Table 471. tmr_output_enable function 361
Table 472. tmr_internal_clock_set function 361
Table 473. tmr_output_channel_polarity_set function 362
Table 474. tmr_external_clock_config function 362
Table 475. tmr_external_clock_mode1_config function 363
Table 476. tmr_external_clock_mode2_config function 364
Table 477. tmr_encoder_mode_config function 365
Table 478. tmr_force_output_set function 366
Table 479. tmr_dma_control_config function 366
Table 480. tmr_brkdt_config function 367
Table 481. Summary of USART registers 370
Table 482. Summary of USART library functions 370
Table 483. usart_reset function 371
Table 484. usart_init function 372
Table 485. usart_parity_selection_config function 373
Table 486. usart_enable function 373
Table 487. usart_transmitter_enable function 374
Table 488. usart_receiver_enable function 374

Table 489. usart_clock_config function 375
Table 490. usart_clock_enable function 376
Table 491. usart_interrupt_enable function 376
Table 492. usart_dma_transmitter_enable function 377
Table 493. usart_dma_receiver_enable function 377
Table 494. usart_wakeup_id_set function 378
Table 495. usart_wakeup_mode_set function 378
Table 496. usart_receiver_mute_enable function 379
Table 497. usart_break_bit_num_set function 379
Table 498. usart_lin_mode_enable function 380
Table 499. usart_data_transmit function 380
Table 500. usart_data_receive function 381
Table 501. usart_break_send function 381
Table 502. usart_smartcard_guard_time_set function 382
Table 503. usart_irda_smartcard_division_set function 382
Table 504. usart_smartcard_mode_enable function 383
Table 505. usart_smartcard_nack_set function 383
Table 506. usart_single_line_halfduplex_select function 384
Table 507. usart_irda_mode_enable function 384
Table 508. usart_irda_low_power_enable function 385
Table 509. usart_hardware_flow_control_set function 385
Table 510. usart_flag_get function 386
Table 511. usart_interrupt_flag_get function 387
Table 512. usart_flag_clear function 388
Table 513. Summary of WDT registers 389
Table 514. Summary of WDT library functions 389
Table 515. wdt_enable function 390
Table 516. wdt_counter_reload function 390
Table 517. wdt_reload_value_set function 391
Table 518. wdt_divider_set function 391
Table 519. wdt_register_write_enable function 392
Table 520. wdt_flag_get function 392
Table 521. Summary of WWDT registers 393
Table 522. Summary of WWDT library functions 393
Table 523. wwdt_reset function 394

Table 524. wwdt_divider_set function 394
Table 525. wwdt_enable function 395
Table 526. wwdt_interrupt_enable function 395
Table 527. wwdt_counter_set function 395
Table 528. wwdt_window_counter_set function 396
Table 529. wwdt_flag_get function 396
Table 530. wwdt_interrupt_flag_clear function 397
Table 531. wwdt_flag_clear function 397
Table 532. Summary of XMC registers 399
Table 533. Summary of XMC library functions 399
Table 534. xmc_nor_sram_reset function 400
Table 535. xmc_nor_sram_init function 400
Table 536. scfg_mem_map_get function 403
Table 537. xmc_ norsram_default_para_init function 405
Table 538. xmc_nor_sram_init_struct default values 406
Table 539. xmc_norsram_timing_default_para_init function 406
Table 540. xmc_rw_timing_struct and xmc_w_timing_struct 407
Table 541. xmc_nor_sram_enable function 407
Table 542. xmc_ext_timing_configfunction 408
Table 543. xmc_nand_reset function 409
Table 544. xmc_nand_init function 409
Table 545. xmc_nand_timing_config function 411
Table 546. xmc_nand_default_para_init 412
Table 547. xmc_nand_init_struct 413
Table 548. xmc_nand_timing_default_para_init function 413
Table 549. xmc_common_spacetiming_struct and xmc_attribute_spacetiming_struct 413
Table 550. xmc_nand_enable function 414
Table 551. xmc_nand_ecc_enable function 414
Table 552. xmc_ecc_get function 415
Table 553. xmc_interrupt_enable function 415
Table 554. xmc_flag_status_get function 416
Table 555. xmc_interrupt_flag_clear function 417
Table 556. xmc_flag_clear function 418
Table 557. Clock configuration guideline 424
Table 558. Document revision history 425

List of figures

Figure 1. Pack kit 37
Figure 2. IAR Pack installation window 37
Figure 3. IAR Pack installation window 38
Figure 4. View IAR Pack installation status 39
Figure 5. View Keil_v5 Pack installation status 40
Figure 6. Keil_v4 Pack installation status 41
Figure 7. Keil_v4 Pack installation process 41
Figure 8. Keil_v4 Pack installation complete 42
Figure 9. View Keil_v4 Pack installation status 43
Figure 10. Segger pack installation window 44
Figure 11. Segger pack installation process 44
Figure 12. Open J-Flash 45
Figure 13. Create a new project using J-Flash 45
Figure 14. View Device information 46
Figure 15. Keil algorithm file settings 47
Figure 16. Keil algorithm file configuration 48
Figure 17. Select algorithm files using Keil 48
Figure 18. Add algorithm files using Keil 49
Figure 19. IAR project name 50
Figure 20. IAR algorithm file configuration 50
Figure 21. IAR Flash Loader Overview 51
Figure 22. IAR Flash Loader configuration 51
Figure 23. IAR Flash Loader configuration success 52
Figure 24. Template content 53
Figure 25. Keil_v5 template project example 54
Figure 26. Peripheral enable macro definitions 55
Figure 27. BSP folder structure 61
Figure 28. BSP function library structure 62
Figure 29. Change device part number in Keil 419
Figure 30. Change macro definition in Keil 420
Figure 31. Change device part number in IAR 421
Figure 32. Change macro definition in IAR 422
Figure 33. Error warning 1 422
Figure 34. Error warning 2 422

Figure 35. Error warning 3 423
Figure 36. JLinkLog and JLinkSettings 423
Figure 37. Unspecified Cortex–M4 423
Figure 38. AT32_New_Clock_Configuration window 424

1 Overview

In order to allow users to make efficient use of Artery MCU, we provide a complete set of BSP & Pack tools to speed up development. They include peripheral driver library, core-related documents and application cases as well as Pack documents supporting a variety of development environments such as Keil_v5, Keil_v4, IAR_v6, IAR_v7 and IAR_v8. The BSP and Pack are available on Artery official website.

This application note is written to present how to use BSP and Pack.

2 How to install Pack

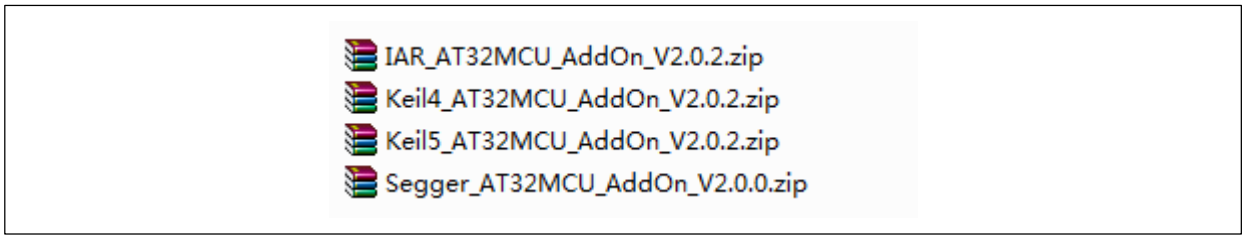
Artery Pack supports various development environment such as Keil_v5, Keil_v4, IAR_v6, IAR_v7 and IAR_v8.

Double click on the corresponding Pack to finish installation.

Note: This section takes AT32F403A as an example, and other AT32 MCUs have similar Pack installation methods.

The installation package is shown in Figure 1 (the specific version information is subject to the actual conditions).

Figure 1. Pack kit



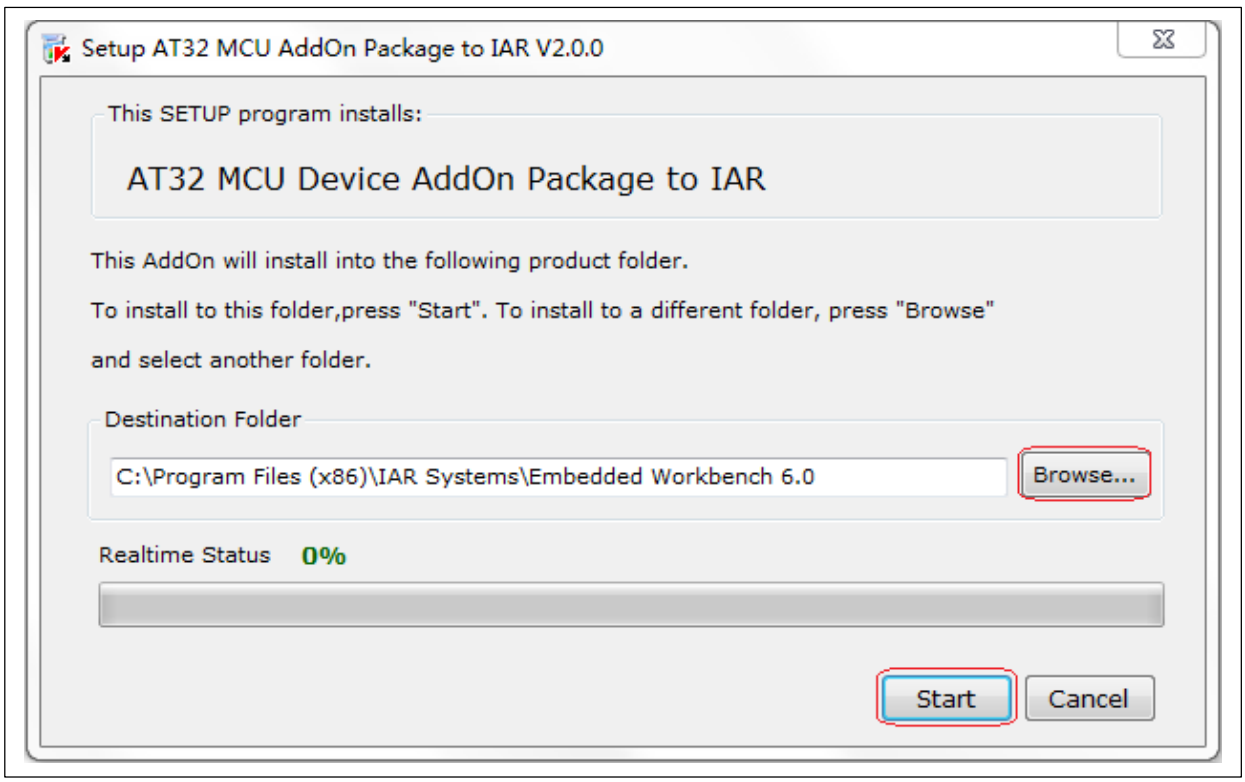
2.1 IAR Pack installation

IAR_AT32MCU_AddOn.zip: This is a zip file supporting IAR_V6, IAR_V7 and IAR_V8. Follow the steps below to install:

① Unzip *IAR_AT32MCU_AddOn.zip*

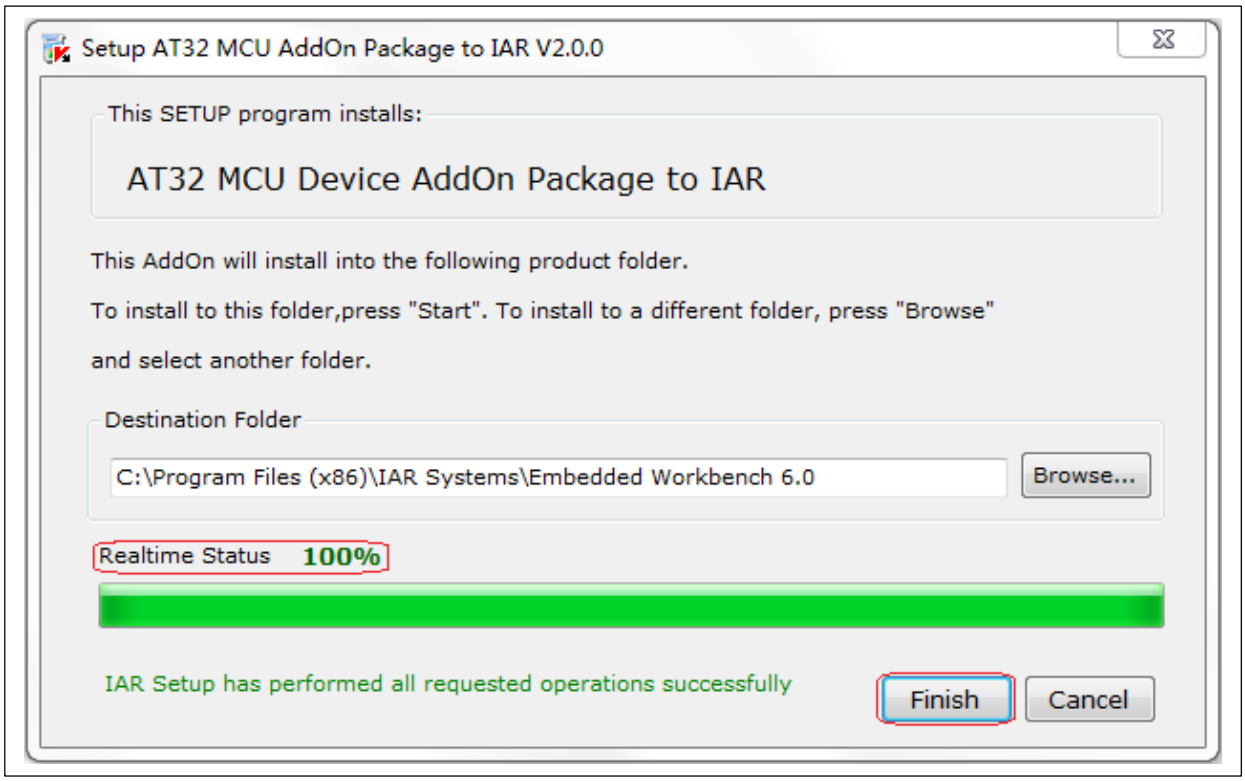
② Double click on *IAR_AT32MCU_AddOn.exe*, and a dialogue box is displayed below (the specific version information is subject to the actual conditions).

Figure 2. IAR Pack installation window



Note: If the installation path of IAR does not match the Destination Folder, click on "Browse" to select a correct path, then click on "Start", as shown below.

Figure 3. IAR Pack installation window

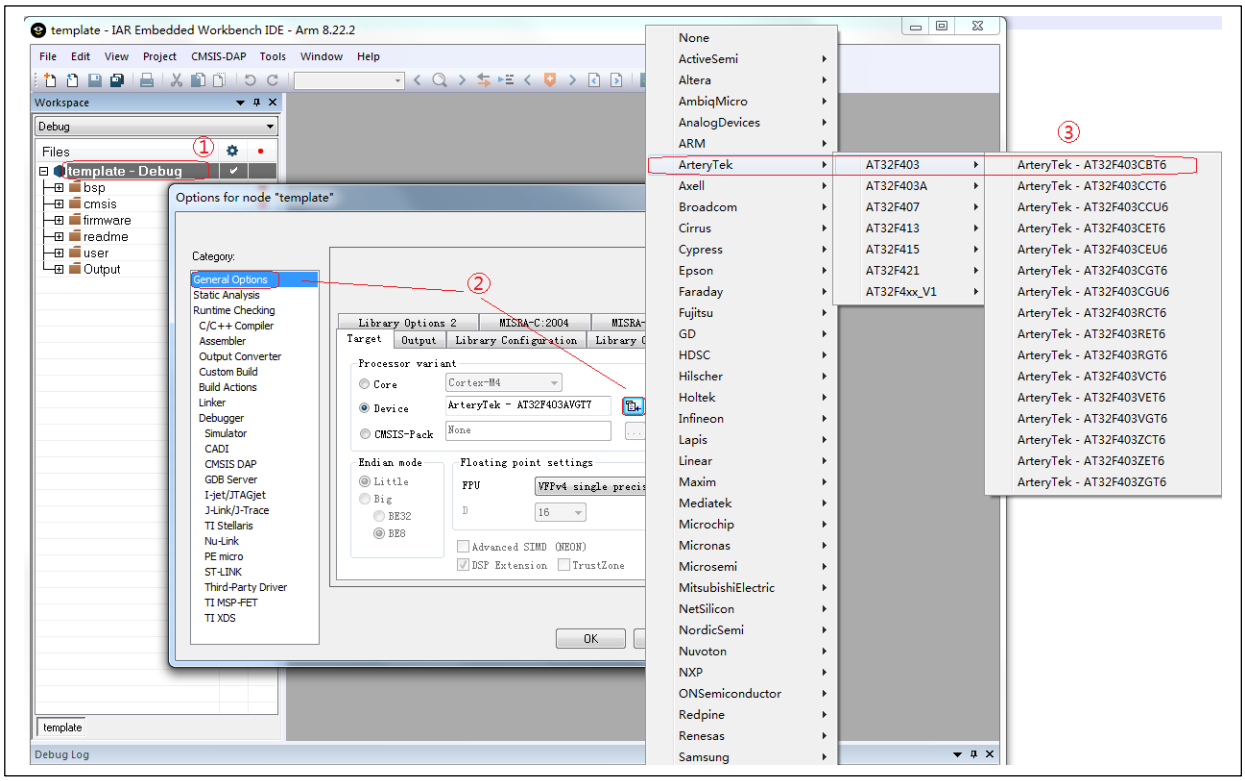


③ Click on “Finish”

④ To check whether the IAR Pack is installed successfully or not, open an IAR project and follow the steps below:

- Right click on a project name, and select “Options…”
- Select “General Options”, and click on the check box
- Click on “ArteryTek” and view AT32 MCU-related information.

Figure 4. View IAR Pack installation status



2.2 Keil_v5 Pack installation

Keil5_AT32MCU_AddOn.zip: This is a zip file supporting Keil_v5. Follow the steps below to install:

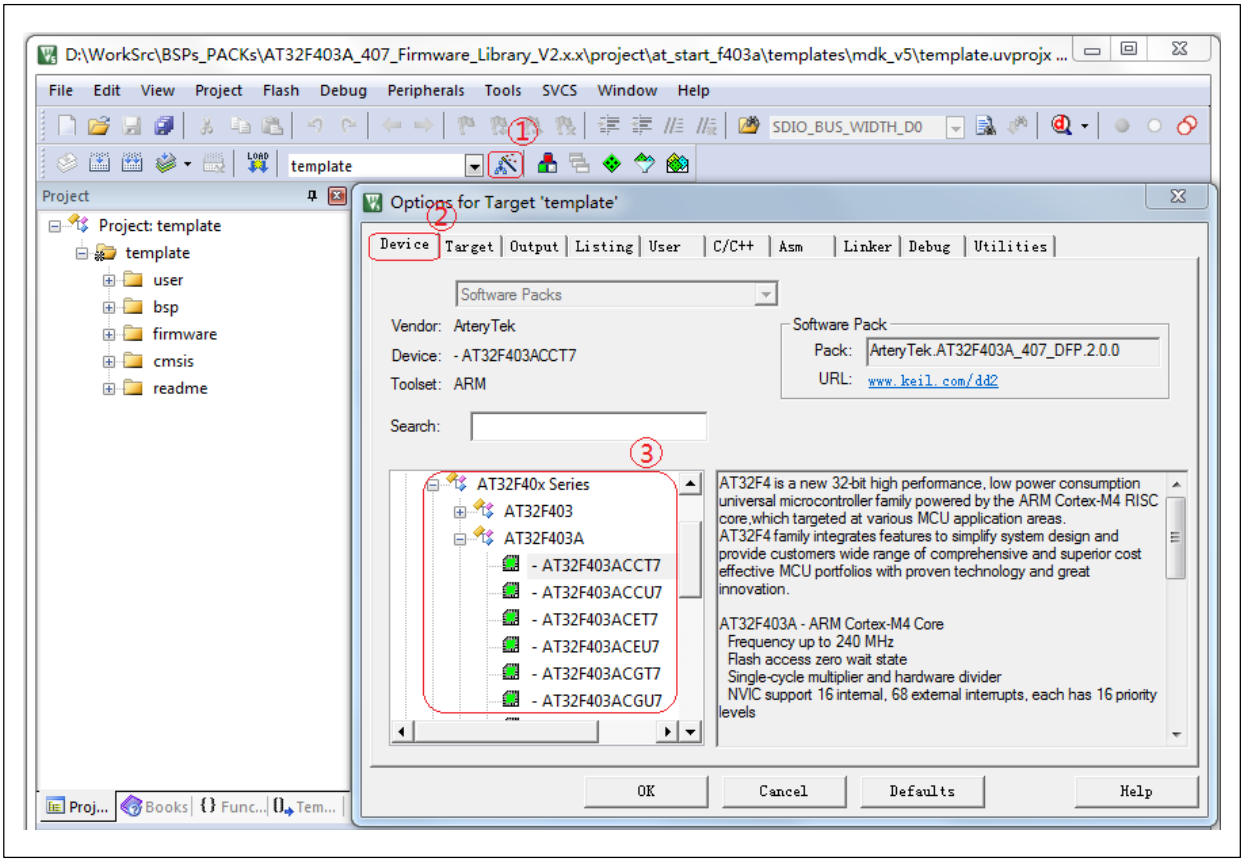
① Unzip *Keil5_AT32MCU_AddOn.zip*. This zip file includes all Keil5 packs supported, all of which are standard Keil_v5 DFP installation files.

② Select the desired Pack, and double click on *ArteryTek.AT32xxxx_DFP.2.x.x.pack to get one-stop installation.*

⑤ To check whether the Keil_v5 Pack is installed successfully or not, follow the steps below:

- Click on wand
- Select “Device”
- View AT32 MCU-related information.

Figure 5. View Keil_v5 Pack installation status



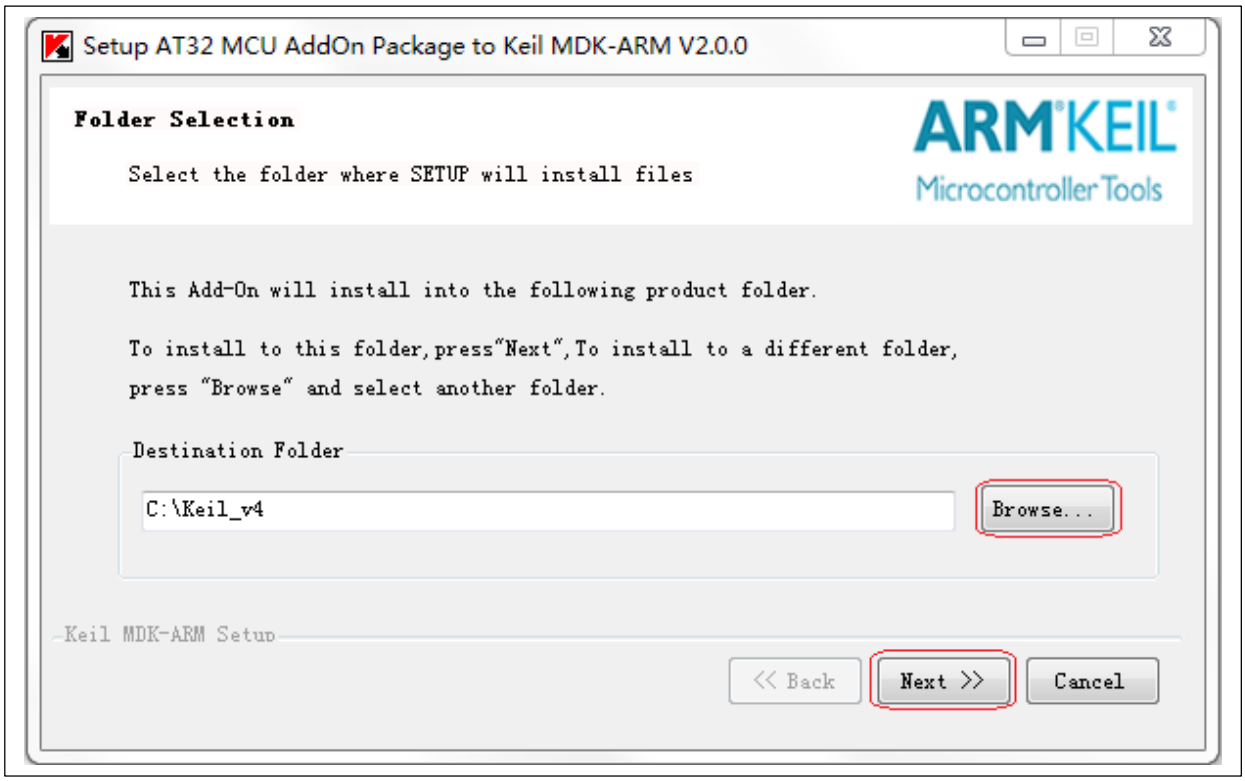
2.3 Keil_v4 Pack installation

Keil4_AT32MCU_AddOn.zip:This is a zip file supporting Keil_v4, follow the steps below to finish installation:

① Unzip Keil4_AT32MCU_AddOn.zip

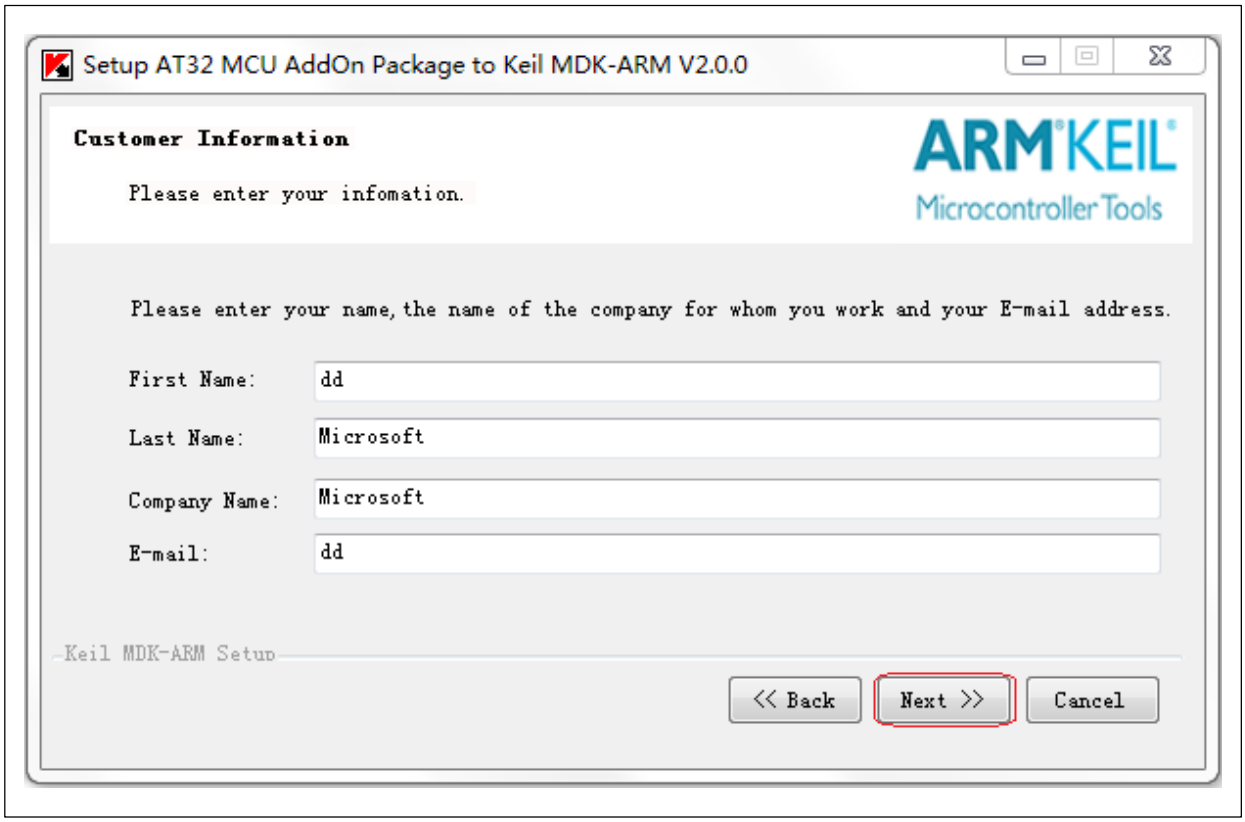
② Click on Keil4_AT32MCU_AddOn.exe, and a dialog box pops up below (the specific version information is subject to the actual conditions).

Figure 6. Keil_v4 Pack installation status



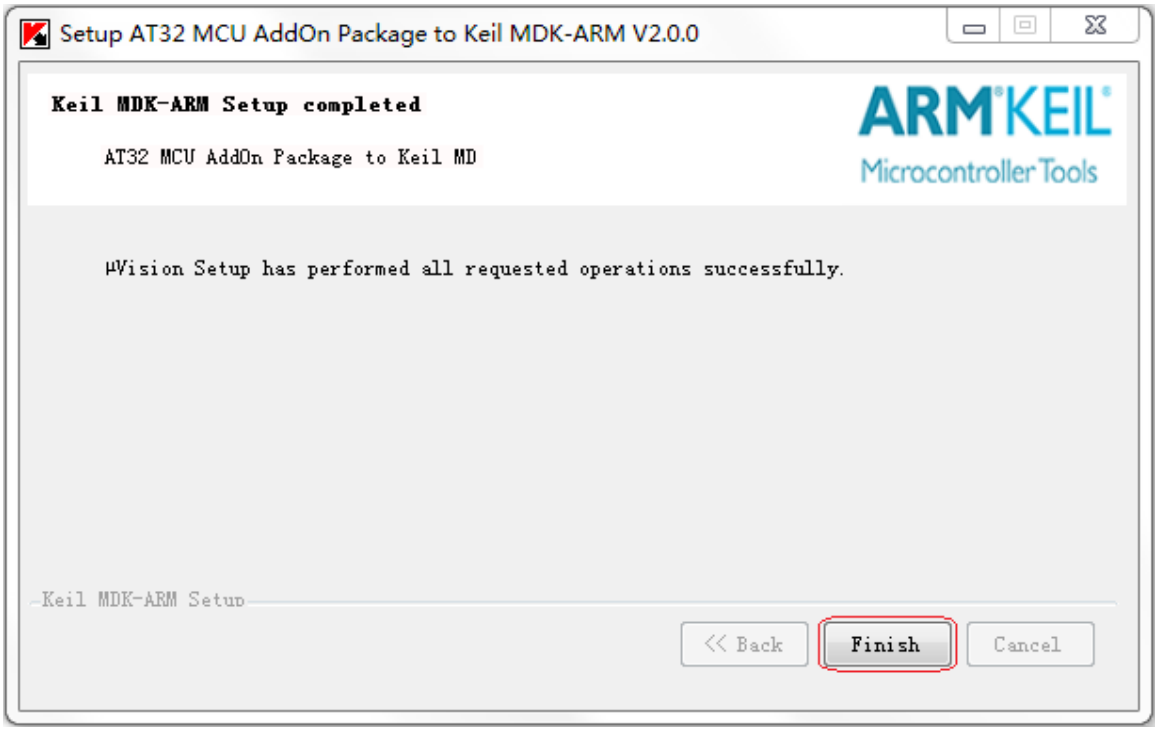
③ If the installation path of Keil_v4 does not match the “Destination Folder”, click on “Browse” to select the actual correct path, then click on “Next”, as shown below.

Figure 7. Keil_v4 Pack installation process



④ In the above “Customer Information” window, you can make some changes, but usually it is unnecessary. Then click on “Next” to start installation. The installation result is as follows.

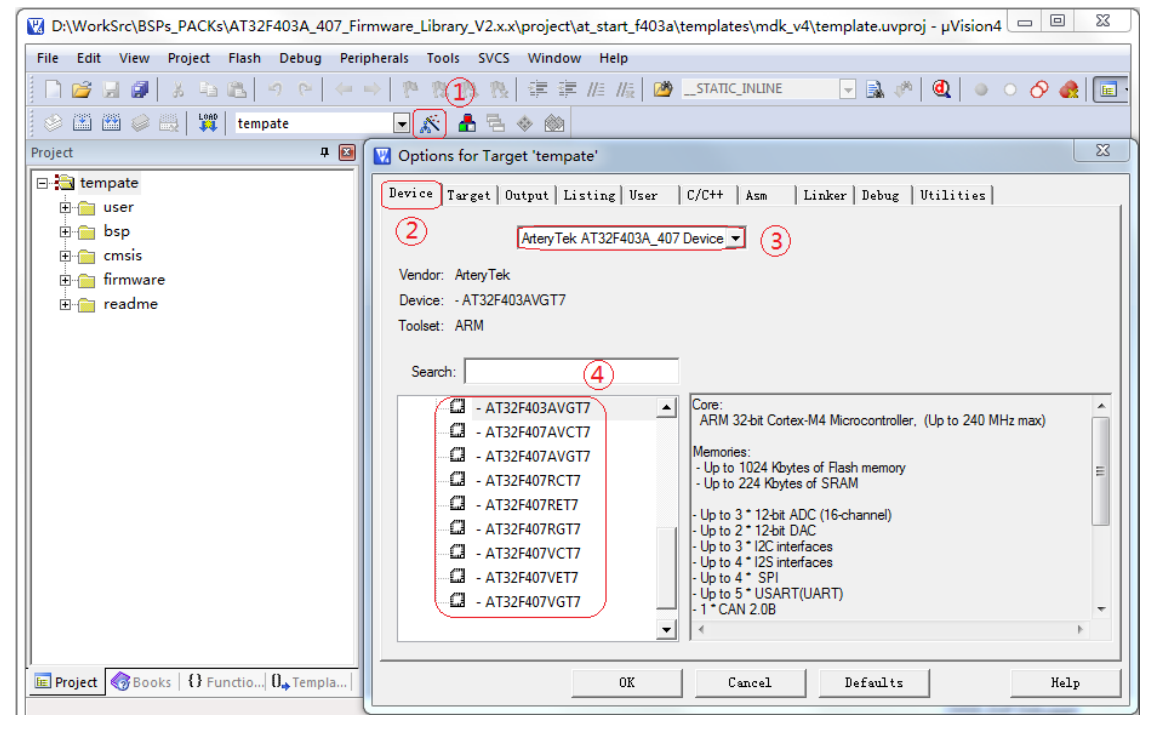
Figure 8. Keil_v4 Pack installation complete



⑤ Click on “Finish”. To check whether Keil_v4 Pack is installed successfully or not, follow the below steps:

- Click on “wand”
- Select “Device”
- Select the desired pack file
- View ArteryTek-related information.

Figure 9. View Keil_v4 Pack installation status



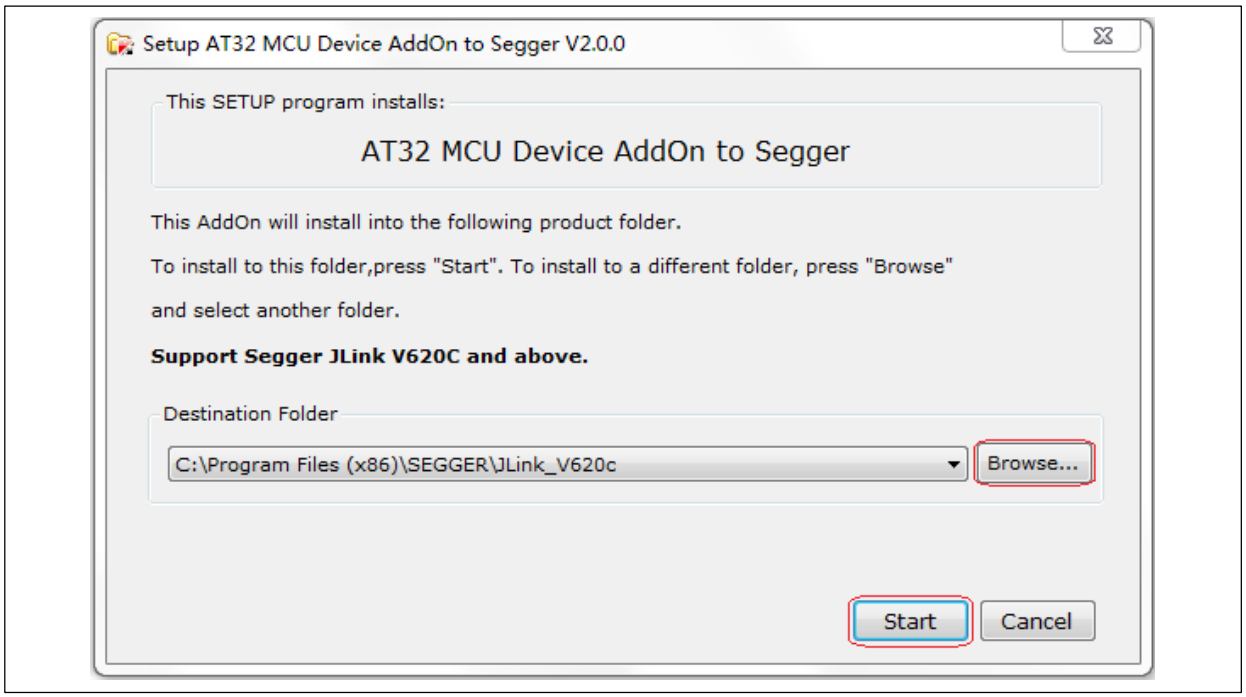
2.4 Segger Pack installation

Segger_AT32MCU_AddOn.zip: This is used to download J-Flash, follow the steps below to install:

① Unzip Segger_AT32MCU_AddOn.zip

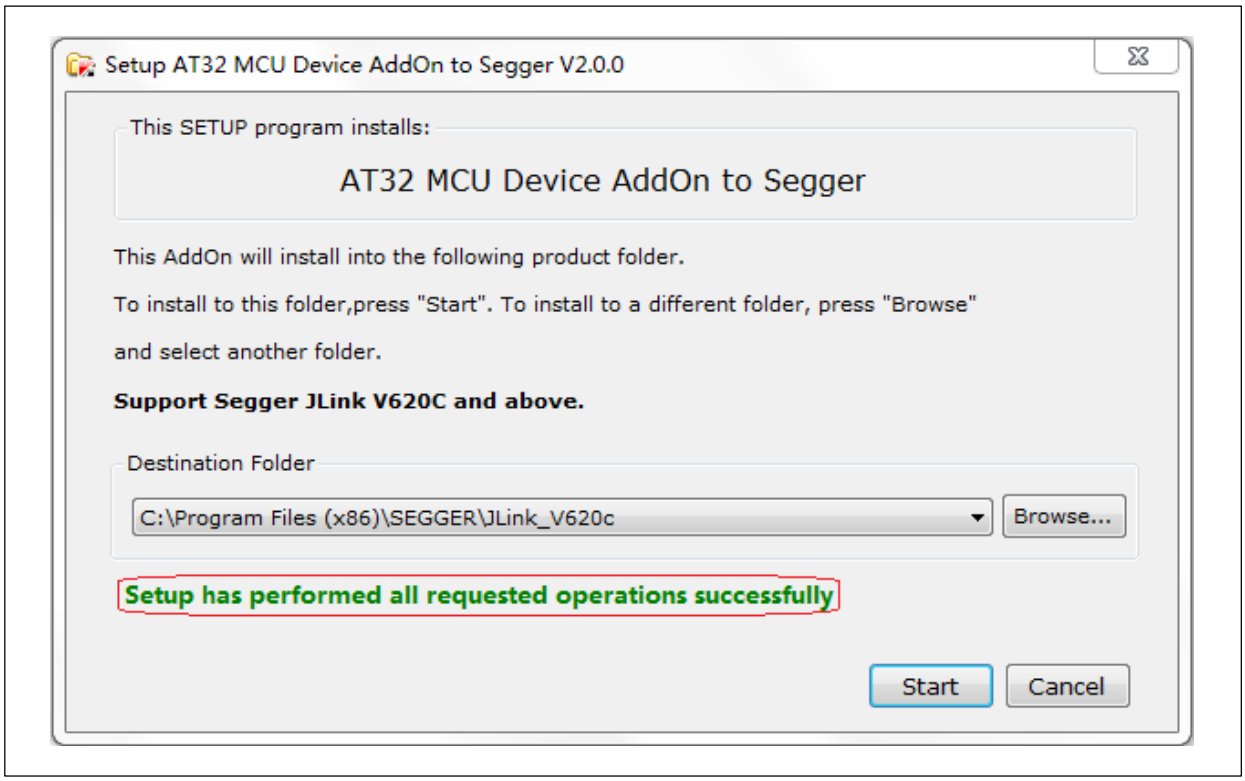
② Click on Segger_AT32MCU_AddOn.exe, and a dialog box pops up below (the specific version information is subject to the actual conditions)

Figure 10. Segger pack installation window



Note: If the installation path of Segger does not match the "Destination Folder", click on "Browse" to select a correct path, then click on "Start", as shown below.

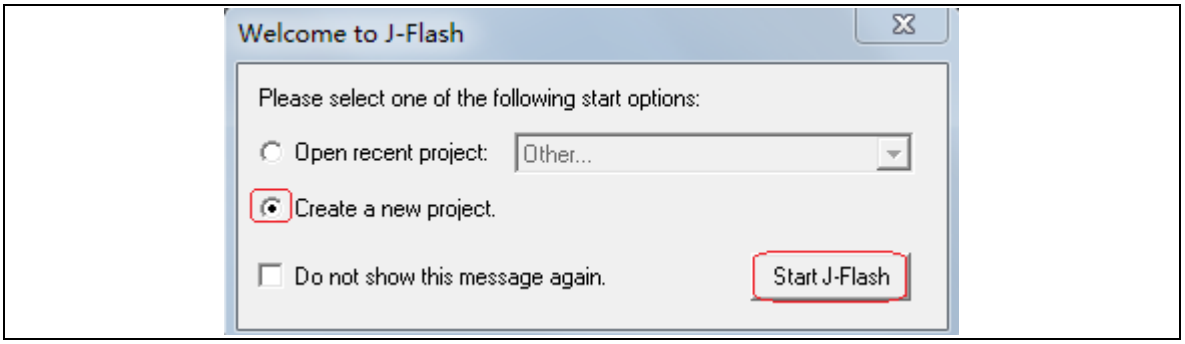
Figure 11. Segger pack installation process



③ If the “Setup has performed all requested operations successfully” appears, it indicates successful installation. To check whether the installation is successful or not, follow the steps below:

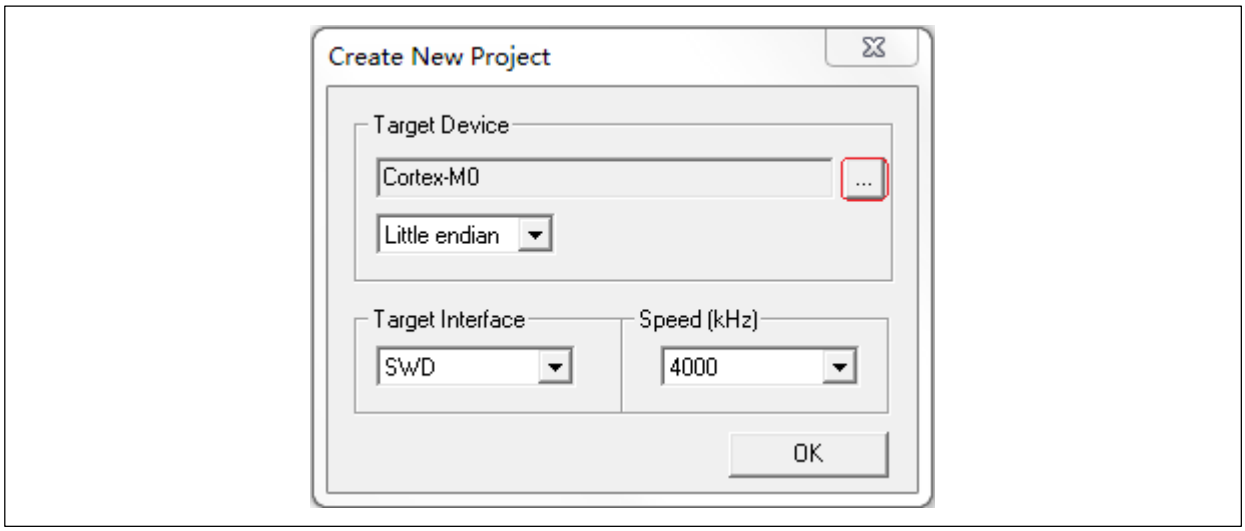
- Open J-Flash.exe, a dialog box appears, tick “Create a new project” and click on “Start J-Flash”:

Figure 12. Open J-Flash



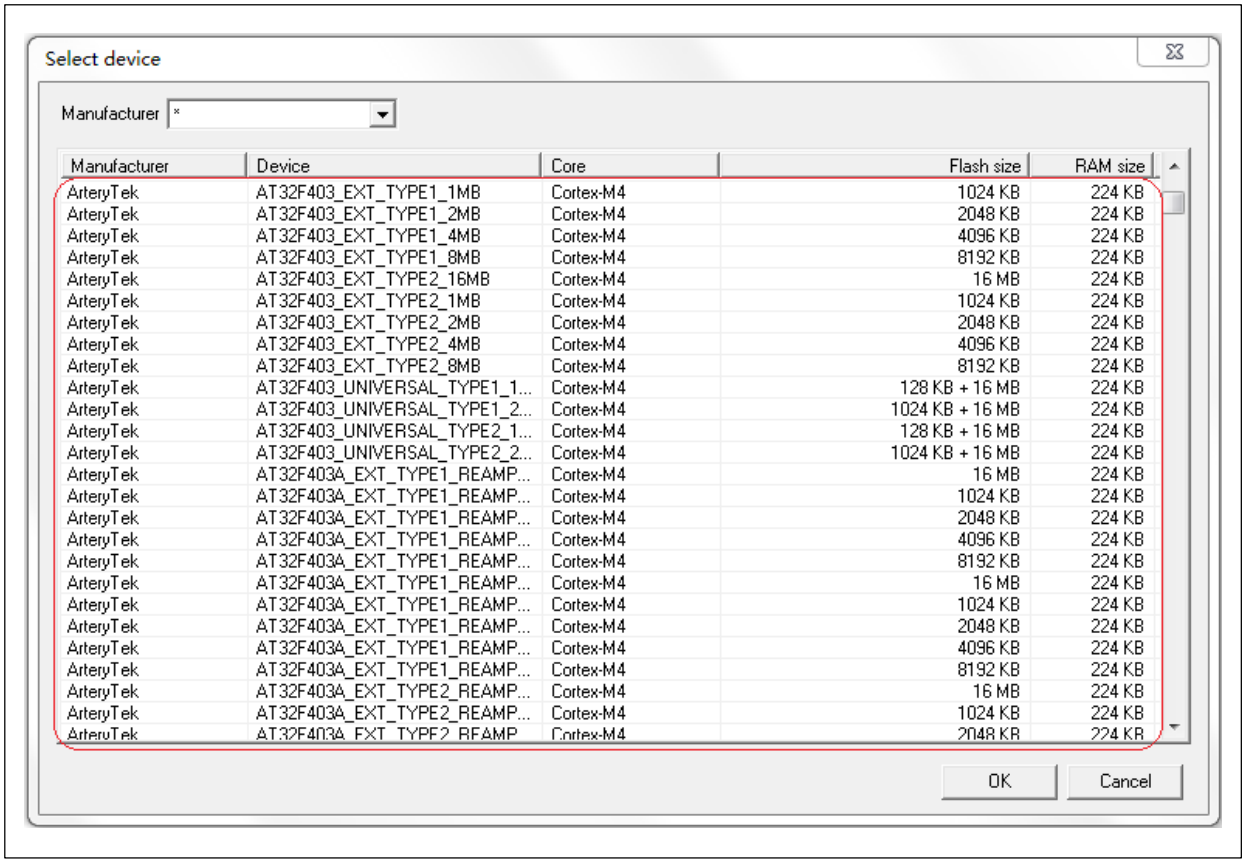
- After “Start J-Flash”, click on the check box under “Target Device”:

Figure 13. Create a new project using J-Flash



– Drag the scroll bar up and down in the check box. If the ArteryTek-related information and algorithm documents can be found, the installation is successful, as shown below:

Figure 14. View Device information



3 Flash algorithm file

Flash algorithm files are included in the Pack for online download through IDE tools such as KEIL/IAR. This section describes how to use Flash algorithm files.

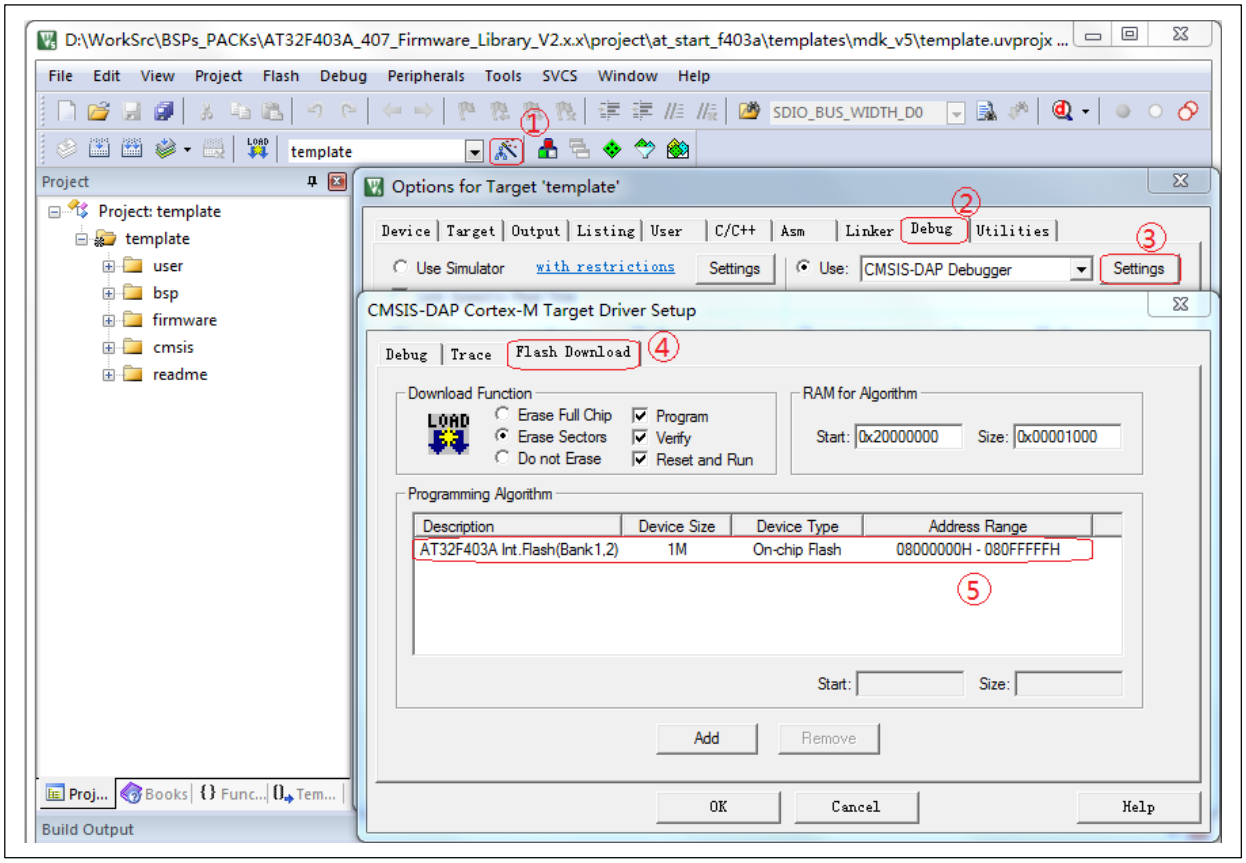
Note: AT32 MUCs have similar Flash algorithms, and this section uses AT32F403A as an example.

3.1 How to use Keil algorithm file

Common IDE tools such as Keil_v4 and Keil_v5 adopt a similar method to select and use the algorithm files. Here we take Keil_v5 as an example.

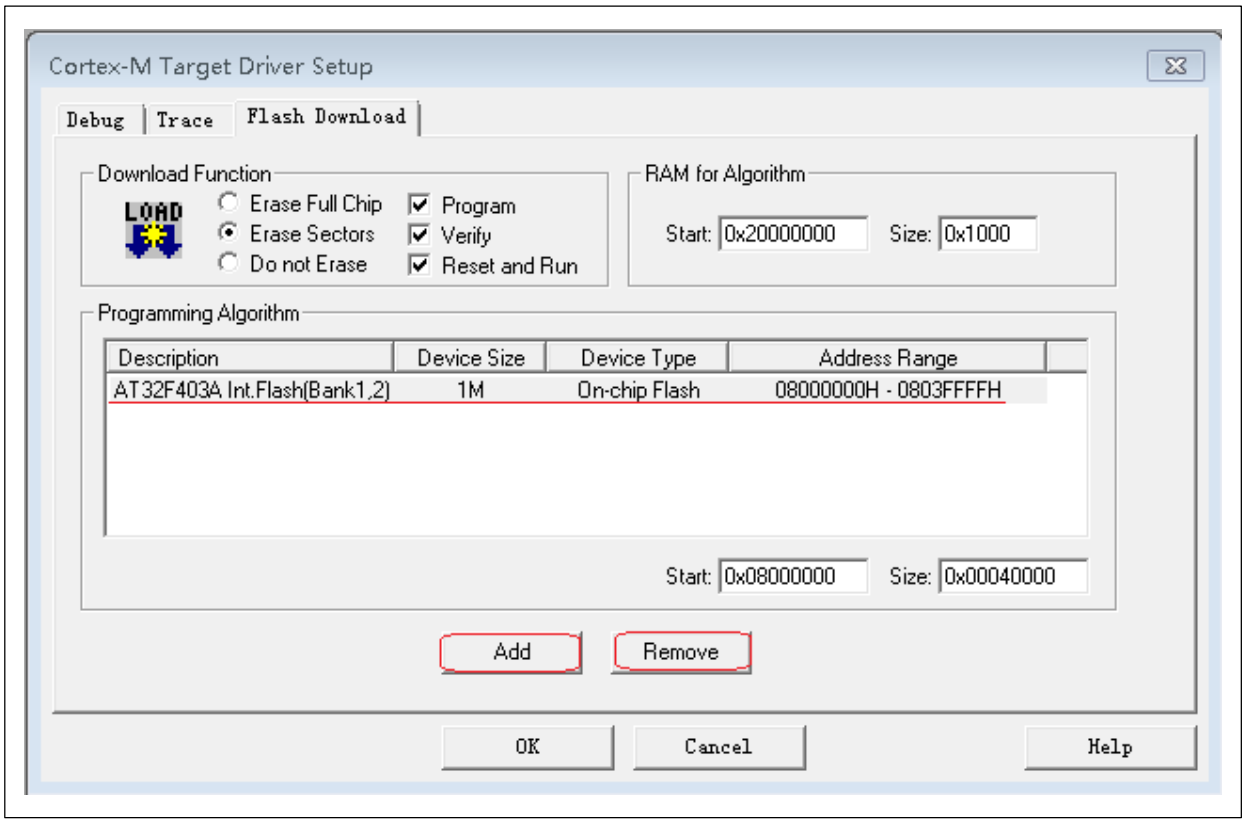
After creating a Keil IDE development tool project, the user can start Debug configuration and select the Flash algorithms. Go to *wand —> Debug —> Settings —> Flash Download*, as shown below:

Figure 15. Keil algorithm file settings



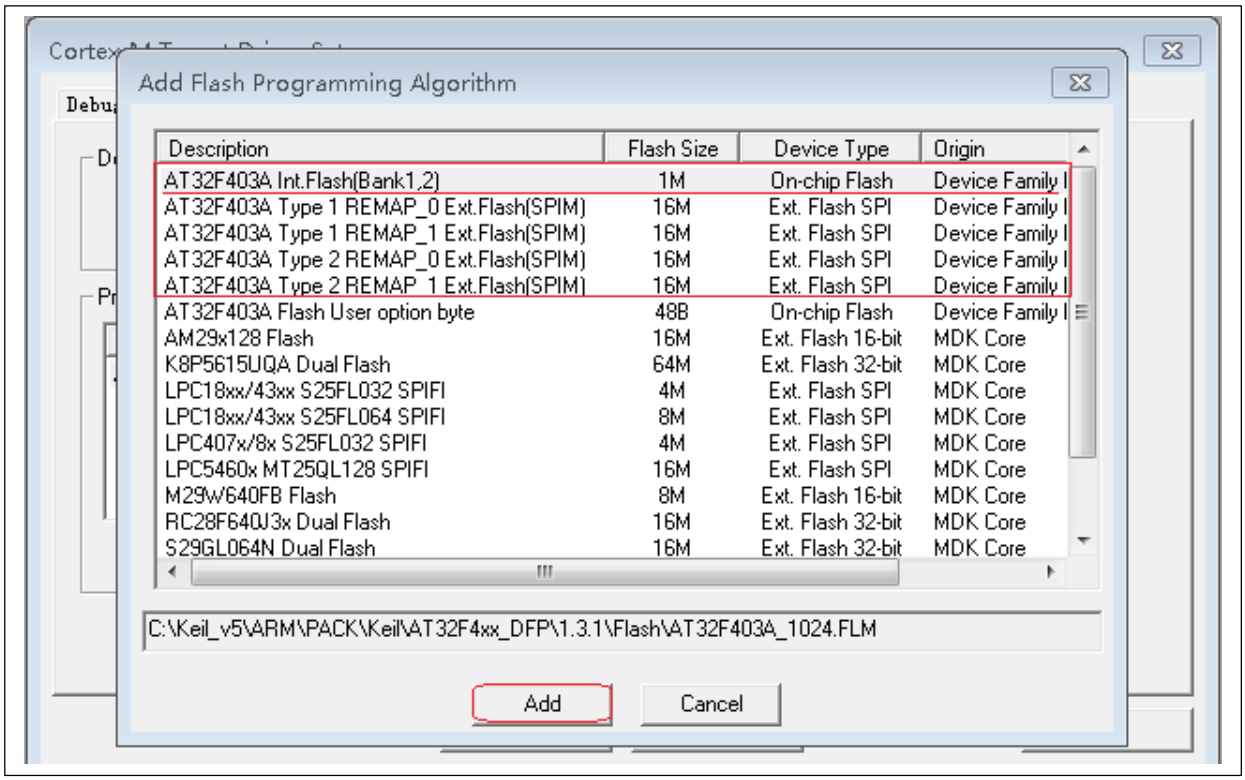
In this example, the selected Flash algorithm file is the default one. To change or remove it, click on this algorithm file, then click on *Add* or *Remove*. If the selected algorithm does not match the MCU, please follow the method below to modify.

Figure 16. Keil algorithm file configuration



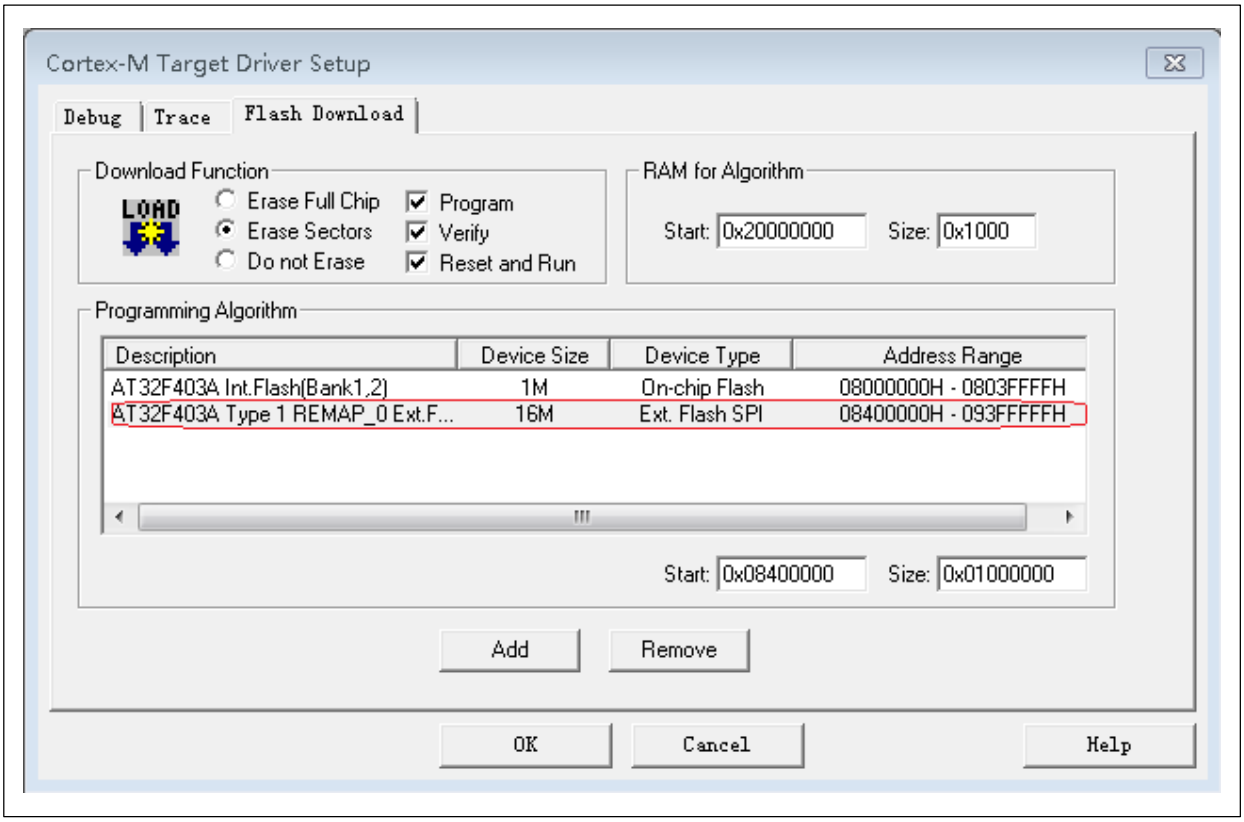
Click on *Remove* to remove the existing algorithm from the configuration, then click on *Add* to view the algorithm files associated with a MCU model and select them, as shown below:

Figure 17. Select algorithm files using Keil



After selection, click on *Add* to add the selected algorithm files into the current configuration. For example, a new SPIM algorithm is added into the project.

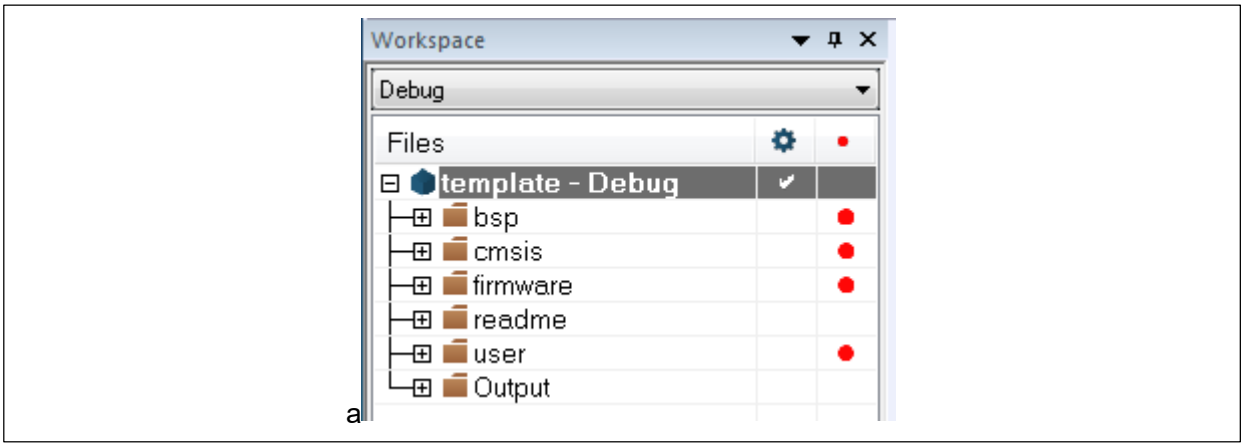
Figure 18. Add algorithm files using Keil



3.2 How to use IAR algorithm files

In IAR environment, the Flash algorithm files are automatically selected according to the selected MCU model during a new project configuration. To configure/modify an algorithm file manually, right-click on the file name (after an IAR project is created) in the following gray box:

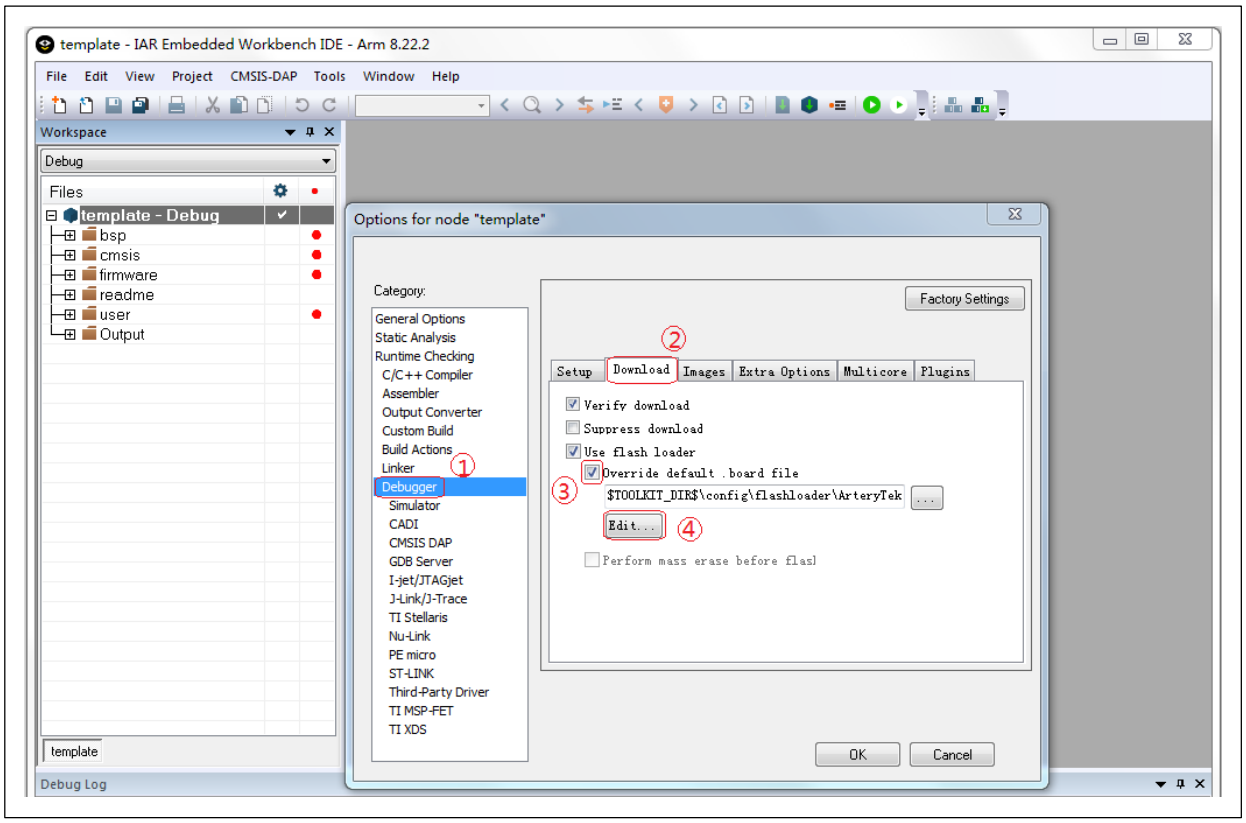
Figure 19. IAR project name



a

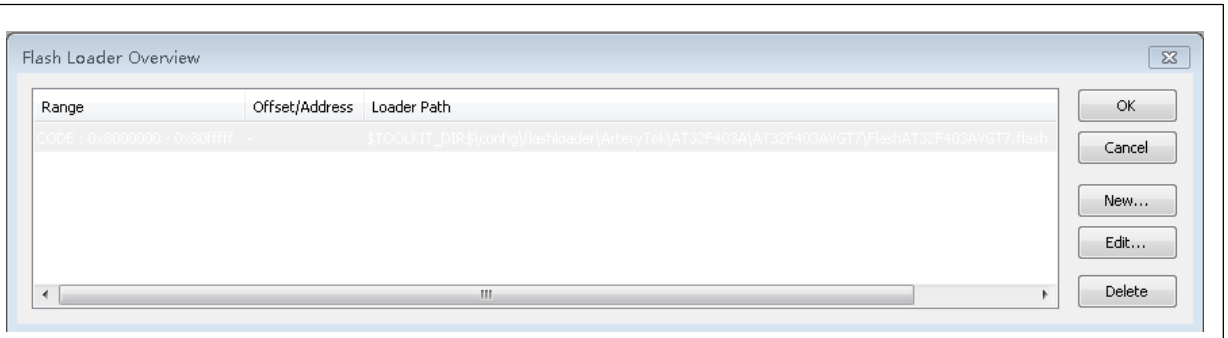
Go to *Options—> Debugger —> Download —>* Tick *Override default .board file —>* Click on *Edit*, as shown below:

Figure 20. IAR algorithm file configuration



Then the following window will be displayed.

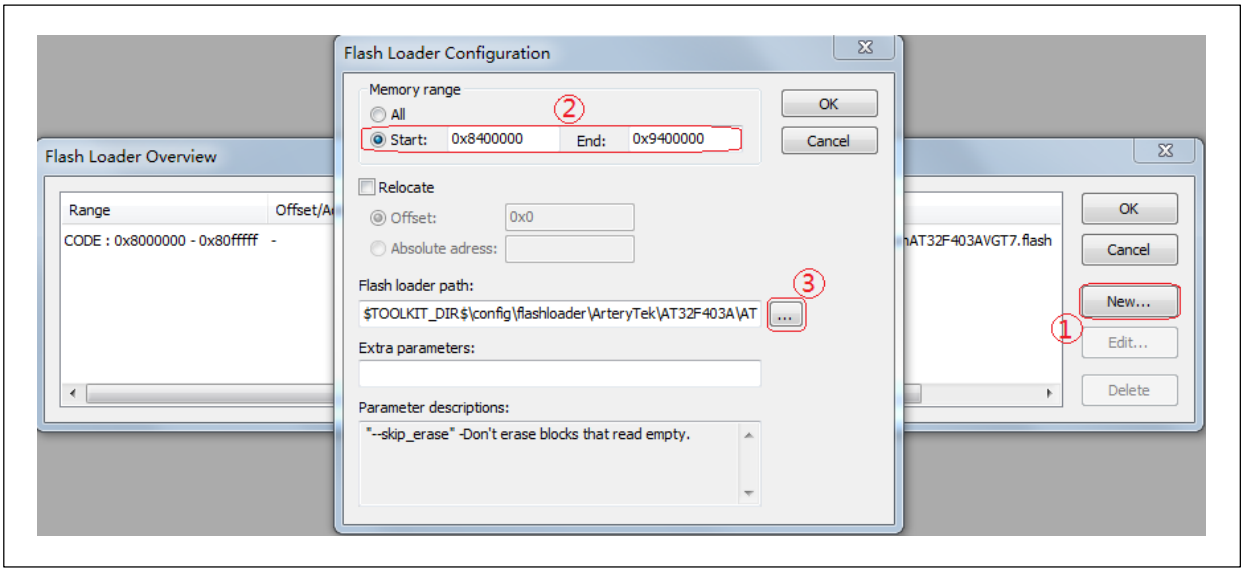
Figure 21. IAR Flash Loader Overview



Flash algorithm configuration is designated by default after selecting a MCU part number. To modify it, click on *New/Edit/Delete*.

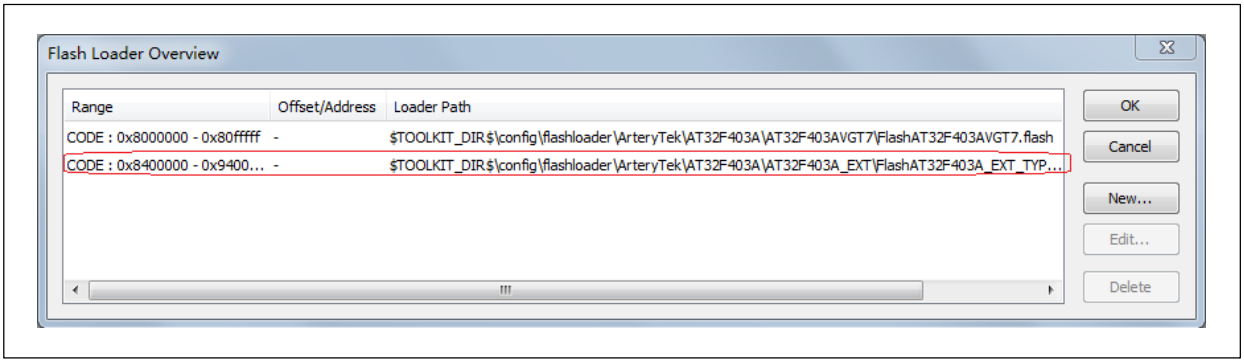
For example, click on *New* —> *Memory range* —> Select a Flash algorithm file, as shown below:

Figure 22. IAR Flash Loader configuration



This example shows how to add a SPIM Flash algorithm file. The user needs to select the corresponding MCU part number and a correct Flash algorithm file. The selected Flash algorithm configuration file is installed into IAR development environment using IAR_AT32F4xx_AddOn tool. After a successful configuration, a new SPIM Flash algorithm is shown below:

Figure 23. IAR Flash Loader configuration success



1. Description of SPIM algorithms

Some Artery MCUs support Bank3 (refer to the Reference Manual or Datasheet on Artery official website for details), which can be used as an expansion of Flash memory in case of insufficient internal Flash or special application requirements. When the compiling addresses of some code or data are stored in the SPIM, these algorithm files are used for external Flash programming during online IDE tool download.

Naming rules of Artery SPIM algorithm file: AT32F4xxTypeNREMAP_P Ext.Flash

N=1,2

P=0,1

TYPEN: External SPI Flash, select it according to the external Flash type and part number. Refer to the FLASH_SELECT register section of the corresponding MCU Reference Manual.

REMAP_P: Select multiplex-function MCU SPIM PIN, select it according to the connection method of pins connected to external Flash. Refer to the external SPIF remapping section in the corresponding MCU reference manual.

REMAP0:EXT_SPIF_GRMP=000

REMAP1:EXT_SPIF_GRMP=001

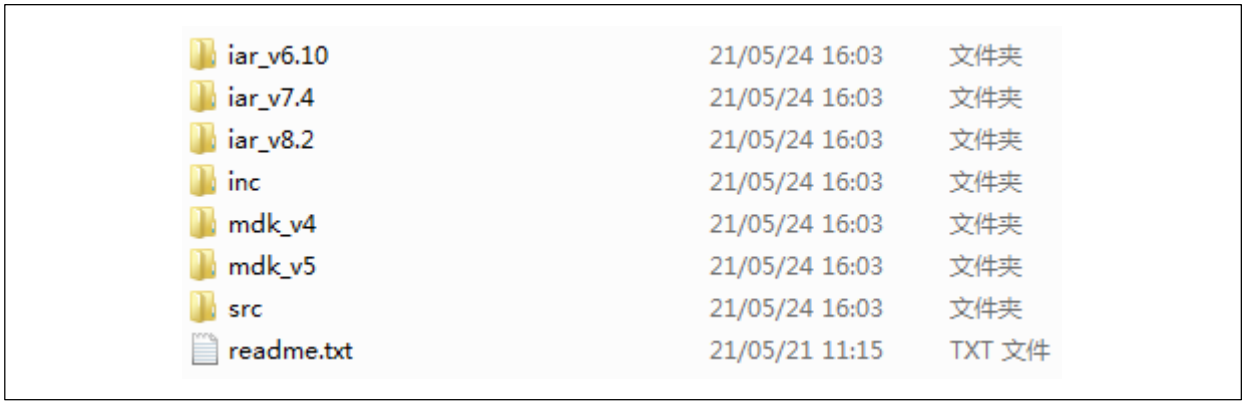
4 BSP introduction

4.1 Quick start

4.1.1 Template project

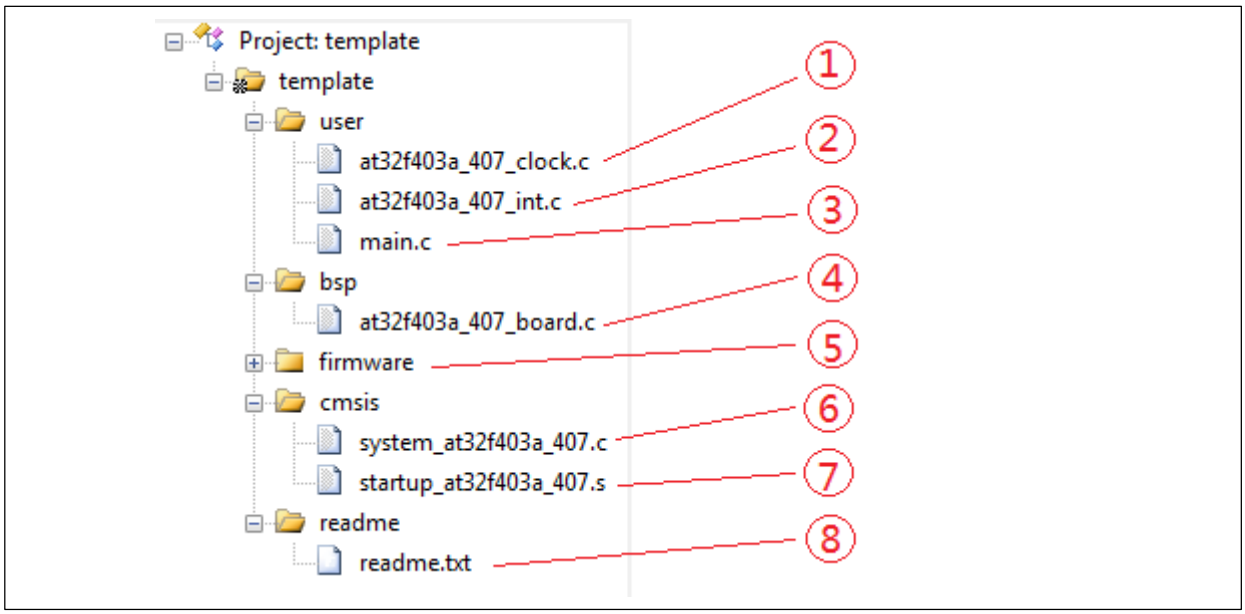
Artery firmware library BSP comes with a series of template projects built around Keil and IAR. For example, the template project of AT32F403A/407 is located in *AT32F403A_407_Firmware_Library_V2.x.x/project/at_start_xxx/templates*.

Figure 24. Template content



The above template project includes various versions such as Keil_v5, Keil_v4, IAR_6.10, IAR_7.4 and IAR_8.2. Of those, “inc” and “src” folders contain header files and source code files. Open a corresponding folder and click on the corresponding file to open an IDE project. Figure 25 presents an example of Keil_v5 template project (its details and version are subject to the actual firmware library).

Figure 25. Keil_v5 template project example



The contents in a project include: (using AT32F403A/407 as an example, other products are similar)

① at32f403a_407_clock.c (clock configuration file) defines the default clock frequency and clock paths

② at32f403a_407_int.c (interrupt file) contains some interrupt handling codes

③ main.c contains the main code files.

④ at32f403a_407_board.c (board configuration file) contains common hardware configurations such as buttons and LEDs on the AT-START-Evaluation Board.

⑤ at32f403a_407_xx.c under firmware folder contains driver files of on-chip peripherals

⑥ system_at32f403a_407.c is the system initialization file

⑦ startup_at32f403a_407.s is a startup file

⑧ readme.txt is a readme file, containing functional description and configuration information

Note: AT32 MUCs share similar BSP usage method, and this section uses AT32F403A as an example.

4.1.2 BSP macro definitions

① To create a project, it is necessary to enable a startup code (startup_at32f403a_407.s) and open the appropriate macro definitions according to MCU part number before compiling code. Table 1 presents the correspondence between the MCU and their macro definitions.

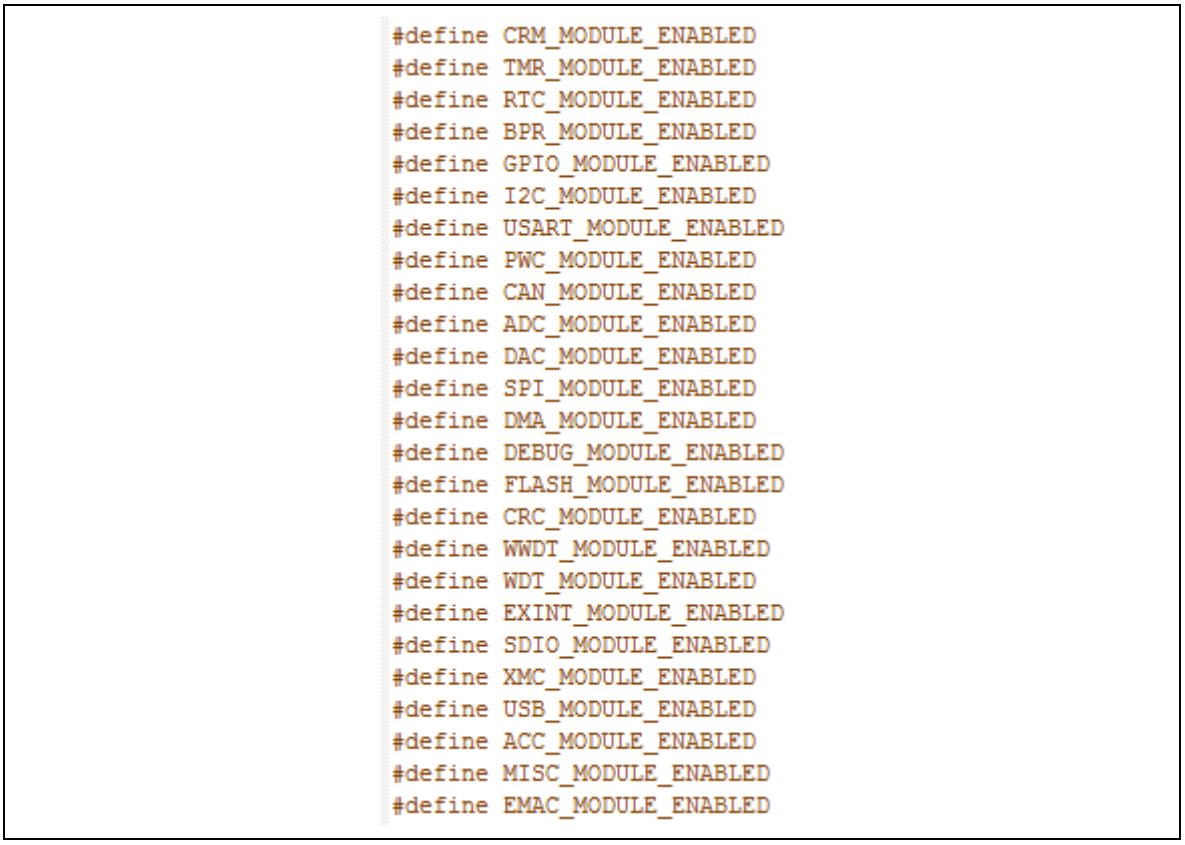
Table 1. Summary of macro definitions

MCU part numbers	Macro definitions	PINs	Flash size (KB)
AT32F403ACCT7	AT32F403ACCT7	48	256
AT32F403ACET7	AT32F403ACET7	48	512
AT32F403ACGT7	AT32F403ACGT7	48	1024
AT32F403ACCU7	AT32F403ACCU7	48	256
AT32F403ACEU7	AT32F403ACEU7	48	512
AT32F403ACGU7	AT32F403ACGU7	48	1024
AT32F403ARCT7	AT32F403ARCT7	64	256
AT32F403ARET7	AT32F403ARET7	64	512
AT32F403ARGT7	AT32F403ARGT7	64	1024
AT32F403AVCT7	AT32F403AVCT7	100	256
AT32F403AVET7	AT32F403AVET7	100	512
AT32F403AVGT7	AT32F403AVGT7	100	1024
AT32F407RCT7	AT32F407RCT7	64	256
AT32F407RET7	AT32F407RET7	64	512
AT32F407RGT7	AT32F407RGT7	64	1024
AT32F407VCT7	AT32F407VCT7	100	256
AT32F407VET7	AT32F407VET7	100	512
AT32F407VGT7	AT32F407VGT7	100	1024
AT32F407AVCT7	AT32F407AVCT7	100	256
AT32F407AVGT7	AT32F407AVGT7	100	1024

② In the header file (at32f403a_407.h), USE_STDPERIPH_DRIVER (macro definition) is used to determine whether the Keil RTE feature is used or not. Enabling this definition while Keil RTE is unused can prevent some versions of Keil-MDK from opening _RTE_ accidentally.

③ The configuration header file (at32f403a_407_conf.h) defines macro definitions that enable peripherals. The file can be used to control the use of peripherals. The peripherals can be disabled simply by masking _MODULE_ENABLED pertaining to peripherals, as shown below:

Figure 26. Peripheral enable macro definitions



```
#define CRM_MODULE_ENABLED
#define TMR_MODULE_ENABLED
#define RTC_MODULE_ENABLED
#define BPR_MODULE_ENABLED
#define GPIO_MODULE_ENABLED
#define I2C_MODULE_ENABLED
#define USART_MODULE_ENABLED
#define PWC_MODULE_ENABLED
#define CAN_MODULE_ENABLED
#define ADC_MODULE_ENABLED
#define DAC_MODULE_ENABLED
#define SPI_MODULE_ENABLED
#define DMA_MODULE_ENABLED
#define DEBUG_MODULE_ENABLED
#define FLASH_MODULE_ENABLED
#define CRC_MODULE_ENABLED
#define WWDT_MODULE_ENABLED
#define WDT_MODULE_ENABLED
#define EXINT_MODULE_ENABLED
#define SDIO_MODULE_ENABLED
#define XMC_MODULE_ENABLED
#define USB_MODULE_ENABLED
#define ACC_MODULE_ENABLED
#define MISC_MODULE_ENABLED
#define EMAC_MODULE_ENABLED
```

at32f403a_407_conf.h also defines the HEXT_VALUE (high-speed external clock value), which should be modified accordingly when changing an external high-speed crystal oscillator.

④ The system clock configuration file (at32f403a_407_clock.c/.h) defines the default system clock frequency and clock paths. The user, if needed, can customize the frequency multiplication process and factors, or generate corresponding clock configuration files using the clock configuration host of ArteryTek.

4.2 BSP specifications

The subsequent sections give a description of BSP specifications.

4.2.1 List of abbreviations for peripherals

Table 2. List of abbreviations for peripherals

Abbreviations	Description
ADC	Analog-to-digital converter
BPR	Battery powered register
CAN	Controller area network
CRC	CRC calculation unit
CRM	Clock and reset manage
DAC	Digital-to analog converter
DMA	Direct memory access
DEBUG	Debug
EXINT	External interrupt/event controller
GPIO	General-purpose I/Os
IOMUX	Multiplexed I/Os
I2C	Inter-integrated circuit interface
NVIC	Nested vectored interrupt controller
PWC	Power controller
RTC	Real-time clock
SPI	Serial peripheral interface
I2S	Inter-IC Sound
SysTick	System tick timer
TMR	Timer
USART	Universal synchronous asynchronous receiver transmitter
WDT	Watchdog timer
WWDT	Window watchdog timer
XMC	External memory controller

4.2.2 Naming rules

The naming rules for BSP are described as follows:
"ip" indicates an abbreviation of a peripheral, for example, ADC, TMR, GPIO, etc, regardless of upper and lower case letters, such as, adc,tmr,gpio…

- **Source code file**
  The file name starts with **"**at32fxxx_ip.c**"**, for example, at32f403a_407_adc.c
- **Header file**
  The file name starts with "at32fxxx_ip.h", such as, at32f403a_407_adc.h
- **Constant**
  If it is used in a single one file, the constant is then defined in this file; if it is used in multiple files, the constant is defined in corresponding header file.
  All constants are in written in English capital letters.
- **Variable**
  If it is used in a single one file, the variable is then defined in this file; if it is used in multiple files, the variable is declared with extern in the corresponding header file.
- **Naming rules for functions**
  The peripheral functions are named based on the rule of "peripheral abbreviation_attribute_action" or "peripheral abbreviation_action".
  The commonly used functions are as follows:

Function type	Naming rule	Example
Peripheral reset	ip_reset	such as, adc_reset
Peripheral enable	ip_enable	adc_enable
Peripheral structure parameter initialize	ip_default_para_init	spi_default_para_init
Peripheral initialize	ip_init	spi_init
Peripheral interrupt enable	ip_interrupt_enable	adc_interrupt_enable
Peripheral flag get	ip_flag_get	adc_flag_get
Peripheral flag clear	ip_flag_clear	adc_flag_clear

4.2.3 Encoding rules

This section describes the encoding rules related to firmware function library.

Type of variables:

```
typedef int32_t  INT32;
typedef int16_t  INT16;
typedef int8_t   INT8;
typedef uint32_t UINT32;
typedef uint16_t UINT16;
typedef uint8_t  UINT8;

typedef int32_t  s32;
typedef int16_t  s16;
typedef int8_t   s8;

typedef const int32_t sc32;   /*!< read only */
typedef const int16_t sc16;   /*!< read only */
typedef const int8_t  sc8;    /*!< read only */

typedef __IO int32_t  vs32;
typedef __IO int16_t  vs16;
typedef __IO int8_t   vs8;

typedef __I int32_t vsc32;    /*!< read only */
typedef __I int16_t vsc16;    /*!< read only */
typedef __I int8_t  vsc8;     /*!< read only */

typedef uint32_t u32;
typedef uint16_t u16;
typedef uint8_t  u8;

typedef const uint32_t uc32;  /*!< read only */
typedef const uint16_t uc16;  /*!< read only */
typedef const uint8_t  uc8;   /*!< read only */

typedef __IO uint32_t vu32;
typedef __IO uint16_t vu16;
typedef __IO uint8_t  vu8;

typedef __I uint32_t vuc32;   /*!< read only */
typedef __I uint16_t vuc16;   /*!< read only */
typedef __I uint8_t  vuc8;    /*!< read only */
```

4.2.3.1 Flag type

typedef enum {RESET = 0, SET = !RESET} flag_status;

4.2.3.2 Function status type

typedef enum {FALSE = 0, TRUE = !FALSE} confirm_state;

4.2.3.3 Error status type

typedef enum {ERROR = 0, SUCCESS = !ERROR} error_status;

4.2.3.4 Peripheral type

① Peripherals

Define the base address of peripheral in the at32fxxx_ip.h, for example, in the at32f403a_407.h:

```
#define ADC1_BASE                       (APB2PERIPH_BASE + 0x2400)
#define ADC2_BASE                       (APB2PERIPH_BASE + 0x2800)
```

Define the type of a peripheral in the at32fxxx_ip.h, for example, in the at32f403a_407_adc.h:

```
#define ADC1                            ((adc_type *) ADC1_BASE)
#define ADC2                            ((adc_type *) ADC2_BASE)
```

② Peripheral registers and bits

Define the type of a peripheral in the at32fxxx_ip.h, for example, in the at32f403a_407_adc.h

```
/**
  * @brief type define adc register all
  */
typedef struct
{

  /**
    * @brief adc sts register, offset:0x00
    */
  union
  {
    __IO uint32_t sts;
    struct
    {
      __IO uint32_t vmor                :1; /* [0] */
      __IO uint32_t cce                 :1; /* [1] */
      __IO uint32_t pcce                :1; /* [2] */
      __IO uint32_t pccs                :1; /* [3] */
      __IO uint32_t occs                :1; /* [4] */
      __IO uint32_t reserved1           :27;/* [31:5] */
    } sts_bit;
  };
…
```

```
…
…
  /**
  * @brief adc odt register, offset:0x4C
  */
  union
  {
    __IO uint32_t odt;
    struct
    {
      __IO uint32_t odt                    :16; /* [15:0] */
      __IO uint32_t adc2odt                :16; /* [31:16] */
    } odt_bit;
  };

} adc_type;
```

③ Examples of peripheral register access

Read peripheral	i = ADC1-> ctrl1;
Write peripheral	ADC1-> ctrl1 = i;
Read bit 5 in bit-field mode	i = ADC1-> ctrl1. cceien;
Write 1 to bit 5 in bit-field mode	ADC1-> ctrl1. cceien= TRUE;
Write 1 to bit 5	ADC1-> ctrl1 |= 1<<5;
Write 0 to bit 5	ADC1-> ctrl1&= ~(1<<5) ;

4.3 BSP structure

4.3.1 BSP folder structure

BSP (Board Support Package) structure is shown in Figure 27.

Figure 27. BSP folder structure



Document:

- AT32Fxxx firmware library BSP&Pack user guide.pdf: refer to BSP/Pack user manual
- ReleaseNotes_AT32F403A_407_Firmware_Library.pdf: document revision history

Libraries:

- **Drivers:** driver library for peripherals
  Src folder: low-level driver source file for peripherals, such as, at32fxxx_ip.c
  Inc folder: low-level driver head file for peripherals, such as, at32fxxx_ip.h
- **Cmsis:** Core-related files
  cm4 folder: core-related files, including cortex-m4 library, system initialization file, startup file, etc.
  dsp folder: dsp-related files

Middlewares:

Third-party software or public protocols, including USB protocol layer driver, network protocol driver, operating system source code, etc.

Project:

Examples: demo

Templates: template project, including Keil4, keil5, IAR6, IAR7, IAR8 and eclipse_gcc

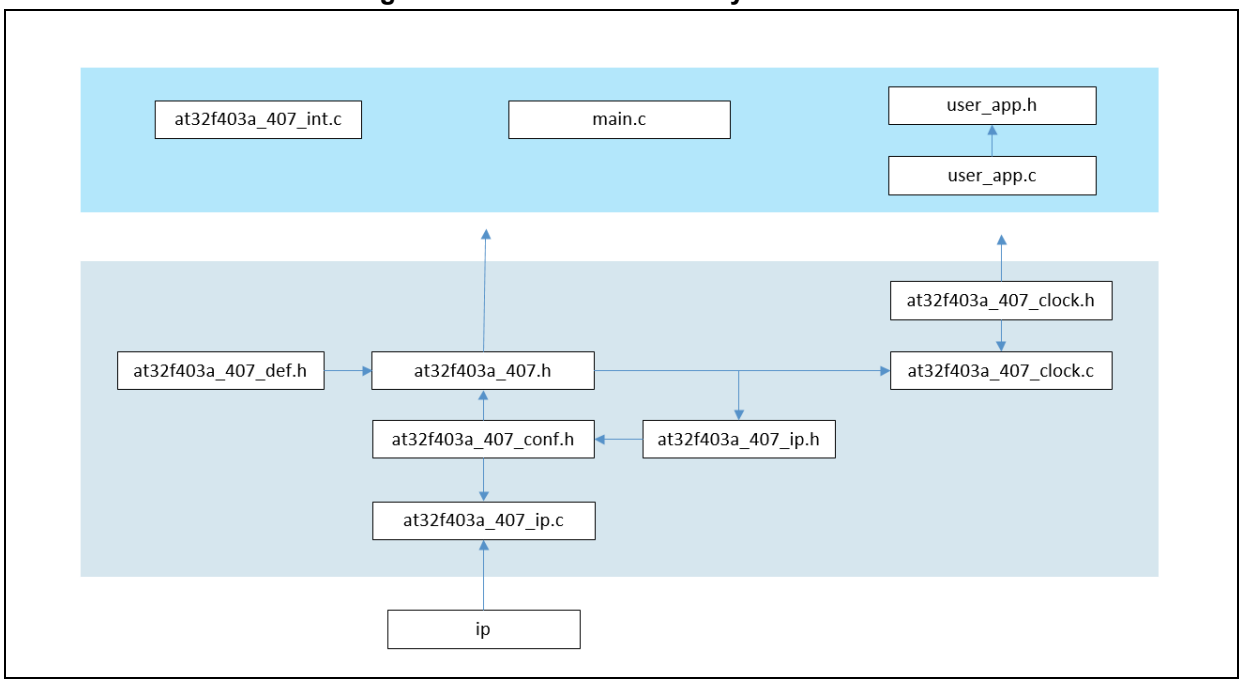
Utilities:

Store application cases

4.3.2 BSP function library structure

Figure 28 shows the architecture of BSP function library.

Figure 28. BSP function library structure



BSP function library files are described in Table 3.

Table 3. Summary of BSP function library files

File name	Description
at32f403a_407_conf.h	Macro definition for peripheral enable, and external high-speed clock HEXT_VALUE
main.c	Main function
at32f403a_407_ip.c	Driver source file for a peripheral, for example, at32f403a_407_adc.c
at32f403a_407_ip.h	Driver header file for a peripheral, for example, at32f403a_407_adc.h
at32f403a_407.h	In the header file (at32f403a_407.h), the definition USE_STDPERIPH_DRIVER is used to determine whether the Keil RTE is used or not. Enabling the definition while Keil RTE is unused can prevent Keil-MDK from enabling _RTE_ accidentally.
at32f403a_407_clock.c	This is a clock configuration file used to configure default clock frequency and clock path
at32f403a_407_clock.h	This is a clock configure header file.
at32f403a_407_int.c	This is a source file for interrupt functions that programs interrupt handling code.
at32f403a_407_int.h	This is a header file for interrupt functions
at32f403a_407_misc.c	This is a source file for other configurations, such as, nvic configuration function, systick clock source selection.
at32f403a_407_misc.h	This is header file for other configurations
startup_at32f403a_407.s	This is a startup file.

4.3.3 Initialization and configuration for peripherals

This section describes how to initialize and configure peripherals using GPIO as an example.

GPIO initialization

Step 1: Define the structure gpio_init_type, for example, gpio_init_type gpio_init_struct;

Step 2: Enable GPIO clock using the function crm_periph_clock_enable

Step 3: De-initialize the structure gpio_init_struct to allow the values of other members (mostly default values) to be correctly written, for example,
gpio_default_para_init(&gpio_init_struct);

Step 4: Configure member of the structure, and write structure parameters into GPIO registers through the gpio_init, for example,

```
gpio_init_struct.gpio_pins = GPIO_PINS_2 | GPIO_PINS_3;
gpio_init_struct.gpio_mode = GPIO_MODE_OUTPUT;
gpio_init_struct.gpio_out_type = GPIO_OUTPUT_PUSH_PULL;
gpio_init_struct.gpio_pull = GPIO_PULL_NONE;
gpio_init_struct.gpio_drive_strength    = GPIO_DRIVE_STRENGTH_STRONGER;
gpio_init(GPIOA, &gpio_init_struct);
```

For more information on peripheral initialization procedure, refer to the section of peripherals of the reference manual, and the section of peripherals of the AT32Fxxx_Firmware_Library_V2.x.x.zip\project\at_start_fxxx\examples.

4.3.4 Peripheral functions description format

Table 4. Function format description for peripherals

Name	Description
Function name	The name of a peripheral function
Function prototype	Prototype declaration
Function description	Brief description of how the function is executed
Input parameter (n)	Description of the input parameters
Output parameter (n)	Description of the output parameters
Return value	Value returned by the function
Required preconditions	Requirements before calling the function
Called functions	Other library functions called

5 AT32F403A/407 peripheral library functions

5.1 HICK automatic clock calibration (ACC)

The ACC register structure acc_type is defined in the "at32f403a_407_acc.h":

```
/**
  * @brief type define acc register all
  */
typedef struct
{
      ……
} acc_type;
```

The table below gives a list of the ACC registers.

Table 5. Summary of ACC registers

Register	Description
acc_sts	ACC status register
acc_ctrl1	ACC control register 1
acc_ctrl2	ACC control register 2
acc_c1	ACC compare value 1
acc_c2	ACC compare value 2
acc_c3	ACC compare value 3

Table 6. Summary of ACC library functions

Function name	Description
acc_calibration_mode_enable	ACC calibration mode enable
acc_step_set	Configure ACC calibration step length
acc_interrupt_enable	ACC interrupt enable
acc_hicktrim_get	Get ACC trimming calibration value
acc_hickcal_get	Get ACC coarse calibration value
acc_write_c1	Write ACC C1 register value
acc_write_c2	Write ACC C2 register value
acc_write_c3	Write ACC C3 register value
acc_read_c1	Read ACC C1 register value
acc_read_c2	Read ACC C2 register value
acc_read_c3	Read ACC C3 register value
acc_flag_get	Get ACC interrupt flag
acc_flag_clear	Clear ACC interrupt flag

5.1.1 acc_calibration_mode_enable function

The table below describes the function acc_calibration_mode_enable.

Table 7. acc_calibration_mode_enable function

Name	Description
Function name	acc_calibration_mode_enable
Function prototype	void acc_calibration_mode_enable(uint16_t acc_trim, confirm_state new_state);
Function description	ACC calibration mode enable
Input parameter 1	acc_trim: calibration mode selection This parameter can be ACC_CAL_HICKCAL or ACC_CAL_HICKTRIM
Input parameter 2	new_state: Enable or disable ACC
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

acc_trim

Calibration mode selection.

ACC_CAL_HICKCAL: Coarse calibration mode

ACC_CAL_HICKTRIM: Fine calibration mode

new_state

Enable or disable ACC

FALSE: Disable interrupts

TRUE: Enable interrupts

Example:

```
/* open acc calibration */
acc_calibration_mode_enable(ACC_CAL_HICKTRIM, TRUE);
```

5.1.2 acc_step_set function

The table below describes the function acc_step_set.

Table 8. acc_step_set function

Name	Description
Function name	acc_step_set
Function prototype	void acc_step_set(uint8_t step_value);
Function description	Configure ACC calibration step length
Input parameter 1	step_value: step value for calibration
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

step_value

This 4-bit field defines the value to be changed for each calibration.

Note: To obtain better calibration accuracy, it is recommended to set the step value to 1. When ENTRIM=0, only the HICKCAL is calibrated. If the step value is incremented or decremented by one, the corresponding HICKCAL follows the change rule (increased or decreased by one), and the HICK frequency will increase or decrease by 40KHz (design value), meaning a positive relationship between them. When ENTRIM=1, only the HICKTRIM is calibrated. If the step value is incremented or decremented by one, the corresponding HICKTRIM follows the change rule (increased or decreased by one), and the HICK frequency will increase or decrease by 20KHz (design value), meaning a positive relationship between them.

Example:

```
/* set acc step value */
acc_step_set(0x1);
```

5.1.3 acc_interrupt_enable function

The table below describes the function acc_interrupt_enable.

Table 9. acc_interrupt_enable function

Name	Description
Function name	dma_ interrupt_enable
Function prototype	void acc_interrupt_enable(uint16_t acc_int, confirm_state new_state);
Function description	Enable ACC interrupts
Input parameter 1	acc_int: interrupt source selection
Input parameter 2	new_state: enable or disable interrupts
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

acc_int

Interrupt source selection.

ACC_CALRDYIEN_INT: Calibration complete interrupt

ACC_EIEN_INT: Reference signal lost interrupt

new_state

Enable or disable interrupts.

FALSE: Interrupt disable

TRUE: Interruupt enable

Example:

```
/* enable the acc reference signal lost interrupt */
acc_interrupt_enable(ACC_EIEN_INT, TRUE);
```

5.1.4 acc_hicktrim_get function

The table below describes the function acc_hicktrim_get.

Table 10. acc_hicktrim_get function

Name	Description
Function name	acc_hicktrim_get
Function prototype	uint8_t acc_hicktrim_get(void);
Function description	Get ACC trimming value
Input parameter	NA
Output parameter	NA
Return value	Return ACC trimming value
Required preconditions	NA
Called functions	NA

Example:

```
/* get trim value*/
uint8_t trim_value;
trim_value = acc_hicktrim_get();
```

5.1.5 acc_hickcal_get function

The table below describes the function acc_hickcal_get.

Table 11. acc_hickcal_get function

Name	Description
Function name	acc_hickcal_get
Function prototype	uint8_t acc_hickcal_get(void);
Function description	Get ACC coarse calibration value
Input parameter	NA
Output parameter	NA
Return value	Return ACC coarse calibration value
Required preconditions	NA
Called functions	NA

Example:

```
/* get cal value*/
uint8_t cal_value;
cal_value = acc_hickcal_get ();
```

5.1.6 acc_write_c1 function

The table below describes the function acc_write_c1.

Table 12. acc_write_c1 function

Name	Description
Function name	acc_write_c1
Function prototype	void acc_write_c1(uint16_t acc_c1_value);
Function description	Write ACC C1 register value
Input parameter	acc_c1_value: the value to be written in ACC C1 register
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* update the c1 value */
acc_c2_value = 8000;
acc_write_c1(acc_c2_value - 10);
```

5.1.7 acc_write_c2 function

The table below describes the function acc_write_c2.

Table 13. acc_write_c2 function

Name	Description
Function name	acc_write_c2
Function prototype	void acc_write_c2(uint16_t acc_c2_value);
Function description	Write ACC C2 register value
Input parameter	acc_c2_value: the value to be written in ACC C2 register
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* update the c2 value */
acc_c2_value = 8000;
acc_write_c2(acc_c2_value - 10);
```

5.1.8 acc_write_c3 function

The table below describes the function acc_write_c3.

Table 14. acc_write_c3 function

Name	Description
Function name	acc_write_c3
Function prototype	void acc_write_c3(uint16_t acc_c3_value);
Function description	Write ACC C3 register value
Input parameter	acc_c3_value: the value to be written in ACC C3 register
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* update the c3 value */
acc_c2_value = 8000;
acc_write_c3(acc_c2_value - 10);
```

5.1.9 acc_read_c1 function

The table below describes the function acc_read_c1.

Table 15. acc_read_c1 function

Name	Description
Function name	acc_read_c1
Function prototype	uint16_t acc_read_c1(void);
Function description	Read ACC C1 register value
Input parameter	NA
Output parameter	NA
Return value	ACC C1 register value
Required preconditions	NA
Called functions	NA

Example:

```
/* get the c1 value */
uint16_t  acc_c1_value;
acc_c1_value = acc_read_c1();
```

5.1.10 acc_read_c2 function

The table below describes the function acc_read_c1.

Table 16. acc_read_c2 function

Name	Description
Function name	acc_read_c2
Function prototype	uint16_t acc_read_c2(void);
Function description	Read ACC C2 register value
Input parameter	NA
Output parameter	NA
Return value	ACC C2 register value
Required preconditions	NA
Called functions	NA

Example:

```
/* get the c2 value */
uint16_t  acc_c2_value;
acc_c2_value = acc_read_c2();
```

5.1.11 acc_read_c3 function

The table below describes the function acc_read_c1.

Table 17. acc_read_c3 function

Name	Description
Function name	acc_read_c3
Function prototype	uint16_t acc_read_c3(void);
Function description	Read ACC C3 register value
Input parameter	NA
Output parameter	NA
Return value	ACC C3 register value
Required preconditions	NA
Called functions	NA

Example:

```
/* get the c3 value */
uint16_t   acc_c3_value;
acc_c3_value = acc_read_c3();
```

5.1.12 acc_flag_get function

The table below describes the function acc_flag_get.

Table 18. acc_flag_get function

Name	Description
Function name	acc_flag_get
Function prototype	flag_status acc_flag_get(uint16_t acc_flag);
Function description	Get ACC flag
Input parameter 1	acc_flag: ACC flag selection
Output parameter	NA
Return value	flag_status: indicates whether or not the flag has been set
Required preconditions	NA
Called functions	NA

acc_flag

The acc_flag is used for flag selection, including:

ACC_RSLOST_FLAG: Reference signal lost interrupt

ACC_CALRDY_FLAG: Calibration complete interrupt

flag_status

RESET: Flag bit is not set

SET: Flag bit is set

Example:

```
if(acc_flag_get(ACC_CALRDY_FLAG) != RESET)
{
  at32_led_toggle(LED2);
  /* clear acc calibration ready flag */
  acc_flag_clear(ACC_CALRDY_FLAG);
}
```

5.1.13 acc_interrupt_flag_get function

The table below describes the function acc_interrupt_flag_get.

Table 19. acc_interrupt_flag_get function

Name	Description
Function name	acc_interrupt_flag_get
Function prototype	flag_status acc_interrupt_flag_get(uint16_t acc_flag);
Function description	Get acc interrupt flag
Input parameter 1	acc_flag: flag selection
Output parameter	NA
Return value	flag_status: indicates whether or not the flag has been set
Required preconditions	NA
Called functions	NA

acc_flag

The acc_flag is used for flag selection, including:

ACC_RSLOST_FLAG: Reference signal lost interrupt

ACC_CALRDY_FLAG: Calibration complete interrupt

flag_status

RESET: Flag bit is not set

SET: Flag bit is set

Example

```
if(acc_interrupt_flag_get(ACC_CALRDY_FLAG) != RESET)
{
  at32_led_toggle(LED2);
  /* clear acc calibration ready flag */
  acc_flag_clear(ACC_CALRDY_FLAG);
}
```

5.1.14 acc_flag_clear function

The table below describes the function acc_flag_clear.

Table 20. acc_flag_clear function

Name	Description
Function name	acc_flag_clear
Function prototype	void acc_flag_clear(uint16_t acc_flag);
Function description	Clear ACC flag
Input parameter 1	acc_flag: ACC flag selection
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

acc_flag

The acc_flag is used for flag selection, including:

ACC_RSLOST_FLAG: Reference signal lost interrupt

ACC_CALRDY_FLAG: Calibration complete interrupt

Example:

```
if(acc_flag_get(ACC_CALRDY_FLAG) != RESET)
{
  at32_led_toggle(LED2);
  /* clear acc calibration ready flag */
  acc_flag_clear(ACC_CALRDY_FLAG);
}
```

5.2 Analog-to-digital converter (ADC)

ADC register structure adc_type is defined in the "at32f403a_407_acc.h":

```
/**
  * @brief type define adc register all
  */
typedef struct
{
      ……
} adc_type;
```

The table below gives a list of the ADC registers.

Table 21. Summary of ADC registers

Register	Description
sts	ADC status register
ctrl1	ADC control register 1
ctrl2	ADC control register 2
spt1	ADC sample time register 1
spt2	ADC sample time register 2
pcdto1	ADC preempted channel data offset register 1
pcdto2	ADC preempted channel data offset register 2
pcdto3	ADC preempted channel data offset register 3
pcdto4	ADC preempted channel data offset register 4
vmhb	ADC voltage monitor high boundary register
vmlb	ADC voltage monitor low boundary register
osq1	ADC ordinary sequence register 1
osq2	ADC ordinary sequence register 2
osq3	ADC ordinary sequence register 3
psq	ADC preempted sequence register
pdt1	ADC preempted data register 1
pdt2	ADC preempted data register 2
pdt3	ADC preempted data register 3
pdt4	ADC preempted data register 4
odt	ADC ordinary data register

The table below gives a list of ADC library functions.

Table 22. Summary of ADC library functions

Function name	Description
adc_reset	Reset all ADC registers to their reset values
adc_enable	Enable A/D converter
adc_combine_mode_select	Master/slave mode selection
adc_base_default_para_init	Define an initial value for adc_base_struct
adc_base_config	Configure ADC registers with the initialized parameters of the adc_base_struct
adc_dma_mode_enable	Enable DMA transfer for ordinary group
adc_interrupt_enable	Enable the selected ADC event interrupt
adc_calibration_init	Initialization calibration
adc_calibration_init_status_get	Get initialization calibration status
adc_calibration_start	Start calibration
adc_calibration_status_get	Get calibration status
adc_voltage_monitor_enable	Enable voltage monitoring for ordinary/preempted channels and a single channel
adc_voltage_monitor_threshold_value_set	Set the threshold of voltage monitoring
adc_voltage_monitor_single_channel_select	Select a single channel for voltage monitoring
adc_ordinary_channel_set	Configure ordinary channels, including channel selection, conversion sequence number and sampling time
adc_preempt_channel_length_set	Configure the length of preempted group conversion sequence
adc_preempt_channel_set	Configure preempted channels, including channel selection, conversion sequence number and sampling time
adc_ordinary_conversion_trigger_set	Enable trigger mode and trigger event selection for ordinary conversion
adc_preempt_conversion_trigger_set	Enable trigger mode and trigger event selection for preempted conversion
adc_preempt_offset_value_set	Set data offset for preempted conversion
adc_ordinary_part_count_set	Set the number of ordinary channels for each triggered conversion in partition mode
adc_ordinary_part_mode_enable	Enable partition mode for ordinary channels
adc_preempt_part_mode_enable	Enable partition mode for preempted channels
adc_preempt_auto_mode_enable	Enable auto conversion of preempted group at the end of ordinary conversion
adc_tempersensor_vintrv_enable	Enable internal temperature sensor and V_{INTRV}
adc_ordinary_software_trigger_enable	Software trigger ordinary group conversion
adc_ordinary_software_trigger_status_get	Get the status of ordinary group conversion triggered by software
adc_preempt_software_trigger_enable	Software trigger preempted group conversion
adc_preempt_software_trigger_status_get	Get the status of preempted group conversion triggered by software
adc_ordinary_conversion_data_get	Get data of ordinary group conversion in non-master-slave mode
adc_combine_ordinary_conversion_data_get	Get the converted data of ordinary group in master/slave combined mode
adc_preempt_conversion_data_get	Get the converted data of preempted group

Function name	Description
adc_flag_get	Get the status of flag bits
adc_flag_clear	Clear flag bits

5.2.1 adc_reset function

The table below describes the function adc_reset.

Table 23. adc_reset function

Name	Description
Function name	adc_reset
Function prototype	void adc_reset(adc_type *adc_x)
Function description	Reset all ADC registers to their reset values
Input parameter	adc_x: indicates the selected ADC This parameter is used to select one of ADC peripherals: ADC1, ADC2, ADC3
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	crm_periph_reset()

Example:

```
/* deinitialize adc1 */
adc_reset(ADC1);
```

5.2.2 adc_enable function

The table below describes the function adc_enable.

Table 24. adc_enable function

Name	Description
Function name	adc_enable
Function prototype	void adc_enable(adc_type *adc_x, confirm_state new_state)
Function description	Enable/disable A/D converter
Input parameter 1	adc_x: indicates the selected ADC This parameter is used to select one of ADC peripherals: ADC1, ADC2, ADC3
Input parameter 2	new_state: indicates the pre-configured status of A/D converter This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable adc1 */
adc_enable(ADC1, TRUE);
```

Note: Calling the function adc_enable while the ADC is enabled triggers the conversion of ordinary channels.

5.2.3 adc_combine_mode_select function

The table below describes the function adc_combine_mode_select.

Table 25. adc_combine_mode_select function

Name	Description
Function name	adc_combine_mode_select
Function prototype	void adc_combine_mode_select(adc_combine_mode_type combine_mode)
Function description	Select master/slave combined mode for ADC1
Input parameter	combine_mode: master/slave combined mode for ADC1 This parameter can be any enumerated value in the adc_combine_mode_type.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

combine_mode

The combine_mode is used to select master/slave mode, including:

ADC_INDEPENDENT_MODE: Independent mode

ADC_ORDINARY_SMLT_PREEMPT_SMLT_MODE:

Ordinary group simultaneous mode + preempted group simultaneous mode

ADC_ORDINARY_SMLT_PREEMPT_INTERLTRIG_MODE:

Ordinary group simultaneous mode + preempted group alternate trigger mode

ADC_ORDINARY_SHORTSHIFT_PREEMPT_SMLT_MODE:

Preempted group simultaneous mode + ordinary group short shift mode

ADC_ORDINARY_LONGSHIFT_PREEMPT_SMLT_MODE:

Preempted group simultaneous mode + ordinary group long shift mode

ADC_PREEMPT_SMLT_ONLY_MODE: Preempted group simultaneous mode

ADC_ORDINARY_SMLT_ONLY_MODE: Ordinary group simultaneous mode

ADC_ORDINARY_SHORTSHIFT_ONLY_MODE: Ordinary group short shift mode

ADC_ORDINARY_LONGSHIFT_ONLY_MODE: Ordinary group long shift mode

ADC_PREEMPT_INTERLTRIG_ONLY_MODE: Preempted group alternate trigger mode

Example:

```
/* select combine mode as independent mode */
  adc_combine_mode_select(ADC_INDEPENDENT_MODE);
```

Note: adc_combine_mode_select applies to ADC1 only, no effect on ADC2 and ADC3.

5.2.4 adc_base_default_para_init function

The table below describes the function adc_base_default_para_init.

Table 26. adc_base_default_para_init function

Name	Description
Function name	adc_ base_default_para_init
Function prototype	void adc_base_default_para_init(adc_base_config_type *adc_base_struct)
Function description	Set the initial default value for the adc_base_struct
Input parameter	adc_base_struct: adc_base_config_type structure pointer
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

The initial default values of members of the adc_base_struct are as follows:

sequence_mode: FALSE

repeat_mode: FALSE

data_align: ADC_RIGHT_ALIGNMENT

ordinary_channel_length: 1

Example:

```
/* initialize a adc_base_config_type structure */
adc_base_config_type adc_base_struct;
adc_base_default_para_init(&adc_base_struct);
```

5.2.5 adc_base_config function

The table below describes the function adc_base_config.

Table 27. adc_base_config function

Name	Description
Function name	adc_base_config
Function prototype	void adc_base_config(adc_type *adc_x, adc_base_config_type *adc_base_struct);
Function description	Initialize ADC registers with the specified parameters in the adc_base_struc
Input parameter 1	adc_x: indicates the selected ADC peripheral This parameter can be ADC1, ADC2, ADC3
Input parameter 2	adc_base_struct: adc_base_config_type structure pointer
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

adc_base_config_type structure

The adc_base_config_type is defined in the at32f403a_407_adc.h:

```
typedef struct
{
  confirm_state        sequence_mode;
  confirm_state        repeat_mode;
```

```
  adc_data_align_type        data_align;
  uint8_t                    ordinary_channel_length;
} adc_base_config_type; the member parameters are described as follows
```

sequence_mode

Set ADC sequence mode.

FALSE: Select a single channel for conversion

TRUE: Select multiple channels for conversion

repeat_mode

Set ADC repeat mode.

FALSE: when SQEN=0, trigger a single channel conversion each time; when SQEN=1, trigger the conversion of a group of channels each time

TRUE: when SQEN =0, repeatedly convert a single channel at each trigger; when SQEN=1, repeatedly convert a group of channels at each trigger until the ADCEN bit is cleared.

data_align

Set data alignment of ADC

ADC_RIGHT_ALIGNMENT: right-aligned

ADC_LEFT_ALIGNMENT: left-aligned

ordinary_channel_length

Set the length of ordinary group ADC conversion

Example:

```
adc_base_config_type adc_base_struct;
adc_base_struct.sequence_mode = TRUE;
adc_base_struct.repeat_mode = FALSE;
adc_base_struct.data_align = ADC_RIGHT_ALIGNMENT;
adc_base_struct.ordinary_channel_length = 3;
adc_base_config(ADC1, &adc_base_struct);
```

5.2.6 adc_dma_mode_enable function

The table below describes the function adc_dma_mode_enable.

Table 28. adc_dma_mode_enable function

Name	Description
Function name	adc_dma_mode_enable
Function prototype	void adc_dma_mode_enable(adc_type *adc_x, confirm_state new_state)
Function description	Enable DMA transfer for ordinary group conversion
Input parameter 1	adc_x: indicates the selected ADC peripheral This parameter can be ADC1, ADC2, ADC3
Input parameter 2	new_state: pre-configured status of ordinary group in DMA transfer mode This parameter can be TRUE or FALSE
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable dma transfer adc ordinary conversion data */
adc_dma_mode_enable(ADC1, TRUE);
```

Note: adc_combine_mode_select applies to ADC1 and ADC3 only, no effect on ADC2

5.2.7 adc_interrupt_enable function

The table below describes the function adc_interrupt_enable.

Table 29. adc_interrupt_enable function

Name	Description
Function name	adc_interrupt_enable
Function prototype	void adc_interrupt_enable(adc_type *adc_x, uint32_t adc_int, confirm_state new_state)
Function description	Enable the selected ADC event interrupt
Input parameter 1	adc_x: indicates the selected ADC peripheral This parameter can be ADC1, ADC2, ADC3
Input parameter 2	adc_int: ADC event interrupt selection This parameter is used to select any event interrupt supported by ADC
Input parameter 3	new_state: indicates the pre-configured status of ADC event interrupts This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

adc_int

The adc_int is used to select and set event interrupts, with the following parameters:

ADC_CCE_INT: Interrupt enabled at the end of channel conversion

ADC_VMOR_INT: Interrupt enabled when voltage monitor is outside a threshold

ADC_PCCE_INT: Interrpt enabled at the end of preempted group conversion

Example:

```
/* enable voltage monitoring out of range interrupt */
adc_interrupt_enable(ADC1, ADC_VMOR_INT, TRUE);
```

5.2.8 adc_calibration_init function

The table below describes the function adc_calibration_init.

Table 30. adc_calibration_init function

Name	Description
Function name	adc_calibration_init
Function prototype	void adc_calibration_init(adc_type *adc_x)
Function description	Initialize calibration
Input parameter	adc_x: indicates the selected ADC peripheral This parameter can be ADC1, ADC2, ADC3
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* initialize A/D calibration */
adc_calibration_init(ADC1);
```

5.2.9 adc_calibration_init_status_get function

The table below describes the function adc_calibration_init_status_get.

Table 31. adc_calibration_init_status_get function

Name	Description
Function name	adc_calibration_init_status_get
Function prototype	flag_status adc_calibration_init_status_get(adc_type *adc_x)
Function description	Get the status of initialization calibration
Input parameter	adc_x: indicates the selected ADC peripheral This parameter can be ADC1, ADC2, ADC3
Output parameter	NA
Return value	flag_status: indicates the status of calibration initialization Return SET or RESET.
Required preconditions	NA
Called functions	NA

Example:

```
/* wait initialize A/D calibration success */
while(adc_calibration_init_status_get(ADC1));
```

5.2.10 adc_calibration_start function

The table below describes the function adc_calibration_start.

Table 32. adc_calibration_start function

Name	Description
Function name	adc_calibration_start
Function prototype	void adc_calibration_start(adc_type *adc_x)
Function description	Start calibration
Input parameter	adc_x: indicates the selected ADC peripheral This parameter can be ADC1, ADC2, ADC3
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* start calibration process */
adc_calibration_start(ADC1);
```

5.2.11 adc_calibration_status_get function

The table below describes the function adc_calibration_status_get.

Table 33. adc_calibration_status_get function

Name	Description
Function name	adc_calibration_status_get
Function prototype	flag_status adc_calibration_status_get(adc_type *adc_x)
Function description	Get the status of calibration
Input parameter	adc_x: indicates the selected ADC peripheral This parameter can be ADC1, ADC2, ADC3
Output parameter	NA
Return value	flag_status: indicates the status of calibration Return SET or RESET
Required preconditions	NA
Called functions	NA

Example:

```
/* wait calibration success */
while(adc_calibration_status_get(ADC1));
```

5.2.12 adc_voltage_monitor_enable function

The table below describes the function adc_voltage_monitor_enable.

Table 34. adc_voltage_monitor_enable function

Name	Description
Function name	adc_voltage_monitor_enable
Function prototype	void adc_voltage_monitor_enable(adc_type *adc_x, adc_voltage_monitoring_type adc_voltage_monitoring)
Function description	Enable voltage monitor for ordinary/preempted group and a single channel
Input parameter 1	adc_x: indicates the selected ADC peripheral This parameter can be ADC1, ADC2, ADC3
Input parameter 2	adc_voltage_monitoring: select ordinary group, preempted group or a single channel for voltage monitoring This parameter can be any enumerated value in the adc_voltage_monitoring_type.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

adc_voltage_monitoring

adc_voltage_monitoring is used to select one or more channels of ordinary group/preempted group for voltage monitoring, including:

ADC_VMONITOR_SINGLE_ORDINARY:

Select a single ordinary channel for voltage monitoring

ADC_VMONITOR_SINGLE_PREEMPT:

Select a single preempted channel for voltage monitoring

ADC_VMONITOR_SINGLE_ORDINARY_PREEMPT:
Select a single channel from ordinary or preempted group for voltage monitoring
ADC_VMONITOR_ALL_ORDINARY:
Select all ordinary channels for voltage monitoring
ADC_VMONITOR_ALL_PREEMPT:
Select all preempted channels for voltage monitoring
ADC_VMONITOR_ALL_ORDINARY_PREEMPT:
Selct all ordinary and preempted channels for voltage monitoring
ADC_VMONITOR_NONE:
No channels need voltage monitoring

Example:

```
/* enable the voltage monitoring on all ordinary and preempt channels */
adc_voltage_monitor_enable(ADC1, ADC_VMONITOR_ALL_ORDINARY_PREEMPT);
```

5.2.13 adc_voltage_monitor_threshold_value_set function

The table below describes the function adc_voltage_monitor_threshold_value_set.

Table 35. adc_voltage_monitor_threshold_value_set function

Name	Description
Function name	adc_voltage_monitor_threshold_value_set
Function prototype	void adc_voltage_monitor_threshold_value_set(adc_type *adc_x, uint16_t adc_high_threshold, uint16_t adc_low_threshold)
Function description	Configure the threshold of voltage monitoring
Input parameter 1	adc_x: indicates the selected ADC peripheral This parameter can be ADC1, ADC2, ADC3
Input parameter 2	adc_high_threshold: indicates the upper limit for voltage monitoring This parameter can be any value between 0x000 and 0xFFF
Input parameter 3	adc_low_threshold: indicates the lower limit for voltage monitoring This parameter can be any value lower than that of the adc_high_threshold in the range of 0x000~0xFFF
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* set voltage monitoring's high and low thresholds value */
adc_voltage_monitor_threshold_value_set(ADC1, 0xBBB, 0xAAA);
```

5.2.14 adc_voltage_monitor_single_channel_select function

The table below describes the function adc_voltage_monitor_single_channel_select.

Table 36. adc_voltage_monitor_single_channel_select function

Name	Description
Function name	adc_voltage_monitor_single_channel_select
Function prototype	void adc_voltage_monitor_single_channel_select(adc_type *adc_x, adc_channel_select_type adc_channel)
Function description	Select a single channel for voltage monitoring
Input parameter 1	adc_x: indicates the selected ADC peripheral This parameter can be ADC1, ADC2, ADC3
Input parameter 2	adc_channel: select a single channel for voltage monitoring Refer to *adc_channel* for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

adc_channel

The adc_channel is used to select a single channel for voltage monitoring, including:

ADC_CHANNEL_0: ADC channel 0

ADC_CHANNEL_1: ADC channel 1

……

ADC_CHANNEL_16: ADC channel 16

ADC_CHANNEL_17: ADC channel 17

Example:

```
/* select the voltage monitoring's channel */
adc_voltage_monitor_single_channel_select(ADC1, ADC_CHANNEL_5);
```

5.2.15 adc_ordinary_channel_set function

The table below describes the function adc_ordinary_channel_set.

Table 37. adc_ordinary_channel_set function

Name	Description
Function name	adc_ordinary_channel_set
Function prototype	void adc_ordinary_channel_set(adc_type *adc_x, adc_channel_select_type adc_channel, uint8_t adc_sequence, adc_sampletime_select_type adc_sampletime)
Function description	Configure ordinary channels, including such parameters as channel selection, conversion sequence number and sampling time
Input parameter 1	adc_x: indicates the selected ADC peripheral This parameter can be ADC1, ADC2, ADC3
Input parameter 2	adc_channel: indicates the selected channel Refer to *adc_channel* for details.
Input parameter 3	adc_sequence: defines the sequence of channel conversion This parameter can be any value from 1 to 16.
Input parameter 4	adc_sampletime: defines the sampling time for channels

Name	Description
	Refer to *adc_sampletime* for details
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

adc_sampletime

adc_sampletime is used to configure the sampling time of channels, including:

ADC_SAMPLETIME_1_5: sampling time is 1.5 ADCCLK cycles

ADC_SAMPLETIME_7_5: sampling time is 7.5 ADCCLK cycles

ADC_SAMPLETIME_13_5: sampling time is 13.5 ADCCLK cycles

ADC_SAMPLETIME_28_5: sampling time is 28.5 ADCCLK cycles

ADC_SAMPLETIME_41_5: sampling time is 41.5 ADCCLK cycles

ADC_SAMPLETIME_55_5: sampling time is 55.5 ADCCLK cycles

ADC_SAMPLETIME_71_5: sampling time is 71.5 ADCCLK cycles

ADC_SAMPLETIME_239_5: sampling time is 239.5 ADCCLK cycles

Example:

```
/* set ordinary channel's corresponding rank in the sequencer and sample time */
adc_ordinary_channel_set(ADC1, ADC_CHANNEL_4, 1, ADC_SAMPLETIME_239_5);
adc_ordinary_channel_set(ADC1, ADC_CHANNEL_5, 2, ADC_SAMPLETIME_239_5);
```

5.2.16 adc_preempt_channel_length_set function

The table below describes the function adc_preempt_channel_length_set.

Table 38. adc_preempt_channel_length_set function

Name	Description
Function name	adc_preempt_channel_length_set
Function prototype	void adc_preempt_channel_length_set(adc_type *adc_x, uint8_t adc_channel_lenght)
Function description	set the length of preempted channel conversion
Input parameter 1	adc_x: indictes the selected ADC This parameter can be any ADC1, ADC2, ADC3
Input parameter 2	adc_channel_length: set the length of preempted channel conversion This parameter can be any value from 0x1 to 0x4.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* set preempt channel lenghth */
adc_preempt_channel_length_set(ADC1, 3);
```

5.2.17 adc_preempt_channel_set function

The table below describes the function adc_preempt_channel_set.

Table 39. adc_preempt_channel_set function

Name	Description
Function name	adc_preempt_channel_set
Function prototype	void adc_preempt_channel_set(adc_type *adc_x, adc_channel_select_type adc_channel, uint8_t adc_sequence, adc_sampletime_select_type adc_sampletime)
Function description	Configure preempted group, including such parameters as channel selection, conversion sequence number and sampling time
Input parameter 1	adc_x: indicates the selected ADC This parameter can be any ADC1, ADC2, ADC3
Input parameter 2	adc_channel: indicates the selected channel Refer to *adc_channel* for details
Input parameter 3	adc_sequence: set the sequence number for channel conversion This parameter can be any value from 1 to 4
Input parameter 4	adc_sampletime: set the sampling time for channels Refer to *adc_sampletime* for details
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* set ordinary channel's corresponding rank in the sequencer and sample time */
adc_preempt_channel_set(ADC1, ADC_CHANNEL_7, 1, ADC_SAMPLETIME_239_5);
adc_preempt_channel_set(ADC1, ADC_CHANNEL_8, 2, ADC_SAMPLETIME_239_5);
```

5.2.18 adc_ordinary_conversion_trigger_set function

The table below describes the function adc_ordinary_conversion_trigger_set.

Table 40. adc_ordinary_conversion_trigger_set function

Name	Description
Function name	adc_ordinary_conversion_trigger_set
Function prototype	void adc_ordinary_conversion_trigger_set(adc_type *adc_x, adc_ordinary_trig_select_type adc_ordinary_trig, confirm_state new_state)
Function description	Enable trigger mode and select trigger events for ordinary group conversion
Input parameter 1	adc_x: indicates the selected ADC This parameter can be any ADC1, ADC2, ADC3
Input parameter 2	adc_ordinary_trig: indicates the selected trigger event for ordinary group This parameter can be any enumerated value in the adc_ordinary_trig_select_type
Input parameter 3	new_state: indicates the pre-configured status of trigger mode This parameter can be TRUE or FALSE
Output parameter	NA
Return value	NA

Name	Description
Required preconditions	NA
Called functions	NA

adc_ordinary_trig

adc_ordinary_trig is used to select a trigger event for ordinary group conversion, including:

Trigger events for ADC1 &ADC2:

ADC12_ORDINARY_TRIG_TMR1CH1: TMR1 CH1 event
ADC12_ORDINARY_TRIG_TMR1CH2: TMR1 CH2 event
ADC12_ORDINARY_TRIG_TMR1CH3: TMR1 CH3 event
ADC12_ORDINARY_TRIG_TMR2CH2: TMR2 CH2 event
ADC12_ORDINARY_TRIG_TMR3TRGOUT: TMR3 TRGOUT event
ADC12_ORDINARY_TRIG_TMR4CH4: TMR4 CH4 event
ADC12_ORDINARY_TRIG_EXINT11_TMR8TRGOUT: EXINT11/TMR8 TRGOUT event
ADC12_ORDINARY_TRIG_SOFTWARE: Software-triggered event
ADC12_ORDINARY_TRIG_TMR1TRGOUT: TMR1 TRGOUT event
ADC12_ORDINARY_TRIG_TMR8CH1: TMR8 CH1 event
ADC12_ORDINARY_TRIG_TMR8CH2: TMR8 CH2 event

Trigger events for ADC3:

ADC3_ORDINARY_TRIG_TMR3CH1: TMR3 CH1 event
ADC3_ORDINARY_TRIG_TMR2CH3: TMR2 CH3 event
ADC3_ORDINARY_TRIG_TMR1CH3: TMR1 CH3 event
ADC3_ORDINARY_TRIG_TMR8CH1: TMR8 CH1 event
ADC3_ORDINARY_TRIG_TMR8TRGOUT: TMR8 TRGOUT event
ADC3_ORDINARY_TRIG_TMR5CH1: TMR5 CH1 event
ADC3_ORDINARY_TRIG_TMR5CH3: TMR5 CH3 event
ADC3_ORDINARY_TRIG_SOFTWARE: Software-triggered event
ADC3_ORDINARY_TRIG_TMR1TRGOUT: TMR1 TRGOUT event
ADC3_ORDINARY_TRIG_TMR1CH1: TMR1 CH1 event
ADC3_ORDINARY_TRIG_TMR8CH3: TMR8 CH3 event

Example:

```
/* set ordinary external trigger event */
adc_ordinary_conversion_trigger_set(ADC1, ADC12_ORDINARY_TRIG_TMR1CH1, TRUE);
```

5.2.19 adc_preempt_conversion_trigger_set function

The table below describes the function adc_preempt_conversion_trigger_set.

Table 41. adc_preempt_conversion_trigger_set function

Name	Description
Function name	adc_preempt_conversion_trigger_set
Function prototype	void adc_preempt_conversion_trigger_set(adc_type *adc_x, adc_preempt_trig_select_type adc_preempt_trig, confirm_state new_state)
Function description	Enable trigger mode and trigger events for preempted group conversion
Input parameter 1	adc_x: indicates the selected ADC This parameter can be any ADC1, ADC2, ADC3
Input parameter 2	adc_ preempt _trig: indicates the selected trigger event for preempted group This parameter can be any enumerated value in the adc_preempt_trig_select_type
Input parameter 3	new_state: indicates the pre-configured status of trigger mode This parameter can be TRUE or FALSE
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

adc_preempt _trig

adc_preempt _trig is used to select a trigger event for preempted group conversion, including:

Trigger events for ADC1 &ADC2:

ADC12_PREEMPT_TRIG_TMR1TRGOUT: TMR1 TRGOUT event
ADC12_PREEMPT_TRIG_TMR1CH4: TMR1 CH4 event
ADC12_PREEMPT_TRIG_TMR2TRGOUT: TMR2 TRGOUT event
ADC12_PREEMPT_TRIG_TMR2CH1: TMR2 CH1 event
ADC12_PREEMPT_TRIG_TMR3CH4: TMR3 CH4 event
ADC12_PREEMPT_TRIG_TMR4TRGOUT: TMR4 TRGOUT event
ADC12_PREEMPT_TRIG_EXINT15_TMR8CH4: EXINT 15/TMR8 CH4 event
ADC12_PREEMPT_TRIG_SOFTWARE: Software-triggered event
ADC12_PREEMPT_TRIG_TMR1CH1: TMR1 CH1 event
ADC12_PREEMPT_TRIG_TMR8CH1: TMR8 CH1 event
ADC12_PREEMPT_TRIG_TMR8TRGOUT: TMR8 TRGOUT event

Trigger events for ADC3:

ADC3_PREEMPT_TRIG_TMR1TRGOUT: TMR3 TRGOUT event
ADC3_PREEMPT_TRIG_TMR1CH4: TMR1 CH4 event
ADC3_PREEMPT_TRIG_TMR4CH3: TMR4 CH3 event
ADC3_PREEMPT_TRIG_TMR8CH2: TMR8 CH2 event
ADC3_PREEMPT_TRIG_TMR8CH4: TMR8 CH4 event
ADC3_PREEMPT_TRIG_TMR5TRGOUT: TMR5 TRGOUT event
ADC3_PREEMPT_TRIG_TMR5CH4: TMR5 CH4 event
ADC3_PREEMPT_TRIG_SOFTWARE: Software-triggered event
ADC3_PREEMPT_TRIG_TMR1CH1: TMR1 CH1 event
ADC3_PREEMPT_TRIG_TMR1CH2: TMR1 CH2 event
ADC3_PREEMPT_TRIG_TMR8TRGOUT: TMR8 TRGOUT event

Example:

```
/* set preempt external trigger event */
adc_preempt_conversion_trigger_set(ADC1, ADC12_PREEMPT_TRIG_SOFTWARE, TRUE);
```

5.2.20 adc_preempt_offset_value_set function

The table below describes the function adc_preempt_offset_value_set.

Table 42. adc_preempt_offset_value_set function

Name	Description
Function name	adc_preempt_offset_value_set
Function prototype	void adc_preempt_offset_value_set(adc_type *adc_x, adc_preempt_channel_type adc_preempt_channel, uint16_t adc_offset_value)
Function description	set the offset value of preempted group conversion
Input parameter 1	adc_x: selected ADC This parameter can be ADC1, ADC2, ADC3
Input parameter 2	adc_preempt_channel: indicates the selected channel Refer to *adc_preempt_channel* for details
Input parameter 3	adc_offset_value: set the offset value for the selected channel This parameter can be any value from 0x000 to 0xFFF.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

adc_preempt_channel

adc_preempt_channel is used to set an offset value for the selected channel, including:

ADC_PREEMPT_CHANNEL_1: Preempted channel 1
ADC_PREEMPT_CHANNEL_2: Preempted channel 2
ADC_PREEMPT_CHANNEL_3: Preempted channel 3
ADC_PREEMPT_CHANNEL_4: Preempted channel 4

Example:

```
/* set preempt channel's conversion value offset */
adc_preempt_offset_value_set(ADC1, ADC_PREEMPT_CHANNEL_1, 0x111);
adc_preempt_offset_value_set(ADC1, ADC_PREEMPT_CHANNEL_2, 0x222);
adc_preempt_offset_value_set(ADC1, ADC_PREEMPT_CHANNEL_3, 0x333);
```

5.2.21 adc_ordinary_part_count_set function

The table below describes the function adc_ordinary_part_count_set.

Table 43. adc_ordinary_part_count_set function

Name	Description
Function name	adc_ordinary_part_count_set
Function prototype	void adc_ordinary_part_count_set(adc_type *adc_x, uint8_t adc_channel_count)
Function description	Set the number of ordinary channels at each triggered conversion in partition mode
Input parameter 1	adc_x: indicates the selected ADC This parameter can be ADC1, ADC2, ADC3
Input parameter 2	adc_channel_count: indicates the number of ordinary group in partition mode This parameter can be any value from 0x1 to 0x8
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* set partitioned mode channel count */
adc_ordinary_part_count_set(ADC1, 2);
```

Note: In partition mode, only the number of ordinary group is settable, and that of preempted group is fixed 1.

5.2.22 adc_ordinary_part_mode_enable function

The table below describes the function adc_ordinary_part_mode_enable.

Table 44. adc_ordinary_part_mode_enable function

Name	Description
Function name	adc_ordinary_part_mode_enable
Function prototype	void adc_ordinary_part_mode_enable(adc_type *adc_x, confirm_state new_state)
Function description	Enable partition mode for ordinary channels
Input parameter 1	adc_x: indicates the selected ADC This parameter can be ADC1, ADC2, ADC3
Input parameter 2	new_state: indicates the pre-configured status for partition mode of ordinary channels This parameter can be TRUE or FALSE
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable the partitioned mode on ordinary channel */
adc_ordinary_part_mode_enable(ADC1, TRUE);
```

5.2.23 adc_preempt_part_mode_enable function

The table below describes the function adc_preempt_part_mode_enable.

Table 45. adc_preempt_part_mode_enable

Name	Description
Function name	adc_preempt_part_mode_enable
Function prototype	void adc_preempt_part_mode_enable(adc_type *adc_x, confirm_state new_state)
Function description	Enable partition mode for preempted channels
Input parameter 1	adc_x: indicates the selected ADC This parameter can be ADC1, ADC2, ADC3.
Input parameter 2	new_state: indicates the pre-configured status for partition mode of preempted channels This parameter can be TRUE or FALSE
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable the partitioned mode on preempt channel */
adc_preempt_part_mode_enable(ADC1, TRUE);
```

5.2.24 adc_preempt_auto_mode_enable function

The table below describes the function adc_preempt_auto_mode_enable.

Table 46. adc_preempt_auto_mode_enable function

Name	Description
Function name	adc_preempt_auto_mode_enable
Function prototype	void adc_preempt_auto_mode_enable(adc_type *adc_x, confirm_state,new_state)
Function description	Enable auto preempted group conversion at the end of ordinary group conversion
Input parameter 1	adc_x: indicates the selected ADC This parameter can be ADC1, ADC2, ADC3
Input parameter 2	new_state: indicates the pre-configured status for auto preempted group conversion This parameter can be TRUE or FALSE
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable automatic preempt group conversion */
adc_preempt_auto_mode_enable(ADC1, TRUE);
```

5.2.25 adc_tempersensor_vintrv_enable function

The table below describes the function adc_tempersensor_vintrv_enable.

Table 47. adc_tempersensor_vintrv_enable function

Name	Description
Function name	adc_tempersensor_vintrv_enable
Function prototype	void adc_tempersensor_vintrv_enable(confirm_state new_state)
Function description	Enable internal temperature sensor and V_{INTRV}
Input parameter	new_state: indicates the pre-configured status for internal temperature sensor and V_{INTRV} This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable the temperature sensor and vintrv channel */
adc_tempersensor_vintrv_enable(TRUE);
```

5.2.26 adc_ordinary_software_trigger_enable function

The table below describes the function adc_ordinary_software_trigger_enable.

Table 48. adc_ordinary_software_trigger_enable function

Name	Description
Function name	adc_ordinary_software_trigger_enable
Function prototype	void adc_ordinary_software_trigger_enable(adc_type *adc_x, confirm_state new_state)
Function description	Trigger ordinary group conversion by software
Input parameter 1	adc_x: indicates the selected ADC This parameter can be ADC1, ADC2, ADC3
Input parameter 2	new_state: indicates the pre-configured status for software-triggered ordinary group conversion This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable ordinary software start conversion */
adc_ordinary_software_trigger_enable(ADC1, TRUE);
```

5.2.27 adc_ordinary_software_trigger_status_get function

The table below describes the function adc_ordinary_software_trigger_status_get.

Table 49. adc_ordinary_software_trigger_status_get function

Name	Description
Function name	adc_ordinary_software_trigger_status_get
Function prototype	flag_status adc_ordinary_software_trigger_status_get(adc_type *adc_x)
Function description	Get the status of software-triggered ordinary group conversion
Input parameter	adc_x: indicates the selected ADC This parameter can be ADC1, ADC2, ADC3
Output parameter	NA
Return value	flag_status: indicates the status of software-triggered ordinary group conversion This parameter can be SET or RESET.
Required preconditions	NA
Called functions	NA

Example:

```
/* wait ordinary software start conversion */
while(adc_ordinary_software_trigger_status_get(ADC1));
```

5.2.28 adc_preempt_software_trigger_enable function

The table below describes the function adc_preempt_software_trigger_enable.

Table 50. adc_preempt_software_trigger_enable function

Name	Description
Function name	adc_preempt_software_trigger_enable
Function prototype	void adc_preempt_software_trigger_enable(adc_type *adc_x, confirm_state new_state)
Function description	Preempted group conversion triggered by software event
Input parameter 1	adc_x: indicates the selected ADC This parameter can be ADC1, ADC2, ADC3.
Input parameter 2	new_state: indicates the pre-configured status of software-triggered preempted group conversion This parameter can be TRUE or FALSE
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable preempt software start conversion */
adc_preempt_software_trigger_enable(ADC1, TRUE);
```

5.2.29 adc_preempt_software_trigger_status_get function

The table below describes the function adc_preempt_software_trigger_status_get.

Table 51. adc_preempt_software_trigger_status_get function

Name	Description
Function name	adc_preempt_software_trigger_status_get
Function prototype	flag_status adc_preempt_software_trigger_status_get(adc_type *adc_x)
Function description	Get the status of software-triggered preempted group conversion
Input parameter	adc_x: indicates the selected ADC This parameter can be ADC1, ADC2, ADC3
Output parameter	NA
Return value	flag_status: indicates the status of software-triggered preempted group conversion This parameter can be SET or RESET
Required preconditions	NA
Called functions	NA

Example:

```
/* wait preempt software start conversion */
while(adc_preempt_software_trigger_status_get(ADC1));
```

5.2.30 adc_ordinary_conversion_data_get function

The table below describes the function adc_ordinary_conversion_data_get.

Table 52. adc_ordinary_conversion_data_get function

Name	Description
Function name	adc_ordinary_conversion_data_get
Function prototype	uint16_t adc_ordinary_conversion_data_get(adc_type *adc_x)
Function description	Get the converted data of ordinary group in non-master/slave mode
Input parameter	adc_x: indicates the selected ADC This parameter can be ADC1, ADC2, ADC3
Output parameter	NA
Return value	16-bit converted data by ordinary group
Required preconditions	NA
Called functions	NA

Example:

```
uint16_t adc1_ordinary_index = 0;
adc1_ordinary_index = adc_ordinary_conversion_data_get(ADC1);
```

Note: This function can be used only when the ADC is configured with a single channel.

5.2.31 adc_combine_ordinary_conversion_data_get function

The table below describes the function adc_combine_ordinary_conversion_data_get

Table 53. adc_combine_ordinary_conversion_data_get function

Name	Description
Function name	adc_combine_ordinary_conversion_data_get
Function prototype	uint32_t adc_combine_ordinary_conversion_data_get(void)
Function description	Get the converted data of ordinary group in master/slave combine mode
Input parameter	NA
Output parameter	NA
Return value	32-bit converted data of ordinary group (upper 16 bits are for ADC2, the lower 16bits for ADC1)
Required preconditions	NA
Called functions	NA

Example:

```
uint32_t common_ordinary_index = 0;
  common_ordinary_index = adc_combine_ordinary_conversion_data_get();
```

Note: This function can be used only when the ADC is configured in master/slave combined mode and each of ADCs is configured with a single channel.

5.2.32 adc_preempt_conversion_data_get function

The table below describes the function adc_preempt_conversion_data_get.

Table 54. adc_preempt_conversion_data_get function

Name	Description
Function name	adc_preempt_conversion_data_get
Function prototype	uint16_t adc_preempt_conversion_data_get(adc_type *adc_x, adc_preempt_channel_type adc_preempt_channel)
Function description	Get the converted data of preempted group
Input parameter 1	adc_x: selected ADC This parameter can be ADC1, ADC2, ADC3
Input parameter 2	adc_preempt_channel: the selected preempted channel Refer to *adc_preempt_channel* for details
Output parameter	NA
Return value	16-bit converted data by preempted group
Required preconditions	NA
Called functions	NA

Example:

```
uint16_t adc1_preempt_valuetab[3] = {0};
adc1_preempt_valuetab[0] = adc_preempt_conversion_data_get(ADC1, ADC_PREEMPT_CHANNEL_1);
adc1_preempt_valuetab[1] = adc_preempt_conversion_data_get(ADC1, ADC_PREEMPT_CHANNEL_2);
adc1_preempt_valuetab[2] = adc_preempt_conversion_data_get(ADC1, ADC_PREEMPT_CHANNEL_3);
```

5.2.33 adc_flag_get function

The table below describes the function adc_flag_get.

Table 55. adc_flag_get function

Name	Description
Function name	adc_flag_get
Function prototype	flag_status adc_flag_get(adc_type *adc_x, uint8_t adc_flag)
Function description	Get the status of the flag bit
Input parameter 1	adc_x: indicates the selected ADC This parameter can be ADC1, ADC2, ADC3.
Input parameter 2	adc_flag: indicates the selected flag Refer to *adc_flag* for details.
Output parameter	NA
Return value	flag_status: the status for the selected flag bit Return value can be SET or RESET.
Required preconditions	NA
Called functions	NA

adc_flag

The adc_flag is used to select a flag to get its status, including:

ADC_VMOR_FLAG: Voltage monitor outside threshold

ADC_CCE_FLAG: End of channel conversion

ADC_PCCE_FLAG: End of preempted group conversion

ADC_PCCS_FLAG: Start of preempted group conversion

ADC_OCCS_FLAG: Start of ordinary group conversion

Example:

```
/* check if wakeup preempted channelsconversion end flag is set */
if(adc_flag_get(ADC1, ADC_PCCE_FLAG) != RESET)
```

5.2.34 adc_interrupt_flag_get function

The table below describes the function adc_interrupt_flag_get.

Table 56. adc_interrupt_flag_get function

Name	Description
Function name	adc_interrupt_flag_get
Function prototype	flag_status adc_interrupt_flag_get(adc_type *adc_x, uint8_t adc_flag)
Function description	Get the status of the interrupt flag bit
Input parameter 1	adc_x: indicates the selected ADC This parameter can be ADC1, ADC2, ADC3.
Input parameter 2	adc_flag: indicates the selected flag Refer to *adc_flag* for details.
Output parameter	NA
Return value	flag_status: the status for the selected flag bit Return value can be SET or RESET.
Required preconditions	NA
Called functions	NA

adc_flag

The adc_flag is used to select a flag to get its status, including:

ADC_VMOR_FLAG: Voltage monitor outside threshold

ADC_CCE_FLAG: End of channel conversion

ADC_PCCE_FLAG: End of preempted group conversion

Example

```
/* check if wakeup preempted channelsconversion end flag is set */
  if(adc_interrupt_flag_get(ADC1, ADC_PCCE_FLAG) != RESET)
```

5.2.35 adc_flag_clear function

The table below describes the function adc_flag_clear.

Table 57. adc_flag_clear function

Name	Description
Function name	adc_flag_clear
Function prototype	void adc_flag_clear(adc_type *adc_x, uint32_t adc_flag)
Function description	Clear the flag bits that have been set.
Input parameter 1	adc_x: indicates the selected ADC This parameter can be ADC1, ADC2, ADC3.
Input parameter 2	adc_flag: select a flag to be clera Refer to *adc_flag* for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* preempted channelsconversion end flag clear */
adc_flag_clear(ADC1, ADC_PCCE_FLAG);
```

5.3 Battery powered domain (BPR)

The BPR register structure bpr_type is defined in the "at32f403a_407_bpr.h":

```
/**
  * @brief type define bpr register all
  */
typedef struct
{


} bpr_type;
```

The table below gives a list of the BPR registers.

Table 58. Summary of BPR registers

Register	Description
dt1	Battery powered data register 1
dt2	Battery powered data register 2
dt3	Battery powered data register 3
dt4	Battery powered data register 4
dt5	Battery powered data register 5
dt6	Battery powered data register 6
dt7	Battery powered data register 7
dt8	Battery powered data register 8
dt9	Battery powered data register 9
dt10	Battery powered data register 10
rtccal	RTC calibration register
ctrl	BPR control register
ctrlsts	BPR control/status register
dt11	Battery powered data register 11
dt12	Battery powered data register 12
dt13	Battery powered data register 13
dt14	Battery powered data register 14
dt15	Battery powered data register 15
dt16	Battery powered data register 16
dt17	Battery powered data register 17
dt18	Battery powered data register 18
dt19	Battery powered data register 19
dt20	Battery powered data register 20
dt21	Battery powered data register 21
dt22	Battery powered data register 22
dt23	Battery powered data register 23
dt24	Battery powered data register 24
dt25	Battery powered data register 25
dt26	Battery powered data register 26
dt27	Battery powered data register 27

Register	Description
dt28	Battery powered data register 28
dt29	Battery powered data register 29
dt30	Battery powered data register 30
dt31	Battery powered data register 31
dt32	Battery powered data register 32
dt33	Battery powered data register 33
dt34	Battery powered data register 34
dt35	Battery powered data register 35
dt36	Battery powered data register 36
dt37	Battery powered data register 37
dt38	Battery powered data register 38
dt39	Battery powered data register 39
dt40	Battery powered data register 40
dt41	Battery powered data register 41
dt42	Battery powered data register 42

The table below gives a list of BP library functions.

Table 59. Summary of BP library functions

Function name	Description
bpr_reset	Reset all BPR registers to their default values
bpr_flag_get	Get flag status
bpr_flag_clear	Clear flag status
bpr_interrupt_enable	Enable tamper detection interrupt
bpr_data_read	Read BPR data registers
bpr_data_write	Write BPR data registers
bpr_rtc_output_select	Select event output
bpr_rtc_clock_calibration_value_set	Set clock calibration
bpr_tamper_pin_enable	Enable tamper detection
bpr_tamper_pin_active_level_set	Select tamper detection active level

5.3.1 bpr_reset function

The table below describes the function bpr_reset.

Table 60. bpr_reset function

Name	Description
Function name	bpr_reset
Function prototype	void bpr_reset(void);
Function description	Reset all BPR data registers to their default values
Input parameter 1	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	void crm_battery_powered_domain_reset(confirm_state new_state);

Example:

```
bpr_reset();
```

5.3.2 bpr_flag_get function

The table below describes the function bpr_flag_get.

Table 61. bpr_flag_get function

Name	Description
Function name	bpr_flag_get
Function prototype	flag_status bpr_flag_get(uint32_t flag);
Function description	Get flat status
Input parameter 1	Flag: flag selection Refer to the “flag” descriptions below for details.
Output parameter	NA
Return value	flag_status: flag status Return SET or RESET.
Required preconditions	NA
Called functions	NA

flag

This is used for flag selection, including:

BPR_TAMPER_INTERRUPT_FLAG: tamper detection interrupt

BPR_TAMPER_EVENT_FLAG: tamper detection event

Example:

```
bpr_flag_get(BPR_TAMPER_INTERRUPT_FLAG);
```

5.3.3 bpr_interrupt_flag_get function

The table below describes the function bpr_interrupt_flag_get function.

Table 62. bpr_interrupt_flag_get function

Name	Description
Function name	bpr_interrupt_flag_get
Function prototype	flag_status bpr_interrupt_flag_get(uint32_t flag);
Function description	Get flat status and judge the corresponding interrupt flag bit
Input parameter 1	flag: flag selection Refer to the “flag” descriptions below for details.
Output parameter	NA
Return value	flag_status: flag status Return SET or RESET.
Required preconditions	NA
Called functions	NA

flag

This is used for flag selection, including:

BPR_TAMPER_INTERRUPT_FLAG: tamper detection interrupt

BPR_TAMPER_EVENT_FLAG: tamper detection event

Example

```
bpr_interrupt_flag_get(BPR_TAMPER_INTERRUPT_FLAG);
```

5.3.4 bpr_flag_clear function

The table below describes the function bpr_flag_clear.

Table 63. bpr_flag_clear function

Name	Description
Function name	bpr_flag_clear
Function prototype	void bpr_flag_clear(uint32_t flag);
Function description	Clear flag
Input parameter 1	Flag: flag selection Refer to the “flag” descriptions below for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

flag

This is used for flag selection, including:

BPR_TAMPER_INTERRUPT_FLAG: tamper detection interrupt

BPR_TAMPER_EVENT_FLAG: tamper detection event

Example:

```
bpr_flag_clear(BPR_TAMPER_INTERRUPT_FLAG);
```

5.3.5 bpr_interrupt_enable function

The table below describes the function bpr_interrupt_enable.

Table 64. bpr_interrupt_enable function

Name	Description
Function name	bpr_interrupt_enable
Function prototype	void bpr_interrupt_enable(confirm_state new_state);
Function description	Enable tamper detection interrupt
Input parameter 1	new_state: tamper detection interrupt status This parameter can be TRUE or FALSE
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
bpr_interrupt_enable(TRUE);
```

5.3.6 bpr_data_read function

The table below describes the function bpr_data_read.

Table 65. bpr_data_read function

Name	Description
Function name	bpr_data_read
Function prototype	uint16_t bpr_data_read(bpr_data_type bpr_data);
Function description	Read BPR data registers
Input parameter 1	bpr_data: data register Refer to the “bpr_data” descriptions below for details.
Output parameter	NA
Return value	Return BPR data register value
Required preconditions	NA
Called functions	NA

bpr_data

Data registers.

BPR_DATA1: Data register 1

BPR_DATA2: Data register 2

BPR_DATA41: Data register 41

BPR_DATA42: Data register 42

Example:

```
bpr_data_read(BPR_DATA1);
```

5.3.7 bpr_data_write function

The table below describes the function bpr_data_write.

Table 66. bpr_data_write function

Name	Description
Function name	bpr_data_write
Function prototype	void bpr_data_write(bpr_data_type bpr_data, uint16_t data_value);
Function description	Write BPR data registers
Input parameter 1	bpr_data: data register Refer to the “bpr_data” descriptions below for details.
Input parameter 2	data_value: 16-bit data
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

bpr_data

Data registers.

BPR_DATA1: Data register 1

BPR_DATA2: Data register 2

BPR_DATA41: Data register 41

BPR_DATA42: Data register 42

Example:

```
bpr_data_write(BPR_DATA1, 0x5A5A);
```

5.3.8 bpr_rtc_output_select function

The table below describes the function bpr_rtc_output_select.

Table 67. bpr_rtc_output_select function

Name	Description
Function name	bpr_rtc_output_select
Function prototype	void bpr_rtc_output_select(bpr_rtc_output_type output_source);
Function description	Select event output
Input parameter 1	output_source: output event Refer to the “output_source” descriptions below for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

output_source

Select an event output.

BPR_RTC_OUTPUT_NONE: No output

BPR_RTC_OUTPUT_CLOCK_CAL_BEFORE: divided-by-64 clock output before calibration

BPR_RTC_OUTPUT_ALARM: Pulse alarm event output

BPR_RTC_OUTPUT_SECOND: Pulse second event output

BPR_RTC_OUTPUT_CLOCK_CAL_AFTER: divided-by-64 clock output after calibration

BPR_RTC_OUTPUT_ALARM_TOGGLE: Toggle output alarm event

BPR_RTC_OUTPUT_SECOND_TOGGLE: Toggle output second event

Example:

```
bpr_rtc_output_select(BPR_RTC_OUTPUT_ALARM);
```

5.3.9 bpr_rtc_clock_calibration_value_set function

The table below describes the function bpr_rtc_clock_calibration_value_set.

Table 68. bpr_rtc_clock_calibration_value_set function

Name	Description
Function name	bpr_rtc_clock_calibration_value_set
Function prototype	void bpr_rtc_clock_calibration_value_set(uint8_t calibration_value);
Function description	Set clock calibration
Input parameter 1	Value: calibration value ranges from 0 to 0x7F
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
bpr_rtc_clock_calibration_value_set(0x7F);
```

5.3.10 bpr_tamper_pin_enable function

The table below describes the function bpr_tamper_pin_enable.

Table 69. bpr_tamper_pin_enable function

Name	Description
Function name	bpr_tamper_pin_enable
Function prototype	void bpr_tamper_pin_enable(confirm_state new_state);
Function description	Enable tamper detection
Input parameter 1	new_state: interrupt enable status This parameter can be TRUE or FALSE
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
bpr_tamper_pin_enable(TRUE);
```

5.3.11 bpr_tamper_pin_active_level_set function

The table below describes the function bpr_tamper_pin_active_level_set.

Table 70. bpr_tamper_pin_active_level_set function

Name	Description
Function name	bpr_tamper_pin_active_level_set
Function prototype	void bpr_tamper_pin_active_level_set(bpr_tamper_pin_active_level_type active_level);
Function description	Set tamper detection active level
Input parameter 1	active_level: tamper detection active level Refer to the “active_level” descriptions below for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

active_level

Select tamer detection active level.

BPR_TAMPER_PIN_ACTIVE_HIGH: High

BPR_TAMPER_PIN_ACTIVE_LOW: Low

Example:

```
bpr_tamper_pin_active_level_set(BPR_TAMPER_PIN_ACTIVE_HIGH);
```

5.4 Controller area network (CAN)

CAN register structure can_type is defined in the “at32f403a_407_can.h”:

```
/**
  * @brief type define can register all
  */
typedef struct
{
  ...
} can_type;
```

The table below gives a list of the CAN registers.

Table 71. Summary of ADC registers

Register	Description
mctrl	CAN master control register
msts	CAN master status register
tsts	CAN transmit status register
rf0	CAN receive FIFO 0 register
fr1	CAN receive FIFO 1 register
inten	CAN interrupt enable register
ests	CAN error status register
btmg	CAN bit timing register
tmi0	Transmit mailbox 0 identifier register
tmc0	Transmit mailbox 0 data length and time stamp register
tmdtl0	Transmit mailbox 0 data byte low register
tmdth0	Transmit mailbox 0 data byte high register
tmi1	Transmit mailbox 1 identifier register
tmc1	Transmit mailbox 1 data length and time stamp register
tmdtl1	Transmit mailbox 1 data byte low register
tmdth1	Transmit mailbox 1 data byte high register
tmi2	Transmit mailbox 2 identifier register
tmc2	Transmit mailbox 2 data length and time stamp register
tmdtl2	Transmit mailbox 2 data byte low register
tmdth2	Transmit mailbox 2 data byte high register
rfi0	Receive FIFO0 mailbox identifier register
rfc0	Receive FIFO0 mailbox data length and time stamp register
rfdtl0	Receive FIFO0 mailbox data byte low register
rfdth0	Receive FIFO0 mailbox data byte high register
rfi1	Receive FIFO1 mailbox identifier register
rfc1	Receive FIFO1 mailbox data length and time stamp register
rfdtl1	Receive FIFO1 mailbox data byte low register
rfdth1	Receive FIFO1 mailbox data byte high register
fctrl	CAN filter control register
fmcfg	CAN filter mode configuration register
fscfg	CAN filter size configuration register

Register	Description
frf	CAN filter FIFO assosication register
facfg	CAN filter activate control register
fb0f1	CAN filter bank 0 filter register 1
fb0f2	CAN filter bank 0 filter register 2
fb1f1	CAN filter bank 1 filter register 1
fb1f2	CAN filter bank 1 filter register 2
…	…
fb13f1	CAN filter bank 13 filter register 1
fb13f2	CAN filter bank 13 filter register 2

The table below gives a list of CAN library functions.

Table 72. Summary of CAN library functions

Function name	Description
can_reset	Reset all CAN registers to their reset values
can_baudrate_default_para_init	Configure the CAN baud rate initial structure with the initial value
can_baudrate_set	Configure CAN baud rate
can_default_para_init	Configure the CAN initial structure with the initial value
can_base_init	Initialize CAN registers with the specified parameters in the can_base_struct
can_filter_default_para_init	Configure the CAN filter initial structure with the initial value
can_filter_init	Initialize CAN registers with the specified parameters in the can_filter_init_struct
can_debug_transmission_prohibit	Select to 104isable/enable message reception and transmission when debug
can_ttc_mode_enable	Enable time-triggered mode
can_message_transmit	Transmit a frame of message
can_transmit_status_get	Get the status of transmission
can_transmit_cancel	Cancel transmission
can_message_receive	Receive a frame of message
can_receive_fifo_release	Release receive FIFO
can_receive_message_pending_get	Get the count of pending messages in FIFO
can_operating_mode_set	Configure CAN operating mode
can_doze_mode_enter	Enter sleep mode
can_doze_mode_exit	Exit sleep mode
can_error_type_record_get	Read CAN error type
can_receive_error_counter_get	Read CAN receive error counter
can_transmit_error_counter_get	Read CAN transmit error counter
can_interrupt_enable	Enable the selected CAN interrupt
can_flag_get	Read the selected CAN flag
Can_interrupt_flag_get	Read the selected CAN interrupt flag
can_flag_clear	Clear the selected CAN flag

5.4.1 can_reset function

The table below describes the function can_reset.

Table 73. can_reset function

Name	Description
Function name	can_reset
Function prototype	void can_reset(can_type* can_x);
Function description	Reset CAN registers to their default values
Input parameter 1	adc_x: indicates the selected CAN This parameter can be CAN1, CAN2
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	crm_periph_reset();

Example:

```
can_reset(CAN1);
```

5.4.2 can_baudrate_default_para_init function

The table below describes the function can_baudrate_default_para_init.

Table 74. can_baudrate_default_para_init function

Name	Description
Function name	can_baudrate_default_para_init
Function prototype	void can_baudrate_default_para_init(can_baudrate_type* can_baudrate_struct);
Function description	Configure the CAN baud rate initial structure with the initial value
Input parameter 1	can_baudrate_struct: *can_baudrate_type* pointer
Output parameter	NA
Return value	NA
Required preconditions	It is necessary to define a variable of the can_baudrate_type before starting.
Called functions	NA

Example:

```
can_baudrate_type can_baudrate_struct;
can_baudrate_default_para_init(&can_baudrate_struct);
```

5.4.3 can_baudrate_set function

The table below describes the function can_baudrate_set.

Table 75. can_baudrate_set function

Name	Description
Function name	can_baudrate_set
Function prototype	error_status can_baudrate_set(can_type* can_x, can_baudrate_type* can_baudrate_struct);
Function description	Set baud rate for CAN
Input parameter 1	adc_x: indicates the selected CAN This parameter can be CAN1, CAN2
Input parameter 2	can_baudrate_struct: *can_baudrate_type* pointer
Output parameter	NA
Return value	status_index: check if baud rate is configured successfully
Required preconditions	It is necessary to define a variable of the can_baudrate_type before starting.
Called functions	NA

can_baudrate_type is defined in the at32f403a_407_can.h:

```
typedef struct
{
  uint16_t          baudrate_div;
  can_rsaw_type     rsaw_size;
  can_bts1_type     bts1_size;
  can_bts2_type     bts2_size;
} can_baudrate_type;
```

baudrate_div

CAN clock division factor

Value range: 0x001~0x400

rsaw_size

Defines the maximum of time unit that the CAN is allowed to lengthen or shorten in a bit

CAN_RSAW_1TQ: Resynchronization width is 1 time unit

CAN_RSAW_2TQ: Resynchronization width is 2 time units

CAN_RSAW_3TQ: Resynchronization width is 3 time units

CAN_RSAW_4TQ: Resynchronization width is 4 time units

bts1_size

segment1 time duration

bts1_size description

CAN_BTS1_1TQ: the bit time segment 1 has 1 time unit

…

CAN_BTS1_16TQ: the bit time segment 16 time units

bts2_size

segment2 time duration

CAN_BTS2_1TQ: the bit time segment 2 has 1 time unit

…

CAN_BTS2_8TQ: the bit time segment 2 has 8 time units

Example:

```
/* can baudrate, set baudrate = pclk/(baudrate_div *(1 + bts1_size + bts2_size)) */
can_baudrate_struct.baudrate_div = 10;
can_baudrate_struct.rsaw_size = CAN_RSAW_3TQ;
can_baudrate_struct.bts1_size = CAN_BTS1_8TQ;
can_baudrate_struct.bts2_size = CAN_BTS2_3TQ;
can_baudrate_set(CAN1, &can_baudrate_struct);
```

5.4.4 can_default_para_init function

The table below describes the function can_default_para_init.

Table 76. can_default_para_init function

Name	Description
Function name	can_default_para_init
Function prototype	void can_default_para_init(can_base_type* can_base_struct);
Function description	Set an initial value for CAN initial structure
Input parameter 1	can_base_struct: *can_base_type* pointer
Output parameter	NA
Return value	NA
Required preconditions	It is necessary to define a variable of the can_base_type before starting.
Called functions	NA

Example:

```
can_base_type can_base_struct;
can_default_para_init (&can_base_struct);
```

5.4.5 can_base_init function

The table below describes the function can_base_init.

Table 77. can_base_init function

Name	Description
Function name	can_base_init
Function prototype	error_status can_base_init(can_type* can_x, can_base_type* can_base_struct);
Function description	Initialize CAN rgisters with the specified parameters in the can_base_struct
Input parameter 1	can_base_struct: *can_base_type* pointer
Output parameter	NA
Return value	NA
Required preconditions	It is necessary to define a variable of can_base_type before starting.
Called functions	NA

The can_base_type is defined in the at32f403a_407_can.h:

```
typedef struct
{
  can_mode_type          mode_selection;
  confirm_state          ttc_enable;
  confirm_state          aebo_enable;
  confirm_state          aed_enable;
```

```
  confirm_state                 prsf_enable;
  can_msg_discarding_rule_type  mdrsel_selection;
  can_msg_sending_rule_type     mmssr_selection;
} can_base_type;
```

mode_selection

Test mode selection

CAN_MODE_COMMUNICATE:	Communication mode
CAN_MODE_LOOPBACK:	Loopback mode
CAN_MODE_LISTENONLY:	Listen only mode
CAN_MODE_LISTENONLY_LOOPBACK:	Loopback + listen only mode

ttc_enable

Enable/disable time-triggered communication mode.

FALSE: Disable time-triggered communication mode

TRUE: Enable time-triggered communication mode (while receiving/sending messages, capture time stamp and store it into the CAN_RFCx and CAN_TMCx registers)

aebo_enable

Enable auto exit of bus-off mode.

FALSE: Automatic exit of bus-off mode is disabled

TRUE: Automatic exit of bus-off mode is enabled

aed_enable

Enable auto exit of sleep mode.

FALSE: Auto exit of sleep mode is disabled

TRUE: Auto exit of sleep mode is enabled

prsf_enable

Disable retransmission when transmit failed.

FALSE: Retransmission is enabled.

TRUE: Retransmission is disabled

mdrsel_selection

Define message discard rule when reception overflows.

CAN_DISCARDING_FIRST_RECEIVED: The previous message is discarded.

CAN_DISCARDING_LAST_RECEIVED: The new incoming message is discarded.

mmssr_selection

Define multiple message transmit sequence rule.

CAN_SENDING_BY_ID: The message with the smallest identifier is first transmitted.

CAN_SENDING_BY_REQUEST: The message with the first request order is first transmitted.

Example:

```
/* can base init */
can_base_struct.mode_selection = CAN_MODE_COMMUNICATE;
can_base_struct.ttc_enable = FALSE;
can_base_struct.aebo_enable = TRUE;
can_base_struct.aed_enable = TRUE;
can_base_struct.prsf_enable = FALSE;
can_base_struct.mdrsel_selection = CAN_DISCARDING_FIRST_RECEIVED;
can_base_struct.mmssr_selection = CAN_SENDING_BY_ID;
can_base_init(CAN1, &can_base_struct);
```

5.4.6 can_filter_default_para_init function

The table below describes the function can_filter_default_para_init.

Table 78. can_filter_default_para_init function

Name	Description
Function name	can_filter_default_para_init
Function prototype	void can_filter_default_para_init(can_filter_init_type* can_filter_init_struct);
Function description	Configure CAN filter initialization structure with the initial value
Input parameter 1	can_filter_init_struct: *can_filter_init_type* pointer
Output parameter	NA
Return value	NA
Required preconditions	It is necessary to define a variable of can_filter_init_type before starting.
Called functions	NA

Example:

```
can_filter_init_type can_filter_init_struct;
can_filter_default_para_init(&can_filter_init_struct);
```

5.4.7 can_filter_init function

The table below describes the function can_filter_init.

Table 79. can_filter_init function

Name	Description
Function name	can_filter_init
Function prototype	void can_filter_init(can_type* can_x, can_filter_init_type* can_filter_init_struct);
Function description	Initialize all CAN registers with the specified parameters in the can_base_struct
Input parameter 1	adc_x: indicates the selected CAN This parameter can be CAN1, CAN2
Input parameter 2	can_filter_init_struct: *can_filter_init_type* pointer
Output parameter	NA
Return value	NA
Required preconditions	It is required to define a variable of the can_filter_init_type before starting.
Called functions	NA

can_filter_init_type is defined in the at32f403a_407_can.h:

```
typedef struct
{
  confirm_state              ilter_activate_enable;
  can_filter_mode_type       filter_mode;
  can_filter_fifo_type       filter_fifo;
  uint8_t                    filter_number;
  can_filter_bit_width_type  filter_bit;
  uint16_t                   filter_id_high;
  uint16_t                   filter_id_low;
  uint16_t                   filter_mask_high;
  uint16_t                   filter_mask_low;
} can_filter_init_type;
```

filter_activate_enable
Enable/disable filter bank
FALSE: Disable filter bank
TRUE: Enable filter bank
filter_mode
Select filter mode.
CAN_FILTER_MODE_ID_MASK: Identifier mask mode
CAN_FILTER_MODE_ID_LIST: Identifier list mode
filter_fifo
Select filter associated FIFO.
CAN_FILTER_FIFO0: Associated with FIFO0
CAN_FILTER_FIFO1: Associated with FIFO1
filter_number
Select filter bank.
Value range: 0~13
filter_bit
Select filter bit width
CAN_FILTER_16BIT: 16-bit
CAN_FILTER_32BIT: 32-bit
filter_id_high
The filter_id_high is used to configure the upper 16 bits (32-bit width, Mask/List mode) of the filter identifier 1, the filter identifier 2 (16-bit width, List mode) or the filter mask identifier 1 (16-bit width, Mask mode)
Value range: 0x0000~0xFFFF
filter_id_low
The filter_id_low is used to configure the lower 16 bits of the filter identifier 1 (32-bit width, Mask/List mode), or the filter identifier 1 (16-bit width, List mode).
Value range: 0x0000~0xFFFF
filter_mask_high
The filter_mask_high is used to configure the upper 16 bits of the filter mask identifier 1 (32-bit width, Mask mode), the filter mask identifier 2 (16-bit width, Mask mode), the upper 16 bits of the filter identifier 2 (32-bit width, List mode) or the filter identifier 4 (16-bit width, List mode)
Value range: 0x0000~0xFFFF
filter_mask_low
The filter_mask_low is used to configure the lower 16 bits of the filter mask identifier 1 (32-bit width, Mask mode), the filter identifier 2 (16-bit width, Mask mode), the lower 16 bits of the filter identifier 2 (32-bit width ,List mode) or the filter identifier 3 (16-bit width, List mode)
Value range: 0x0000~0xFFFF
Example:

```
/* can filter init */
can_filter_init_struct.filter_activate_enable = TRUE;
can_filter_init_struct.filter_mode = CAN_FILTER_MODE_ID_MASK;
can_filter_init_struct.filter_fifo = CAN_FILTER_FIFO0;
can_filter_init_struct.filter_number = 0;
can_filter_init_struct.filter_bit = CAN_FILTER_32BIT;
can_filter_init_struct.filter_id_high = 0;
```

```
can_filter_init_struct.filter_id_low = 0;
can_filter_init_struct.filter_mask_high = 0;
can_filter_init_struct.filter_mask_low = 0;
can_filter_init(CAN1, &can_filter_init_struct);
```

5.4.8 can_debug_transmission_prohibit function

The table below describes the function can_debug_transmission_prohibit.

Table 80. can_debug_transmission_prohibit function

Name	Description
Function name	can_debug_transmission_prohibit
Function prototype	void can_debug_transmission_prohibit(can_type* can_x, confirm_state new_state);
Function description	Disable/enable message transceiver when debugging
Input parameter 1	adc_x: indicates the selected CAN This parameter can be any CAN1, CAN2
Input parameter 2	new_state: Enable or disable This parameter can be FALSE or TRUE
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* prohibit can trans when debug*/
can_debug_transmission_prohibit(CAN1, TRUE);
```

5.4.9 can_ttc_mode_enable function

The table below describes the function can_ttc_mode_enable.

Table 81. can_ttc_mode_enable function

Name	Description
Function name	can_ttc_mode_enable
Function prototype	void can_ttc_mode_enable(can_type* can_x, confirm_state new_state);
Function description	Enable time-triggered mode
Input parameter 1	adc_x: indicates the selected CAN This parameter can be CAN1, CAN2
Input parameter 2	new_state: Enable or disable This parameter can be FALSE or TRUE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* can time trigger operation communication mode enable*/
can_ttc_mode_enable (CAN1, TRUE);
```

Note: When the ttc_enable is enabled in the can_base_init, it indicates that only the time stamp is enabled (During message receive and transmit, the time stamp is captured and stored in the CAN_RFCx

and CAN_TMCx registers). But when the can_ttc_mode_enable is enabled, not only the time stamp is enabled, and but the time stamp transmission feature is enabled (During message transmission, the time stamp is sent on the 7th and 8th data byte).

5.4.10 can_message_transmit function

The table below describes the function can_message_transmit.

Table 82. can_message_transmit function

Name	Description
Function name	can_message_transmit
Function prototype	uint8_t can_message_transmit(can_type* can_x, can_tx_message_type* tx_message_struct);
Function description	Transmit a frame of message
Input parameter 1	adc_x: indicates the selected CAN This parameter can be any CAN1, CAN2
Input parameter 2	tx_message_struct: message pending for transmission, refer to *can_tx_message_type*
Output parameter	NA
Return value	transmit_mailbox: indicates the mailbox number required to send message
Required preconditions	Write the to-be-sent message in the tx_message_struct
Called functions	NA

can_tx_message_type is defined in the at32f403a_407_can.h:

```
typedef struct
{
  uint32_t                standard_id;
  uint32_t                extended_id;
  can_identifier_type     id_type;
  can_trans_frame_type    frame_type;
  uint8_t                 dlc;
  uint8_t                 data[8];
} can_tx_message_type;
```

standard_id

Standard identifier (11 bits active)

Value range: 0x000~0x7FF

extended_id

Extended identifier (29 bits active)

Value range: 0x000~0x1FFFFFFF

id_type

Identifier type

CAN_ID_STANDARD: Standard identifier

CAN_ID_EXTENDED: Extended identifier

frame_type

Frame type

CAN_TFT_DATA: Data frame

CAN_TFT_REMOTE: Remote frame

dlc

Data length (in byte)

Value range: 0~8

data[8]

Data pending for transmission

Value range: 0x00~0xFF

Example:

```
/* can transmit data */
static void can_transmit_data(void)
{
  uint8_t transmit_mailbox;
  can_tx_message_type tx_message_struct;
  tx_message_struct.standard_id = 0x400;
  tx_message_struct.extended_id = 0;
  tx_message_struct.id_type = CAN_ID_STANDARD;
  tx_message_struct.frame_type = CAN_TFT_DATA;
  tx_message_struct.dlc = 8;
  tx_message_struct.data[0] = 0x11;
  tx_message_struct.data[1] = 0x22;
  tx_message_struct.data[2] = 0x33;
  tx_message_struct.data[3] = 0x44;
  tx_message_struct.data[4] = 0x55;
  tx_message_struct.data[5] = 0x66;
  tx_message_struct.data[6] = 0x77;
  tx_message_struct.data[7] = 0x88;
  transmit_mailbox = can_message_transmit(CAN1, &tx_message_struct);
  while(can_transmit_status_get(CAN1, (can_tx_mailbox_num_type)transmit_mailbox) !=
CAN_TX_STATUS_SUCCESSFUL);
}
```

5.4.11 can_transmit_status_get function

The table below describes the function can_transmit_status_get.

Table 83. can_transmit_status_get function

Name	Description
Function name	can_transmit_status_get
Function prototype	can_transmit_status_type can_transmit_status_get(can_type* can_x, can_tx_mailbox_num_type transmit_mailbox);
Function description	Get the status of transmission
Input parameter 1	adc_x: indicates the selected CAN This parameter can be any CAN1, CAN2
Input parameter 2	transmit_mailbox: indicates the mailbox number required to send messge
Output parameter	NA
Return value	state_index: transmission status
Required preconditions	First send a frame of message and get a transmit mailbox number
Called functions	NA

Example:

```
/* can transmit data */
```

```
static void can_transmit_data(void)
{
  uint8_t transmit_mailbox;
  can_tx_message_type tx_message_struct;
  tx_message_struct.standard_id = 0x400;
  tx_message_struct.extended_id = 0;
  tx_message_struct.id_type = CAN_ID_STANDARD;
  tx_message_struct.frame_type = CAN_TFT_DATA;
  tx_message_struct.dlc = 8;
  tx_message_struct.data[0] = 0x11;
  tx_message_struct.data[1] = 0x22;
  tx_message_struct.data[2] = 0x33;
  tx_message_struct.data[3] = 0x44;
  tx_message_struct.data[4] = 0x55;
  tx_message_struct.data[5] = 0x66;
  tx_message_struct.data[6] = 0x77;
  tx_message_struct.data[7] = 0x88;
  transmit_mailbox = can_message_transmit(CAN1, &tx_message_struct);
  while(can_transmit_status_get(CAN1, (can_tx_mailbox_num_type)transmit_mailbox) !=
CAN_TX_STATUS_SUCCESSFUL);
}
```

5.4.12 can_transmit_cancel function

The table below describes the function can_transmit_cancel.

Table 84. can_transmit_cancel function

Name	Description
Function name	can_transmit_cancel
Function prototype	void can_transmit_cancel(can_type* can_x, can_tx_mailbox_num_type transmit_mailbox);
Function description	Cancel transmission
Input parameter 1	adc_x: indicates the selected CAN This parameter can be CAN1, CAN2
Input parameter 2	transmit_mailbox: indicates the mailbox number required to send message
Output parameter	NA
Return value	NA
Required preconditions	First send a frame of message and get a transmit mailbox number
Called functions	NA

Example:

```
/* cancel a transmit request */
uint8_t transmit_mailbox;
transmit_mailbox = can_message_transmit(CAN1, &tx_message_struct);
can_transmit_cancel(CAN1, (can_tx_mailbox_num_type)transmit_mailbox);
```

5.4.13 can_message_receive function

The table below describes the function can_message_receive.

Table 85. can_message_receive function

Name	Description
Function name	can_message_receive
Function prototype	void can_message_receive(can_type* can_x, can_rx_fifo_num_type fifo_number, can_rx_message_type* rx_message_struct);
Function description	Receive message
Input parameter 1	adc_x: indicates the selected CAN This parameter can be CAN1, CAN2
Input parameter 2	fifo_number: receive FIFO This parameter can be CAN_RX_FIFO0 or CAN_RX_FIFO1.
Output parameter	rx_message_struct: indicates the received message, refer to *can_rx_message_type*
Return value	NA
Required preconditions	Receive FIFO not empty (FIFO message count is not zero)
Called functions	void can_receive_fifo_release(can_type* can_x, can_rx_fifo_num_type fifo_number);

can_rx_message_type is defined in the at32f403a_407_can.h:

```
typedef struct
{
    uint32_t                standard_id;
    uint32_t                extended_id;
    can_identifier_type     id_type;
    can_trans_frame_type    frame_type;
    uint8_t                 dlc;
    uint8_t                 data[8];
    uint8_t                 filter_index;
} can_rx_message_type;
```

standard_id

Standard identifier (11 bit active)

Value range: 0x000~0x7FF

extended_id

Extended identifier (29 bits active)

Value range: 0x000~0x1FFFFFFF

id_type

Identifier type

CAN_ID_STANDARD: Standard identifier

CAN_ID_EXTENDED: Extended identifier

frame_type

Frame type

CAN_TFT_DATA: Data frame

CAN_TFT_REMOTE: Remote frame

dlc

Data length (in byte)

Value range: 0~8

data[8]

Data pending for transmission
Value range: 0x00~0xFF

filter_index

Filter match index (indicating the filter number that a message has passed through.)
Value range: 0x00~0xFF

Example:

```
/* can receive message */
can_rx_message_type rx_message_struct;
can_message_receive(CAN1, CAN_RX_FIFO0, &rx_message_struct);
```

5.4.14 can_receive_fifo_release function

The table below describes the function can_receive_fifo_release.

Table 86. can_receive_fifo_release function

Name	Description
Function name	can_receive_fifo_release
Function prototype	void can_receive_fifo_release(can_type* can_x, can_rx_fifo_num_type fifo_number);
Function description	Release receive FIFO
Input parameter 1	adc_x: indicates the selected CAN This parameter can be CAN1, CAN2
Input parameter 2	fifo_number: receive FIFO number This parameter can be either CAN_RX_FIFO0 or CAN_RX_FIFO1
Output parameter	NA
Return value	NA
Required preconditions	Message in FIFO has already been read
Called functions	NA

Example:

```
/* can receive message */
void can_message_receive(can_type* can_x, can_rx_fifo_num_type fifo_number, can_rx_message_type*
rx_message_struct)
{
  /* get the id type */
  rx_message_struct->id_type = (can_identifier_type)can_x->fifo_mailbox[fifo_number].rfi_bit.rfidi;
  …

  /* get the data field */
  rx_message_struct->data[0] = can_x->fifo_mailbox[fifo_number].rfdtl_bit.rfdt0;
  …
  rx_message_struct->data[7] = can_x->fifo_mailbox[fifo_number].rfdth_bit.rfdt7;

  /*FIFO must be read before releasing FIFO */
  /* release the fifo */
  can_receive_fifo_release(can_x, fifo_number);
}
```

5.4.15 can_receive_message_pending_get function

The table below describes the function can_receive_message_pending_get.

Table 87. can_receive_message_pending_get function

Name	Description
Function name	can_receive_message_pending_get
Function prototype	uint8_t can_receive_message_pending_get(can_type* can_x, can_rx_fifo_num_type fifo_number);
Function description	Get the number of message pending for read in FIFO
Input parameter 1	adc_x: indicates the selected CAN This parameter can be CAN1, CAN2
Input parameter 2	fifo_number: receive FIFO number This parameter can be either CAN_RX_FIFO0 or CAN_RX_FIFO1
Output parameter	NA
Return value	message_pending: the count of message pending for read in FIFO
Required preconditions	NA
Called functions	NA

Example:

```
/* return the number of pending messages of */
can_receive_message_pending_get (CAN1, CAN_RX_FIFO0);
```

5.4.16 can_operating_mode_set function

The table below describes the function can_operating_mode_set.

Table 88. can_operating_mode_set function

Name	Description
Function name	can_operating_mode_set
Function prototype	error_status can_operating_mode_set(can_type* can_x, can_operating_mode_type can_operating_mode);
Function description	Configure CAN operating modes
Input parameter 1	adc_x: indicates the selected CAN This parameter can be CAN1, CAN2
Input parameter 2	*can_operating_mode*: CAN operating mode selection
Output parameter	NA
Return value	Status: indicates whether configuration is successful or not
Required preconditions	NA
Called functions	NA

can_operating_mode

CAN_OPERATINGMODE_FREEZE: Freeze mode—for CAN controller initialization

CAN_OPERATINGMODE_DOZE: Sleep mode—CAN clock stopped to save power consumption

CAN_OPERATINGMODE_COMMUNICATE: Communication mode—for communication

Example:

```
/* set the operation mode –enter freeze mode*/
can_operating_mode_set (CAN1, CAN_OPERATINGMODE_FREEZE);
```

```
/*Initialize CAN controller*/
…

/* set the operation mode –enter communicate mode*/
can_operating_mode_set (CAN1, CAN_OPERATINGMODE_COMMUNICATE);

/*Starts communication: send and receive message */
…
```

5.4.17 can_doze_mode_enter function

The table below describes the function can_doze_mode_enter

Table 89. can_doze_mode_enter function

Name	Description
Function name	can_doze_mode_enter
Function prototype	can_enter_doze_status_type can_doze_mode_enter(can_type* can_x);
Function description	Enter sleep mode
Input parameter 1	adc_x: indicates the selected CAN This parameter can be CAN1, CAN2
Output parameter	NA
Return value	*can_enter_doze_status*: indicates wheter the Sleep mode is entered
Required preconditions	NA
Called functions	NA

can_enter_doze_status

Indicates whether the Sleep mode is entered or not

CAN_ENTER_DOZE_FAILED: Sleep mode entry failure

CAN_ENTER_DOZE_SUCCESSFUL: Sleep mode entry success

Example:

```
/* can enter the low power mode */
can_enter_doze_status_type can_enter_doze_status;
can_enter_doze_status = can_doze_mode_enter(CAN1);
```

5.4.18 can_doze_mode_exit function

The table below describes the function can_doze_mode_exit.

Table 90. can_doze_mode_exit function

Name	Description
Function name	can_doze_mode_exit
Function prototype	can_quit_doze_status_type can_doze_mode_exit(can_type* can_x);
Function description	Exit Sleep mode
Input parameter 1	adc_x: indicates the selected CAN This parameter can be CAN1, CAN2
Output parameter	NA
Return value	*can_quit_doze_status*: indicates whethe the Sleep mode has been left
Required preconditions	NA
Called functions	NA

can_quit_doze_status

Indicates whethe the Sleep mode has been left successfully.

CAN_QUIT_DOZE_FAILED: Sleep mode exit failure

CAN_QUIT_DOZE_SUCCESSFUL: Sleep mode exit success

Example:

```
/* can exit the low power mode */
can_quit_doze_status_type can_quit_doze_status;
can_quit_doze_status = can_doze_mode_exit (CAN1);
```

5.4.19 can_error_type_record_get function

The table below describes the function can_error_type_record_get.

Table 91. can_error_type_record_get function

Name	Description
Function name	can_error_type_record_get
Function prototype	can_error_record_type can_error_type_record_get(can_type* can_x);
Function description	Read CAN error tyype
Input parameter 1	adc_x: indicates the selected CAN This parameter can be CAN1, CAN2
Output parameter	NA
Return value	*can_error_record*: Error type
Required preconditions	NA
Called functions	NA

can_error_record

CAN error record

CAN_ERRORRECORD_NOERR:	No error
CAN_ERRORRECORD_STUFFERR:	Bit stuffing error
CAN_ERRORRECORD_FORMERR:	Format error
CAN_ERRORRECORD_ACKERR:	Acknowledge error
CAN_ERRORRECORD_BITRECESSIVEERR:	Recessive bit error
CAN_ERRORRECORD_BITDOMINANTERR:	Dominant bit error

CAN_ERRORRECORD_CRCERR: CRC error
CAN_ERRORRECORD_SOFTWARESETERR: Set by software

Example:

```
/* get the error type record (etr) */
can_error_record_type can_error_record;
can_error_record = can_error_type_record_get (CAN1);
```

5.4.20 can_receive_error_counter_get function

The table below describes the function can_receive_error_counter_get.

Table 92. can_receive_error_counter_get function

Name	Description
Function name	can_receive_error_counter_get
Function prototype	uint8_t can_receive_error_counter_get(can_type* can_x);
Function description	Read CAN receive error counter
Input parameter 1	adc_x: indicates the selected CAN This parameter can be CAN1, CAN2
Output parameter	NA
Return value	receive_error_counter: Receive error counter Value range: 0x00~0xFF
Required preconditions	NA
Called functions	NA

Example:

```
/* get the receive error counter (rec) */
uint8_t receive_error_counter;
receive_error_counter = can_receive_error_counter_get (CAN1);
```

5.4.21 can_transmit_error_counter_get function

The table below describes the function can_transmit_error_counter_get.

Table 93. can_transmit_error_counter_get function

Name	Description
Function name	can_transmit_error_counter_get
Function prototype	uint8_t can_transmit_error_counter_get(can_type* can_x);
Function description	Read CAN transmit error counter
Input parameter 1	adc_x: indicates the selected CAN This parameter can be CAN1, CAN2
Output parameter	NA
Return value	transmit_error_counter: Transmit error counter Value range:0x00~0xFF
Required preconditions	NA
Called functions	NA

Example:

```
/* get the transmit error counter (tec) */
uint8_t transmit_error_counter;
```

```
transmit_error_counter = can_transmit_error_counter_get (CAN1);
```

5.4.22 can_interrupt_enable function

The table below describes the function can_interrupt_enable.

Table 94. can_interrupt_enable function

Name	Description
Function name	can_interrupt_enable
Function prototype	void can_interrupt_enable(can_type* can_x, uint32_t can_int, confirm_state new_state);
Function description	Enable the selected CAN interrupt
Input parameter 1	adc_x: indicates the selected CAN This parameter can be CAN1, CAN2
Input parameter 2	*can_int*: Select CAN interruupts
Input parameter 3	new_state: Enable or disable This parameter can be FALSE or TRUE
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

can_int

CAN interrupt select

CAN_TCIEN_INT: Transmit mailbox empty interrupt enable
CAN_RF0MIEN_INT: FIFO 0 receive message interrupt enable
CAN_RF0FIEN_INT: Receive FIFO0 full interrupt enable
CAN_RF0OIEN_INT: Receive FIFO0 overflow interrupt enable
CAN_RF1MIEN_INT: FIFO 1 receive message interrupt enable
CAN_RF1FIEN_INT: Receive FIFO1 full interrupt enable
CAN_RF1OIEN_INT: Receive FIFO1 overflow interrupt enable
CAN_EAIEN_INT: Error active interrupt enable
CAN_EPIEN_INT: Error passive interrupt enable
CAN_BOIEN_INT: Bus-off interrupt enable
CAN_ETRIEN_INT: Error type record interrupt enable
CAN_EOIEN_INT: Error occur interrupt enable
CAN_QDZIEN_INT: Quit Sleep mode interrupt enable
CAN_EDZIEN_INT: Enter Sleep mode interrupt enable

Example

```
/* can interrupt config */
nvic_irq_enable(CAN1_SE_IRQn, 0x00, 0x00);/*CAN1 error/status change interrut*/
nvic_irq_enable(USBFS_L_CAN1_RX0_IRQn, 0x00, 0x00);/*CAN1 FIFO0 receive interrrupt*/

/* FIFO 0 receive message interrupt enable */
can_interrupt_enable(CAN1, CAN_RF0MIEN_INT, TRUE);
/* error type record interrupt enable */
can_interrupt_enable(CAN1, CAN_ETRIEN_INT, TRUE);
```

```
/*This parameter is an error interrupt controller and it is enabled before error-related interrupts */
can_interrupt_enable(CAN1, CAN_EOIEN_INT, TRUE);
```

5.4.23 can_flag_get function

The table below describes the function can_flag_get.

Table 95. can_flag_get function

Name	Description
Function name	can_flag_get
Function prototype	flag_status can_flag_get(can_type* can_x, uint32_t can_flag);
Function description	Get the status of the selected CAN flag
Input parameter 1	adc_x: indicates the selected CAN This parameter can be CAN1, CAN2
Input parameter 2	*can_flag:* indicates the selected flag Refer to the “can_flag” description below for details.
Output parameter	NA
Return value	flag_status: the status of the selected flag Return value can be RESET or SET.
Required preconditions	NA
Called functions	NA

can_flag

This is used to select a flag and get its status, including:

CAN_EAF_FLAG: Error active flag
CAN_EPF_FLAG: Error passive flag
CAN_BOF_FLAG: Bus-off flag
CAN_ETR_FLAG: Error type record (non-zero Error type flag)
CAN_EOIF_FLAG: Error occur interrupt flag
CAN_TM0TCF_FLAG: Mailbox 0 transmission complete flag
CAN_TM1TCF_FLAG: Mailbox 1 transmission complete flag
CAN_TM2TCF_FLAG: Mailbox 2 transmission complete flag
CAN_RF0MN_FLAG: Receive FIFO0 non-empty flag
CAN_RF0FF_FLAG: FIFO0 full flag
CAN_RF0OF_FLAG: FIFO0 overflow flag
CAN_RF1MN_FLAG: FIFO1 non-empty flag
CAN_RF1FF_FLAG: FIFO1 full flag
CAN_RF1OF_FLAG: FIFO1 overflow flag
CAN_QDZIF_FLAG: Quite Sleep mode flag
CAN_EDZC_FLAG: Enter Sleep mode flag
CAN_TMEF_FLAG: Transmit mailbox empty flag (any one of three transmit mailboxes is empty)

Example:

```
/* get receive fifo 0 message num flag */
flag_status bit_status = RESET;
bit_status = can_flag_get (CAN1, CAN_RF0MN_FLAG);
```

5.4.24 can_interrupt_flag_get function

The table below describes the function can_interrupt_flag_get.

Table 96. can_interrupt_flag_get function

Name	Description
Function name	can_interrupt_flag_get
Function prototype	flag_status can_interrupt_flag_get(can_type* can_x, uint32_t can_flag);
Function description	Get CAN interrupt flag
Input parameter 1	can_x: indicates the selected CAN This parameter can be CAN1, CAN2.
Input parameter 2	*can_flag:* indicates the selected flag Refer to the “can_flag” description below for details.
Output parameter	NA
Return value	flag_status: the status of the selected flag Return value can be RESET or SET
Required preconditions	NA
Called functions	NA

can_flag

This is used to select a flag and get its status, including:

CAN_EAF_FLAG: Error active flag
CAN_EPF_FLAG: Error passive flag
CAN_BOF_FLAG: Bus-off flag
CAN_ETR_FLAG: Error type record (non-zero Error type flag)
CAN_EOIF_FLAG: Error occur interrupt flag
CAN_TM0TCF_FLAG: Mailbox 0 transmission complete flag
CAN_TM1TCF_FLAG: Mailbox 1 transmission complete flag
CAN_TM2TCF_FLAG: Mailbox 2 transmission complete flag
CAN_RF0MN_FLAG: FIFO0 non-empty flag
CAN_RF0FF_FLAG: FIFO0 full flag
CAN_RF0OF_FLAG: FIFO0 overflow flag
CAN_RF1MN_FLAG: FIFO1 non-empty flag
CAN_RF1FF_FLAG: FIFO1 full flag
CAN_RF1OF_FLAG: FIFO1 overflow flag
CAN_QDZIF_FLAG: Quite Sleep mode flag
CAN_EDZC_FLAG: Enter Sleep mode flag
CAN_TMEF_FLAG: Transmit mailbox empty flag (any one of three transmit mailboxes is empty)

Example

```
/* check receive fifo 0 message num interrupt flag */
if(can_interrupt_flag_get(CAN1, CAN_RF0MN_FLAG) != RESET)
{
}
```

5.4.25 can_flag_clear function

The table below describes the function can_flag_clear.

Table 97. can_flag_clear function

Name	Description
Function name	can_flag_clear
Function prototype	void can_flag_clear(can_type* can_x, uint32_t can_flag);
Function description	Clear the selected CAN flag
Input parameter 1	adc_x: indicates the selected CAN This parameter can be CAN1, CAN2
Input parameter 2	can_flag: indicates the selected flag Refer to the “can_flag” description below for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

can_flag:

This is used to clear the selected flag, including:

CAN_EAF_FLAG: Error active flag
CAN_EPF_FLAG: Error passive flag
CAN_BOF_FLAG: Bus-off flag
CAN_ETR_FLAG: Error type record (non-zero Error type flag)
CAN_EOIF_FLAG: Error occur interrupt flag
CAN_TM0TCF_FLAG: Mailbox 0 transmission complete flag
CAN_TM1TCF_FLAG: Mailbox 1 transmission complete flag
CAN_TM2TCF_FLAG: Mailbox 2 transmission complete flag
CAN_RF0FF_FLAG: FIFO0 full flag
CAN_RF0OF_FLAG: FIFO0 overflow flag
CAN_RF1FF_FLAG: FIFO1 full flag
CAN_RF1OF_FLAG: FIFO1 overflow flag
CAN_QDZIF_FLAG: Leave Sleep mode flag
CAN_EDZC_FLAG: Enter Sleep mode flag
CAN_TMEF_FLAG: Transmit mailbox empty flag (any one of three transmit mailboxes is empty)

Note: The CAN_RF0MN_FLAG (FIFO0 non-empty flag) and CAN_RF1MN_FLAG (FIFO1 non-empty flag) have no clear operations since both are defined by software.

Example:

```
/* clear receive fifo 0 overflow flag */
can_flag_clear (CAN1, CAN_RF1OF_FLAG);
```

5.5 CRC calculation unit (CRC)

The CRC register structure crc_type is defined in the "at32f403a_407_crc.h":

```
/**
  * @brief type define crc register all
  */
typedef struct
{
      ...

} crc_type;
```

The table below gives a list of the CRC registers.

Table 98. Summary of CRC registers

Register	Description
dt	Data register
cdt	General-purpose data register
ctrl	Control register
idt	Initialization register
poly	Polynomial generation register

The table below gives a list of CRC library functions.

Table 99. Summary of CRC library functions

Function name	Description
crc_data_reset	Data register reset
crc_one_word_calculate	Calulate the CRC value using combination of a new 32-bit data and the previous CRC value
crc_block_calculate	Write a data block in order into CRC check and return the calculated result
crc_data_get	Get the currently calculated CRC result
crc_common_data_set	Configure common registers
crc_common_date_get	Get the value of common registers
crc_init_data_set	Set the CRC initialization register
crc_reverse_input_data_set	Set CRC input data bit reverse type
crc_reverse_output_data_set	Set CRC output data reverse type
crc_poly_value_set	Set polynomial value
crc_poly_value_get	Get polynomial value
crc_poly_size_set	Set polynomial size
crc_poly_size_get	Get polynomial size

5.5.1 crc_data_reset function

The table below describes the function crc_data_reset.

Table 100. crc_data_reset function

Name	Description
Function name	crc_data_reset
Function prototype	void crc_data_reset(void);
Function description	When the data register is reset, the value of the initialization register is added into the data register as an initiai value. The default reset value is 0xFFFFFFFF.
Input parameter 1	NA
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* reset crc data register */
crc_data_reset();
```

5.5.2 crc_one_word_calculate function

The table below describes the function crc_one_word_calculate.

Table 101. crc_one_word_calculate function

Name	Description
Function name	crc_one_word_calculate
Function prototype	uint32_t crc_one_word_calculate(uint32_t data);
Function description	Calculate the CRC value using combination of a new 32-bit data and the previous CRC value
Input parameter 1	Data: input a 32-bit data
Input parameter 2	NA
Output parameter	NA
Return value	uint32_t: return CRC calculation result
Required preconditions	NA
Called functions	NA

Example:

```
/* calculate and return result */
uint32_t data = 0x12345678, result = 0;
result = crc_one_word_calculate (data);
```

5.5.3 crc_block_calculate function

The table below describes the function crc_block_calculate.

Table 102. crc_block_calculate function

Name	Description
Function name	crc_block_calculate
Function prototype	uint32_t crc_block_calculate(uint32_t *pbuffer, uint32_t length);
Function description	Input a data block in sequence to go through CRC calculation and return a result
Input parameter 1	Pbuffer: point to the data block pending for CRC check
Input parameter 2	Length: data block length pending for CRC check, in terms of 32-bit
Output parameter	NA
Return value	uint32_t: return CRC calculation result
Required preconditions	NA
Called functions	NA

Example:

```
/* calculate and return result */
uint32_t pbuffer[2] = {0x12345678, 0x87654321};
uint32_t result = 0;
result = crc_block_calculate (pbuffer, 2);
```

5.5.4 crc_data_get function

The table below describes the function crc_data_get.

Table 103. crc_data_get function

Name	Description
Function name	crc_data_get
Function prototype	uint32_t crc_data_get(void);
Function description	Return the current CRC calculation result
Input parameter 1	NA
Input parameter 2	NA
Output parameter	NA
Return value	uint32_t: return CRC calculation result
Required preconditions	NA
Called functions	NA

Example:

```
/* get result */
uint32_t result = 0;
result = crc_data_get ();
```

5.5.5 crc_common_data_set function

The table below describes the function crc_common_data_set.

Table 104. crc_common_data_set function

Name	Description
Function name	crc_common_data_set
Function prototype	void crc_common_data_set(uint8_t cdt_value);
Function description	Configure common data register
Input parameter 1	cdt_value: 8-bit common data that can be used as temporary storage data
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* set common data */
crc_common_data_set (0x88);
```

5.5.6 crc_common_date_get function

The table below describes the function crc_common_data_get.

Table 105. crc_common_date_get function

Name	Description
Function name	crc_common_data_get
Function prototype	uint8_t crc_common_data_get(void);
Function description	Return the value of the command data register
Input parameter 1	NA
Input parameter 2	NA
Output parameter	NA
Return value	uint8_t: return the value of the previously programmed common data register
Required preconditions	NA
Called functions	NA

Example:

```
/* get common data */
uint8_t cdt_value = 0;
cdt_value = crc_common_data_set ();
```

5.5.7 crc_init_data_set function

The table below describes the function crc_init_data_set.

Table 106. crc_init_data_set function

Name	Description
Function name	crc_init_data_set
Function prototype	void crc_init_data_set(uint32_t value);
Function description	Set the value of the CRC initialization register
Input parameter 1	Value: the value of the CRC initialization register
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

After the value of the CRC initialization register is programmed, the CRC data register is updated with this value whenever the crc_data_reset function is called.

Example:

```
/* set initial data */
uint32_t init_value = 0x11223344;
crc_init_data_set (init_value);
```

5.5.8 crc_reverse_input_data_set function

The table below describes the function crc_reverse_input_data_set.

Table 107. crc_reverse_input_data_set function

Name	Description
Function name	crc_reverse_input_data_set
Function prototype	void crc_reverse_input_data_set(crc_reverse_input_type value);
Function description	Define the CRC input data bit reverse type
Input parameter 1	Value: input data bit reverse type
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

value

Defines the reverse type of input data bit.

CRC_REVERSE_INPUT_NO_AFFECTE: No effect
CRC_REVERSE_INPUT_BY_BYTE: Byte reverse
CRC_REVERSE_INPUT_BY_HALFWORD: Half-word reverse
CRC_REVERSE_INPUT_BY_WORD: Word reverse

Example:

```
/* set input data reversing type */
crc_reverse_input_data_set(CRC_REVERSE_INPUT_BY_WORD);
```

5.5.9 crc_reverse_output_data_set function

The table below describes the function crc_reverse_output_data_set.

Table 108. crc_reverse_output_data_set function

Name	Description
Function name	crc_reverse_output_data_set
Function prototype	void crc_reverse_output_data_set(crc_reverse_output_type value);
Function description	Define the CRC output data reverse type
Input parameter 1	Value: output data bit reverse type
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

value

Defines the reverse type of output data bit.

CRC_REVERSE_OUTPUT_NO_AFFECTE: No effect

CRC_REVERSE_OUTPUT_DATA: Word reverse

Example:

```
/* set output data reversing type */
crc_reverse_output_data_set (CRC_REVERSE_OUTPUT_DATA);
```

5.5.10 crc_poly_value_set function

The table below describes the function crc_poly_value_set.

Table 109. crc_poly_value_set function

Name	Description
Function name	crc_poly_value_set
Function prototype	void crc_poly_value_set(uint32_t value);
Function description	Set CRC polynomial value
Input parameter 1	value: polynomial value
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example

```
/* set poly value */
crc_poly_value_set(0x12345671);
```

5.5.11 crc_poly_value_get function

The table below describes the function crc_poly_value_get.

Table 110. crc_poly_value_get function

Name	Description
Function name	crc_poly_value_get
Function prototype	uint32_t crc_poly_value_get(void);
Function description	Get CRC polynomial value
Input parameter 1	NA
Input parameter 2	NA
Output parameter	NA
Return value	uint32_t: current polynomial value
Required preconditions	NA
Called functions	NA

Example

```
/* get poly value */
uint32_t poly = 0;
poly = crc_poly_value_get();
```

5.5.12 crc_poly_size_set function

The table below describes the function crc_poly_size_set.

Table 111. crc_poly_size_set function

Name	Description
Function name	crc_poly_size_set
Function prototype	void crc_poly_size_set(crc_poly_size_type size);
Function description	Set CRC polynomial size
Input parameter 1	size: valid polynomial width
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

size

This bit is used to define valid polynomial width.

CRC_POLY_SIZE_32B: 32-bit valid width
CRC_POLY_SIZE_16B: 16-bit valid width
CRC_POLY_SIZE_8B: 8-bit valid width
CRC_POLY_SIZE_7B: 7-bit valid width

Example

```
/* set poly size 32-bit */
crc_poly_size_set(CRC_POLY_SIZE_32B);
```

5.5.13 crc_poly_size_get function

The table below describes the function crc_poly_size_get.

Table 112. crc_poly_size_get function

Name	Description
Function name	crc_poly_size_get
Function prototype	crc_poly_size_type crc_poly_size_get(void);
Function description	Get CRC polynomial size
Input parameter 1	NA
Input parameter 2	NA
Output parameter	NA
Return value	crc_poly_size_type: valid polynomial width
Required preconditions	NA
Called functions	NA

crc_poly_size_type

This bit is used to define valid polynomial width.

CRC_POLY_SIZE_32B: 32-bit valid width

CRC_POLY_SIZE_16B: 16-bit valid width

CRC_POLY_SIZE_8B: 8-bit valid width

CRC_POLY_SIZE_7B: 7-bit valid width

Example

```
/* get poly size */
crc_poly_size_type size;
size = crc_poly_size_get();
```

5.6 Clock and reset management (CRM)

The CRM register structure crm_type is defined in the "at32f403a_407_crm.h":

```
/**
  * @brief type define crm register all
  */
typedef struct
{
      ...

} crm_type;
```

The table below gives a list of the CRM registers.

Table 113. Summary of CRM registers

Register	Description
ctrl	Clock control register
cfg	Clock configuration register
clkint	Clock interrupt register
apb2rst	APB2 peripheral reset register
apb1rst	APB1 peripheral reset register
ahben	AHB peripheral clock enable register
apb2en	APB2 peripheral clock register
apb1en	APB1 peripheral clock register
bpdc	Battery powered domain control register
ctrlsts	Control/status register
ahbrst	AHB peripheral reset register
misc1	Extra register 1
misc2	Extra register 2
misc3	Extra register 3

The table below gives a list of CRM library functions.

Table 114. Summary of CRM library functions

Function name	Description
crm_reset	Reset clock reset management register and control status
crm_lext_bypass	Configure low-speed external clock bypass
crm_hext_bypass	Configure high-speed external clock bypass
crm_flag_get	Check if the selected flag is set or not
crm_hext_stable_wait	Wait HEXT to get stable
crm_hick_clock_trimming_set	High speed internal clock trimming
crm_hick_clock_calibration_set	High speed internal clock calibration
crm_periph_clock_enable	Peripheral clock enable
crm_periph_reset	Peripheral set
crm_periph_sleep_mode_clock_enable	Peripheral clock enable in Sleep mode
crm_clock_source_enable	Clock source enable
crm_flag_clear	Clear flag

crm_rtc_clock_select	RTC clock source selection
crm_rtc_clock_enable	RTC clock enable
crm_ahb_div_set	AHB clock division
crm_apb1_div_set	APB1 clock division
crm_apb2_div_set	APB2 clock division
crm_adc_clock_div_set	ADC clock division
crm_usb_clock_div_set	USB clock division
crm_clock_failure_detection_enable	Clock failure detection enable
crm_battery_powered_domain_reset	Battery powered domain reset
crm_pll_config	PLL clock source and frequency multiplication factor
crm_sysclk_switch	System clock source switch
crm_sysclk_switch_status_get	Get the status of system clock source
crm_clocks_freq_get	Get clock frequency
crm_clock_out_set	Clock output clock source
crm_interrupt_enable	Interrupt enable
crm_auto_step_mode_enable	Auto step-by-step mode enable
crm_usb_interrupt_remapping_set	USB interrupt number remap
crm_hick_sclk_frequency_select	Set system clock frequency as 8M or 48M when HICK is used as system clock
crm_usb_clock_source_select	Select PLL or interal high-speed clock (48M) as USB clock source
crm_clkout_to_tmr10_enable	Connect the clkout to tmr10 channel 1
crm_hext_clock_div_set	Configure HEXT frequency division when it is used as PLL input clock
crm_clkout_div_set	Set clkout output clock frequency division
crm_emac_output_pulse_set	Configure emac output pulse width

5.6.1 crm_reset function

The table below describes the function crm_reset.

Table 115. crm_reset function

Name	Description
Function name	crm_reset
Function prototype	void crm_reset(void);
Function description	Reset the clock reset management register and control status
Input parameter 1	NA
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

1. This function does not change the HICKTRIM[5:0] bit of the CRM_CTRL register.
2. Modifying the function does not reset the CRM_BPDC and CRM_CTRLSTS registers.

Example:

```
/* reset crm */
crm_reset();
```

5.6.2 crm_lext_bypass function

The table below describes the function crm_lext_bypass.

Table 116. crm_lext_bypass function

Name	Description
Function name	crm_lext_bypass
Function prototype	void crm_lext_bypass(confirm_state new_state);
Function description	Configure low-speed external clock bypass
Input parameter 1	new_state: Enable bypass (TRUE), disable bypass (FALSE)
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	The LEXT configuration must be done before being enabled.
Called functions	NA

Example:

```
/* enable lext bypass mode */
crm_lext_bypass(TRUE);
```

5.6.3 crm_hext_bypass function

The table below describes the function crm_hext_bypass.

Table 117. crm_hext_bypass function

Name	Description
Function name	crm_hext_bypass
Function prototype	void crm_hext_bypass(confirm_state new_state);
Function description	Configure high-speed external clock bypass
Input parameter 1	new_state: Enable bypass (TRUE), disable bypass (FALSE)
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	The HEXT configuration must be done before being enabled.
Called functions	NA

Example:

```
/* enable hext bypass mode */
crm_hext_bypass(TRUE);
```

5.6.4 crm_flag_get function

The table below describes the function crm_flag_get.

Table 118. crm_flag_get function

Name	Description
Function name	crm_flag_get
Function prototype	flag_status crm_flag_get(uint32_t flag);
Function description	Check if the selected flag has been set.
Input parameter 1	Flag: flag selection
Input parameter 2	NA
Output parameter	NA
Return value	flag_status: check the status of the selected flag. (SET or RESET)
Required preconditions	NA
Called functions	NA

flag

Select a flag to read, including:

CRM_HICK_STABLE_FLAG: HICK clock stable flag
CRM_HEXT_STABLE_FLAG: HEXT clock stable flag
CRM_PLL_STABLE_FLAG: PLL clock stable flag
CRM_LEXT_STABLE_FLAG: LEXT clock stable flag
CRM_LICK_STABLE_FLAG: LICK clock stable flag
CRM_NRST_RESET_FLAG: NRST pin reset flag
CRM_POR_RESET_FLAG: Power-on/low voltage reset flag
CRM_SW_RESET_FLAG: Software reset flag
CRM_WDT_RESET_FLAG: Watchdog reset flag
CRM_WWDT_RESET_FLAG: Window watchdog reset flag
CRM_LOWPOWER_RESET_FLAG: Low-power consumption reset flag

CRM_LICK_READY_INT_FLAG: LICK clock ready interrupt flag
CRM_LEXT_READY_INT_FLAG: LEXT clock ready interrupt flag
CRM_HICK_READY_INT_FLAG: HICK clock ready interrupt flag
CRM_HEXT_READY_INT_FLAG: HEXT clock ready interrupt flag
CRM_PLL_READY_INT_FLAG: PLL clock ready interrupt flag
CRM_CLOCK_FAILURE_INT_FLAG: Clock failure interrupt flag

Example:

```
/* wait till pll is ready */
while(crm_flag_get(CRM_PLL_STABLE_FLAG) != SET)
{
}
```

5.6.5 crm_interrupt_flag_get function

The table below describes the function crm_interrupt_flag_get.

Table 119. crm_interrupt_flag_get function

Name	Description
Function name	crm_interrupt_flag_get
Function prototype	flag_status crm_interrupt_flag_get(uint32_t flag);
Function description	Check if the selected flag has been set.
Input parameter 1	flag: flag selection refer to the “flag” description below for details.
Input parameter 2	NA
Output parameter	NA
Return value	flag_status: the status of the selected flag. (SET or RESET)
Required preconditions	NA
Called functions	NA

flag

Select a flag to read, including:

CRM_LICK_READY_INT_FLAG: LICK clock ready interrupt flag
CRM_LEXT_READY_INT_FLAG: LEXT clock ready interrupt flag
CRM_HICK_READY_INT_FLAG: HICK clock ready interrupt flag
CRM_HEXT_READY_INT_FLAG: HEXT clock ready interrupt flag
CRM_PLL_READY_INT_FLAG: PLL clock ready interrupt flag
CRM_CLOCK_FAILURE_INT_FLAG: Clock failure interrupt flag

Example

```
/* check pll ready interrupt flag */
if(crm_interrupt_flag_get(CRM_PLL_READY_INT_FLAG) != RESET)
{
}
```

5.6.6 crm_hext_stable_wait function

The table below describes the function crm_hext_stable_wait.

Table 120. crm_hext_stable_wait function

Name	Description
Function name	crm_hext_stable_wait
Function prototype	error_status crm_hext_stable_wait(void);
Function description	Wait for HEXT to activate and become stable
Input parameter 1	NA
Input parameter 2	NA
Output parameter	NA
Return value	error_status: Return the status of HEXT. (SUCCESS or ERROR)
Required preconditions	NA
Called functions	NA

Example:

```
/* wait till hext is ready */
while(crm_hext_stable_wait() == ERROR)
{
}
```

5.6.7 crm_hick_clock_trimming_set function

The table below describes the function crm_hick_clock_trimming_set.

Table 121. crm_hick_clock_trimming_set function

Name	Description
Function name	crm_hick_clock_trimming_set
Function prototype	void crm_hick_clock_trimming_set(uint8_t trim_value);
Function description	Trim HICK clock
Input parameter 1	trim_value: trimming value. Default value is 0x20, configurable range is from 0 to 0x3F.
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* set trimming value */
crm_hick_clock_trimming_set(0x1F);
```

5.6.8 crm_hick_clock_calibration_set function

The table below describes the function crm_hick_clock_calibration_set.

Table 122. crm_hick_clock_calibration_set function

Name	Description
Function name	crm_hick_clock_calibration_set
Function prototype	void crm_hick_clock_calibration_set(uint8_t cali_value);
Function description	Set HICK clock calibration value
Input parameter 1	cali_value: calibration compensation value. The factory gate value is the default value, Its configurable range is from 0 to 0xFF.
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* set trimming value */
crm_hick_clock_trimming_set(0x80);
```

5.6.9 crm_periph_clock_enable function

The table below describes the function crm_periph_clock_enable.

Table 123. crm_periph_clock_enable function

Name	Description
Function name	crm_periph_clock_enable
Function prototype	void crm_periph_clock_enable(crm_periph_clock_type value, confirm_state new_state);
Function description	Enable peripheral clock
Input parameter 1	Value: defines peripheral clock type
Input parameter 2	new_state: TRUE or FALSE
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

value

The crm_periph_clock_type is defined in the at32f403a_407_crm.h. The naming rule of this parameter is: CRM_peripheral_PERIPH_CLOCK.

CRM_DMA1_PERIPH_CLOCK: DMA1 peripheral clock enable
CRM_DMA2_PERIPH_CLOCK: DMA2 peripheral clock enable
…
CRM_PWC_PERIPH_CLOCK: PWC peripheral clock enable
CRM_DAC_PERIPH_CLOCK: DAC peripheral clock enable

Example:

```
/* enable gpioa periph clock */
```

```
crm_periph_clock_enable(CRM_GPIOA_PERIPH_CLOCK, TRUE);
```

5.6.10 crm_periph_reset function

The table below describes the function crm_periph_reset.

Table 124. crm_periph_reset function

Name	Description
Function name	crm_periph_reset
Function prototype	void crm_periph_reset(crm_periph_reset_type value, confirm_state new_state);
Function description	Reset peripherals
Input parameter 1	Value: Peripheral reset type Refer to the “value” description below for details.
Input parameter 2	new_state: TRUE or FALSE
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

value

This indicates the selected peripheral. The crm_periph_reset_type is defined in the at32f403a_407_crm.h. The naming rule of this parameter is defined as CRM_peripheral name_PERIPH_RESET.

CRM_DMA1_PERIPH_RESET: DMA1 peripheral reset

CRM_DMA2_PERIPH_RESET: DMA 2 peripheral reset

…

CRM_PWC_PERIPH_RESET: PWC peripheral reset

CRM_DAC_PERIPH_RESET: DAC peripheral reset

Example:

```
/* reset gpioa periph */
crm_periph_reset(CRM_GPIOA_PERIPH_RESET, TRUE);
```

5.6.11 crm_periph_sleep_mode_clock_enable function

The table below describes the function crm_periph_sleep_mode_clock_enable.

Table 125. crm_periph_sleep_mode_clock_enable function

Name	Description
Function name	crm_periph_sleep_mode_clock_enable
Function prototype	void crm_periph_sleep_mode_clock_enable(crm_periph_clock_sleepmd_type value, confirm_state new_state);
Function description	Enable peripheral clock in Sleep mode
Input parameter 1	Value: indicates peripheral clock type in Sleep mode Refer to the “value” description below for details.
Input parameter 2	new_state: TRUE or FALSE
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

value

This indicates the selected peripheral. The crm_periph_clock_sleepmd_type is defined in the at32f403a_407_crm.h. The naming rule of this parameter is defined as:

CRM_peripheral name_PERIPH_CLOCK_SLEEP_MODE

CRM_SRAM_PERIPH_RESET: SRAM clock reset in Sleep mode

CRM_FLASH_PERIPH_RESET: Flash clock reset in Sleep mode

Example:

```
/* disable flash clock when entry sleep mode */
crm_periph_sleep_mode_clock_enable (CRM_FLASH_PERIPH_CLOCK_SLEEP_MODE, FALSE);
```

5.6.12 crm_clock_source_enable function

The table below describes the function crm_clock_source_enable.

Table 126. crm_clock_source_enable function

Name	Description
Function name	crm_clock_source_enable
Function prototype	void crm_clock_source_enable(crm_clock_source_type source, confirm_state new_state);
Function description	Enable clock source
Input parameter 1	Source: Clock type
Input parameter 2	new_state: TRUE or FALSE
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

source

Clock source selection.

CRM_CLOCK_SOURCE_HICK: HICK
CRM_CLOCK_SOURCE_HEXT: HEXT
CRM_CLOCK_SOURCE_PLL: PLL
CRM_CLOCK_SOURCE_LEXT: LEXT
CRM_CLOCK_SOURCE_LICK: LICK

Example:

```
/* enable hext */
crm_clock_source_enable (CRM_CLOCK_SOURCE_HEXT, FALSE);
```

5.6.13 crm_flag_clear function

The table below describes the function crm_flag_clear.

Table 127. crm_flag_clear function

Name	Description
Function name	crm_flag_clear
Function prototype	void crm_flag_clear(uint32_t flag);
Function description	Clear the selected flags
Input parameter 1	flag: Indicates the flag to clear Refer to the “flag” description below for details.
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

flag

Select a falg to clear.

CRM_NRST_RESET_FLAG: NRST pin reset flag
CRM_POR_RESET_FLAG: Power-on/low voltage reset flag

CRM_SW_RESET_FLAG: Software reset flag
CRM_WDT_RESET_FLAG: Watchdog reset flag
CRM_WWDT_RESET_FLAG: Window watchdog reset flag
CRM_LOWPOWER_RESET_FLAG: Low-power reset flag
CRM_ALL_RESET_FLAG: All reset flags
CRM_LICK_READY_INT_FLAG: LICK clock ready interrupt flag
CRM_LEXT_READY_INT_FLAG: LEXT clock ready interrupt flag
CRM_HICK_READY_INT_FLAG: HICK clock ready interrupt flag
CRM_HEXT_READY_INT_FLAG: HEXT clock ready interrupt flag
CRM_PLL_READY_INT_FLAG: PLL clock ready interrupt flag
CRM_CLOCK_FAILURE_INT_FLAG: Clock failure interrupt flag

Example:

```
/* clear clock failure detection flag */
crm_flag_clear(CRM_CLOCK_FAILURE_INT_FLAG);
```

5.6.14 crm_rtc_clock_select function

The table below describes the function crm_ertc_clock_select.

Table 128. crm_ertc_clock_select function

Name	Description
Function name	crm_rtc_clock_select
Function prototype	void crm_rtc_clock_select(crm_rtc_clock_type value);
Function description	Select RTC clock source
Input parameter 1	Value: indicates ertc clock source type
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

value

ERTC clock source selection.

CRM_RTC_CLOCK_NOCLK: No clock source for RTC
CRM_RTC_CLOCK_LEXT: LEXT selected as RTC clock
CRM_RTC_CLOCK_LICK: LICK selected as RTC clock
CRM_RTC_CLOCK_HEXT_DIV: HEXT/128 selected as RTC clock

Example:

```
/* config lext as rtc clock */
crm_rtc_clock_select (CRM_RTC_CLOCK_LEXT);
```

5.6.15 crm_rtc_clock_enable function

The table below describes the function crm_ertc_clock_enable.

Table 129. crm_ertc_clock_enable function

Name	Description
Function name	crm_rtc_clock_enable
Function prototype	void crm_rtc_clock_enable(confirm_state new_state);
Function description	Enable RTC clock
Input parameter 1	new_state: TRUE or FALSE
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable rtc clock */
crm_rtc_clock_enable (TRUE);
```

5.6.16 crm_ahb_div_set function

The table below describes the function crm_ahb_div_set.

Table 130. crm_ahb_div_set function

Name	Description
Function name	crm_ahb_div_set
Function prototype	void crm_ahb_div_set(crm_ahb_div_type value);
Function description	Configure AHB clock division
Input parameter 1	Value: indicates the division factor Refer to the “value” description below for details.
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

value

CRM_AHB_DIV_1: SCLK/1 used as AHB clock
CRM_AHB_DIV_2: SCLK/2 used as AHB clock
CRM_AHB_DIV_4: SCLK/4 used as AHB clock
CRM_AHB_DIV_8: SCLK/8 used as AHB clock
CRM_AHB_DIV_16: SCLK/16 used as AHB clock
CRM_AHB_DIV_64: SCLK/64 used as AHB clock
CRM_AHB_DIV_128: SCLK/128 used as AHB clock
CRM_AHB_DIV_256: SCLK/256 used as AHB clock
CRM_AHB_DIV_512: SCLK/512 used as AHB clock

Example:

```
/* config ahbclk */
```

```
crm_ahb_div_set(CRM_AHB_DIV_1);
```

5.6.17 crm_apb1_div_set function

The table below describes the function crm_apb1_div_set.

Table 131. crm_apb1_div_set function

Name	Description
Function name	crm_apb1_div_set
Function prototype	void crm_apb1_div_set(crm_apb1_div_type value);
Function description	Configure APB1 clock division
Input parameter 1	Value: indicates the division factor
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

value

CRM_APB1_DIV_1: AHB/1 used as APB1 clock
CRM_APB1_DIV_2: AHB/2 used as APB1 clock
CRM_APB1_DIV_4: AHB/4 used as APB1 clock
CRM_APB1_DIV_8: AHB/8 used as APB1 clock
CRM_APB1_DIV_16: AHB/16 used as APB1 clock

Example:

```
/* config apb1clk */
crm_apb1_div_set(CRM_APB1_DIV_2);
```

5.6.18 crm_apb2_div_set function

The table below describes the function crm_apb2_div_set.

Table 132. crm_apb2_div_set function

Name	Description
Function name	crm_apb2_div_set
Function prototype	void crm_apb2_div_set(crm_apb2_div_type value);
Function description	Configure APB2 clock division
Input parameter 1	Value: indicates the division factor
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

value

CRM_APB2_DIV_1: AHB/1 used as APB2 clock
CRM_APB2_DIV_2: AHB/2 used as APB2 clock
CRM_APB2_DIV_4: AHB/4 used as APB2 clock
CRM_APB2_DIV_8: AHB/8 used as APB2 clock
CRM_APB2_DIV_16: AHB/16 used as APB2 clock

Example:

```
/* config apb2clk */
crm_apb2_div_set(CRM_APB2_DIV_2);
```

5.6.19 crm_adc_clock_div_set function

The table below describes the function crm_adc_clock_div_set.

Table 133. crm_adc_clock_div_set function

Name	Description
Function name	crm_adc_clock_div_set
Function prototype	void crm_adc_clock_div_set(crm_adc_div_type div_value);
Function description	Configure ADC clock division
Input parameter 1	div_value: indicate the division factor
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

div_value

CRM_ADC_DIV_2: APB/2 used as ADC clock
CRM_ADC_DIV_4: APB/4 used as ADC clock
CRM_ADC_DIV_6: APB/6 used as ADC clock
CRM_ADC_DIV_8: APB/8 used as ADC clock
CRM_ADC_DIV_12: APB/12 used as ADC clock
CRM_ADC_DIV_16: APB/16 used as ADC clock

Example:

```
/* config adc div 4 */
crm_adc_clock_div_set (CRM_ADC_DIV_4);
```

5.6.20 crm_usb_clock_div_set function

The table below describes the function crm_usb_clock_div_set.

Table 134. crm_usb_clock_div_set function

Name	Description
Function name	crm_usb_clock_div_set
Function prototype	void crm_usb_clock_div_set(crm_usb_div_type div_value);
Function description	Configure USB clock division
Input parameter 1	div_value: division factor Refer to the “div_value” descriptions below for details.
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

div_value

CRM_USB_DIV_1_5: PLL/1.5 as USB clock
CRM_USB_DIV_1: PLL/1 as USB clock
CRM_USB_DIV_2_5: PLL/2.5 as USB clock
CRM_USB_DIV_2: PLL/2 as USB clock
CRM_USB_DIV_3_5: PLL/3.5 as USB clock
CRM_USB_DIV_3: PLL/3 as USB clock
CRM_USB_DIV_4: PLL/4 as USB clock

Example:

```
/* config usb div 2 */
crm_usb_clock_div_set (CRM_USB_DIV_2);
```

5.6.21 crm_clock_failure_detection_enable function

The table below describes the function crm_clock_failure_detection_enable.

Table 135. crm_clock_failure_detection_enable function

Name	Description
Function name	crm_clock_failure_detection_enable
Function prototype	void crm_clock_failure_detection_enable(confirm_state new_state);
Function description	Enable clock failure detection
Input parameter 1	new_state: TRUE or FALSE
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable clock failure detection */
crm_clock_failure_detection_enable(TRUE);
```

5.6.22 crm_battery_powered_domain_reset function

The table below describes the function crm_battery_powered_domain_reset.

Table 136. crm_battery_powered_domain_reset function

Name	Description
Function name	crm_battery_powered_domain_reset
Function prototype	void crm_battery_powered_domain_reset(confirm_state new_state);
Function description	Reset battery powered domain
Input parameter 1	new_state: Reset (TRUE), Not reset (FALSE)
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

When it comes to resetting battery powered domain, it is usually necessary to reset battery powered

domain through TRUE operation and then disable battery powered domain reset through FALSE operation after the completion of reset.

Example:

```
/* reset battery powered domain */
crm_battery_powered_domain_reset (TRUE);
```

5.6.23 crm_pll_config function

The table below describes the function crm_pll_config.

Table 137. crm_pll_config function

Name	Description
Function name	crm_pll_config
Function prototype	void crm_pll_config(crm_pll_clock_source_type clock_source, crm_pll_mult_type mult_value, crm_pll_output_range_type pll_range);
Function description	Configure PLL clock source and frequency multiplication factor
Input parameter 1	clock_source: clock source for PLL frequency multiplication Refer to the “clock_source” descriptions below for details.
Input parameter 2	mult_value: frequency multiplication factor Refer to the “mult_value” descriptions below for details.
Input parameter 3	pll_range: set it depending on whether PLL output range is greather than 72 MHz or not
Output parameter	NA
Return value	NA
Required preconditions	PLL clock source must be enabled and stable before enabling PLL
Called functions	NA

clock_source

CRM_PLL_SOURCE_HICK: HICK as PLL clock
CRM_PLL_SOURCE_HEXT: HEXT as PLL clock
CRM_PLL_SOURCE_HEXT_DIV: Divided HEXT as PLL clock

mult_value

CRM_PLL_MULT_2: PLL input clock x2
CRM_PLL_MULT_3: PLL input clock x3
…
CRM_PLL_MULT_63: PLL input clock x63
CRM_PLL_MULT_64: PLL input clock x64

pll_range

CRM_PLL_OUTPUT_RANGE_LE72MHZ: PLL output frequency is lower than 72MHz
CRM_PLL_OUTPUT_RANGE_GT72MHZ: PLL output frequency is greater than 72MHz

Example:

```
/* config pll clock resource */
crm_pll_config(CRM_PLL_SOURCE_HEXT_DIV, CRM_PLL_MULT_60,
CRM_PLL_OUTPUT_RANGE_GT72MHZ);
```

5.6.24 crm_sysclk_switch function

The table below describes the function crm_sysclk_switch.

Table 138. crm_sysclk_switch function

Name	Description
Function name	crm_sysclk_switch
Function prototype	void crm_sysclk_switch(crm_sclk_type value);
Function description	Switch system clock source
Input parameter 1	Value: indicates the clock source for system clock Refer to the “value” descriptions below for details.
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

value

CRM_SCLK_HICK: HICK as system clock

CRM_SCLK_HEXT: HEXT as system clock

CRM_SCLK_PLL: PLL as system clock

Example:

```
/* select pll as system clock source */
crm_sysclk_switch(CRM_SCLK_PLL);
```

5.6.25 crm_sysclk_switch_status_get function

The table below describes the function crm_sysclk_switch_status_get.

Table 139. crm_sysclk_switch_status_get function

Name	Description
Function name	crm_sysclk_switch_status_get
Function prototype	crm_sclk_type crm_sysclk_switch_status_get(void);
Function description	Get the clock source of System clock
Input parameter 1	NA
Input parameter 2	NA
Output parameter	NA
Return value	crm_sclk_type: return value is the clock source of system clock
Required preconditions	NA
Called functions	NA

Example:

```
/* wait till pll is used as system clock source */
while(crm_sysclk_switch_status_get() != CRM_SCLK_PLL)
{
}
```

5.6.26 crm_clocks_freq_get function

The table below describes the function crm_clocks_freq_get.

Table 140. crm_clocks_freq_get function

Name	Description
Function name	crm_clocks_freq_get
Function prototype	void crm_clocks_freq_get(crm_clocks_freq_type *clocks_struct);
Function description	Get clock frequency
Input parameter 1	clocks_struct: crm_clocks_freq_type pointer, including clock frequency Refer to the “crm_clocks_freq_type” for details.
Input parameter 2	NA
Output parameter	NA
Return value	crm_sclk_type: return the clock source for system clock
Required preconditions	NA
Called functions	NA

crm_clocks_freq_type

The crm_clocks_freq_type is defined in the at32f403a_407_crm.h:

```
typedef struct
{
  uint32_t      sclk_freq;
  uint32_t      ahb_freq;
  uint32_t      apb2_freq;
  uint32_t      apb1_freq;
  uint32_t      adc_freq;
} crm_clocks_freq_type;
```

sclk_freq

Get the system clock frequency, in Hz

ahb_freq

Get the clock frequency of AHB, in Hz

apb2_freq

Get the clock frequency of APB2, in Hz

apb1_freq

Get the clock frequency of APB1, in Hz

adc_freq

Get the clock frequency of ADC, in Hz

Example:

```
/* get frequency */
crm_clocks_freq_type clocks_struct;
crm_clocks_freq_get(&clocks_struct);
```

5.6.27 crm_clock_out_set function

The table below describes the function crm_clock_out_set.

Table 141. crm_clock_out_set function

Name	Description
Function name	crm_clock_out_set
Function prototype	void crm_clock_out_set(crm_clkout_select_type clkout);
Function description	Configure clock output source on the clkout pin
Input parameter 1	Clkout: the clock output source on the clkcout pin
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* config PA8 output pll/4 */
crm_clock_out_set(CRM_CLKOUT_PLL_DIV_4);
```

5.6.28 crm_interrupt_enable function

The table below describes the function crm_interrupt_enable.

Table 142. crm_interrupt_enable function

Name	Description
Function name	crm_interrupt_enable
Function prototype	void crm_interrupt_enable(uint32_t crm_int, confirm_state new_state);
Function description	Enable interrupts
Input parameter 1	crm_int: indicates the selected interrupt Refer to the “crm_int” for details.
Input parameter 2	new_state: Enable (TRUE), disable (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

crm_int

CRM_LICK_STABLE_INT: LICK stable interrupt
CRM_LEXT_STABLE_INT: LEXT stable interrupt
CRM_HICK_STABLE_INT: HICK stable interrupt
CRM_HEXT_STABLE_INT: HEXT stable interrupt
CRM_PLL_STABLE_INT: PLL stable interrupt
CRM_CLOCK_FAILURE_INT: Clock failure interrupt

Example:

```
/* enable pll stable interrupt */
crm_interrupt_enable (CRM_PLL_STABLE_INT);
```

5.6.29 crm_auto_step_mode_enable function

The table below describes the function crm_auto_step_mode_enable.

Table 143. crm_auto_step_mode_enable function

Name	Description
Function name	crm_auto_step_mode_enable
Function prototype	void crm_auto_step_mode_enable(confirm_state new_state);
Function description	Enable auto step-by-step mode
Input parameter 1	new_state: Enable (TRUE), disable (FALSE)
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable auto step mode */
crm_auto_step_mode_enable(TRUE);
```

5.6.30 crm_usb_interrupt_remapping_set function

The table below describes the function crm_usb_interrupt_remapping_set.

Table 144. crm_usb_interrupt_remapping_set function

Name	Description
Function name	crm_usb_interrupt_remapping_set
Function prototype	void crm_usb_interrupt_remapping_set(crm_usb_int_map_type int_remap);
Function description	USB interrupt vector number remap
Input parameter 1	int_remap: interrupt vector number for USB Refer to the “int_remap” descriptions below for details.
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

int_remap

CRM_USB_INT19_INT20: Interrupt vector number 19 and 20 for USB

CRM_USB_INT73_INT74: Interrupt vector number 73 and 74 for USB

Example:

```
/* config usb IRQ number with 73/74 */
crm_usb_interrupt_remapping_set (CRM_USB_INT73_INT74);
```

5.6.31 crm_hick_sclk_frequency_select function

The table below describes the function crm_hick_sclk_frequency_select.

Table 145. crm_hick_sclk_frequency_select function

Name	Description
Function name	crm_hick_sclk_frequency_select
Function prototype	void crm_hick_sclk_frequency_select(crm_hick_sclk_frequency_type value);
Function description	Select 8M or 48M system clock frequency when HICK is used as system clock
Input parameter 1	Value: 8M or 48M HICK
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

value

CRM_HICK_SCLK_8MHZ: 8MHz HICK used as system clock

CRM_HICK_SCLK_48MHZ: 48 MHz HICK used as system clock

Example:

```
/* config sysclk with hick 48mhz */
crm_hick_sclk_frequency_select (CRM_HICK_SCLK_48MHZ);
```

5.6.32 crm_usb_clock_source_select function

The table below describes the function crm_usb_clock_source_select.

Table 146. crm_usb_clock_source_select function

Name	Description
Function name	crm_usb_clock_source_select
Function prototype	void crm_usb_clock_source_select(crm_usb_clock_source_type value);
Function description	Select PLL or internal high-speed clock (48M) as USB clock source
Input parameter 1	Value: PLL or internal high-speed clock (48M) Refer to the “value” descriptions below for details.
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

value

CRM_USB_CLOCK_SOURCE_PLL: PLL is used as USB clock source

CRM_USB_CLOCK_SOURCE_HICK: HICK is used as PLL clock source

Example:

```
/* select hick48 as usb clock */
crm_usb_clock_source_select (CRM_USB_CLOCK_SOURCE_HICK);
```

5.6.33 crm_clkout_to_tmr10_enable function

The table below describes the function crm_clkout_to_tmr10_enable.

Table 147. crm_clkout_to_tmr10_enable function

Name	Description
Function name	crm_clkout_to_tmr10_enable
Function prototype	void crm_clkout_to_tmr10_enable(confirm_state new_state);
Function description	Clkout is internally connected to tmr10 channel 1
Input parameter 1	new_state: Enable (TRUE) or disable (FALSE)
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* config clkout internal connect to tmr10 channel1 */
crm_clkout_to_tmr10_enable (TRUE);
```

5.6.34 crm_hext_clock_div_set function

The table below describes the function crm_hext_clock_div_set.

Table 148. crm_hext_clock_div_set function

Name	Description
Function name	crm_hext_clock_div_set
Function prototype	void crm_hext_clock_div_set(crm_hext_div_type value);
Function description	HEXT clock frequency division when used as PLL input clock
Input parameter 1	Value: hext clock frequency division value when used as pll input clock Refer to the “value” descriptions below for details.
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

value

CRM_HEXT_DIV_2:
HEXT/2 when the CRM_PLL_SOURCE_HEXT_DIV is selected as PLL clock source
CRM_HEXT_DIV_3:
HEXT/3 when the CRM_PLL_SOURCE_HEXT_DIV is selected as PLL clock source
CRM_HEXT_DIV_4:
HEXT/4 when the CRM_PLL_SOURCE_HEXT_DIV is selected as PLL clock source
CRM_HEXT_DIV_5:
HEXT/5 when the CRM_PLL_SOURCE_HEXT_DIV is selected as PLL clock source

Example:

```
/* config hext division */
crm_hext_clock_div_set(CRM_HEXT_DIV_2);
```

5.6.35 crm_clkout_div_set function

The table below describes the function crm_clkout_div_set.

Table 149. crm_clkout_div_set function

Name	Description
Function name	crm_clkout_div_set
Function prototype	void crm_clkout_div_set(crm_clkout_div_type clkout_div);
Function description	Configure clkout output clock division
Input parameter 1	clkout_div: indicates the clkout output frequency division factor Refer to the “clkout_div” for details.
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

value

CRM_CLKOUT_DIV_1: CLKOUT divided by 1
CRM_CLKOUT_DIV_2: CLKOUT divided by 2
CRM_CLKOUT_DIV_4: CLKOUT divided by 4
CRM_CLKOUT_DIV_8: CLKOUT divided by 8
CRM_CLKOUT_DIV_16: CLKOUT divided by 16
CRM_CLKOUT_DIV_64: CLKOUT divided by 64
CRM_CLKOUT_DIV_128: CLKOUT divided by 128
CRM_CLKOUT_DIV_256: CLKOUT divided by 256
CRM_CLKOUT_DIV_512: CLKOUT divided by 512

Example:

```
/* config clkout division */
crm_clkout_div_set(CRM_CLKOUT_DIV_1);
```

5.6.36 crm_emac_output_pulse_set function

The table below describes the function crm_emac_output_pulse_set.

Table 150. crm_emac_output_pulse_set function

Name	Description
Function name	crm_emac_output_pulse_set
Function prototype	void crm_emac_output_pulse_set(crm_emac_output_pulse_type width);
Function description	Configure emac output pulse width
Input parameter 1	Width: emac emac output pulse width Refer to the “width” descriptions below for details.
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

width

CRM_EMAC_PULSE_125MS: EMAC output pulse width is 125 ms

CRM_EMAC_PULSE_1SCLK:EMAC output pulse width is one system clock cycle

Example:

```
/* config emac output pulse 125ms */
crm_emac_output_pulse_set (CRM_EMAC_PULSE_125MS);
```

5.7 Digital-to-analog converter (DAC)

The DAC register structure dac_type is defined in the "at32f403a_407_dac.h":

```
/**
  * @brief type define dac register all
  */
typedef struct
{
  ...
} dac_type;
```

The table below gives a list of the DAC registers.

Table 151. Summary of DAC registers

Register	Description
ctrl	DACcontrol register
swtrg	DAC software trigger register
d1dth12r	DAC1 12-bit right-aligned data holding register
d1dth12l	DAC1 12-bit left-aligned data holding register
d1dth8r	DAC1 8-bit right-aligned data holding register
d2dth12r	DAC2 12-bit right-aligned data holding register
d2dth12l	DAC2 12-bit left-aligned data holding register
d2dth8r	DAC2 8-bit right-aligned data holding register
ddth12r	Dual DAC 12-bit right-aligned data holding register
ddth12l	Dual DAC 12-bit left-aligned data holding register
ddth8r	Dual DAC 8-bit right-aligned data holding register
d1odt	DAC1data output register
d2odt	DAC2 data output register

The table below gives a list of DAC library functions.

Table 152. Summary of DAC library functions

Function name	Description
dac_reset	Reset all DAC registers to their reset values
dac_enable	Enable DAC
dac_output_buffer_enable	Enable DAC output buffer
dac_trigger_enable	Enable DAC trigger
dac_trigger_select	Select DAC trigger source
dac_software_trigger_generate	Trigger DAC by software
dac_dual_software_trigger_generate	Simultaneous rrigger DAC1 and DAC2 by software
dac_wave_generate	Select DAC output waveform
dac_mask_amplitude_select	Select DAC noise /triangle-wave amplitude
dac_dma_enable	DAC DMA enable
dac_data_output_get	Get DAC output value
dac_1_data_set	Set DAC1 output value
dac_2_data_set	Set DAC2 output value
dac_dual_data_set	Set DAC1 and DAC2 output values

5.7.1 dac_reset function

The table below describes the function dac_reset.

Table 153. dac_reset function

Name	Description
Function name	dac_reset
Function prototype	void dac_reset(void);
Function description	Reset all DAC registers to their reset values
Input parameter	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	crm_periph_reset();

Example:

```
dac_reset ();
```

5.7.2 dac_enable function

The table below describes the function dac_enable.

Table 154. dac_enable function

Name	Description
Function name	dac_enable
Function prototype	void dac_enable(dac_select_type dac_select, confirm_state new_state);
Function description	Enable DAC
Input parameter 1	dac_select: Select a DAC This parameter can be DAC1_SELECT or DAC2_SELECT.
Input parameter 2	new_state: Enable or disable This parameter can be FALSE or TURE
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
dac_enable(DAC1_SELECT, TRUE);
```

5.7.3 dac_output_buffer_enable function

The table below describes the function dac_output_buffer_enable.

Table 155. dac_output_buffer_enable function

Name	Description
Function name	dac_output_buffer_enable
Function prototype	void dac_output_buffer_enable(dac_select_type dac_select, confirm_state new_state);
Function description	Enable DAC output buffer
Input parameter 1	dac_select: Select a DAC This parameter can be DAC1_SELECT or DAC2_SELECT.
Input parameter 2	new_state: Enable or disable This parameter can be FALSE or TURE
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
dac_output_buffer_enable (DAC1_SELECT, TRUE);
```

5.7.4 dac_trigger_enable function

The table below describes the function dac_trigger_enable.

Table 156. dac_trigger_enable function

Name	Description
Function name	dac_trigger_enable
Function prototype	void dac_trigger_enable(dac_select_type dac_select, confirm_state new_state);
Function description	Enable DAC trigger
Input parameter 1	dac_select: Select a DAC This parameter can be DAC1_SELECT or DAC2_SELECT.
Input parameter 2	new_state: Enable or disable This parameter can be FALSE or TURE
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
dac_trigger_enable (DAC1_SELECT, TRUE);
```

5.7.5 dac_trigger_select function

The table below describes the function dac_trigger_select.

Table 157. dac_trigger_select function

Name	Description
Function name	dac_trigger_select
Function prototype	void dac_trigger_select(dac_select_type dac_select, dac_trigger_type dac_trigger_source);
Function description	Select DAC trigger source
Input parameter 1	dac_select: Select a DAC This parameter can be DAC1_SELECT or DAC2_SELECT.
Input parameter 2	*dac_trigger_source:* trigger source selected
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

dac_trigger_source

Trigger source selection.

DAC_TMR6_TRGOUT_EVENT: TMR6 TRGOUT event triggers DAC
DAC_TMR8_TRGOUT_EVENT: TMR8 TRGOUT event triggers DAC
DAC_TMR7_TRGOUT_EVENT: TMR7 TRGOUT event triggers DAC
DAC_TMR5_TRGOUT_EVENT: TMR5 TRGOUT event triggers DAC
DAC_TMR2_TRGOUT_EVENT: TMR2 TRGOUT event triggers DAC
DAC_TMR4_TRGOUT_EVENT: TMR4 TRGOUT event triggers DAC
DAC_EXTERNAL_INTERRUPT_LINE_9: EXINT LINE 9 event triggers DAC
DAC_SOFTWARE_TRIGGER: Software triggers DAC

Example:

```
dac_trigger_select(DAC1_SELECT, DAC_TMR2_TRGOUT_EVENT);
dac_trigger_select(DAC2_SELECT, DAC_TMR2_TRGOUT_EVENT);
```

5.7.6 dac_software_trigger_generate function

The table below describes the function dac_software_trigger_generate.

Table 158. dac_software_trigger_generate function

Name	Description
Function name	dac_software_trigger_generate
Function prototype	void dac_software_trigger_generate(dac_select_type dac_select);
Function description	Trigger DAC by software
Input parameter 1	dac_select: Select a DAC This parameter can be DAC1_SELECT or DAC2_SELECT.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
dac_software_trigger_generate (DAC1_SELECT);
```

5.7.7 dac_dual_software_trigger_generate function

The table below describes the function dac_dual_software_trigger_generate.

Table 159. dac_dual_software_trigger_generate function

Name	Description
Function name	dac_dual_software_trigger_generate
Function prototype	void dac_dual_software_trigger_generate(void);
Function description	Trigger DAC1 and DAC2 by software simultaneously
Input parameter	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
dac_dual_software_trigger_generate ();
```

5.7.8 dac_wave_generate function

The table below describes the function dac_wave_generate.

Table 160. dac_wave_generate function

Name	Description
Function name	dac_wave_generate
Function prototype	void dac_wave_generate(dac_select_type dac_select, dac_wave_type dac_wave);
Function description	DAC wave generation enable
Input parameter 1	dac_select: Select a DAC This parameter can be DAC1_SELECT or DAC2_SELECT.
Input parameter 2	*dac_wave:* wave selected
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

dac_wave

Wave generation enable.

DAC_WAVE_GENERATE_NONE: Wave generation disabled (output fixed register value)

DAC_WAVE_GENERATE_NOISE: Noise wave generation

DAC_WAVE_GENERATE_TRIANGLE: Triangle wave generation

Example:

```
dac_wave_generate(DAC1_SELECT, DAC_WAVE_GENERATE_NONE);
dac_wave_generate(DAC2_SELECT, DAC_WAVE_GENERATE_NOISE);
```

5.7.9 dac_mask_amplitude_select function

The table below describes the function dac_mask_amplitude_select.

Table 161. dac_mask_amplitude_select function

Name	Description
Function name	dac_mask_amplitude_select
Function prototype	void dac_mask_amplitude_select(dac_select_type dac_select, dac_mask_amplitude_type dac_mask_amplitude);
Function description	DAC noise bit width/triangle amplitude selection
Input parameter 1	dac_select: Select a DAC This parameter can be DAC1_SELECT or DAC2_SELECT.
Input parameter 2	*dac_mask_amplitude*: noise bit width/triangle amplitude selection
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

dac_mask_amplitude

Noise bit width/triangle amplitude selection.

DAC_LSFR_BIT0_AMPLITUDE_1:

LSFR[0] in noise mode/triangle amplitude equal to 1

DAC_LSFR_BIT10_AMPLITUDE_3:
LSFR[1:0] in noise mode/triangle amplitude equal to 3
DAC_LSFR_BIT20_AMPLITUDE_7:
LSFR[2:0] in noise mode/triangle amplitude equal to 7
DAC_LSFR_BIT30_AMPLITUDE_15:
LSFR[3:0] in noise mode/triangle amplitude equal to 15
DAC_LSFR_BIT40_AMPLITUDE_31:
LSFR[4:0] in noise mode/triangle amplitude equal to 31
DAC_LSFR_BIT50_AMPLITUDE_63:
LSFR[5:0] in noise mode/triangle amplitude equal to 63
DAC_LSFR_BIT60_AMPLITUDE_127:
LSFR[6:0] in noise mode/triangle amplitude equal to 127
DAC_LSFR_BIT70_AMPLITUDE_255:
LSFR[7:0] in noise mode/triangle amplitude equal to 255
DAC_LSFR_BIT80_AMPLITUDE_511:
LSFR[8:0] in noise mode/triangle amplitude equal to 511
DAC_LSFR_BIT90_AMPLITUDE_1023:
LSFR[9:0] in noise mode/triangle amplitude equal to 1023
DAC_LSFR_BITA0_AMPLITUDE_2047:
LSFR[10:0] in noise mode/triangle amplitude equal to 2047
DAC_LSFR_BITB0_AMPLITUDE_4095:
LSFR[11:0] in noise mode/triangle amplitude equal to 4095

Example:

```
dac_mask_amplitude_select (DAC1_SELECT, DAC_LSFR_BIT60_AMPLITUDE_127);
dac_mask_amplitude_select (DAC2_SELECT, DAC_LSFR_BIT80_AMPLITUDE_511);
```

5.7.10 dac_dma_enable function

The table below describes the function dac_dma_enable.

Table 162. dac_dma_enable function

Name	Description
Function name	dac_dma_enable
Function prototype	void dac_dma_enable(dac_select_type dac_select, confirm_state new_state);
Function description	DAC DMA enable
Input parameter 1	dac_select: Select a DAC This parameter can be DAC1_SELECT or DAC2_SELECT.
Input parameter 2	new_state: Enable or disable This parameter can be FALSE or TURE
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
dac_dma_enable (DAC1_SELECT, TRUE);
```

5.7.11 dac_data_output_get function

The table below describes the function dac_data_output_get.

Table 163. dac_data_output_get function

Name	Description
Function name	dac_data_output_get
Function prototype	uint16_t dac_data_output_get(dac_select_type dac_select);
Function description	Get DAC output value
Input parameter 1	dac_select: Select a DAC This parameter can be DAC1_SELECT or DAC2_SELECT.
Output parameter	NA
Return value	dacx_data: dac1/dac2 output value
Required preconditions	NA
Called functions	NA

Example:

```
uint16_t dac1_data;
dac1_data = dac_data_output_get (DAC1_SELECT);
```

5.7.12 dac_1_data_set function

The table below describes the function dac_1_data_set.

Table 164. dac_1_data_set function

Name	Description
Function name	dac_1_data_set
Function prototype	void dac_1_data_set(dac1_aligned_data_type dac1_aligned, uint16_t dac1_data);
Function description	Set DAC1 output data format and range
Input parameter 1	*dac1_aligned*: data format selection
Input parameter 2	*dac1_data*: DAC output value
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

dac1_aligned

Data format selector.

DAC1_12BIT_RIGHT: 12-bit right-aligned

DAC1_12BIT_LEFT: 12-bit left-aligned

DAC1_8BIT_RIGHT: 8-bit right-aligned

dac1_data

DAC output range settings. Value range varies from one format to another.

DAC1_12BIT_RIGHT: 0x000~0xFFF

DAC1_12BIT_LEFT: 0x0000~0xFFF0

DAC1_8BIT_RIGHT: 0x00~0xFF

Example:

```
dac_1_data_set (DAC1_12BIT_RIGHT, 0x666);
```

5.7.13 dac_2_data_set function

The table below describes the function dac_2_data_set.

Table 165. dac_2_data_set function

Name	Description
Function name	dac_2_data_set
Function prototype	void dac_2_data_set(dac2_aligned_data_type dac2_aligned, uint16_t dac2_data);
Function description	Set DAC2 output data format and range
Input parameter 1	*dac2_aligned* : data format selection
Input parameter 2	*dac2_data*: DAC output value
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

dac2_aligned

Data format selector.

DAC2_12BIT_RIGHT: 12-bit right-aligned

DAC2_12BIT_LEFT: 12-bit left-aligned

DAC2_8BIT_RIGHT: 8-bit right-aligned

dac2_data

DAC output range settings. Value range varies from one format to another.

DAC2_12BIT_RIGHT: 0x000~0xFFF

DAC2_12BIT_LEFT: 0x0000~0xFFF0

DAC2_8BIT_RIGHT: 0x00~0xFF

Example:

```
dac_2_data_set (DAC2_12BIT_RIGHT, 0x666);
```

5.7.14 dac_dual_data_set function

The table below describes the function dac_dual_data_set

Table 166. dac_dual_data_set function

Name	Description
Function name	dac_dual_data_set
Function prototype	void dac_dual_data_set(dac_dual_data_type dac_dual, uint16_t data1, uint16_t data2);
Function description	DAC1/DAC2 output data format and range
Input parameter 1	*dac_dual:* data format selection
Input parameter 2	*data1*: DAC1 output value
Input parameter 3	*data2*: DAC2 output value
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

dac_dual

Data format selector.

DAC_DUAL_12BIT_RIGHT: 12-bit right-aligned

DAC_DUAL_12BIT_LEFT: 12-bit left-aligned

DAC_DUAL_8BIT_RIGHT: 8-bit right-aligned

data1/data2

DAC output range settings. Value range varies from one format to another.

DAC2_12BIT_RIGHT: 0x000~0xFFF

DAC2_12BIT_LEFT: 0x0000~0xFFF0

DAC2_8BIT_RIGHT: 0x00~0xFF

Example:

```
dac_dual_data_set (DAC2_12BIT_RIGHT, 0x666, 0x777);
```

5.8 DMA controller

The DMA register structure dma_type is defined in the "at32f403a_407_dma.h":

```
/**
  * @brief type define dma register
  */
typedef struct
{
      …

} dma_type;
```

DMA channel register structure dma_channel_type is defined in the "at32f403a_407_dma.h":

```
/**
  * @brief type define dma channel register all
  */
typedef struct
{
      …

} dma_channel_type;
```

The table below gives a list of the DMA registers.

Table 167. Summary of DMA registers

Register	Description
dma_sts	DMA status register
dma_clr	DMA status clear register
dma_c1ctrl	DMA channel 1 configuration register
dma_c1dtcnt	DMA channel 1 number of data register
dma_c1paddr	DMA channel 1 peripheral address register
dma_c1maddr	DMA channel 1 memory address register
dma_c2ctrl	DMA channel 2 configuration register
dma_c2dtcnt	DMA channel 2 number of data register
dma_c2paddr	DMA channel 2 peripheral address register
dma_c2maddr	DMA channel 2 memory address register
dma_c3ctrl	DMA channel 3 configuration register
dma_c3dtcnt	DMA channel 3 number of data register
dma_c3paddr	DMA channel 3 peripheral address register
dma_c3maddr	DMA channel 3 memory address register
dma_c4ctrl	DMA channel 4 configuration register
dma_c4dtcnt	DMA channel 4 number of data register
dma_c4paddr	DMA channel 4 peripheral address register
dma_c4maddr	DMA channel 4 memory address register
dma_c5ctrl	DMA channel 5 configuration register
dma_c5dtcnt	DMA channel 5 number of data register

Register	Description
dma_c5paddr	DMA channel 5 peripheral address register
dma_c5maddr	DMA channel 5 memory address register
dma_c6ctrl	DMA channel 6 configuration register
dma_c6dtcnt	DMA channel 6 number of data register
dma_c6paddr	DMA channel 6 peripheral address register
dma_c6maddr	DMA channel 6 memory address register
dma_c7ctrl	DMA channel 7 configuration register
dma_c7dtcnt	DMA channel 7 number of data register
dma_c7paddr	DMA channel 7 peripheral address register
dma_c7maddr	DMA channel 7 memory address register
dma_src_sel0	Channel source register 0
dma_src_sel1	Channel source register 1

The table below gives a list of DMA library functions.

Table 168. Summary of DMA library functions

Function name	Description
dma_default_para_init	Initialize the parameters of the dma_init_struct
dma_init	Initialize the selectec channel
dma_reset	Reset the selected DMA channel
dma_data_number_set	Set the number of data transfer of a given channel
dma_data_number_get	Get the number of data transfer of a given channel
dma_interrupt_enable	Enable DMA channel interrupt
dma_channel_enable	Enable DMA channel
dma_flexible_config	Configure flexible DMA request mapping
dma_flag_get	Get the flag of DMA channels
dma_flag_clear	Clear the flag of DMA channels

5.8.1 dma_default_para_init function

The table below describes the function dma_default_para_init

Table 169. dma_default_para_init function

Name	Description
Function name	dma_default_para_init
Function prototype	void dma_default_para_init(dma_init_type* dma_init_struct);
Function description	Initialize the parameters of the dma_init_struct
Input parameter 1	dma_init_struct: dma_init_type pointer
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

The table below describes the default values of the dma_init_struct members.

Table 170.dma_init_struct default values

Member	Default values
peripheral_base_addr	0x0
memory_base_addr	0x0
direction	DMA_DIR_PERIPHERAL_TO_MEMORY
buffer_size	0x0
peripheral_inc_enable	FALSE
memory_inc_enable	FALSE
peripheral_data_width	DMA_PERIPHERAL_DATA_WIDTH_BYTE
memory_data_width	DMA_MEMORY_DATA_WIDTH_BYTE
loop_mode_enable	FALSE
priority	DMA_PRIORITY_LOW

Example:

```
/* dma init config with its default value */
dma_init_type dma_init_struct = {0};
dma_default_para_init(&dma_init_struct);
```

5.8.2 dma_init function

The table below describes the function dma_init

Table 171. dma_init function

Name	Description
Function name	dma_init
Function prototype	void dma_init(dma_channel_type* dmax_channely, dma_init_type* dma_init_struct)
Function description	Initialize the selected DMA channel
Input parameter 1	dmax_channely: DMAx_CHANNELy defines a DMA channel number, x=1 or 2, y=1...7
Input parameter 2	dma_init_struct: dma_init_type pointer
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

dma_init_type structure

The dma_init_type is defined in the at32f403a_407_dma.h:

```
typedef struct
{
    uint32_t                        peripheral_base_addr;
    uint32_t                        memory_base_addr;
    dma_dir_type                    direction;
    uint16_t                        buffer_size;
    confirm_state                   peripheral_inc_enable;
    confirm_state                   memory_inc_enable;
    dma_peripheral_data_size_type   peripheral_data_width;
    dma_memory_data_size_type       memory_data_width;
    confirm_state                   loop_mode_enable;
    dma_priority_level_type         priority;
} dma_init_type;
```

peripheral_base_addr

Set the peripheral address of a DMA channel

memory_base_addr

Set the memory address of a DMA channel

direction

Set the transfer direction of a DMA channel

DMA_DIR_PERIPHERAL_TO_MEMORY: Peripheral to memory
DMA_DIR_MEMORY_TO_PERIPHERAL: Memory to peripheral
DMA_DIR_MEMORY_TO_MEMORY: Memory to memory

buffer_size

Set the number of data transfer of a DMA channel

peripheral_inc_enable

Enable/disable DMA channel peripheral address auto increment

FALSE: Peripheral address is not incremented
TRUE: Peripheral address is incremented

memory_inc_enable

Enable/disable DMA channel memory address auto increment

FALSE: Memory address is not incremented

TRUE: Memoryaddress is incremented

peripheral_data_width

Set DMA peripheral data width

DMA_PERIPHERAL_DATA_WIDTH_BYTE:	Byte
DMA_PERIPHERAL_DATA_WIDTH_HALFWORD:	Half-word
DMA_PERIPHERAL_DATA_WIDTH_WORD:	Word

memory_data_width

Set DMA memory data width

DMA_MEMORY_DATA_WIDTH_BYTE:	Byte
DMA_MEMORY_DATA_WIDTH_HALFWORD:	Half-word
DMA_MEMORY_DATA_WIDTH_WORD:	Word

loop_mode_enable

Set DMA loop mode

FALSE: DMA single mode

TRUE: DMA loop mode

priority

Set DMA channel priority

DMA_PRIORITY_LOW:	Low
DMA_PRIORITY_MEDIUM:	Medium
DMA_PRIORITY_HIGH:	High
DMA_PRIORITY_VERY_HIGH:	Very high

Example:

```
dma_init_type dma_init_struct = {0};
/* dma2 channel1 configuration */
dma_init_struct.buffer_size = BUFFER_SIZE;
dma_init_struct.direction = DMA_DIR_MEMORY_TO_PERIPHERAL;
dma_init_struct.memory_base_addr = (uint32_t)src_buffer;
dma_init_struct.memory_data_width = DMA_MEMORY_DATA_WIDTH_HALFWORD;
dma_init_struct.memory_inc_enable = TRUE;
dma_init_struct.peripheral_base_addr = (uint32_t)0x4001100C;
dma_init_struct.peripheral_data_width = DMA_PERIPHERAL_DATA_WIDTH_HALFWORD;
dma_init_struct.peripheral_inc_enable = FALSE;
dma_init_struct.priority = DMA_PRIORITY_MEDIUM;
dma_init_struct.loop_mode_enable = FALSE;
dma_init(DMA2_CHANNEL1, &dma_init_struct);
```

5.8.3 dma_reset function

The table below describes the function dma_reset

Table 172. dma_reset function

Name	Description
Function name	dma_reset
Function prototype	void dma_reset(dma_channel_type* dmax_channely);
Function description	Reset the selected DMA channel
Input parameter 1	dmax_channely: DMAx_CHANNELy defines a DMA channel number, x=1 or 2, y=1...7
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* reset dma2 channel1 */
dma_reset(DMA2_CHANNEL1);
```

5.8.4 dma_data_number_set function

The table below describes the function dma_data_number_set

Table 173. dma_data_number_set function

Name	Description
Function name	dma_ data_number_set
Function prototype	void dma_data_number_set(dma_channel_type* dmax_channely, uint16_t data_number);
Function description	Set the number of data transfer of the selected DMA channel
Input parameter 1	dmax_channely: DMAx_CHANNELy defines a DMA channel number, x=1 or 2, y=1...7
Input parameter 2	data_number: indicates the number of data transfer, up to 65535
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* set dma2 channel1 data count is 0x100*/
dma_data_number_set(DMA2_CHANNEL1, 0x100);
```

5.8.5 dma_data_number_get function

The table below describes the function dma_data_number_get

Table 174. dma_data_number_get function

Name	Description
Function name	dma_ data_number_get
Function prototype	uint16_t dma_data_number_get(dma_channel_type* dmax_channely);
Function description	Get the number of data transfer of the selected DMA channel
Input parameter 1	dmax_channely: DMAx_CHANNELy defines a DMA channel number, x=1 or 2, y=1...7
Output parameter	NA
Return value	Get the number of data transfer of a DMA channel
Required preconditions	NA
Called functions	NA

Example:

```
/* get dma2 channel1 data count*/
uint16_t data_counter;
data_counter = dma_data_number_set(DMA2_CHANNEL1);
```

5.8.6 dma_interrupt_enable function

The table below describes the function dma_interrupt_enable

Table 175. dma_interrupt_enable function

Name	Description
Function name	dma_ interrupt_enable
Function prototype	void dma_interrupt_enable(dma_channel_type* dmax_channely, uint32_t dma_int, confirm_state new_state);
Function description	Enable DMA channels interrupt
Input parameter 1	dmax_channely: DMAx_CHANNELy defines a DMA channel number, x=1 or 2, y=1...7
Input parameter 2	dma_int: interrupt source selection
Input parameter 3	new_state: interrupt enable/disable
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

dma_int

Select DMA interrupt source

DMA_FDT_INT: Transfer complete interrupt
DMA_HDT_INT: Half transfer complete interrupt
DMA_DTERR_INT: Transfer error interrupt

new_state

Enable or disable DMA channel interrupt

FALSE: Disabled
TRUE: Enabled

Example:

```
/* enable dma2 channel1 transfer full data intterrupt */
dma_interrupt_enable(DMA2_CHANNEL1, DMA_FDT_INT, TRUE);
```

5.8.7 dma_channel_enable function

The table below describes the function dma_channel_enable

Table 176. dma_channel_enable function

Name	Description
Function name	dma_channel_enable
Function prototype	void dma_channel_enable(dma_channel_type* dmax_channely, confirm_state new_state);
Function description	Enable the selected DMA channel
Input parameter 1	dmax_channely: DMAx_CHANNELy defines a DMA channel number, x=1 or 2, y=1...7
Input parameter 2	new_state: Enable or disable the selected DMA channel
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

new_state

Enable or disable DMA channels

FALSE: Disable

TRUE: Enable

Example:

```
/* enable dma channel */
dma_channel_enable(DMA2_CHANNEL1, TRUE);
```

5.8.8 dma_ flexible_config function

The table below describes the function dma_ flexible_config.

Table 177. dma_interrupt_enable function

Name	Description
Function name	dma_ flexible_config
Function prototype	void dma_flexible_config(dma_type* dma_x, uint8_t flex_channelx, dma_flexible_request_type flexible_request);
Function description	Configure flexible DMA request mapping
Input parameter 1	dma_x: DMAx, x=1 or 2
Input parameter 2	flex_channelx: FLEX_CHANNELx indicates a channel number, x=1…7
Input parameter 3	flexible_request: flexible request mapping source ID
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

flexible_request

The table below gives a list of flexible mapping request source ID.

Table 178. Flexible mapping request source ID

Request source ID	Description	Request source ID	Description
0x01	DMA_FLEXIBLE_ADC1	0x03	DMA_FLEXIBLE_ADC3
0x05	DMA_FLEXIBLE_DAC1	0x06	DMA_FLEXIBLE_DAC2
0x09	DMA_FLEXIBLE_SPI1_RX	0x0A	DMA_FLEXIBLE_SPI1_TX
0x0B	DMA_FLEXIBLE_SPI2_RX	0x0C	DMA_FLEXIBLE_SPI2_TX
0x0D	DMA_FLEXIBLE_SPI3_RX	0x0E	DMA_FLEXIBLE_SPI3_TX
0x0F	DMA_FLEXIBLE_SPI4_RX	0x10	DMA_FLEXIBLE_SPI4_TX
0x11	DMA_FLEXIBLE_I2S2EXT_RX	0x12	DMA_FLEXIBLE_I2S2EXT_TX
0x13	DMA_FLEXIBLE_I2S3EXT_RX	0x14	DMA_FLEXIBLE_I2S3EXT_TX
0x19	DMA_FLEXIBLE_UART1_RX	0x1A	DMA_FLEXIBLE_UART1_TX
0x1B	DMA_FLEXIBLE_UART2_RX	0x1C	DMA_FLEXIBLE_UART2_TX
0x1D	DMA_FLEXIBLE_UART3_RX	0x1E	DMA_FLEXIBLE_UART3_TX
0x1F	DMA_FLEXIBLE_UART4_RX	0x20	DMA_FLEXIBLE_UART4_TX
0x21	DMA_FLEXIBLE_UART5_RX	0x22	DMA_FLEXIBLE_UART5_TX
0x23	DMA_FLEXIBLE_UART6_RX	0x24	DMA_FLEXIBLE_UART6_TX
0x25	DMA_FLEXIBLE_UART7_RX	0x26	DMA_FLEXIBLE_UART7_TX
0x27	DMA_FLEXIBLE_UART8_RX	0x28	DMA_FLEXIBLE_UART8_TX
0x29	DMA_FLEXIBLE_I2C1_RX	0x2A	DMA_FLEXIBLE_I2C1_TX
0x2B	DMA_FLEXIBLE_I2C2_RX	0x2C	DMA_FLEXIBLE_I2C2_TX
0x2D	DMA_FLEXIBLE_I2C3_RX	0x2E	DMA_FLEXIBLE_I2C3_TX
0x31	DMA_FLEXIBLE_SDIO1	0x32	DMA_FLEXIBLE_SDIO2
0x35	DMA_FLEXIBLE_TMR1_TRIG	0x36	DMA_FLEXIBLE_TMR1_HALL
0x37	DMA_FLEXIBLE_TMR1_OVERFLOW	0x38	DMA_FLEXIBLE_TMR1_CH1
0x39	DMA_FLEXIBLE_TMR1_CH2	0x3A	DMA_FLEXIBLE_TMR1_CH3
0x3B	DMA_FLEXIBLE_TMR1_CH4	0x3D	DMA_FLEXIBLE_TMR2_TRIG
0x3F	DMA_FLEXIBLE_TMR2_OVERFLOW	0x40	DMA_FLEXIBLE_TMR2_CH1
0x41	DMA_FLEXIBLE_TMR2_CH2	0x42	DMA_FLEXIBLE_TMR2_CH3
0x43	DMA_FLEXIBLE_TMR2_CH4	0x45	DMA_FLEXIBLE_TMR3_TRIG
0x47	DMA_FLEXIBLE_TMR3_OVERFLOW	0x48	DMA_FLEXIBLE_TMR3_CH1
0x49	DMA_FLEXIBLE_TMR3_CH2	0x4A	DMA_FLEXIBLE_TMR3_CH3
0x4B	DMA_FLEXIBLE_TMR3_CH4	0x4D	DMA_FLEXIBLE_TMR4_TRIG
0x4F	DMA_FLEXIBLE_TMR4_OVERFLOW	0x50	DMA_FLEXIBLE_TMR4_CH1
0x51	DMA_FLEXIBLE_TMR4_CH2	0x52	DMA_FLEXIBLE_TMR4_CH3
0x53	DMA_FLEXIBLE_TMR4_CH4	0x55	DMA_FLEXIBLE_TMR5_TRIG
0x57	DMA_FLEXIBLE_TMR5_OVERFLOW	0x58	DMA_FLEXIBLE_TMR5_CH1
0x59	DMA_FLEXIBLE_TMR5_CH2	0x5A	DMA_FLEXIBLE_TMR5_CH3
0x5B	DMA_FLEXIBLE_TMR5_CH4	0x5F	DMA_FLEXIBLE_TMR6_OVERFLOW
0x67	DMA_FLEXIBLE_TMR7_OVERFLOW	0x6D	DMA_FLEXIBLE_TMR8_TRIG
0x6E	DMA_FLEXIBLE_TMR8_HALL	0x6F	DMA_FLEXIBLE_TMR8_OVERFLOW
0x70	DMA_FLEXIBLE_TMR8_CH1	0x71	DMA_FLEXIBLE_TMR8_CH2
0x72	DMA_FLEXIBLE_TMR8_CH3	0x73	DMA_FLEXIBLE_TMR8_CH4

Example:

```
/* tmr2 flexible function enable */
dma_flexible_config(DMA2, FLEX_CHANNEL1, DMA_FLEXIBLE_TMR2_OVERFLOW);
```

5.8.9 dma_flag_get function

The table below describes the function dma_flag_get.

Table 179. dma_flag_get function

Name	Description
Function name	dma_flag_get
Function prototype	flag_status dma_flag_get(uint32_t dmax_flag);
Function description	Get the flag of the selected DMA channel
Input parameter 1	*dmax_flag:* select the desired flag
Output parameter	NA
Return value	flag_status: indicates whether the desired flag is set or not
Required preconditions	NA
Called functions	NA

dmax_flag

dmax_flag is used for flag section, including:

DMA1_GL1_FLAG: DMA1 channel 1 global flag
DMA1_FDT1_FLAG: DMA1 channel 1 transfer complete flag
DMA1_HDT1_FLAG: DMA1 channel 1 half transfer complete flag
DMA1_DTERR1_FLAG: DMA1 channel 1 transfer error flag
DMA1_GL2_FLAG: DMA1 channel 2 global flag
DMA1_FDT2_FLAG: DMA1 channel 2 transfer complete flag
DMA1_HDT2_FLAG: DMA1 channel 2 half transfer complete flag
DMA1_DTERR2_FLAG: DMA1 channel 2 transfer error flag
DMA1_GL3_FLAG: DMA1 channel 3 global flag
DMA1_FDT3_FLAG: DMA1 channel 3 transfer complete flag
DMA1_HDT3_FLAG: DMA1 channel 3 half transfer complete flag
DMA1_DTERR3_FLAG: DMA1 channel 3 transfer error flag
DMA1_GL4_FLAG: DMA1 channel 4 global flag
DMA1_FDT4_FLAG: DMA1 channel 4 transfer complete flag
DMA1_HDT4_FLAG: DMA1 channel 4 half transfer complete flag
DMA1_DTERR4_FLAG: DMA1 channel 4 transfer error flag
DMA1_GL5_FLAG: DMA1 channel 5 global flag
DMA1_FDT5_FLAG: DMA1 channel 5 transfer complete flag
DMA1_HDT5_FLAG: DMA1 channel 5 half transfer complete flag
DMA1_DTERR5_FLAG: DMA1 channel 5 transfer error flag
DMA1_GL6_FLAG: DMA1 channel 6 global flag
DMA1_FDT6_FLAG: DMA1 channel 6 transfer complete flag
DMA1_HDT6_FLAG: DMA1 channel 6 half transfer complete flag
DMA1_DTERR6_FLAG: DMA1 channel 6 transfer error flag
DMA1_GL7_FLAG: DMA1 channel 7 global flag
DMA1_FDT7_FLAG: DMA1 channel 7 transfer complete flag
DMA1_HDT7_FLAG: DMA1 channel 7 half transfer complete flag

DMA1_DTERR7_FLAG: DMA1 channel 7 transfer error flag
DMA2_GL1_FLAG: DMA2 channel 1 global flag
DMA2_FDT1_FLAG: DMA2 channel 1 transfer complete flag
DMA2_HDT1_FLAG: DMA2 channel 1 half transfer complete flag
DMA2_DTERR1_FLAG: DMA2 channel 1 transfer error flag
DMA2_GL2_FLAG: DMA2 channel 2 global flag
DMA2_FDT2_FLAG: DMA2 channel 2 transfer complete flag
DMA2_HDT2_FLAG: DMA2 channel 2 half transfer complete flag
DMA2_DTERR2_FLAG: DMA2 channel 2 transfer error flag
DMA2_GL3_FLAG: DMA2 channel 3 global flag
DMA2_FDT3_FLAG: DMA2 channel 3 transfer complete flag
DMA2_HDT3_FLAG: DMA2 channel 3 half transfer complete flag
DMA2_DTERR3_FLAG: DMA2 channel 3 transfer error flag
DMA2_GL4_FLAG: DMA2 channel 4 global flag
DMA2_FDT4_FLAG: DMA2 channel 4 transfer complete flag
DMA2_HDT4_FLAG: DMA2 channel 4 half transfer complete flag
DMA2_DTERR4_FLAG: DMA2 channel 4 transfer error flag
DMA2_GL5_FLAG: DMA2 channel 5 global flag
DMA2_FDT5_FLAG: DMA2 channel 5 transfer complete flag
DMA2_HDT5_FLAG: DMA2 channel 5 half transfer complete flag
DMA2_DTERR5_FLAG: DMA2 channel 5 transfer error flag
DMA2_GL6_FLAG: DMA2 channel 6 global flag
DMA2_FDT6_FLAG: DMA2 channel 6 transfer complete flag
DMA2_HDT6_FLAG: DMA2 channel 6 half transfer complete flag
DMA2_DTERR6_FLAG: DMA2 channel 6 transfer error flag
DMA2_GL7_FLAG: DMA2 channel 7 global flag
DMA2_FDT7_FLAG: DMA2 channel 7 transfer complete flag
DMA2_HDT7_FLAG: DMA2 channel 7 half transfer complete flag
DMA2_DTERR7_FLAG: DMA2 channel 7 transfer error flag

flag_status

RESET: Flag is reset
SET: Flag is set

Example:

```
if(dma_flag_get(DMA1_FDT1_FLAG) != RESET)
{
    /* turn led2/led3/led4 on */
    at32_led_on(LED2);
    at32_led_on(LED3);
    at32_led_on(LED4);
}
```

5.8.10 dma_flag_clear function

The table below describes the function dma_flag_clear

Table 180. dma_flag_clear function

Name	Description
Function name	dma_flag_clear
Function prototype	void dma_flag_clear(uint32_t dmax_flag);
Function description	Clear the selected flag
Input parameter 1	*dmax_flag*: a flag that needs to be cleared
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

dmax_flag

dmax_flag is used to select the desired flag, including:

DMA1_GL1_FLAG: DMA1 channel 1 global flag
DMA1_FDT1_FLAG: DMA1 channel 1 transfer complete flag
DMA1_HDT1_FLAG: DMA1 channel 1 half transfer complete flag
DMA1_DTERR1_FLAG: DMA1 channel 1 transfer error flag
DMA1_GL2_FLAG: DMA1 channel 2 global flag
DMA1_FDT2_FLAG: DMA1 channel 2 transfer complete flag
DMA1_HDT2_FLAG: DMA1 channel 2 half transfer complete flag
DMA1_DTERR2_FLAG: DMA1 channel 2 transfer error flag
DMA1_GL3_FLAG: DMA1 channel 3 global flag
DMA1_FDT3_FLAG: DMA1 channel 3 transfer complete flag
DMA1_HDT3_FLAG: DMA1 channel 3 half transfer complete flag
DMA1_DTERR3_FLAG: DMA1 channel 3 transfer error flag
DMA1_GL4_FLAG: DMA1 channel 4 global flag
DMA1_FDT4_FLAG: DMA1 channel 4 transfer complete flag
DMA1_HDT4_FLAG: DMA1 channel 4 half transfer complete flag
DMA1_DTERR4_FLAG: DMA1 channel 4 transfer error flag
DMA1_GL5_FLAG: DMA1 channel 5 global flag
DMA1_FDT5_FLAG: DMA1 channel 5 transfer complete flag
DMA1_HDT5_FLAG: DMA1 channel 5 half transfer complete flag
DMA1_DTERR5_FLAG: DMA1 channel 5 transfer error flag
DMA1_GL6_FLAG: DMA1 channel 6 global flag
DMA1_FDT6_FLAG: DMA1 channel 6 transfer complete flag
DMA1_HDT6_FLAG: DMA1 channel 6 half transfer complete flag
DMA1_DTERR6_FLAG: DMA1 channel 6 transfer error flag
DMA1_GL7_FLAG: DMA1 channel 7 global flag
DMA1_FDT7_FLAG: DMA1 channel 7 transfer complete flag
DMA1_HDT7_FLAG: DMA1 channel 7 half transfer complete flag
DMA1_DTERR7_FLAG: DMA1 channel 7 transfer error flag
DMA2_GL1_FLAG: DMA2 channel 1 global flag
DMA2_FDT1_FLAG: DMA2 channel 1 transfer complete flag
DMA2_HDT1_FLAG: DMA2 channel 1 half transfer complete flag

DMA2_DTERR1_FLAG:	DMA2 channel 1 transfer error flag
DMA2_GL2_FLAG:	DMA2 channel 2 global flag
DMA2_FDT2_FLAG:	DMA2 channel 2 transfer complete flag
DMA2_HDT2_FLAG:	DMA2 channel 2 half transfer complete flag
DMA2_DTERR2_FLAG:	DMA2 channel 2 transfer error flag
DMA2_GL3_FLAG:	DMA2 channel 3 global flag
DMA2_FDT3_FLAG:	DMA2 channel 3 transfer complete flag
DMA2_HDT3_FLAG:	DMA2 channel 3 half transfer complete flag
DMA2_DTERR3_FLAG:	DMA2 channel 3 transfer error flag
DMA2_GL4_FLAG:	DMA2 channel 4 global flag
DMA2_FDT4_FLAG:	DMA2 channel 4 transfer complete flag
DMA2_HDT4_FLAG:	DMA2 channel 4 half transfer complete flag
DMA2_DTERR4_FLAG:	DMA2 channel 4 transfer error flag
DMA2_GL5_FLAG:	DMA2 channel 5 global flag
DMA2_FDT5_FLAG:	DMA2 channel 5 transfer complete flag
DMA2_HDT5_FLAG:	DMA2 channel 5 half transfer complete flag
DMA2_DTERR5_FLAG:	DMA2 channel 5 transfer error flag
DMA2_GL6_FLAG:	DMA2 channel 6 global flag
DMA2_FDT6_FLAG:	DMA2 channel 6 transfer complete flag
DMA2_HDT6_FLAG:	DMA2 channel 6 half transfer complete flag
DMA2_DTERR6_FLAG:	DMA2 channel 6 transfer error flag
DMA2_GL7_FLAG:	DMA2 channel 7 global flag
DMA2_FDT7_FLAG:	DMA2 channel 7 transfer complete flag
DMA2_HDT7_FLAG:	DMA2 channel 7 half transfer complete flag
DMA2_DTERR7_FLAG:	DMA2 channel 7 transfer error flag

Example:

```
if(dma_flag_get(DMA2_FDT1_FLAG) != RESET)
{
    /* turn led2/led3/led4 on */
    at32_led_on(LED2);
    at32_led_on(LED3);
    at32_led_on(LED4);
    dma_flag_clear(DMA2_FDT1_FLAG);
}
```

5.9 Debug

The DEBUG register structure debug_type is defined in the *"*at32f403a_407_debug.h*"*:

```
/**
  * @brief type define debug register all
  */
typedef struct
{
      ...

} debug_type;
```

The table below gives a list of the DEBUG registers.

Table 181. Summary of debug registers

Register	Description
idcode	Device ID
ctrl	Control register

The table below gives a list of DEBUG library functions.

Table 182. Summary of debug library functions

Function name	Description
debug_device_id_get	Read device idcode
debug_periph_mode_set	Peripheral debug mode configuration

5.9.1 debug_device_id_get function

The table below describes the function debug_device_id_get

Table 183. debug_device_id_get function

Name	Description
Function name	debug_device_id_get
Function prototype	uint32_t debug_device_id_get(void);
Function description	Read device ID code
Input parameter 1	NA
Input parameter 2	NA
Output parameter	NA
Return value	Return 32-bit ID code
Required preconditions	NA
Called functions	NA

Example:

```
/* get idcode */
uint32_t idcode = 0;
idcode = debug_device_id_get();
```

5.9.2 debug_periph_mode_set function

The table below describes the function debug_periph_mode_set

Table 184. debug_periph_mode_set function

Name	Description
Function name	debug_periph_mode_set
Function prototype	void debug_periph_mode_set(uint32_t periph_debug_mode, confirm_state new_state);
Function description	Select a peripheral or mode to debug
Input parameter 1	periph_debug_mode: Select a peripheral or mode
Input parameter 2	new_state: Enable (TRUE) or disable (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

periph_debug_mode

Select a peripheral or mode to debug

DEBUG_SLEEP:	Debug in Sleep mode
DEBUG_DEEPSLEEP:	Debug in Deepsleep mode
DEBUG_STANDBY:	Debug in Standby mode
DEBUG_WDT_PAUSE:	Watchdog pause control bit
DEBUG_WWDT_PAUSE:	Window watchdog pause control bit
DEBUG_TMR1_PAUSE:	TMR1 pause control bit
DEBUG_TMR3_PAUSE:	TMR3 pause control bit
DEBUG_I2C1_SMBUS_TIMEOUT:	I2C1 SMBUS TIMEOUT pause control bit
DEBUG_I2C2_SMBUS_TIMEOUT:	I2C2 SMBUS TIMEOUT pause control bit
DEBUG_I2C3_SMBUS_TIMEOUT:	I2C3 SMBUS TIMEOUT pause control bit
DEBUG_TMR2_PAUSE:	TMR2 pause control bit
DEBUG_TMR4_PAUSE:	TMR4 pause control bit
DEBUG_CAN1_PAUSE:	CAN1 receive register pause control bit
DEBUG_TMR8_PAUSE:	TMR8 pause control bit
DEBUG_TMR5_PAUSE:	TMR5 pause control bit
DEBUG_TMR6_PAUSE:	TMR6 pause control bit
DEBUG_TMR7_PAUSE:	TMR7 pause control bit
DEBUG_CAN2_PAUSE:	CAN2 receive register pause control bit
DEBUG_TMR12_PAUSE:	TMR12 pause control bit
DEBUG_TMR13_PAUSE:	TMR13 pause control bit
DEBUG_TMR14_PAUSE:	TMR14 pause control bit
DEBUG_TMR9_PAUSE:	TMR9 pause control bit
DEBUG_TMR10_PAUSE:	TMR10 pause control bit
DEBUG_TMR11_PAUSE:	TMR11 pause control bit

Example:

```
/* enable tmr1 debug mode */
debug_periph_mode_set(DEBUG_TMR1_PAUSE, TRUE);
```

5.10 External interrupt/event controller (EXINT)

The EXINT register structure exint_type is defined in the "at32f403a_407_exint.h":

```
/**
  * @brief type define exint register all
  */
typedef struct
{
  ...
} exint_type;
```

The table below gives a list of the EXINT registers.

Table 185. Summary of EXINT registers

Register	Description
inten	Interrupt enable register
evten	Event enable register
polcfg1	Polarity configuration register 1
polcfg2	Polarity configuration register 2
swtrg	Software trigger register
intsts	Interrupt status register

The table below gives a list of EXINT library functions.

Table 186. Summary of EXINT library functions

Function name	Description
exint_reset	Reset all EXINT registers to their reset values.
exint_default_para_init	Configure the EXINT initial structure with the initial value
exint_init	Initialize EXINT
exint_flag_clear	Clear the selected EXINT interrupt flag
exint_flag_get	Read the selected EXINT interrupt flag
exint_software_interrupt_event_generate	Software interrupt event generation
exint_interrupt_enable	Enable the selected EXINT interrupt
exint_event_enable	Enable the selected EXINT event

5.10.1 exint_reset function

The table below describes the function exint_reset.

Table 187. exint_reset function

Name	Description
Function name	exint_reset
Function prototype	void exint_reset(void);
Function description	Reset all EXINT registers to their reset values.
Input parameter	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	crm_periph_reset();

Example:

```
exint_reset ();
```

5.10.2 exint_default_para_init function

The table below describes the function exint_default_para_init

Table 188. exint_default_para_init function

Name	Description
Function name	exint_default_para_init
Function prototype	void exint_default_para_init(exint_init_type *exint_struct);
Function description	Configure the EXINT initial structure with the initiali value
Input parameter 1	exint_struct: *exint_init_type* pointer
Output parameter	NA
Return value	NA
Required preconditions	It is necessary to define a variable of exint_init_type before starting.
Called functions	NA

Example:

```
exint_init_type exint_init_struct;
exint_default_para_init(&exint_init_struct);
```

5.10.3 exint_init function

The table below describes the function exint_init

Table 189. exint_init function

Name	Description
Function name	exint_init
Function prototype	void exint_init(exint_init_type *exint_struct);
Function description	Initialize EXINT
Input parameter 1	*exint_init_type:* exint_init_struct pointer
Output parameter	NA
Return value	NA
Required preconditions	It is necessary to define a variable of exint_init_type before starting.
Called functions	NA

exint_init_type is defined in the at32f421_exint.h:

```
typedef struct
{
  exint_line_mode_type          line_mode;
  uint32_t                      line_select;
  exint_polarity_config_type    line_polarity;
  confirm_state                 line_enable;
} exint_init_type;
```

line_mode

Select event mode or interrupt mode

EXINT_LINE_INTERRUPUT: Interrupt mode

EXINT_LINE_EVENT: Event mode

line_select

Line selection.

EXINT_LINE_NONE: No line

EXINT_LINE_0: line0

EXINT_LINE_1: line1

…

EXINT_LINE_18: line18

EXINT_LINE_19: line19

line_polarity

Trigger edge selection.

EXINT_TRIGGER_RISING_EDGE: Rising edge

EXINT_TRIGGER_FALLING_EDGE: Falling edge

EXINT_TRIGGER_BOTH_EDGE: Rising/falling edge

line_enable

Enable/disable line

FALSE: Disable line

TRUE: Enable line

Example:

```
exint_init_type exint_init_struct;
exint_default_para_init(&exint_init_struct);
```

```
exint_init_struct.line_enable = TRUE;
exint_init_struct.line_mode = EXINT_LINE_INTERRUPUT;
exint_init_struct.line_select = EXINT_LINE_0;
exint_init_struct.line_polarity = EXINT_TRIGGER_RISING_EDGE;
exint_init(&exint_init_struct);
```

5.10.4 exint_flag_clear function

The table below describes the function exint_flag_clear

Table 190. exint_flag_clear function

Name	Description
Function name	exint_flag_clear
Function prototype	void exint_flag_clear(uint32_t exint_line);
Function description	Clear the selected EXINT interrupt flag
Input parameter	exint_line: line selection Refer to the "*line_select*" for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
exint_flag_clear(EXINT_LINE_0);
```

5.10.5 exint_flag_get function

The table below describes the function exint_flag_get.

Table 191. exint_flag_get function

Name	Description
Function name	exint_flag_get
Function prototype	flag_status exint_flag_get(uint32_t exint_line);
Function description	Get the selected EXINT interrupt flag
Input parameter	exint_line: line selection Refer to the "*line_select*" for details.
Output parameter	NA
Return value	flag_status: indicates the status of the selected flag This parameter can be SET or RESET.
Required preconditions	NA
Called functions	NA

Example:

```
flag_status status = RESET;
status = exint_flag_get(EXINT_LINE_0);
```

5.10.6 exint_interrupt_flag_get function

The table below describes the function exint_interrupt_flag_get.

Table 192. exint_interrupt_flag_get function

Name	Description
Function name	exint_interrupt_flag_get
Function prototype	flag_status exint_interrupt_flag_get(uint32_t exint_line)
Function description	Get the selected EXINT interrupt flag
Input parameter	exint_line: line selection Refer to the *line_select* for details.
Output parameter	NA
Return value	flag_status: indicates the status of the selected flag This parameter can be SET or RESET.
Required preconditions	NA
Called functions	NA

Example

```
flag_status status = RESET;
status = exint_interrupt_flag_get (EXINT_LINE_0);
```

5.10.7 exint_software_interrupt_event_generate function

The table below describes the function exint_software_interrupt_event_generate.

Table 193. exint_software_interrupt_event_generate function

Name	Description
Function name	exint_software_interrupt_event_generate
Function prototype	void exint_software_interrupt_event_generate(uint32_t exint_line);
Function description	Generate software interrupt event
Input parameter	exint_line: line selection Refer to the "*line_select*" for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
exint_software_interrupt_event_generate (EXINT_LINE_0);
```

5.10.8 exint_interrupt_enable function

The table below describes the function exint_interrupt_enable.

Table 194. exint_interrupt_enable function

Name	Description
Function name	exint_interrupt_enable
Function prototype	void exint_interrupt_enable(uint32_t exint_line, confirm_state new_state);
Function description	Enable the sected EXINT interrupt
Input parameter 1	exint_line: line selection Refer to the "*line_select*" for details.
Input parameter 2	new_state: Enable or disable This parameter can be FALSE or TRUE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
exint_interrupt_enable (EXINT_LINE_0);
```

5.10.9 exint_event_enable function

The table below describes the function exint_event_enable.

Table 195. exint_event_enable function

Name	Description
Function name	exint_event_enable
Function prototype	void exint_event_enable(uint32_t exint_line, confirm_state new_state);
Function description	Enable the sected EXINT event
Input parameter 1	exint_line: line selection Refer to the "*line_select*" for details
Input parameter 2	new_state: Enable or disable This parameter can be FALSE or TRUE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
exint_event_enable (EXINT_LINE_0);
```

5.11 Flash memory controller (FLASH)

The FLASH register structure flash_type is defined in the “at32f403a_407_flash.h”:

```
/**
  * @brief type define flash register all
  */
typedef struct
{
      …
} flash_type;
```

The table below gives a list of the FLASH registers.

Table 196. Summary of FLASH registers

Register	Description
flash_psr	Flash performance select register
flash_unlock	Flash unlock register
flash_usd_unlock	Flash user system data unlock register
flash_sts	Flash status register
flash_ctrl	Flash control register
flash_addr	Flash address register
flash_usd	User system data register
flash_epps	Erase/program protection status register
flash_unlock2	Flash unlock register2
flash_sts2	Flash status register2
flash_ctrl2	Flash control register2
flash_addr2	Flash address register2
flash_unlock3	Flash unlock register3
flash_select	Flash select register
flash_sts3	Flash status register3
flash_ctrl3	Flash control register3
flash_addr3	Flash address register3
flash_da	Flash decrption address register
slib_sts0	Flash security library status register 0
slib_sts1	Flash security library status register1
slib_pwd_clr	Flash security library password clear register
slib_misc_sts	Security library additional status register
slib_set_pwd	Security library password setting register
slib_set_range	Security library address setting register
slib_unlock	Security library unlock register
flash_crc_ctrl	Flash CRC check control register
flash_crc_chkr	Flash CRC check result register

The table below gives a list of FLASH library functions.

Table 197. Summary of FLASH library functions

Function name	Description
flash_flag_get	Get flag status
flash_flag_clear	Clear flag
flash_operation_status_get	Get operation status (Flash memory bank 1)
flash_operation_wait_for	Wait for operation complete (Flash memory bank 1)
flash_bank1_operation_status_get	Get Flash bank1 operation status
flash_bank2_operation_status_get	Get Flash bank2 operation status
flash_spim_operation_status_get	Get SPIM operation status
flash_operation_wait_for	Wait for operation complete (Flash bank 1)
flash_bank1_operation_wait_for	Wait for Flash bank 1 operation complete
flash_bank2_operation_wait_for	Wait for Flash bank 2 operation complete
flash_spim_operation_wait_for	Wait for SPIM peration complete
flash_unlock	Unlock Flash (Flash memory bank 1 and 2)
flash_bank1_unlock	Unlock Flash bank 1
flash_bank2_unlock	Unlock Flash bank 2
flash_spim_unlock	Unlock SPIM
flash_lock	Lock Flash (Flash memory bank 1 and 2)
flash_bank1_lock	Lock 1 Flash bank 1
flash_bank2_lock	Lock Flash bank 2
flash_spim_lock	Lock SPIM
flash_sector_erase	Erase Flash sector
flash_internal_all_erase	Erase interal Flash
flash_bank1_erase	Erase Flash bank 1
flash_bank2_erase	Erase Flash bank 2
flash_spim_all_erase	Erase SPIM
flash_user_system_data_erase	Erase user system data
flash_word_program	Flash word programming
flash_halfword_program	Flash half-word programming
flash_byte_program	Flash byte programming
flash_user_system_data_program	User system data programming
flash_user_system_data_erase	Erase user system data area
flash_word_program	Flash word programming
flash_halfword_program	Flash half-word programming
flash_byte_program	Flash bute programming
flash_epp_set	Erase/programming protection configuration
flash_epp_status_get	Get erase/programming protection status
flash_fap_enable	Flash access protection enable
flash_fap_status_get	Get Flash access protection status
flash_ssb_set	System configuration byte configuration
flash_ssb_status_get	Get system configuration byte configuration status
flash_interrupt_enable	Flash interrupt configuration
flash_spim_model_select	SPIM model selection

Function name	Description
flash_spim_encryption_range_set	SPIM encryption range configuration
flash_slib_enable	sLib enable
flash_slib_disable	sLib disable
flash_slib_remaining_count_get	Remaining usable times for security library function
flash_slib_state_get	Get sLib statys
flash_slib_start_sector_get	Get sLib start sector
flash_slib_datastart_sector_get	Get sKub data area start sector
flash_slib_end_sector_get	Get sLib end sector
flash_crc_calibrate	Flash CRC verify

5.11.1 flash_flag_get function

The table below describes the function flash_flag_get.

Table 198. flash_flag_get function

Name	Description
Function name	flash_flag_get
Function prototype	flag_status flash_flag_get(uint32_t flash_flag);
Function description	Get flag status
Input parameter	flash_flag: Flag selection
Output parameter	NA
Return value	flag_status: indicates the flag status Return RESET or SET.
Required preconditions	NA
Called functions	NA

flash_flag

Flag selection.

FLASH_OBF_FLAG: Flash operation busy (bank 1)
FLASH_ODF_FLAG: Flash operation complete (bank 1)
FLASH_PRGMERR_FLAG: Flash programming error (bank 1)
FLASH_EPPERR_FLAG: Flash erase error (bank 1)
FLASH_BANK1_OBF_FLAG: Flash bank 1 operation busy
FLASH_BANK1_ODF_FLAG: Flash bank 1 operation complete
FLASH_BANK1_PRGMERR_FLAG: Flash bank 1 programming error
FLASH_BANK1_EPPERR_FLAG: Flash bank 1 erase error
FLASH_BANK2_OBF_FLAG: Flash bank 2 operation busy
FLASH_BANK2_ODF_FLAG: Flash bank 2 operation complete
FLASH_BANK2_PRGMERR_FLAG: Flash bank 2 programming error
FLASH_BANK2_EPPERR_FLAG: Flash bank 2 erase error
FLASH_SPIM_OBF_FLAG: SPIM operation busy
FLASH_SPIM_ODF_FLAG: SPIM operation complete
FLASH_SPIM_PRGMERR_FLAG: SPIM programming error
FLASH_SPIM_EPPERR_FLAG: SPIM erase error
FLASH_USDERR_FLAG: User system data area error

Example:

```
flag_status status;
status = flash_flag_get (FLASH_ODF_FLAG);
```

5.11.2 flash_flag_clear function

The table below describes the function flash_flag_clear.

Table 199. flash_flag_clear function

Name	Description
Function name	flash_flag_clear
Function prototype	void flash_flag_clear(uint32_t flash_flag);
Function description	Clear flag
Input parameter	flash_flag: flag selection
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

flash_flag

Flag selection

FLASH_ODF_FLAG:	Flash operation complete (bank 1)
FLASH_PRGMERR_FLAG:	Flash programming error (bank 1)
FLASH_EPPERR_FLAG:	Flash erase error (bank 1)
FLASH_BANK1_ODF_FLAG:	Flash bank 1 operation complete
FLASH_BANK1_PRGMERR_FLAG:	Flash bank 1 programming error
FLASH_BANK1_EPPERR_FLAG:	Flash bank 1 erase error
FLASH_BANK2_ODF_FLAG:	Flash bank 2 operation complete
FLASH_BANK2_PRGMERR_FLAG:	Flash bank 2 programming error
FLASH_BANK2_EPPERR_FLAG:	Flash bank 2 erase error
FLASH_SPIM_ODF_FLAG:	SPIM operation complete
FLASH_SPIM_PRGMERR_FLAG:	SPIM programming error
FLASH_SPIM_EPPERR_FLAG:	SPIM erase error

Example:

```
flash_flag_clear(FLASH_ODF_FLAG);
```

5.11.3 flash_operation_status_get function

The table below describes the function flash_operation_status_get.

Table 200. flash_operation_status_get function

Name	Description
Function name	flash_operation_status_get
Function prototype	flash_status_type flash_operation_status_get(void);
Function description	Get operation status
Input parameter	NA
Output parameter	NA
Return value	Refer to the *flash_status_type* for details.
Required preconditions	NA
Called functions	NA

flash_status_type

FLASH_OPERATE_BUSY	Operate busy
FLASH_PROGRAM_ERROR	Programming error
FLASH_EPP_ERROR	Erase/program protection error
FLASH_OPERATE_DONE	Operation complete
FLASH_OPERATE_TIMEOUT	Operation timeout

Example:

```
flash_status_type status = FLASH_OPERATE_DONE;
/* check for the flash status */
status = flash_operation_status_get();
```

5.11.4 flash_bank1_operation_status_get function

The table below describes the function flash_bank1_operation_status_get

Table 201. flash_bank1_operation_status_get function

Name	Description
Function name	flash_bank1_operation_status_get
Function prototype	flash_status_type flash_bank1_operation_status_get (void);
Function description	Get the status of Flash bank 1
Input parameter	NA
Output parameter	NA
Return value	Return Flash bank 1 operation status, refer to the *flash_status_type* for details
Required preconditions	NA
Called functions	NA

Example:

```
flash_status_type status = FLASH_OPERATE_DONE;
/* check for the flash status */
status = flash_bank1_operation_status_get();
```

5.11.5 flash_bank2_operation_status_get function

The table below describes the function flash_bank2_operation_status_get.

Table 202. flash_bank2_operation_status_get function

Name	Description
Function name	flash_bank2_operation_status_get
Function prototype	flash_status_type flash_bank2_operation_status_get (void);
Function description	Get the status of Flash bank 2
Input parameter	NA
Output parameter	NA
Return value	Return Flash bank 2 operation status, refer to the *flash_status_type* for details
Required preconditions	NA
Called functions	NA

Example:

```
flash_status_type status = FLASH_OPERATE_DONE;
/* check for the flash status */
status = flash_bank2_operation_status_get();
```

5.11.6 flash_spim_operation_status_get function

The table below describes the function flash_spim_operation_status_get.

Table 203. flash_spim_operation_status_get function

Name	Description
Function name	flash_spim_operation_status_get
Function prototype	flash_status_type flash_spim_operation_status_get (void);
Function description	Get the status of SPIM (external Flash)
Input parameter	NA
Output parameter	NA
Return value	Return Flash bank 2 operation status, refer to the *flash_status_type* for details
Required preconditions	NA
Called functions	NA

Example:

```
flash_status_type status = FLASH_OPERATE_DONE;
/* check for the flash status */
status = flash_spim_operation_status_get();
```

5.11.7 flash_operation_wait_for function

The table below describes the function flash_operation_wait_for.

Table 204. flash_operation_wait_for function

Name	Description
Function name	flash_operation_wait_for
Function prototype	flash_status_type flash_operation_wait_for(uint32_t time_out);
Function description	Wait for Flash operation
Input parameter	time_out: wait timeout The timeout value is defined in the flash.h file, refer to *flash_time_out* for details.
Output parameter	NA
Return value	Refer to *flash_status_type* for details.
Required preconditions	NA
Called functions	NA

flash_time_out

ERASE_TIMEOUT	Erase timeout
PROGRAMMING_TIMEOUT	Programming timeout
SPIM_ERASE_TIMEOUT:	SPIM erase timeout
SPIM_PROGRAMMING_TIMEOUT:	SPIM programming timeout
OPERATION_TIMEOUT	General operation timeout

Example:

```
/* wait for operation to be completed */
status = flash_operation_wait_for(PROGRAMMING_TIMEOUT);
```

5.11.8 flash_bank1_operation_wait_for function

The table below describes the function flash_bank1_operation_wait_for.

Table 205. flash_bank1_operation_wait_for function

Name	Description
Function name	flash_bank1_operation_wait_for
Function prototype	flash_status_type flash_bank1_operation_wait_for(uint32_t time_out);
Function description	Wait for Flash bank1 operation
Input parameter	time_out: wait timeout The timeout value is defined in the flash.h file, refer to *flash_time_out* for details.
Output parameter	NA
Return value	Return Flash bank 2 operation status, refer to the *flash_status_type* for details
Required preconditions	NA
Called functions	NA

Example:

```
/* wait for operation to be completed */
status = flash_bank1_operation_wait_for(PROGRAMMING_TIMEOUT);
```

5.11.9 flash_bank2_operation_wait_for function

The table below describes the function flash_bank2_operation_wait_for.

Table 206. flash_bank2_operation_wait_for function

Name	Description
Function name	flash_bank2_operation_wait_for
Function prototype	flash_status_type flash_bank2_operation_wait_for(uint32_t time_out);
Function description	Wait for Flash bank2 operation
Input parameter	time_out: wait timeout The timeout value is defined in the flash.h file, refer to *flash_time_out* for details.
Output parameter	NA
Return value	Return Flash bank 2 operation status, refer to the *flash_status_type* for details
Required preconditions	NA
Called functions	NA

Example:

```
/* wait for operation to be completed */
status = flash_bank2_operation_wait_for(PROGRAMMING_TIMEOUT);
```

5.11.10 flash_spim_operation_wait_for function

The table below describes the function flash_spim_operation_wait_for.

Table 207. flash_spim_operation_wait_for function

Name	Description
Function name	flash_spim_operation_wait_for
Function prototype	flash_status_type flash_spim_operation_wait_for(uint32_t time_out);
Function description	Wait for SPIM (external Flash) operation
Input parameter	time_out: wait timeout The timeout value is defined in the flash.h file, refer to *flash_time_out* for details.
Output parameter	NA
Return value	Return Flash bank 2 operation status, refer to the *flash_status_type* for details
Required preconditions	NA
Called functions	NA

Example:

```
/* wait for operation to be completed */
status = flash_spim_operation_wait_for(PROGRAMMING_TIMEOUT);
```

5.11.11 flash_unlock function

The table below describes the function flash_unlock.

Table 208. flash_unlock function

Name	Description
Function name	flash_unlock
Function prototype	void flash_unlock(void);
Function description	Unlock Flash memory controller
Input parameter	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
flash_unlock();
```

5.11.12 flash_bank1_unlock function

The table below describes the function flash_bank1_unlock.

Table 209. flash_bank1_unlock function

Name	Description
Function name	flash_bank1_unlock
Function prototype	void flash_bank1_unlock(void);
Function description	Unlock Flash bank1 control register
Input parameter	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
flash_bank1_unlock();
```

5.11.13 flash_bank2_unlock function

The table below describes the function flash_bank2_unlock.

Table 210. flash_bank2_unlock function

Name	Description
Function name	flash_bank2_unlock
Function prototype	void flash_bank2_unlock(void);
Function description	Unlock Flash bank2 control register
Input parameter	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
flash_bank2_unlock();
```

5.11.14 flash_spim_unlock function

The table below describes the function flash_spim_unlock.

Table 211. flash_spim_unlock function

Name	Description
Function name	flash_spim_unlock
Function prototype	void flash_spim_unlock(void);
Function description	Unlock SPIM (external Flash) control register
Input parameter	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
flash_spim_unlock();
```

5.11.15 flash_lock function

The table below describes the function flash_lock.

Table 212. flash_lock function

Name	Description
Function name	flash_lock
Function prototype	void flash_lock(void);
Function description	Lock Flash memory controller
Input parameter	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
flash_lock();
```

5.11.16 flash_bank1_lock function

The table below describes the function flash_bank1_lock.

Table 213. flash_bank1_lock function

Name	Description
Function name	flash_bank1_lock
Function prototype	void flash_bank1_lock(void);
Function description	Lock Flash bank1 controll register
Input parameter	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
flash_bank1_lock();
```

5.11.17 flash_bank2_lock function

The table below describes the function flash_bank2_lock.

Table 214. flash_bank2_lock function

Name	Description
Function name	flash_bank2_lock
Function prototype	void flash_bank2_lock(void);
Function description	Lock Flash bank2 controll register
Input parameter	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
flash_bank2_lock();
```

5.11.18 flash_spim_lock function

The table below describes the function flash_spim_lock.

Table 215. flash_spim_lock function

Name	Description
Function name	flash_spim_lock
Function prototype	void flash_spim_lock(void);
Function description	Lock SPIM (external Flash) control register
Input parameter	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
flash_spim_lock();
```

5.11.19 flash_sector_erase function

The table below describes the function flash_sector_erase.

Table 216. flash_sector_erase function

Name	Description
Function name	flash_sector_erase
Function prototype	flash_status_type flash_sector_erase(uint32_t sector_address);
Function description	Erase data in the selected Flash sector address
Input parameter	sector_address: select the Flash sector address to be erased, usually Flash sector start address
Output parameter	NA
Return value	Refer to *flash_status_type* for details.
Required preconditions	NA
Called functions	NA

Example:

```
flash_status_type status = FLASH_OPERATE_DONE;
flash_unlock();
status = flash_sector_erase(0x08001000);
```

5.11.20 flash_internal_all_erase function

The table below describes the function flash_internal_all_erase.

Table 217. flash_internal_all_erase function

Name	Description
Function name	flash_internal_all_erase
Function prototype	flash_status_type flash_internal_all_erase(void);
Function description	Erase internal Flash data
Input parameter	NA
Output parameter	NA
Return value	Refer to *flash_status_type* for details.
Required preconditions	NA
Called functions	NA

Example:

```
flash_status_type status = FLASH_OPERATE_DONE;
flash_unlock();
status = flash_internal_all_erase();
```

5.11.21 flash_bank1_erase function

The table below describes the function flash_bank1_erase.

Table 218. flash_bank1_erase function

Name	Description
Function name	flash_bank1_erase
Function prototype	flash_status_type flash_bank1_erase(void);
Function description	Erase internal Flash bank1 data
Input parameter	NA
Output parameter	NA
Return value	Refer to *flash_status_type* for details.
Required preconditions	NA
Called functions	NA

Example:

```
flash_status_type status = FLASH_OPERATE_DONE;
flash_bank1_unlock();
status = flash_bank1_erase();
```

5.11.22 flash_bank2_erase function

The table below describes the function flash_bank2_erase.

Table 219. flash_bank2_erase function

Name	Description
Function name	flash_bank2_erase
Function prototype	flash_status_type flash_bank2_erase(void);
Function description	Erase internal Flash bank2 data
Input parameter	NA
Output parameter	NA
Return value	Refer to *flash_status_type* for details.
Required preconditions	NA
Called functions	NA

Example:

```
flash_status_type status = FLASH_OPERATE_DONE;
flash_bank2_unlock();
status = flash_bank2_erase();
```

5.11.23 flash_spim_all_erase function

The table below describes the function flash_spim_all_erase.

Table 220. flash_spim_all_erase function

Name	Description
Function name	flash_spim_all_erase
Function prototype	flash_status_type flash_spim_all_erase(void);
Function description	Erase external Flash (SPIM) data
Input parameter	NA
Output parameter	NA
Return value	Refer to *flash_status_type* for details.
Required preconditions	NA
Called functions	NA

Example:

```
flash_status_type status = FLASH_OPERATE_DONE;
flash_spim_unlock();
status = flash_spim_all_erase();
```

5.11.24 flash_user_system_data_erase function

The table below describes the function flash_user_system_data_erase.

Table 221. flash_user_system_data_erase function

Name	Description
Function name	flash_user_system_data_erase
Function prototype	flash_status_type flash_user_system_data_erase(void);
Function description	Erase user system data
Input parameter	NA
Output parameter	NA
Return value	Refer to *flash_status_type* for details.
Required preconditions	NA
Called functions	NA

Note: As this function remains in FAP state, it only erases data except FAP in the user system data area.

Example:

```
flash_status_type status = FLASH_OPERATE_DONE;
flash_unlock();
status = flash_user_system_data_erase();
```

5.11.25 flash_word_program function

The table below describes the function flash_word_program.

Table 222. flash_word_program function

Name	Description
Function name	flash_word_program
Function prototype	flash_status_type flash_word_program(uint32_t address, uint32_t data);
Function description	Program a word data to a given address
Input parameter 1	Address: programming address, word-aligned
Input parameter 2	Data: programming data
Output parameter	NA
Return value	Refer to *flash_status_type* for details.
Required preconditions	The programming operation can be allowed only when data in the address are all 0xFF
Called functions	NA

Example:

```
flash_status_type status = FLASH_OPERATE_DONE;
 uint32_t i;
 flash_unlock();
 status = flash_sector_erase(0x08001000);
 if(status = FLASH_OPERATE_DONE)
 {
   /* program 256 words */
   for(I = 0; I < 256; i++)
   {
     status = flash_word_program(0x08001000 + i*4, i);
   }
 }
```

5.11.26 flash_halfword_program function

The table below describes the function flash_halfword_program.

Table 223. flash_halfword_program function

Name	Description
Function name	flash_halfword_program
Function prototype	flash_status_type flash_halfword_program(uint32_t address, uint16_t data);
Function description	Program a half-word data to a given address
Input parameter 1	Address: programming address, half-word-aligned
Input parameter 2	Data: programming data
Output parameter	NA
Return value	Refer to *flash_status_type* for details.
Required preconditions	The programming operation can be allowed only when data in the address are all 0xFF
Called functions	NA

Example:

```
flash_status_type status = FLASH_OPERATE_DONE;
uint32_t i;
flash_unlock();
status = flash_sector_erase(0x08001000);
if(status = FLASH_OPERATE_DONE)
{
  /* program 256 halfwords */
  for(I = 0; I < 256; i++)
  {
    status = flash_halfword_program(0x08001000 + i*2, (uint16_t)i);
  }
}
```

5.11.27 flash_byte_program function

The table below describes the function flash_byte_program.

Table 224. flash_byte_program function

Name	Description
Function name	flash_byte_program
Function prototype	flash_status_type flash_byte_program(uint32_t address, uint8_t data);
Function description	Program a byte data to a given address
Input parameter 1	Address: programming address
Input parameter 2	Data: programming data
Output parameter	NA
Return value	Refer to *flash_status_type* for details.
Required preconditions	The programming operation can be allowed only when data in the address are all 0xFF
Called functions	NA

Example:

```
flash_status_type status = FLASH_OPERATE_DONE;
uint32_t i;
flash_unlock();
status = flash_sector_erase(0x08001000);
if(status = FLASH_OPERATE_DONE)
{
  /* program 256 bytes */
  for(I = 0; I < 256; i++)
  {
    status = flash_byte_program(0x08001000 + i*2, (uint8_t)i);
  }
}
```

5.11.28 flash_user_system_data_program function

The table below describes the function flash_user_system_data_program.

Table 225. flash_user_system_data_program function

Name	Description
Function name	flash_user_system_data_program
Function prototype	flash_status_type flash_user_system_data_program (uint32_t address, uint8_t data);
Function description	Program a byte data to a given address in the user system data area
Input parameter 1	Address: programmed address
Input parameter 2	Data: programned data
Output parameter	NA
Return value	Refer to *flash_status_type* for details.
Required preconditions	The programming operation can be allowed only when data and its inverse data in the user system data area are all 0xFF
Called functions	NA

Example:

```
flash_status_type status = FLASH_OPERATE_DONE;
  flash_unlock();
  status = flash_user_system_data_erase();
  if(status = FLASH_OPERATE_DONE)
  {
    /* program user system data */
    status = flash_user_system_data_program(0x1FFFF804, 0x55);
  }
```

5.11.29 flash_epp_set function

The table below describes the function flash_epp_set.

Table 226. flash_epp_set function

Name	Description
Function name	flash_epp_set
Function prototype	flash_status_type flash_epp_set(uint32_t *sector_bits);
Function description	Enable erase programming protection
Input parameter	*sector_bits: Erase programming protection sector address pointer. Each bit protects 4KB sectors, and the last bit guards the remaing sectors. When this bit is set to 1, it indicates that the corresponding sector is protected.
Output parameter	NA
Return value	Refer to *flash_status_type* for details.
Required preconditions	NA
Called functions	NA

Example:

```
flash_status_type status = FLASH_OPERATE_DONE;
  uint32_t epp_val;
```

```
flash_unlock();
status = flash_user_system_data_erase();
if(status = FLASH_OPERATE_DONE)
{
  epp_val = 0x00000001;
  /* program epp */
  status = flash_epp_set(&epp_val);
}
```

5.11.30 flash_epp_status_get function

The table below describes the function flash_epp_status_get.

Table 227. flash_epp_status_get function

Name	Description
Function name	flash_epp_status_get
Function prototype	void flash_epp_status_get(uint32_t *sector_bits);
Function description	Get the status of erase programming protection
Input parameter	NA
Output parameter	*sector_bits: Erase programming protection sector address pointer. Each bit protects 4KB sectors, and the last bit guards the remaining sectors. When this bit is set to 1, it indicates that the corresponding sector is protected.
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
uint32_t epp_val[2];
/* get epp status */
flash_epp_status_get(epp_val);
```

5.11.31 flash_fap_enable function

The table below describes the function flash_fap_enable.

Table 228. flash_fap_enable function

Name	Description
Function name	flash_fap_enable
Function prototype	flash_status_type flash_fap_enable(confirm_state new_state);
Function description	Enable Flash access protection
Input parameter	new_state: Flash access protection status This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	Refer to *flash_status_type* for details.
Required preconditions	NA
Called functions	NA

Note: This function will erase the whole user system data area. If there were data programmed in the user system data area before calling this function, they have to be re-programmed after calling this function.

Example:

```
flash_status_type status = FLASH_OPERATE_DONE;
flash_unlock();
status = flash_fap_enable(TRUE);
```

5.11.32 flash_fap_status_get function

The table below describes the function flash_fap_status_get.

Table 229. flash_fap_status_get function

Name	Description
Function name	flash_fap_status_get
Function prototype	flag_status flash_fap_status_get(void);
Function description	Get the status of Flash access protection
Input parameter	NA
Output parameter	NA
Return value	flag_status: flag status This parameter can be SET or RESET.
Required preconditions	NA
Called functions	NA

Example:

```
flag_status status;
status = flash_fap_status_get();
```

5.11.33 flash_ssb_set function

The table below describes the function flash_ssb_set.

Table 230. flash_ssb_set function

Name	Description
Function name	flash_ssb_set
Function prototype	flash_status_type flash_ssb_set(uint8_t usd_ssb);
Function description	Configure system setting bytes
Input parameter	usd_ssb: system setting byte value is a combination of the selected data from all data group, refer to *ssb_data_define* for details.
Output parameter	NA
Return value	Return operation status, refer to the *flash_status_type* for details.
Required preconditions	NA
Called functions	NA

ssb_data_define

type 1:

USD_WDT_ATO_DISABLE: Watchdog auto-start disabled

USD_WDT_ATO_ENABLE: Watchdog auto-start enabled

type 2:

USD_DEPSLP_NO_RST: No reset occurred when entering Deepsleep mode

USD_DEPSLP_RST: Reset occurred when entering Deepsleep mode

type 3:

USD_STDBY_NO_RST: No reset occurred when entering Standby mode

USD_STDBY_RST: Reset occurred when entering Standby mode

type 4:

FLASH_BOOT_FROM_BANK1: Boot from Flash bank 1

FLASH_BOOT_FROM_BANK2: Boot from Flash bank 2

Example:

```
flash_status_type status = FLASH_OPERATE_DONE;
flash_unlock();
status = flash_user_system_data_erase();
if(status = FLASH_OPERATE_DONE)
{
  status = flash_ssb_set(USD_WDT_ATO_DISABLE | USD_DEPSLP_NO_RST | USD_STDBY_RST |
FLASH_BOOT_FROM_BANK1);
}
```

5.11.34 flash_ssb_status_get function

The table below describes the function flash_ssb_status_get.

Table 231. flash_ssb_status_get function

Name	Description
Function name	flash_ssb_status_get
Function prototype	uint8_t flash_ssb_status_get(void);
Function description	Get the status of system setting bytes
Input parameter	NA
Output parameter	NA
Return value	Return system setting byte value, refer to *ssb_data_define* for details.
Required preconditions	NA
Called functions	NA

Example:

```
uint8_t ssb_val;
ssb_val = flash_ssb_status_get();
```

5.11.35 flash_interrupt_enable function

The table below describes the function flash_interrupt_enable.

Table 232. flash_interrupt_enable function

Name	Description
Function name	flash_interrupt_enable
Function prototype	void flash_interrupt_enable(uint32_t flash_int, confirm_state new_state);
Function description	Enable Flash interrupts
Input parameter 1	flash_int: Flash interrupt type. Refer to *flash_interrupt_type* for details.
Input parameter 2	new_state: interrupt status This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

flash_interrupt_type

FLASH_ERR_INT: Flash error interrupt
FLASH_ODF_INT: Flash operation complete interrupt
FLASH_BANK1_ERR_INT: Flash bank 1 error interrupt
FLASH_BANK1_ODF_INT: Flash bank 1 operation complete interrupt
FLASH_BANK2_ERR_INT: Flash bank 2 error interrupt
FLASH_BANK2_ODF_INT: Flash bank 2 operation complete interrupt
FLASH_SPIM_ERR_INT: SPIM error interrupt
FLASH_SPIM_ODF_INT: SPIM operation complete interrupt

Example:

```
flash_interrupt_enable(FLASH_BANK1_ERR_INT | FLASH_BANK1_ODF_INT, TRUE);
```

5.11.36 flash_spim_model_select function

The table below describes the function flash_spim_model_select.

Table 233. flash_spim_model_select function

Name	Description
Function name	flash_spim_model_select
Function prototype	void flash_spim_model_select(flash_spim_model_type mode);
Function description	Select SPIM mode
Input parameter	Mode: SPIM mode, refer to the *flash_spim_mode_type* for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

flash_spim_mode_type

FLASH_SPIM_MODEL1: SPIM mode 1

FLASH_SPIM_MODEL2: SPIM mode 2

Example:

```
flash_spim_model_select(FLASH_SPIM_MODEL1);
```

5.11.37 flash_spim_encryption_range_set function

The table below describes the function flash_spim_encryption_range_set.

Table 234. flash_spim_encryption_range_set function

Name	Description
Function name	flash_spim_encryption_range_set
Function prototype	void flash_spim_encryption_range_set(uint32_t decode_address);
Function description	Configure SPIM data encryption data
Input parameter	decode_address: address range for encription, word-aligned, meaning that the data located before this address are stored as ciphertext.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
flash_spim_encryption_range_set(0x08401000);
```

5.11.38 flash_slib_enable function

The table below describes the function flash_slib_enable.

Table 235. flash_slib_enable function

Name	Description
Function name	flash_slib_enable
Function prototype	flash_status_type flash_slib_enable(uint32_t pwd, uint16_t start_sector, uint16_t data_start_sector, uint16_t end_sector);
Function description	Enable security library (sLib) and its address range
Input parameter 1	Pwd: sLib password. The sLib data are saved as ciphertext, associated with encrypted computing. A correct password is entered in order to unlock encryption.
Input parameter 2	start_sector: sLib start sector number
Input parameter 3	inst_start_sector: sLib data area instruction start sector number
Input parameter 4	end_sector: sLib end sector number
Output parameter	NA
Return value	Refer to *flash_status_type* for details.
Required preconditions	NA
Called functions	NA

Example:

```
flash_status_type status = FLASH_OPERATE_DONE;
status = flash_slib_enable(0x12345678, 0x04 , 0x05, 0x06);
```

5.11.39 flash_slib_disable function

The table below describes the function flash_slib_disable.

Table 236. flash_slib_disable function

Name	Description
Function name	flash_slib_disable
Function prototype	error_status flash_slib_disable(uint32_t pwd);
Function description	Disable security library (sLib)
Input parameter	Pwd: sLib password. it must be entered correctly, otherwise it is not allowed to enter until reset.
Output parameter	NA
Return value	Return error status This parameter can be ERROE or SUCCESS.
Required preconditions	NA
Called functions	NA

Note: Successful calling of this function will erase the whole internal Flash memory.

Example:

```
error_status status;
status = flash_slib_disable(0x12345678);
```

5.11.40 flash_slib_remaining_count_get function

The table below describes the function flash_slib_remaining_count_get.

Table 237. flash_slib_remaining_count_get function

Name	Description
Function name	flash_slib_remaining_count_get
Function prototype	uint32_t flash_slib_remaining_count_get(void);
Function description	Get how many times have been left for security library to use
Input parameter	NA
Output parameter	NA
Return value	Return the remaining usable times for security library function
Required preconditions	NA
Called functions	NA

Example:

```
uint32_t num;
num = flash_slib_remaining_count_get();
```

5.11.41 flash_slib_state_get function

The table below describes the function flash_slib_state_get.

Table 238. flash_slib_state_get function

Name	Description
Function name	flash_slib_state_get
Function prototype	flag_status flash_slib_state_get(void);
Function description	Get the status of sLib
Input parameter	NA
Output parameter	NA
Return value	flag_status: flag status This parameter can be SET or RESET.
Required preconditions	NA
Called functions	NA

Example:

```
flag_status status;
status = flash_slib_state_get();
```

5.11.42 flash_slib_start_sector_get function

The table below describes the function flash_slib_start_sector_get.

Table 239. flash_slib_start_sector_get function

Name	Description
Function name	flash_slib_start_sector_get
Function prototype	uint16_t flash_slib_start_sector_get(void);
Function description	Get the start sector number of sLib
Input parameter	NA
Output parameter	NA
Return value	Return the start sector number of sLib
Required preconditions	NA
Called functions	NA

Example:

```
uint16_t num;
num = flash_slib_start_sector_get();
```

5.11.43 flash_slib_datastart_sector_get function

The table below describes the function flash_slib_datastart_sector_get.

Table 240. flash_slib_datastart_sector_get function

Name	Description
Function name	flash_slib_datastart_sector_get
Function prototype	uint16_t flash_slib_datastart_sector_get(void);
Function description	Get the start sector number of sLib data area
Input parameter	NA
Output parameter	NA
Return value	Return the start sector number of sLib data area
Required preconditions	NA
Called functions	NA

Example:

```
uint16_t num;
num = flash_slib_datastart_sector_get();
```

5.11.44 flash_slib_end_sector_get function

The table below describes the function flash_slib_end_sector_get.

Table 241. flash_slib_end_sector_get function

Name	Description
Function name	flash_slib_end_sector_get
Function prototype	uint16_t flash_slib_end_sector_get(void);
Function description	Get the end sector number of sLib
Input parameter	NA
Output parameter	NA
Return value	Return the end sector number of sLib
Required preconditions	NA
Called functions	NA

Example:

```
uint16_t num;
num = flash_slib_end_sector_get();
```

5.11.45 flash_crc_calibrate function

The table below describes the function flash_crc_calibrate.

Table 242. flash_crc_calibrate function

Name	Description
Function name	flash_crc_calibrate
Function prototype	uint32_t flash_crc_calibrate(uint32_t start_sector, uint32_t sector_cnt);
Function description	Enable Flash CRC check
Input parameter 1	start_addr: CRC check start address
Input parameter 2	sector_cnt: CRC check sector count
Output parameter	NA
Return value	Return CRC calculation result
Required preconditions	NA
Called functions	NA

Note: The sector set to go through CRC check is only allowed to be on a single area, rather than on both security library and common area.

Example:

```
uint32_t crc_val;
crc_val = flash_crc_calibrate(0, 10);
```

5.12 Geneal-purpose I/Os (GPIO)

The GPIO register structuregpio_type is defined in the “at32f403a_407_gpio.h”:

```
/**
  * @brief type define gpio register all
  */
typedef struct
{


} gpio_type;

/**
  * @brief type define iomux register all
  */
typedef struct
{


} iomux_type;
```

The table below gives a list of the GPIO registers.

Table 243. Summary of GPIO registers

Register	Description
cfglr	GPIO configuration register low
cfghr	GPIO configuration register high
idt	GPIO input register
odt	GPIO output register
scr	GPIO set/clear register
clr	GPIO bit clear register
wpr	GPIO write protection register
hdrv	GPIO huge drive capability control register

The table below gives a list of IOMUX library functions.

Table 244. GPIO and IOMUX library functions

Register	Description
evtout	Event output control register
remap	IOMUX remap register
exintc1	IOMUX external interrupt configuration register 1
exintc2	IOMUX external interrupt configuration register 2
exintc3	IOMUX external interrupt configuration register 3
exintc4	IOMUX external interrupt configuration register 4
remap2	IOMUX remap register 2
remap3	IOMUX remap register 3
remap4	IOMUX remap register 4
remap5	IOMUX remap register 5
remap6	IOMUX remap register 6
remap7	IOMUX remap register 7
remap8	IOMUX remap register 8

The table below gives a list of GPIO and IOMUX library functions.

Table 245. GPIO and IOMUX library functions

Function name	Description
gpio_reset	GPIO is reset by CRM reset register
gpio_iomux_reset	IOMUX is reset by CRM reset register
gpio_init	Initialize GPIO peripherlas
gpio_default_para_init	Initialize GPIO default parameters
gpio_input_data_bit_read	Read GPIO input data bit
gpio_input_data_read	Read GPIO input data
gpio_output_data_bit_read	Read GPIO output data bit
gpio_output_data_read	Read GPIO output data
gpio_bits_set	Set GPIO bits
gpio_bits_reset	Reset GPIO bits
gpio_bits_toggle	Toggle GPIO bits
gpio_bits_write	Write GPIO bits
gpio_port_write	Write GPIO ports
gpio_pin_wp_config	Configure GPIO pin write protection
gpio_pins_huge_driven_config	Configure GPIO huge drive capability
gpio_event_output_config	Configure GPIO event output
gpio_event_output_enable	Enable/disable GPIO event output
gpio_pin_remap_config	Configure GPIO pin remapping
gpio_exint_line_config	Configure GPIO external interrupt lines

5.12.1 gpio_reset function

The table below describes the function gpio_reset.

Table 246. gpio_reset function

Name	Description
Function name	gpio_reset
Function prototype	void gpio_reset(gpio_type *gpio_x);
Function description	GPIO is reset by CRM reset register
Input parameter	gpio_x: Select GPIO pheripherals. This parameter can be GPIOA, GPIOB, GPIOC, GPIOD, GPIOE
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	crm_periph_reset();

Example:

```
gpio_reset(GPIOA);
```

5.12.2 gpio_iomux_reset function

The table below describes the function gpio_iomux_reset.

Table 247. gpio_iomux_reset function

Name	Description
Function name	gpio_iomux_reset
Function prototype	void gpio_iomux_reset ();
Function description	IOMUX is reset by CRM reset register.
Input parameter	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	crm_periph_reset();

Example:

```
gpio_iomux_reset();
```

5.12.3 gpio_init function

The table below describes the function gpio_init.

Table 248. gpio_init function

Name	Description
Function name	gpio_init
Function prototype	void gpio_init(gpio_type *gpio_x, gpio_init_type *gpio_init_struct);
Function description	Initialize GPIO peripherals
Input parameter 1	gpio_x: indicates the selected GPIO peripheral This parameter can be GPIOA, GPIOB, GPIOC, GPIOD, GPIOE.
Input parameter 2	gpio_init_struct: gpio_init_type pointer
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

gpio_init_type structure

The gpio_init_type is defined in the at32f403a_407_gpio.h:

```
typedef struct
{
  uint32_t                gpio_pins;
  gpio_output_type        gpio_out_type;
  gpio_pull_type          gpio_pull;
  gpio_mode_type          gpio_mode;
  gpio_drive_type         gpio_drive_strength;
} gpio_init_type;
```

gpio_pins

Select a GPIO pin.

GPIO_PINS_0: GPIO pin 0
GPIO_PINS_1: GPIO pin 1
GPIO_PINS_2: GPIO pin 2
GPIO_PINS_3: GPIO pin 3
GPIO_PINS_4: GPIO pin 4
GPIO_PINS_5: GPIO pin 5
GPIO_PINS_6: GPIO pin 6
GPIO_PINS_7: GPIO pin 7
GPIO_PINS_8: GPIO pin 8
GPIO_PINS_9: GPIO pin 9
GPIO_PINS_10: GPIO pin 10
GPIO_PINS_11: GPIO pin 11
GPIO_PINS_12: GPIO pin 12
GPIO_PINS_13: GPIO pin 13
GPIO_PINS_14: GPIO pin 14
GPIO_PINS_15: GPIO pin 15

gpio_out_type

Set GPIO output type.

GPIO_OUTPUT_PUSH_PULL: GPIO push-pull

GPIO_OUTPUT_OPEN_DRAIN: GPIO open drain

gpio_pull

Set GPIO pull-up or pull-down.

GPIO_PULL_NONE: No GPIO pull-up/pull-down

GPIO_PULL_UP: GPIO pull-up

GPIO_PULL_DOWN: GPIO pull-down

gpio_mode

Set GPIO mode

GPIO_MODE_INPUT: GPIO input

GPIO_MODE_OUTPUT: GPIO output

GPIO_MODE_MUX: GPIO multiplexed mode

GPIO_MODE_ANALOG: GPIO analog mode

gpio_drive_strength

Set GPIO driver capability.

GPIO_DRIVE_STRENGTH_STRONGER: Strong drive strength

GPIO_DRIVE_STRENGTH_MODERATE: Moderate drive strength

Example:

```
gpio_init_type gpio_init_struct;
gpio_init_struct.gpio_pins = GPIO_PINS_0;
gpio_init_struct.gpio_mode = GPIO_MODE_MUX;
gpio_init_struct.gpio_out_type = GPIO_OUTPUT_PUSH_PULL;
gpio_init_struct.gpio_pull = GPIO_PULL_NONE;
gpio_init_struct.gpio_drive_strength = GPIO_DRIVE_STRENGTH_STRONGER;
gpio_init(GPIOA, &gpio_init_struct);
```

5.12.4 gpio_default_para_init function

The table below describes the function gpio_default_para_init.

Table 249. gpio_default_para_init function

Name	Description
Function name	gpio_default_para_init
Function prototype	void gpio_default_para_init(gpio_init_type *gpio_init_struct);
Function description	Initialize GPIO default parameter
Input parameter	gpio_init_struct: gpio_init_type pointer
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

The table below describes the default values of members of the gpio_init_struct.

Table 250. gpio_init_struct default values

Member	Default value
gpio_pins	GPIO_PINS_ALL
gpio_mode	GPIO_MODE_INPUT
gpio_out_type	GPIO_OUTPUT_PUSH_PULL
gpio_pull	GPIO_PULL_NONE
gpio_drive_strength	GPIO_DRIVE_STRENGTH_STRONGER

Example:

```
gpio_init_type gpio_init_struct;
gpio_default_para_init(&gpio_init_struct);
```

5.12.5 gpio_input_data_bit_read function

The table below describes the function gpio_input_data_bit_read.

Table 251. gpio_input_data_bit_read function

Name	Description
Function name	gpio_input_data_bit_read
Function prototype	flag_status gpio_input_data_bit_read(gpio_type *gpio_x, uint16_t pins);
Function description	Read GPIO input port pin
Input parameter 1	gpio_x: indicates the selected GPIO peripheral This parameter can be GPIOA, GPIOB, GPIOC, GPIOD, GPIOE.
Input parameter 2	Pins: indicates the GPIO pins, refer to *gpio_pins* for details.
Output parameter	NA
Return value	Return GPIO input pin status
Required preconditions	NA
Called functions	NA

Example:

```
gpio_input_data_bit_read(GPIOA, GPIO_PINS_0);
```

5.12.6 gpio_input_data_read function

The table below describes the function gpio_input_data_read.

Table 252. gpio_input_data_read function

Name	Description
Function name	gpio_input_data_read
Function prototype	uint16_t gpio_input_data_read(gpio_type *gpio_x);
Function description	Read GPIO input port
Input parameter	gpio_x: indicates the selected GPIO peripheral. This parameter can be GPIOA, GPIOB, GPIOC, GPIOD, GPIOE
Output parameter	NA
Return value	Return GPIO input port status
Required preconditions	NA
Called functions	NA

Example:

```
gpio_input_data_read(GPIOA);
```

5.12.7 gpio_output_data_bit_read function

The table below describes the function gpio_output_data_bit_read.

Table 253. gpio_output_data_bit_read function

Name	Description
Function name	gpio_output_data_bit_read
Function prototype	uint16_t gpio_output_data_bit_read(gpio_type *gpio_x);
Function description	Read GPIO output port pin
Input parameter 1	gpio_x: indicates the selected GPIO peripheral This parameter can be GPIOA, GPIOB, GPIOC, GPIOD, GPIOE.
Input parameter 2	Pins: indicates the GPIO pins, refer to, refer to *gpio_pins* for details.
Output parameter	NA
Return value	Return GPIO output pin status
Required preconditions	NA
Called functions	NA

Example:

```
gpio_output_data_bit_read(GPIOA, GPIO_PINS_0);
```

5.12.8 gpio_output_data_read function

The table below describes the function gpio_output_data_read.

Table 254. gpio_output_data_read function

Name	Description
Function name	gpio_output_data_read
Function prototype	uint16_t gpio_output_data_read(gpio_type *gpio_x);
Function description	Read GPIO output port
Input parameter	gpio_x: indicates the selected GPIO peripheral This parameter can be GPIOA, GPIOB, GPIOC, GPIOD, GPIOE.
Output parameter	NA
Return value	Read GPIO output port status
Required preconditions	NA
Called functions	NA

Example:

```
gpio_output_data_read(GPIOA);
```

5.12.9 gpio_bits_set function

The table below describes the function gpio_bits_set.

Table 255. gpio_bits_set function

Name	Description
Function name	gpio_bits_set
Function prototype	void gpio_bits_set(gpio_type *gpio_x, uint16_t pins);
Function description	Set GPIO pins
Input parameter 1	gpio_x: indicates the selected GPIO peripheral This parameter can be GPIOA, GPIOB, GPIOC, GPIOD, GPIOE
Input parameter 2	Pins: indicates the GPIO pins, refer to *gpio_pins* for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
gpio_bits_set(GPIOA, GPIO_PINS_0);
```

5.12.10 gpio_bits_reset function

The table below describes the function gpio_bits_reset.

Table 256. gpio_bits_reset function

Name	Description
Function name	gpio_bits_reset
Function prototype	void gpio_bits_reset(gpio_type *gpio_x, uint16_t pins);
Function description	Reset GPIO pins
Input parameter 1	gpio_x: indicates the selected GPIO peripheral This parameter can be GPIOA, GPIOB, GPIOC, GPIOD, GPIOE.
Input parameter 2	Pins: indicates the GPIO pins, refer to *gpio_pins* for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
gpio_bits_reset(GPIOA, GPIO_PINS_0);
```

5.12.11 gpio_bits_toggle function

The table below describes the function gpio_bits_toggle.

Table 257. gpio_bits_toggle function

Name	Description
Function name	gpio_bits_toggle
Function prototype	void gpio_bits_toggle(gpio_type *gpio_x, uint16_t pins);
Function description	Toggle GPIO pins
Input parameter 1	gpio_x: indicates the selected GPIO peripheral This parameter can be GPIOA, GPIOB, GPIOC, GPIOD, GPIOE.
Input parameter 2	pins: indicates the GPIO pins, refer to *gpio_pins* for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example

```
gpio_bits_toggle (GPIOA, GPIO_PINS_0);
```

5.12.12 gpio_bits_write function

The table below describes the function gpio_bits_write.

Table 258. gpio_bits_write function

Name	Description
Function name	gpio_bits_write
Function prototype	void gpio_bits_write(gpio_type *gpio_x, uint16_t pins, confirm_state bit_state);
Function description	Write GPIO pins
Input parameter 1	gpio_x: indicates the selected GPIO peripheral This parameter can be GPIOA, GPIOB, GPIOC, GPIOD, GPIOE.
Input parameter 2	pins: indicates the GPIO pins, refer to *gpio_pins* for details.
Input parameter 3	bit_state: indicates GPIO pin value to write This parameter can be 1 (TRUE) or 0 (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
gpio_bits_write(GPIOA, GPIO_PINS_0, TRUE);
```

5.12.13 gpio_port_write function

The table below describes the function gpio_port_write.

Table 259. gpio_port_write function

Name	Description
Function name	gpio_port_write
Function prototype	void gpio_port_write(gpio_type *gpio_x, uint16_t port_value);
Function description	Write GPIO port
Input parameter 1	gpio_x: indicates the selected GPIO peripheral This parameter can be GPIOA, GPIOB, GPIOC, GPIOD, GPIOE
Input parameter 2	port_value: indicates the port value to write This parameter can be 0x0000~0xFFFF.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
gpio_port_write(GPIOA, 0xFFFF);
```

5.12.14 gpio_pin_wp_config function

The table below describes the function gpio_pin_wp_config.

Table 260. gpio_pin_wp_config function

Name	Description
Function name	gpio_pin_wp_config
Function prototype	void gpio_pin_wp_config(gpio_type *gpio_x, uint16_t pins);
Function description	Configure GPIO pin write protection
Input parameter 1	gpio_x: indicates the selected GPIO peripheral This parameter can be GPIOA, GPIOB, GPIOC, GPIOD, GPIOE
Input parameter 2	Pins: indicates the GPIO pins, refer to *gpio_pins* for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
gpio_pin_wp_config(GPIOA, GPIO_PINS_0);
```

5.12.15 gpio_pins_huge_driven_config function

The table below describes the function gpio_pins_huge_driven_config.

Table 261. gpio_pins_huge_driven_config function

Name	Description
Function name	gpio_pins_huge_driven_config
Function prototype	void gpio_pins_huge_driven_config(gpio_type *gpio_x, uint16_t pins, confirm_state new_state);
Function description	Configure huge drive ability of GPIO pins
Input parameter 1	gpio_x: indicates the selected GPIO peripheral This parameter can be GPIOA, GPIOB, GPIOC, GPIOD, GPIOE
Input parameter 2	Pins: refer to the *gpio_pins* for details
Input parameter 3	new_state: Enable (TRUE) or disable (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
gpio_pins_huge_driven_config(GPIOA, GPIO_PINS_0, TRUE);
```

5.12.16 gpio_event_output_config function

The table below describes the function gpio_event_output_config.

Table 262. gpio_event_output_config function

Name	Description
Function name	gpio_event_output_config
Function prototype	void gpio_event_output_config(gpio_port_source_type gpio_port_source, gpio_pins_source_type gpio_pin_source);
Function description	Configure GPIO event output
Input parameter 1	gpio_port_source: indicates a GPIO port
Input parameter 2	gpio_pin_source: indicates a GPIO pin
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

gpio_port_source

Select a GPIO port.

GPIO_PORT_SOURCE_GPIOA: GPIO port A

GPIO_PORT_SOURCE_GPIOB: GPIO port B

GPIO_PORT_SOURCE_GPIOC: GPIO port C

GPIO_PORT_SOURCE_GPIOD: GPIO port D

GPIO_PORT_SOURCE_GPIOE: GPIO port E

gpio_pin_source

Select a GPIO pin.

GPIO_PINS_SOURCE0: GPIO pin 0

GPIO_PINS_SOURCE1: GPIO pin 1
GPIO_PINS_SOURCE2: GPIO pin 2
GPIO_PINS_SOURCE3: GPIO pin 3
GPIO_PINS_SOURCE4: GPIO pin 4
GPIO_PINS_SOURCE5: GPIO pin 5
GPIO_PINS_SOURCE6: GPIO pin 6
GPIO_PINS_SOURCE7: GPIO pin 7
GPIO_PINS_SOURCE8: GPIO pin 8
GPIO_PINS_SOURCE9: GPIO pin 9
GPIO_PINS_SOURCE10: GPIO pin10
GPIO_PINS_SOURCE11: GPIO pin 11
GPIO_PINS_SOURCE12: GPIO pin 12
GPIO_PINS_SOURCE13: GPIO pin 13
GPIO_PINS_SOURCE14: GPIO pin 14
GPIO_PINS_SOURCE15: GPIO pin 15

Example:

```
gpio_event_output_config(GPIO_PORT_SOURCE_GPIOA, GPIO_PINS_SOURCE0);
```

5.12.17 gpio_event_output_enable function

The table below describes the function gpio_event_output_enable.

Table 263. gpio_event_output_enable function

Name	Description
Function name	gpio_event_output_enable
Function prototype	void gpio_event_output_enable(confirm_state new_state);
Function description	Enable/disable GPIO event output
Input parameter	new_state: indicates GPIO event output status This parameter can be Enable (TRUE) or Disable (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
gpio_event_output_enable(TRUE);
```

5.12.18 gpio_pin_remap_config function

The table below describes the function gpio_pin_remap_config.

Table 264. gpio_pin_remap_config function

Name	Description
Function name	gpio_pin_remap_config
Function prototype	void gpio_pin_remap_config(uint32_t gpio_remap, confirm_state new_state);
Function description	Configure IOMUX pin
Input parameter 1	gpio_remap: indicates IOMUX peripherals
Input parameter 2	new_state: indicates IOMUX status

Name	Description
	This parameter can be Enable (TRUE) or Disable (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

gpio_remap

Select IOMUX peripherals. It includes many parameters, to name a few here. For more information, please refer to the corresponding reference manual.

SPI1_MUX_01: spi1_cs/i2s1_ws(pa15), spi1_sck/i2s1_ck(pb3), spi1_miso(pb4), spi1_mosi/i2s1_sd(pb5), i2s1_mck(pb0)

SPI1_MUX_10: spi1_cs/i2s1_ws(pa4), spi1_sck/i2s1_ck(pa5), spi1_miso(pa6), spi1_mosi/i2s1_sd(pa7), i2s1_mck(pb6)

……

USART6_GMUX: usart6_tx(pa4), usart6_rx(pa5)

UART7_GMUX: uart7_tx(pb4), uart7_rx(pb3)

Example:

```
gpio_pin_remap_config(SPI1_MUX_01, TRUE);
```

5.12.19 gpio_exint_line_config function

The table below describes the function gpio_exint_line_config.

Table 265. gpio_exint_line_config function

Name	Description
Function name	gpio_exint_line_config
Function prototype	void gpio_exint_line_config(gpio_port_source_type gpio_port_source, gpio_pins_source_type gpio_pin_source);
Function description	Config GPIO external interrupt lines
Input parameter 1	gpio_port_source: GPIO port selection
Input parameter 2	gpio_pin_source: GPIO pin selection
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

gpio_port_source

For GPIO port selection, refer to *gpio_port_source* for details.

gpio_pin_source

For GPIO pin selection, refer to *gpio_pin_source* for details.

Example:

```
gpio_exint_line_config(GPIO_PORT_SOURCE_GPIOA, GPIO_PINS_SOURCE0);
```

5.13 I2C interfaces

The I2C register structure i2c_type is defined in the “at32f403a_407_i2c.h”:

```
/**
  * @brief type define i2c register all
  */
typedef struct
{


} i2c_type;
```

Table 255 gives a list of the I2C registers.

Table 266. Summary of I2C registera

Register	Description
ctrl1	I2C control register 1
ctrl2	I2C control register 2
oaddr1	I2C own address register 1
oaddr2	I2C own address register 2
dt	I2C data register
sts1	I2C status register 1
sts2	I2C status register 2
clkctrl	I2C clock control register
tmrise	I2C clock rise register

The table below gives a list of I2C library functions.

Table 267. Summary of I2C library functions

Function name	Description
i2c_reset	I2C peripheral reset
i2c_software_reset	I2C software reset
i2c_init	Set I2C bus speed
i2c_own_address1_set	Set I2C own address 1
i2c_own_address2_set	Set I2C own address 2
i2c_own_address2_enable	Enable I2C own address 2
i2c_smbus_enable	Enable Smbus mode
i2c_enable	Enable I2C
i2c_fast_mode_duty_set	Set fast mode duty cycle
i2c_clock_stretch_enable	Enable clock stretching capability
i2c_ack_enable	Enable ACK response
i2c_master_receive_ack_set	Set master receive mode ACK response
i2c_pec_position_set	Set PEC location in Smbus mod and master receive mode
i2c_general_call_enable	Enable general call (broadcast address enable)
i2c_arp_mode_enable	Enable SMBus ARP address
i2c_smbus_mode_set	SMBus device mode selection
i2c_smbus_alert_set	Set SMBus alert pin level

i2c_pec_transmit_enable	Enable PEC transmit
i2c_pec_calculate_enable	Enable PEC calculation
i2c_pec_value_get	Get current PEC value
i2c_dma_end_transfer_set	DMA transfer end indication
i2c_dma_enable	Enable DMA transfer
i2c_interrupt_enable	Enable I2C interrupts
i2c_start_generate	Generate Start condition
i2c_stop_generate	Generate Stop condition
i2c_7bit_address_send	Send 7-bit slave address
i2c_data_send	Send data
i2c_data_receive	Receive data
i2c_flag_get	Get flag
i2c_flag_clear	Clear flag

Table 268. I2C application-layer library functions

Function name	Description
i2c_config	I2C application initialization
i2c_lowlevel_init	I2C low-layer initialization
i2c_wait_end	I2C wait data transmit complete
i2c_wait_flag	I2C wait flag
i2c_master_transmit	I2C master transmits data (polling mode)
i2c_master_receive	I2C master receives data (polling mode)
i2c_slave_transmit	I2C slave transmits data (polling mode)
i2c_slave_receive	I2C slave receives data (polling mode)
i2c_master_transmit_int	I2C master transmits data (interrupt mode)
i2c_master_receive_int	I2C master receives data (interrupt mode)
i2c_slave_transmit_int	I2C slave transmits data (interrupt mode)
i2c_slave_receive_int	I2C slave receives data (interrupt mode)
i2c_master_transmit_dma	I2C master transmits data (DMA mode)
i2c_master_receive_dma	I2C master receives data (DMA mode)
i2c_slave_transmit_dma	I2C slave transmits data (DMA mode)
i2c_slave_receive_dma	I2C slave receives data (DMA mode)
i2c_memory_write	I2C writes data to EEPROM (polling mode)
i2c_memory_write_int	I2C writes data to EEPROM (interrupt mode)
i2c_memory_write_dma	I2C writes data to EEPROM (DMA mode)
i2c_memory_read	I2C reads from EEPROM (polling mode)
i2c_memory_read_int	I2C reads from EEPROM (interrupt mode)
i2c_memory_read_dma	I2C reads from EEPROM (DMA mode)
i2c_evt_irq_handler	I2C event interrupt function
i2c_err_irq_handler	I2C error interrupt function
i2c_dma_tx_irq_handler	I2C DMA Tx interrupt function
i2c_dma_rx_irq_handler	I2C DMA Rx interrupt function

5.13.1 i2c_reset function

The table below describes the function i2c_reset.

Table 269. i2c_reset function

Name	Description
Function name	i2c_reset
Function prototype	void i2c_reset(i2c_type *i2c_x)
Function description	Reset all I2C registers to their initial values through CRM (Clock and reset management)
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	void crm_periph_reset(crm_periph_reset_type value, confirm_state new_state);

Example:

```
i2c_reset(I2C1);
```

5.13.2 i2c_software_reset function

The table below describes the function i2c_software_reset.

Table 270. i2c_software_reset function

Name	Description
Function name	i2c_software_reset
Function prototype	void i2c_software_reset(i2c_type *i2c_x, confirm_state new_state);
Function description	Reset I2C by software, in the same way of the function i2c_reset(i2c_type *i2c_x)
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3
Input parameter 2	new_state: indicates software reset status This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
i2c_software_reset(I2C1, TRUE);
i2c_software_reset(I2C1, FALSE);
```

5.13.3 i2c_init function

The table below describes the function i2c_init.

Table 271. i2c_init function

Name	Description
Function name	i2c_init
Function prototype	void i2c_init(i2c_type *i2c_x, i2c_fsmode_duty_cycle_type duty, uint32_t speed);
Function description	Set I2C bus speed and duty cycle in fast mode
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3
Input parameter 2	Duty: SCL bus duty cycle in fast mode Refer to the “duty” description below for details.
Input parameter 3	Speed: bus speed in Hz
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

duty

SCL duty cycle in fast mode (≥400kHz)

I2C_FSMODE_DUTY_2_1: SCL duty cycle is 2: 1

I2C_FSMODE_DUTY_16_9: SCL duty cycle is 16: 9

Example:

```
i2c_init(I2C1, I2C_FSMODE_DUTY_2_1, 100000);
```

5.13.4 i2c_own_address1_set function

The table below describes the function i2c_own_address1_set.

Table 272. i2c_own_address1_set function

Name	Description
Function name	i2c_own_address1_set
Function prototype	void i2c_own_address1_set(i2c_type *i2c_x, i2c_address_mode_type mode, uint16_t address);
Function description	Set own address 1
Input parameter 1	Mode: Own address 1 address mode Refer to the “mode” description below for details.
Input parameter 2	Address: own address 1
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

mode

Own address 1 address mode

I2C_ADDRESS_MODE_7BIT: 7-bit address

I2C_ADDRESS_MODE_10BIT: 10-bit address

Example:

```
i2c_own_address1_set(I2C1, I2C_ADDRESS_MODE_7BIT, 0xA0);
```

5.13.5 i2c_own_address2_set function

The table below describes the function i2c_own_address2_set.

Table 273. i2c_own_address2_set function

Name	Description
Function name	i2c_own_address2_set
Function prototype	void i2c_own_address2_set(i2c_type *i2c_x, uint8_t address);
Function description	Set own address 2. The address 2 becomes active only after it is enabled. Note: only 7-bit address is supported, not 10-bit address mode
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Input parameter 2	Address: own address 2
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
i2c_own_address2_set(I2C1, 0xB0);
```

5.13.6 i2c_own_address2_enable function

The table below describes the function i2c_own_address2_enable.

Table 274. i2c_own_address2_enable function

Name	Description
Function name	i2c_own_address2_enable
Function prototype	void i2c_own_address2_enable(i2c_type *i2c_x, confirm_state new_state);
Function description	Enable own address 2. The address becomes active only after it is enabled. Note that this function should be used in conjunction with the i2c_own_address2_set.
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Input parameter 2	new_state: indicates address 2 status This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
i2c_own_address2_enable(I2C1, TRUE);
```

5.13.7 i2c_smbus_enable function

The table below describes the function i2c_smbus_enable.

Table 275. i2c_smbus_enable function

Name	Description
Function name	i2c_smbus_enable
Function prototype	void i2c_smbus_enable(i2c_type *i2c_x, confirm_state new_state);
Function description	Enable SMBus mode. After power-on reset, it is I2C mode by default.
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Input parameter 2	new_state: indicates SMBus mode status This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
i2c_smbus_enable(I2C1, TRUE);
```

5.13.8 i2c_enable function

The table below describes the function i2c_enable.

Table 276. i2c_enable function

Name	Description
Function name	i2c_enable
Function prototype	void i2c_enable(i2c_type *i2c_x, confirm_state new_state);
Function description	Enable I2C peripheral
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Input parameter 2	new_state: indicates I2C status This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
i2c_enable(I2C1, TRUE);
```

5.13.9 i2c_fast_mode_duty_set function

The table below describes the function i2c_fast_mode_duty_set.

Table 277. i2c_fast_mode_duty_set function

Name	Description
Function name	i2c_fast_mode_duty_set
Function prototype	void i2c_fast_mode_duty_set(i2c_type *i2c_x, i2c_fsmode_duty_cycle_type duty);
Function description	Configure the ratio of SCL low level to high level in fast mode. This function works in the same way of the duty parameter in the void i2c_init(i2c_type *i2c_x, i2c_fsmode_duty_cycle_type duty, uint32_t speed).
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Input parameter 2	Duty: indicates the duty cycle of SCL in fast mode Refer to the “duty” description below for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

duty

SCL duty cycle in fast mode (≥400kHz).

I2C_FSMODE_DUTY_2_1: SCL duty cycle is 2: 1

I2C_FSMODE_DUTY_16_9: SCL duty cycle is 16: 9

Example:

```
i2c_fast_mode_duty_set(I2C1, I2C_FSMODE_DUTY_2_1);
```

5.13.10 i2c_clock_stretch_enable function

The table below describes the function i2c_clock_stretch_enable.

Table 278. i2c_clock_stretch_enable function

Name	Description
Function name	i2c_clock_stretch_enable
Function prototype	void i2c_clock_stretch_enable(i2c_type *i2c_x, confirm_state new_state);
Function description	Enable clock stretching capability. This function is applicable to slave mode only. In most cases, enabling the clock stretching mode is recommended in order to prevent slave from having no sufficient time to receive or send data due to slow process speed, which causes a loss of data. It should be noted that the host must be able to support clock stretching function before using this mode by slave. For example, some hosts based on IO analog are not equipped with the clock stretching capability.
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Input parameter 2	new_state: indicates clock stretching status This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
i2c_clock_stretch_enable(I2C1, TRUE);
```

5.13.11 i2c_ack_enable function

The table below describes the function i2c_ack_enable.

Table 279. i2c_ack_enable function

Name	Description
Function name	i2c_ack_enable
Function prototype	void i2c_ack_enable(i2c_type *i2c_x, confirm_state new_state);
Function description	Enable ACK and NACK. This function is used to enable ACK or NACK of each byte in master and slave mode. For ACK information on I2C communication protocol, refer to I2C protocol or AT32 reference manual.
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Input parameter 2	new_state: indictes ACK response status This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
i2c_ack_enable(I2C1, TRUE);
```

5.13.12 i2c_master_receive_ack_set function

The table below describes the function i2c_master_receive_ack_set.

Table 280. i2c_master_receive_ack_set function

Name	Description
Function name	i2c_master_receive_ack_set
Function prototype	void i2c_master_receive_ack_set(i2c_type *i2c_x, i2c_master_ack_type pos)
Function description	Enable master receive ACK response. This function is used in master receive mode to define the location where the function void i2c_ack_enable(i2c_type *i2c_x, confirm_state new_state) becomes active. The function is aimed at returing a correct NACK reponse while two bytes are received in master receive mode.
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Input parameter 2	Pos: indicates the location of ACKEN Refer to the “pos” description below fore details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

pos

ACKEN valid position.

I2C_MASTER_ACK_CURRENT: ACKEN bit effective on the current byte being transfered

I2C_MASTER_ACK_NEXT: ACKEN bit effective on the next byte to be transferred

Example:

```
i2c_master_receive_ack_set(I2C1, TRUE);
```

5.13.13 i2c_pec_position_set function

The table below describes the function i2c_pec_position_set.

Table 281. i2c_pec_position_set function

Name	Description
Function name	i2c_pec_position_set
Function prototype	void i2c_pec_position_set(i2c_type *i2c_x, i2c_pec_position_type pos);
Function description	Set PEC location in SMBus and master receive mode. This function is used to receive PEC and return NACK when two bytes are received in master receive mode.
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Input parameter 2	Pos: PEC position Refer to the “pos” description below fore details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

pos

PEC valid position.

I2C_PEC_POSITION_CURRENT: Current byte is PEC

I2C_PEC_POSITION_NEXT: Next byte is PEC

Example:

```
i2c_pec_position_set(I2C1, I2C_PEC_POSITION_CURRENT);
```

5.13.14 i2c_general_call_enable function

The table below describes the function i2c_general_call_enable.

Table 282. i2c_general_call_enable function

Name	Description
Function name	i2c_general_call_enable
Function prototype	void i2c_general_call_enable(i2c_type *i2c_x, confirm_state new_state);
Function description	Enable general call. After enabled, address 0x00 is ACKed or NACKed.
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Input parameter 2	new_state: indicates general call status This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
i2c_general_call_enable(I2C1, TRUE);
```

5.13.15 i2c_arp_mode_enable function

The table below describes the function i2c_arp_mode_enable.

Table 283. i2c_arp_mode_enable function

Name	Description
Function name	i2c_arp_mode_enable
Function prototype	void i2c_arp_mode_enable(i2c_type *i2c_x, confirm_state new_state);
Function description	Enable SMBus ARP In SMBus master mode: respond to master address 0001000x In SMBus device mode: respond to device default address 0001100x For more information on ARP, refer to SMBUS protocol.
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Input parameter 2	new_state: indicates ARP address status This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
i2c_arp_mode_enable(I2C1, TRUE);
```

5.13.16 i2c_smbus_mode_set function

The table below describes the function i2c_smbus_mode_set.

Table 284. i2c_smbus_mode_set function

Name	Description
Function name	i2c_smbus_mode_set
Function prototype	void i2c_smbus_mode_set(i2c_type *i2c_x, i2c_smbus_mode_set_type mode);
Function description	Select SMBus device mode, including SMBus host or SMBus device
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Input parameter 2	Mode: SMBus device mode Refer to the “mode” description below for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

mode

SMBus device mode.

I2C_SMBUS_MODE_DEVICE: SMBus device

I2C_SMBUS_MODE_HOST: SMBus host

Example:

```
i2c_smbus_mode_set(I2C1, I2C_SMBUS_MODE_HOST);
```

5.13.17 i2c_smbus_alert_set function

The table below describes the function i2c_smbus_alert_set.

Table 285. i2c_smbus_alert_set function

Name	Description
Function name	i2c_smbus_alert_set
Function prototype	void i2c_smbus_alert_set(i2c_type *i2c_x, i2c_smbus_alert_set_type level);
Function description	Set SMBus alert pin low or high
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Input parameter 2	Level: SMBus alert pin level Refer to the “level” description below for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

level

SMBus alert pin level

I2C_SMBUS_ALERT_LOW: SMBus alert pin low

I2C_SMBUS_ALERT_HIGH: SMBus alert pin high

Example:

```
i2c_smbus_alert_set(I2C1, I2C_SMBUS_ALERT_LOW);
```

5.13.18 i2c_pec_transmit_enable function

The table below describes the function i2c_pec_transmit_enable.

Table 286. i2c_pec_transmit_enable function

Name	Description
Function name	i2c_pec_transmit_enable
Function prototype	void i2c_pec_transmit_enable(i2c_type *i2c_x, confirm_state new_state);
Function description	Enable PEC transmit (send or receive PEC). After calling this function, the PEC is ready to send or receive.
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Input parameter 2	new_state: PEC transfer status This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
i2c_pec_transmit_enable(I2C1, TRUE);
```

5.13.19 i2c_pec_calculate_enable function

The table below describes the function i2c_pec_calculate_enable.

Table 287. i2c_pec_calculate_enable function

Name	Description
Function name	i2c_pec_calculate_enable
Function prototype	void i2c_pec_calculate_enable(i2c_type *i2c_x, confirm_state new_state);
Function description	Enable PEC calculation
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Input parameter 2	new_state: PEC calculation status This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
i2c_pec_calculate_enable(I2C1, TRUE);
```

5.13.20 i2c_pec_value_get function

The table below describes the function i2c_pec_value_get.

Table 288. i2c_pec_value_get function

Name	Description
Function name	i2c_pec_value_get
Function prototype	uint8_t i2c_pec_value_get(i2c_type *i2c_x);
Function description	Get PEC value
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Output parameter	uint8_t: current PEC value
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
Pec_value = i2c_pec_value_get(I2C1);
```

5.13.21 i2c_dma_end_transfer_set function

The table below describes the function i2c_dma_end_transfer_set.

Table 289. i2c_dma_end_transfer_set function

Name	Description
Function name	i2c_dma_end_transfer_set
Function prototype	void i2c_dma_end_transfer_set(i2c_type *i2c_x, confirm_state new_state);
Function description	Indicates DMA transfer complete, that is, indicating whether the last data is being sent or not.
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Input parameter 2	new_state: indicates whether it is the last data being transferred or not. This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
i2c_dma_end_transfer_set(I2C1, TRUE);
```

5.13.22 i2c_dma_enable function

The table below describes the function i2c_dma_enable.

Table 290. i2c_dma_enable function

Name	Description
Function name	i2c_dma_enable
Function prototype	void i2c_dma_enable(i2c_type *i2c_x, confirm_state new_state);
Function description	Enable DMA transfer
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Input parameter 2	new_state: DMA status This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
i2c_dma_enable(I2C1, TRUE);
```

5.13.23 i2c_interrupt_enable function

The table below describes the function i2c_interrupt_enable.

Table 291. i2c_interrupt_enable function

Name	Description
Function name	i2c_interrupt_enable
Function prototype	void i2c_interrupt_enable(i2c_type *i2c_x, uint16_t source, confirm_state new_state)
Function description	Enable I2C interrupts
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Input parameter 2	Source: Interrupt source Refer to the “source” description below for details.
Input parameter 3	new_state: interrupt status This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

source

Interrupt sources.

I2C_DATA_INT: Data interrupt

I2C_EV_INT: Event interrupt

I2C_ERR_INT: Error interrupt

Example:

```
i2c_interrupt_enable(I2C1, I2C_DATA_INT, TRUE);
```

5.13.24 i2c_start_generate function

The table below describes the function i2c_start_generate.

Table 292. i2c_start_generate function

Name	Description
Function name	i2c_start_generate
Function prototype	void i2c_start_generate(i2c_type *i2c_x);
Function description	Generate Start condition (for host)
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
i2c_start_generate(I2C1);
```

5.13.25 i2c_stop_generate function

The table below describes the function i2c_stop_generate.

Table 293. i2c_stop_generate function

Name	Description
Function name	i2c_stop_generate
Function prototype	void i2c_stop_generate(i2c_type *i2c_x);
Function description	Generate Stop condition
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
i2c_stop_generate(I2C1);
```

5.13.26 i2c_7bit_address_send function

The table below describes the function i2c_7bit_address_send.

Table 294. i2c_7bit_address_send function

Name	Description
Function name	i2c_7bit_address_send
Function prototype	void i2c_7bit_address_send(i2c_type *i2c_x, uint8_t address, i2c_direction_type direction);
Function description	Send 7-bit slave address (for host)
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Input parameter 2	Address: slave address
Input parameter 3	Direction: data transfer direction Refer to the “direction” description below for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

direction

Data transfer direction.

I2C_DIRECTION_TRANSMIT: Master transmit

I2C_DIRECTION_RECEIVE: Master receive

Example:

```
i2c_7bit_address_send(I2C1, 0xB0, I2C_DIRECTION_TRANSMIT);
```

5.13.27 i2c_data_send function

The table below describes the function i2c_data_send.

Table 295. i2c_data_send function

Name	Description
Function name	i2c_data_send
Function prototype	void i2c_data_send(i2c_type *i2c_x, uint8_t data);
Function description	Send data
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Input parameter 2	Data: send data
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
i2c_data_send(I2C1, 0x55);
```

5.13.28 i2c_data_receive function

The table below describes the function i2c_data_receive.

Table 296. i2c_data_receive function

Name	Description
Function name	i2c_data_receive
Function prototype	uint8_t i2c_data_receive(i2c_type *i2c_x);
Function description	Receive data
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Output parameter	uint8_t: receive data
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
data_value = i2c_data_receive(I2C1);
```

5.13.29 i2c_flag_get function

The table below describes the function i2c_flag_get.

Table 297. i2c_flag_get function

Name	Description
Function name	i2c_flag_get
Function prototype	flag_status i2c_flag_get(i2c_type *i2c_x, uint32_t flag);
Function description	Get flag status
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Input parameter 2	Flag: Flag selection Refer to the “flag” description below for details.
Output parameter	NA
Return value	flag_status: indicates Flag status Returns SET or RESET
Required preconditions	NA
Called functions	NA

flag

This is used for flag selection, including:

I2C_STARTF_FLAG: Start condition generation complete flag
I2C_ADDR7F_FLAG: 0~7 bit address match flag
I2C_TDC_FLAG: Data transfer complete flag
I2C_ADDRHF_FLAG: 9~8 bit address head match flag (host)
I2C_STOPF_FLAG: Stop condition generation complete flag
I2C_RDBF_FLAG: Receive data buffer full flag
I2C_TDBE_FLAG: Transmit data buffer empty flag
I2C_BUSERR_FLAG: Bus error flag
I2C_ARLOST_FLAG: Arbitration lost flag

I2C_ACKFAIL_FLAG: Acknowledge failure flag
I2C_OUF_FLAG: Overflow flag
I2C_PECERR_FLAG: PEC receive error flag
I2C_TMOUT_FLAG: SMBus timeout flag
I2C_ALERTF_FLAG: SMBus alert flag
I2C_TRMODE_FLAG: Transfer mode
I2C_BUSYF_FLAG: Bus busy flag
I2C_DIRF_FLAG: Transmission direction flag
I2C_GCADDRF_FLAG: General call address reception flag
I2C_DEVADDRF_FLAG: SMBus device address reception flag
I2C_HOSTADDRF_FLAG: SMBus host address reception flag
I2C_ADDR2_FLAG: Received address 2 flag

Example:

```
i2c_flag_get(I2C1, I2C_STARTF_FLAG);
```

5.13.30 i2c_interrupt_flag_get function

The table below describes the function i2c_interrupt_flag_get.

Table 298. i2c_interrupt_flag_get function

Name	Description
Function name	i2c_interrupt_flag_get
Function prototype	flag_status i2c_interrupt_flag_get(i2c_type *i2c_x, uint32_t flag);
Function description	Get flag status and judge the corresponding interrupt enable bit
Input parameter 1	i2c_x indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Input parameter 2	flag: flag selection Refer to the “flag” description below for details.
Output parameter	NA
Return value	flag_status: indicates Flag status Returns SET or RESET
Required preconditions	NA
Called functions	NA

flag

This is used for flag selection, including:

I2C_STARTF_FLAG: Start condition generation complete flag
I2C_ADDR7F_FLAG: 0~7 bit address match flag
I2C_TDC_FLAG: Data transfer complete flag
I2C_ADDRHF_FLAG: 9~8 bit address head match flag (host)
I2C_STOPF_FLAG: Stop condition generation complete flag
I2C_RDBF_FLAG: Receive data buffer full flag
I2C_TDBE_FLAG: Transmit data buffer full flag
I2C_BUSERR_FLAG: Bus error flag
I2C_ARLOST_FLAG: Arbitration lost flag
I2C_ACKFAIL_FLAG: Acknowledge failure flag
I2C_OUF_FLAG: Overflow flag
I2C_PECERR_FLAG: PEC receive error flag
I2C_TMOUT_FLAG: SMBus timeout flag

I2C_ALERTF_FLAG: SMBus alert flag

Example

```
i2c_interrupt_flag_get(I2C1, I2C_STARTF_FLAG);
```

5.13.31 i2c_flag_clear function

The table below describes the function i2c_flag_clear.

Table 299. i2c_flag_clear function

Name	Description
Function name	i2c_flag_clear
Function prototype	void i2c_flag_clear(i2c_type *i2c_x, uint32_t flag);
Function description	Clear fag
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Input parameter 2	Flag: indicates the selected flag Refer to the “flag” description below for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

flag

This is used for flag selection, including:

I2C_BUSERR_FLAG: Bus error flag
I2C_ARLOST_FLAG: Arbitration lost flag
I2C_ACKFAIL_FLAG: Acknowledge failure flag
I2C_OUF_FLAG: Overflow flag
I2C_PECERR_FLAG: PEC receive error flag
I2C_TMOUT_FLAG: SMBus timeout flag
I2C_ALERTF_FLAG: SMBus alert flag
I2C_ADDR7F_FLAG: 0~7 bit address match flag
I2C_STOPF_FLAG: Stop condition generation complete flag

Example:

```
i2c_flag_clear(I2C1, I2C_ACKFAIL_FLAG);
```

5.13.32 i2c_config function

The table below describes the function i2c_config.

Table 300. i2c_config function

Name	Description
Function name	i2c_config
Function prototype	void i2c_config(i2c_handle_type* hi2c);
Function description	Initialize I2C. The function i2c_lowlevel_init() is called to initialize I2C, GPIO, DMA, interrupts, and so on.
Input parameter 1	hi2c: i2c_handle_type pointer Refer to *i2c_handle_type* for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	void i2c_lowlevel_init(i2c_handle_type* hi2c);

i2c_handle_type* hi2c

The i2c_handle_type is defined in the i2c_application.h:

```
typedef struct
{
    i2c_type              *i2cx;
    uint8_t               *pbuff;
    __IO uint16_t         pcount;
    __IO uint32_t         mode;
    __IO uint32_t         timeout;
    __IO uint32_t         status;
    __IO i2c_status_type  error_code;
    dma_channel_type      *dma_tx_channel;
    dma_channel_type      *dma_rx_channel;
    dma_init_type         dma_init_struct;
}i2c_handle_type;
```

i2cx

Indicates the selected I2C peripheral, and this parameter can be I2C1 or I2C2.

pbuff

array of receive/send data

pcount

number of receive/send data

mode

I2C communication mode for interal state machine, users don't need to care about

timeout

communication timeout

status

Transfer status for interal state machine, users don't need to care about

error_code

Enumerate error code of the i2c_status_type. It records the erroneous code when an error occurs during communication.

I2C_OK:	Communication OK
I2C_ERR_STEP_1:	Step 1 error
I2C_ERR_STEP_2:	Step 2 error
I2C_ERR_STEP_3:	Step 3 error
I2C_ERR_STEP_4:	Step 4 error
I2C_ERR_STEP_5:	Step 5 error
I2C_ERR_STEP_6:	Step 6 error
I2C_ERR_STEP_7:	Step 7 error
I2C_ERR_STEP_8:	Step 8 error
I2C_ERR_STEP_9:	Step 9 error
I2C_ERR_STEP_10:	Step 10 error
I2C_ERR_STEP_11:	Step 11 error
I2C_ERR_STEP_12:	Step 12 error
I2C_ERR_START:	START condition error
I2C_ERR_ADDR10:	10-bit address header (bit9~8) error
I2C_ERR_ADDR:	Address error
I2C_ERR_STOP:	STOP condition error
I2C_ERR_ACKFAIL:	Acknowledge error
I2C_ERR_TIMEOUT:	Timeout
I2C_ERR_INTERRUPT:	Error event occurred and error interrupt entry

dma_tx_channel

I2C Tx DMA channel

dma_rx_channel

I2C Rx DMA channel

dma_init_struct

DMA initialization structure

Example:

```
i2c_handle_type hi2c;
hi2c.i2cx = I2C1;
i2c_config(&hi2c);
```

5.13.33 i2c_lowlevel_init function

The table below describes the function i2c_lowlevel_init.

Table 301. i2c_lowlevel_init function

Name	Description
Function name	i2c_lowlevel_init
Function prototype	void i2c_lowlevel_init(i2c_handle_type* hi2c);
Function description	I2C low-layer initialization function. It is used in the function i2c_config to initialize I2C, GPIO, DMA, interrupts and so on. The I2C initialization needs to be done in the function.
Input parameter 1	hi2c: i2c_handle_type pointer Refer to *i2c_handle_type* for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
void i2c_lowlevel_init(i2c_handle_type* hi2c)
{
  if(hi2c->i2cx == I2C1)
  {
    Initialize I2C1
  }
  else if(hi2c->i2cx == I2C2)
  {
    Initialize I2C2
  }
}
```

5.13.34 i2c_wait_end function

The table below describes the function i2c_wait_end.

Table 302. i2c_wait_end function

Name	Description
Function name	i2c_wait_end
Function prototype	i2c_status_type i2c_wait_end(i2c_handle_type* hi2c, uint32_t timeout);
Function description	Wait for the completion of communication. This function is used in DMA and interrupt transfer mode as both of them are non-blocking and can be used to wait for the end of transfer.
Input parameter 1	hi2c: i2c_handle_type pointer Refer to *i2c_handle_type* for details.
Input parameter 2	Timeout: wait timeout
Output parameter	NA
Return value	i2c_status_type: error code Refer to *section 5.13.32* for details.
Required preconditions	NA
Called functions	NA

Example:

```
if (i2c_master_transmit_dma(&hi2c, 0xB0, tx_buf, 8, 0xFFFFFFFF) != I2C_OK)
{
  error_handler(i2c_status);
}

/* Wait for the end of communication */
if(i2c_wait_end(&hi2c, 0xFFFFFFFF) != I2C_OK)
{
  error_handler(i2c_status);
}
```

5.13.35 i2c_wait_flag function

The table below describes the function i2c_wait_flag.

Table 303. i2c_wait_flag function

Name	Description
Function name	i2c_wait_flag
Function prototype	i2c_status_type i2c_wait_flag(i2c_handle_type* hi2c, uint32_t flag, uint32_t event_check, uint32_t timeout)
Function description	Wait for the flag to set or reset Users need to wait for BUSYF flag to reset, and other flags except for BUSYF to set.
Input parameter 1	hi2c: i2c_handle_type pointer Refer to *section 5.13.32* for details.
Input parameter 2	hi2c: i2c_handle_type pointer Refer to *i2c_handle_type* for details.
Input parameter 3	event_check: check whether an event is generated or not while waiting for flags Refer to the “event_check” description below for details.
Input parameter 4	Timeout: wait timeout
Output parameter	NA
Return value	i2c_status_type: error code Refer to *section 5.13.32* for details.
Required preconditions	NA
Called functions	NA

flag

Flag selection.

I2C_STARTF_FLAG: Start condition generation complete flag
I2C_ADDR7F_FLAG: 0~7 bit address match flag
I2C_TDC_FLAG: Data transfer complete flag
I2C_ADDRHF_FLAG: 9~8 bit address head match flag (host)
I2C_STOPF_FLAG: Stop condition generation complete flag
I2C_RDBF_FLAG: Receive data buffer full flag
I2C_TDBE_FLAG: Transmit data buffer empty flag
I2C_BUSERR_FLAG: Bus error flag
I2C_ARLOST_FLAG: Arbitration lost flag
I2C_ACKFAIL_FLAG: Acknowledge failure flag
I2C_OUF_FLAG: Overflow flag
I2C_PECERR_FLAG: PEC receive error flag
I2C_TMOUT_FLAG: SMBus timeout flag
I2C_ALERTF_FLAG: SMBus alert flag
I2C_TRMODE_FLAG: Transfer mode
I2C_BUSYF_FLAG: Bus busy flag
I2C_DIRF_FLAG: Transmission direction flag
I2C_GCADDRF_FLAG: General call address reception flag
I2C_DEVADDRF_FLAG: SMBus device address reception flag
I2C_HOSTADDRF_FLAG: SMBus host address reception flag
I2C_ADDR2_FLAG: Received address 2 flag

event_check

While waitng for a flag, check if an event occurred or not.

I2C_EVENT_CHECK_NONE: None event check

I2C_EVENT_CHECK_ACKFAIL: ACKFAIL event check

I2C_EVENT_CHECK_STOP: STOP event check

Example:

```
i2c_wait_flag(&hi2c, I2C_BUSYF_FLAG, I2C_EVENT_CHECK_NONE, 0xFFFFFFFF);
```

5.13.36 i2c_master_transmit function

The table below describes the function i2c_master_transmit.

Table 304. i2c_master_transmit function

Name	Description
Function name	i2c_master_transmit
Function prototype	i2c_status_type i2c_master_transmit(i2c_handle_type* hi2c, uint16_t address, uint8_t* pdata, uint16_t size, uint32_t timeout);
Function description	Master transmits data (polling mode). This function is in blocking mode, which means that I2C transfer comes to end after the execution of this function.
Input parameter 1	hi2c: i2c_handle_type pointer Refer to *i2c_handle_type* for details.
Input parameter 2	Address: slave address
Input parameter 3	Pdata: array address of data to send
Input parameter 4	Size: number of data transmit
Input parameter 5	Timeout: wait timeout
Output parameter	NA
Return value	i2c_status_type: error code Refer to *section 5.13.32* for details.
Required preconditions	NA
Called functions	NA

Example:

```
i2c_master_transmit(&hi2c, 0xB0, tx_buf, 8, 0xFFFFFFFF);
```

5.13.37 i2c_master_receive function

The table below describes the function i2c_master_receive.

Table 305. i2c_master_receive function

Name	Description
Function name	i2c_master_receive
Function prototype	i2c_status_type i2c_master_receive(i2c_handle_type* hi2c, uint16_t address, uint8_t* pdata, uint16_t size, uint32_t timeout);
Function description	Master receives data (polling mode). This function is in blocking mode, which means that I2C transfer comes to end after the execution of this function
Input parameter 1	hi2c: i2c_handle_type pointer Refer to *i2c_handle_type* for details.
Input parameter 2	address: slave address
Input parameter 3	pdata: array address of data to send
Input parameter 4	Size: number of data received
Input parameter 5	timeout: wait timeout
Output parameter	NA
Return value	i2c_status_type: error code Refer to *section 5.13.32* for details.
Required preconditions	NA
Called functions	NA

Example:

```
i2c_master_receive(&hi2c, 0xB0, rx_buf, 8, 0xFFFFFFFF);
```

5.13.38 i2c_slave_transmit function

The table below describes the function i2c_slave_transmit.

Table 306. i2c_slave_transmit function

Name	Description
Function name	i2c_slave_transmit
Function prototype	i2c_status_type i2c_slave_transmit(i2c_handle_type* hi2c, uint8_t* pdata, uint16_t size, uint32_t timeout);
Function description	Slave sends data (polling mode). This function is in blocking mode, which means that I2C transfer comes to end after the execution of this function
Input parameter 1	hi2c: i2c_handle_type pointer Refer to *i2c_handle_type* for details.
Input parameter 2	pdata: array address of data to send
Input parameter 3	size: number of data transmit
Input parameter 4	timeout: wait timeout
Output parameter	NA
Return value	i2c_status_type: error code Refer to *section 5.13.32* for details.
Required preconditions	NA
Called functions	NA

Example:

```
i2c_slave_transmit(&hi2c, tx_buf, 8, 0xFFFFFFFF);
```

5.13.39 i2c_slave_receive function

The table below describes the function i2c_slave_receive.

Table 307. i2c_slave_receive function

Name	Description
Function name	i2c_slave_receive
Function prototype	i2c_status_type i2c_slave_receive(i2c_handle_type* hi2c, uint8_t* pdata, uint16_t size, uint32_t timeout);
Function description	Slave receives data (polling mode). This function is in blocking mode, which means that I2C transfer comes to end after the execution of this function
Input parameter 1	hi2c: i2c_handle_type pointer Refer to *i2c_handle_type* for details.
Input parameter 2	pdata: array address of data to send
Input parameter 3	Size: number of data received
Input parameter 4	timeout: wait timeout
Output parameter	NA
Return value	i2c_status_type: error code Refer to *section 5.13.32* for details.
Required preconditions	NA
Called functions	NA

Example:

```
i2c_slave_receive(&hi2c, rx_buf, 8, 0xFFFFFFFF);
```

5.13.40 i2c_master_transmit_int function

The table below describes the function i2c_master_transmit_int.

Table 308. i2c_master_transmit_int function

Name	Description
Function name	i2c_master_transmit_int
Function prototype	i2c_status_type i2c_master_transmit_int(i2c_handle_type* hi2c, uint16_t address, uint8_t* pdata, uint16_t size, uint32_t timeout);
Function description	Master sends data (interrupt mode). This function is non-blocking, which means that I2C communication has yet to complete after the execution of this function. It is possible to wait for the end of communication by calling the function i2c_wait_end().
Input parameter 1	hi2c: i2c_handle_type pointer. Refer to *i2c_handle_type* for details.
Input parameter 2	Address: slave address
Input parameter 3	pdata: array address of data to send
Input parameter 4	Size: number of data transmit
Input parameter 5	timeout: wait timeout
Output parameter	NA
Return value	i2c_status_type: error code. Refer to *section 5.13.32* for details.
Required preconditions	NA

Name	Description
Called functions	NA

Example:

```
i2c_master_transmit_int(&hi2c, 0xB0, tx_buf, 8, 0xFFFFFFFF);
```

5.13.41 i2c_master_receive_int function

The table below describes the function i2c_master_receive_int.

Table 309. i2c_master_receive_int function

Name	Description
Function name	i2c_master_receive_int
Function prototype	i2c_status_type i2c_master_receive_int(i2c_handle_type* hi2c, uint16_t address, uint8_t* pdata, uint16_t size, uint32_t timeout);
Function description	Master receives data (interrupt mode). This function is non-blocking, which means that I2C communication has yet to complete after the execution of this function. It is possible to wait for the end of communication by calling the function i2c_wait_end().
Input parameter 1	hi2c: i2c_handle_type pointer Refer to *i2c_handle_type* for details.
Input parameter 2	address: slave address
Input parameter 3	Pdata: array address of data to receive
Input parameter 4	Size: number of data received
Input parameter 5	timeout: wait timeout
Output parameter	NA
Return value	i2c_status_type: error code Refer to *section 5.13.32* for details.
Required preconditions	NA
Called functions	NA

Example:

```
i2c_master_receive_int(&hi2c, 0xB0, rx_buf, 8, 0xFFFFFFFF);
```

5.13.42 i2c_slave_transmit_int function

The table below describes the function i2c_slave_transmit_int.

Table 310. 2c_slave_transmit_int function

Name	Description
Function name	i2c_slave_transmit_int
Function prototype	i2c_status_type i2c_slave_transmit_int(i2c_handle_type* hi2c, uint8_t* pdata, uint16_t size, uint32_t timeout);
Function description	Slave sends data (interrupt mode). This function is non-blocking, which means that I2C communication has yet to complete after the execution of this function. It is possible to wait for the end of communication by calling the function i2c_wait_end().
Input parameter 1	hi2c: i2c_handle_type pointer . Refer to *i2c_handle_type* for details.
Input parameter 2	Pdata: array address of data to send
Input parameter 3	size: number of data transmit
Input parameter 4	timeout: wait timeout
Output parameter	NA
Return value	i2c_status_type: error code. Refer to *section 5.13.32* for details.
Required preconditions	NA
Called functions	NA

Example:

```
i2c_slave_transmit_int(&hi2c, tx_buf, 8, 0xFFFFFFFF);
```

5.13.43 i2c_slave_receive_int function

The table below describes the function i2c_slave_receive_int.

Table 311. i2c_slave_receive_int function

Name	Description
Function name	i2c_slave_receive_int
Function prototype	i2c_status_type i2c_slave_receive_int(i2c_handle_type* hi2c, uint8_t* pdata, uint16_t size, uint32_t timeout);
Function description	Slave receives data (interrupt mode). This function is non-blocking, which means that I2C communication has yet to complete after the execution of this function. It is possible to wait for the end of communication by calling the function i2c_wait_end().
Input parameter 1	hi2c: i2c_handle_type pointer. Refer to *i2c_handle_type* for details.
Input parameter 2	Pdata: array address of data to receive
Input parameter 3	Size: number of data to receive
Input parameter 4	timeout: wait timeout
Output parameter	NA
Return value	i2c_status_type: error code. Refer to *section 5.13.32* for details.
Required preconditions	NA
Called functions	NA

Example:

```
i2c_slave_receive_int(&hi2c, rx_buf, 8, 0xFFFFFFFF);
```

5.13.44 i2c_master_transmit_dma function

The table below describes the function i2c_master_transmit_dma.

Table 312. i2c_master_transmit_dma function

Name	Description
Function name	i2c_master_transmit_dma
Function prototype	i2c_status_type i2c_master_transmit_dma(i2c_handle_type* hi2c, uint16_t address, uint8_t* pdata, uint16_t size, uint32_t timeout);
Function description	Master sends data (DMA mode). This function is non-blocking, which means that I2C communication has yet to complete after the execution of this function. It is possible to wait for the end of communication by calling the function i2c_wait_end().
Input parameter 1	hi2c: i2c_handle_type pointer Refer to *i2c_handle_type* for details.
Input parameter 2	Address: slave address
Input parameter 3	Pdata: array address of data to send
Input parameter 4	Size: number of data to send
Input parameter 5	timeout: wait timeout
Output parameter	NA
Return value	i2c_status_type: error code Refer to *section 5.13.32* for details.
Required preconditions	NA
Called functions	NA

Example:

```
i2c_master_transmit_dma(&hi2c, 0xB0, tx_buf, 8, 0xFFFFFFFF);
```

5.13.45 i2c_master_receive_dma function

The table below describes the function i2c_master_receive_dma.

Table 313. i2c_master_receive_dma function

Name	Description
Function name	i2c_master_receive_dma
Function prototype	i2c_status_type i2c_master_receive_dma(i2c_handle_type* hi2c, uint16_t address, uint8_t* pdata, uint16_t size, uint32_t timeout);
Function description	Master receives data (DMA mode). This function is non-blocking, which means that I2C communication has yet to complete after the execution of this function. It is possible to wait for the end of communication by calling the function i2c_wait_end().
Input parameter 1	hi2c: i2c_handle_type pointer. Refer to *i2c_handle_type* for details.
Input parameter 2	address: slave address
Input parameter 3	Pdata: array address of data to receive
Input parameter 4	Size: number of data to receive
Input parameter 5	timeout: wait timeout
Output parameter	NA
Return value	i2c_status_type: error code. Refer to *section 5.13.32* for details.
Required preconditions	NA
Called functions	NA

Example:

```
i2c_master_receive_dma(&hi2c, 0xB0, rx_buf, 8, 0xFFFFFFFF);
```

5.13.46 i2c_slave_transmit_dma function

The table below describes the function i2c_slave_transmit_dma.

Table 314. i2c_slave_transmit_dma function

Name	Description
Function name	i2c_slave_transmit_dma
Function prototype	i2c_status_type i2c_slave_transmit_dma(i2c_handle_type* hi2c, uint8_t* pdata, uint16_t size, uint32_t timeout);
Function description	Slave sends data (DMA mode). This function is non-blocking, which means that I2C communication has yet to complete after the execution of this function. It is possible to wait for the end of communication by calling the function i2c_wait_end().
Input parameter 1	hi2c: i2c_handle_type pointer. Refer to *i2c_handle_type* for details.
Input parameter 2	Pdata: array address of data to send
Input parameter 3	Size: number of data to send
Input parameter 4	timeout: wait timeout
Output parameter	NA
Return value	i2c_status_type: error code. Refer to *section 5.13.32* for details.
Required preconditions	NA
Called functions	NA

Example:

```
i2c_slave_transmit_dma(&hi2c, tx_buf, 8, 0xFFFFFFFF);
```

5.13.47 i2c_slave_receive_dma function

The table below describes the function i2c_slave_receive_dma.

Table 315. i2c_slave_receive_dma function

Name	Description
Function name	i2c_slave_receive_dma
Function prototype	i2c_status_type i2c_slave_receive_dma(i2c_handle_type* hi2c, uint8_t* pdata, uint16_t size, uint32_t timeout);
Function description	Slave receives data (DMA mode). This function is non-blocking, which means that I2C communication has yet to complete after the execution of this function. It is possible to wait for the end of communication by calling the function i2c_wait_end().
Input parameter 1	hi2c: i2c_handle_type pointer Refer to *i2c_handle_type* for details.
Input parameter 2	pdata: array address of data to receive
Input parameter 3	Size: number of data to receive
Input parameter 4	timeout: wait timeout
Output parameter	NA
Return value	i2c_status_type: error code Refer to *section 5.13.32* for details.
Required preconditions	NA
Called functions	NA

Example:

```
i2c_slave_receive_dma(&hi2c, rx_buf, 8, 0xFFFFFFFF);
```

5.13.48 i2c_memory_write function

The table below describes the function i2c_memory_write.

Table 316. i2c_memory_write function

Name	Description
Function name	i2c_memory_write
Function prototype	i2c_status_type i2c_memory_write(i2c_handle_type* hi2c, i2c_mem_address_width_type mem_address_width, uint16_t address, uint16_t mem_address, uint8_t* pdata, uint16_t size, uint32_t timeout);
Function description	Write EEPROM (polling mode). This function is in blocking mode, which means that the I2C transfer comes to an end after the completion of the function.
Input parameter 1	hi2c: i2c_handle_type pointer Refer to *i2c_handle_type* for details.
Input parameter 2	mem_address_width: EEPROM memory address width Refer to the “mem_address_width” description below for details.
Input parameter 3	Address: EEPROM address
Input parameter 4	mem_address: EEPROM data memory address
Input parameter 5	Pdata: array address of data to send
Input parameter 6	Size: number of data to send
Input parameter 7	timeout: wait timeout
Output parameter	NA
Return value	i2c_status_type: error code Refer to *section 5.13.32* for details.
Required preconditions	NA
Called functions	NA

mem_address_width

EEPROM memory address width.

I2C_MEM_ADDR_WIDIH_8: 8-bit

I2C_MEM_ADDR_WIDIH_16: 16-bit

Example:

```
i2c_memory_write(&hi2c, 0xA0, 0x05, tx_buf, 8, 0xFFFFFFFF);
```

5.13.49 i2c_memory_write_int function

The table below describes the function i2c_memory_write_int.

Table 317. i2c_memory_write_int function

Name	Description
Function name	i2c_memory_write_int
Function prototype	i2c_status_type i2c_memory_write_int(i2c_handle_type* hi2c, i2c_mem_address_width_type mem_address_width, uint16_t address, uint16_t mem_address, uint8_t* pdata, uint16_t size, uint32_t timeout);
Function description	Write EEPROM (interrupt mode). This function is non-blocking, which means that the I2C communication has yet to complete after the completion of the function. It is possible to wait for the end of communication by calling the function i2c_wait_end().
Input parameter 1	hi2c: i2c_handle_type pointer Refer to *i2c_handle_type* for details.
Input parameter 2	mem_address_width: EEPROM memory address width Refer to the “mem_address_width” description below for details.
Input parameter 3	Address: EEPROM address
Input parameter 4	mem_address: EEPROM data memory address
Input parameter 5	pdata: array address of data to send
Input parameter 6	Size: number of data to send
Input parameter 7	timeout: wait timeout
Output parameter	NA
Return value	i2c_status_type: error code Refer to *section 5.13.32* for details.
Required preconditions	NA
Called functions	NA

mem_address_width

EEPROM memory address width.

I2C_MEM_ADDR_WIDIH_8: 8-bit

I2C_MEM_ADDR_WIDIH_16: 16-bit

Example:

```
i2c_memory_write_int(&hi2c, 0xA0, 0x05, tx_buf, 8, 0xFFFFFFFF);
```

5.13.50 i2c_memory_write_dma function

The table below describes the function i2c_memory_write_dma.

Table 318. i2c_memory_write_dma function

Name	Description
Function name	i2c_memory_write_dma
Function prototype	i2c_status_type i2c_memory_write_dma(i2c_handle_type* hi2c, i2c_mem_address_width_type mem_address_width, uint16_t address, uint16_t mem_address, uint8_t* pdata, uint16_t size, uint32_t timeout);
Function description	Write EEPROM (DMA mode). This function is non-blocking, which means that the I2C communication has yet to complete after the completion of the function. It is possible to wait for the end of communication by calling the function i2c_wait_end().
Input parameter 1	hi2c: i2c_handle_type pointer Refer to *i2c_handle_type* for details.
Input parameter 2	mem_address_width: EEPROM memory address width Refer to the “mem_address_width” description below for details.
Input parameter 3	Address: EEPROM address
Input parameter 4	mem_address: EEPROM data memory address
Input parameter 5	pdata: array address of data to send
Input parameter 6	size: number of data to send
Input parameter 7	timeout: wait timeout
Output parameter	NA
Return value	i2c_status_type: error code Refer to *section 5.13.32* for details.
Required preconditions	NA
Called functions	NA

mem_address_width

EEPROM memory address width.

I2C_MEM_ADDR_WIDIH_8: 8-bit

I2C_MEM_ADDR_WIDIH_16: 16-bit

Example:

```
i2c_memory_write_dma(&hi2c, 0xA0, 0x05, tx_buf, 8, 0xFFFFFFFF);
```

5.13.51 i2c_memory_read function

The table below describes the function i2c_memory_read.

Table 319. i2c_memory_read function

Name	Description
Function name	i2c_memory_read
Function prototype	i2c_status_type i2c_memory_read(i2c_handle_type* hi2c, i2c_mem_address_width_type mem_address_width, uint16_t address, uint16_t mem_address, uint8_t* pdata, uint16_t size, uint32_t timeout);
Function description	Read EEPROM (polling mode). This function is in blocking mode, which means that the I2C transfer comes to an end after the completion of the function.
Input parameter 1	hi2c: i2c_handle_type pointer Refer to *i2c_handle_type* for details.
Input parameter 2	mem_address_width: EEPROM memory address width Refer to the “mem_address_width” description below for details.
Input parameter 3	Address: EEPROM address
Input parameter 4	mem_address: EEPROM data memory address
Input parameter 5	Pdata: array address of data to read
Input parameter 6	Size: number of data to read
Input parameter 7	timeout: wait timeout
Output parameter	NA
Return value	i2c_status_type: error code Refer to *section 5.13.32* for details.
Required preconditions	NA
Called functions	NA

mem_address_width

EEPROM memory address width.

I2C_MEM_ADDR_WIDIH_8: 8-bit

I2C_MEM_ADDR_WIDIH_16: 16-bit

Example:

```
i2c_memory_read(&hi2c, 0xA0, 0x05, rx_buf, 8, 0xFFFFFFFF);
```

5.13.52 i2c_memory_read_int function

The table below describes the function i2c_memory_read_int.

Table 320. i2c_memory_read_int function

Name	Description
Function name	i2c_memory_read_int
Function prototype	i2c_status_type i2c_memory_read_int(i2c_handle_type* hi2c, i2c_mem_address_width_type mem_address_width, uint16_t address, uint16_t mem_address, uint8_t* pdata, uint16_t size, uint32_t timeout);
Function description	Read EEPROM (interrupt mode). This function is non-blocking, which means that the I2C communication has yet to complete after the completion of the function. It is possible to wait for the end of communication by calling the function
Input parameter 1	hi2c: i2c_handle_type pointer Refer to *i2c_handle_type* for details.
Input parameter 2	mem_address_width: EEPROM memory address width Refer to the “mem_address_width” description below for details.
Input parameter 3	Address: EEPROM address
Input parameter 4	mem_address: EEPROM data memory address
Input parameter 5	Pdata: array address of data to read
Input parameter 6	Size: number of data to read
Input parameter 7	timeout: wait timeout
Output parameter	NA
Return value	i2c_status_type: error code Refer to *section 5.13.32* for details.
Required preconditions	NA
Called functions	NA

mem_address_width

EEPROM memory address width.

I2C_MEM_ADDR_WIDIH_8: 8-bit

I2C_MEM_ADDR_WIDIH_16: 16-bit

Example:

```
i2c_memory_read_int(&hi2c, 0xA0, 0x05, rx_buf, 8, 0xFFFFFFFF);
```

5.13.53 i2c_memory_read_dma function

The table below describes the function i2c_memory_read_dma.

Table 321. i2c_memory_read_dma function

Name	Description
Function name	i2c_memory_read_dma
Function prototype	i2c_status_type i2c_memory_read_dma(i2c_handle_type* hi2c, i2c_mem_address_width_type mem_address_width, uint16_t address, uint16_t mem_address, uint8_t* pdata, uint16_t size, uint32_t timeout);
Function description	Read EEPROM (DMA mode). This function is non-blocking, which means that the I2C communication has yet to complete after the completion of the function. It is possible to wait for the end of communication by calling the function
Input parameter 1	hi2c: i2c_handle_type pointer Refer to *i2c_handle_type* for details.
Input parameter 2	mem_address_width: EEPROM memory address width Refer to the “mem_address_width” description below for details.
Input parameter 3	Address: EEPROM address
Input parameter 4	mem_address: EEPROM data memory address
Input parameter 5	Pdata: array address of data to read
Input parameter 6	Size: number of data to read
Input parameter 7	timeout: wait timeout
Output parameter	NA
Return value	i2c_status_type: error code Refer to *section 5.13.32* for details.
Required preconditions	NA
Called functions	NA

mem_address_width

EEPROM memory address width.

I2C_MEM_ADDR_WIDIH_8: 8-bit

I2C_MEM_ADDR_WIDIH_16: 16-bit

Example:

```
i2c_memory_read_dma(&hi2c, 0xA0, 0x05, rx_buf, 8, 0xFFFFFFFF);
```

5.13.54 i2c_evt_irq_handler function

The table below describes the function i2c_evt_irq_handler.

Table 322. i2c_evt_irq_handler function

Name	Description
Function name	i2c_evt_irq_handler
Function prototype	void i2c_evt_irq_handler(i2c_handle_type* hi2c);
Function description	Event interrupt function for handling I2C event interrupts
Input parameter 1	hi2c: i2c_handle_type pointer Refer to *i2c_handle_type* for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
void I2C1_EVT_IRQHandler(void)
{
  i2c_evt_irq_handler(&hi2c);
}
```

5.13.55 i2c_err_irq_handler function

The table below describes the function i2c_err_irq_handler.

Table 323. i2c_err_irq_handler function

Name	Description
Function name	i2c_err_irq_handler
Function prototype	void i2c_err_irq_handler(i2c_handle_type* hi2c);
Function description	Error interrupt function for handling I2C error interrupts
Input parameter 1	hi2c: i2c_handle_type pointer Refer to *i2c_handle_type* for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
void I2C1_ERR_IRQHandler(void)
{
  i2c_err_irq_handler(&hi2c);
}
```

5.13.56 i2c_dma_tx_irq_handler function

The table below describes the function i2c_dma_tx_irq_handler.

Table 324. i2c_dma_tx_irq_handler function

Name	Description
Function name	i2c_dma_tx_irq_handler
Function prototype	void i2c_dma_tx_irq_handler(i2c_handle_type* hi2c);
Function description	DMA transmit interrupt function for handling DMA transmit interrupts
Input parameter 1	hi2c: i2c_handle_type pointer Refer to *i2c_handle_type* for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
void DMA1_Channel6_IRQHandler(void)
{
  i2c_dma_rx_irq_handler(&hi2c);
}
```

5.13.57 i2c_dma_rx_irq_handler function

The table below describes the function i2c_dma_rx_irq_handler.

Table 325. i2c_dma_rx_irq_handler function

Name	Description
Function name	i2c_dma_rx_irq_handler
Function prototype	void i2c_dma_rx_irq_handler(i2c_handle_type* hi2c);
Function description	DMA receive interrupt function for handling DMA receive interrupts
Input parameter 1	hi2c: i2c_handle_type pointer Refer to *i2c_handle_type* for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
void DMA1_Channel7_IRQHandler(void)
{
  i2c_dma_tx_irq_handler(&hi2c);
}
```

5.14 Nested vectored interrupt controller (NVIC)

The NVIC register structure NVIC_Type is defined in the "core_cm4.h":

```
/**
  * @brief Structure type to access the Nested Vectored Interrupt Controller (NVIC).
  */
typedef struct
{
      ……
} NVIC_Type;
```

The table below gives a list of the NVIC registers.

Table 326. Summary of NVIC registers

Register	Description
iser	nterrupt enable set register
icer	Interrupt enable clear register
ispr	Interrupt suspend set register
icpr	Interrupt suspend clear register
iabr	Interrupt activate bit register
ip	Interrupt priority register
stir	Software trigger interrupt register

The table below gives a list of NVIC library functions.

Table 327. Summary of NVIC library functions

Function name	Description
nvic_system_reset	System software reset
nvic_irq_enable	NVIC interrupt enable and priority enable
nvic_irq_disable	NVIC interrupt disable
nvic_priority_group_config	NVIC interrupt priority grouping configuration
nvic_vector_table_set	NVIC interrupt vector table base address and offset address configuration
nvic_lowpower_mode_config	NVIC low-power mode configuration

5.14.1 nvic_system_reset function

The table below describes the function nvic_system_reset.

Table 328. nvic_system_reset function

Name	Description
Function name	nvic_system_reset
Function prototype	void nvic_system_reset(void)
Function description	System software reset
Input parameter	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NVIC_SystemReset()

Example:

```
/* system reset */
nvic_system_reset();
```

5.14.2 nvic_irq_enable function

The table below describes the function nvic_irq_enable.

Table 329. nvic_irq_enable function

Name	Description
Function name	nvic_irq_enable
Function prototype	void nvic_irq_enable(IRQn_Type irqn, uint32_t preempt_priority, uint32_t sub_priority)
Function description	NVIC interrupt enable and priority configuration
Input parameter 1	Irqn: select interrupt vector Refer to the "irqn" descriptions below for details.
Input parameter 2	preempt_priority: set preemption priority This parameter cannot be greater than the highest preemption priority defined in the NVIC_PRIORITY_GROUP_x
Input parameter 3	sub_priority: set response priority This parameter cannot be greater than the highest response priority defined in the NVIC_PRIORITY_GROUP_x
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NVIC_SetPriority() NVIC_EnableIRQ()

irqn

irqn is used to select interrupt vectors, including:

WWDT_IRQn: Window timer interrupt

PVM_IRQn: PVM interrupt linked to EXINT

……

UART7_IRQn: UART7 global interrupt

UART8_IRQn: UART8 global interrupt

Example:

```
/* enable nvic irq */
nvic_irq_enable(ADC1_2_IRQn, 0, 0);
```

5.14.3 nvic_irq_disable function

The table below describes the function nvic_irq_disable.

Table 330. nvic_irq_disable function

Name	Description
Function name	nvic_irq_disable
Function prototype	void nvic_irq_disable(IRQn_Type irqn)
Function description	NVIC interrupt enable
Input parameter	Irqn: select interrupt vector. Refer to “irqn” description for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NVIC_DisableIRQ()

Example:

```
/* disable nvic irq */
nvic_irq_disable(ADC1_CMP_IRQn);
```

5.14.4 nvic_priority_group_config function

The table below describes the function nvic_priority_group_config.

Table 331. nvic_priority_group_config function

Name	Description
Function name	nvic_priority_group_config
Function prototype	void nvic_priority_group_config(nvic_priority_group_type priority_group)
Function description	NVIC interrupt priority grouping configuration
Input parameter	priority_group: select interrrupt priority group This parameter can be any enumerated value in the nvic_priority_group_type
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NVIC_SetPriorityGrouping()

priority_group

priority_group is used to select priority group, including:

NVIC_PRIORITY_GROUP_0:

Priority group 0 (0 bit for preemption priority, and 4 bits for response priority)

NVIC_PRIORITY_GROUP_1:

Priority group 1 (1 bit for preemption priority, and 3 bits for response priority)

NVIC_PRIORITY_GROUP_2:

Priority group 2 (2 bits for preemption priority, and 2 bits for response priority)

NVIC_PRIORITY_GROUP_3:
Priority group 3 (3 bits for preemption priority, and 1 bit for response priority)
NVIC_PRIORITY_GROUP_4:
Priority group 4 (4 bits for preemption priority, and 0 bit for response priority)

Example:

```
/* config nvic priority group */
nvic_priority_group_config(NVIC_PRIORITY_GROUP_4);
```

5.14.5 nvic_vector_table_set function

The table below describes the function nvic_vector_table_set.

Table 332. nvic_vector_table_set function

Name	Description
Function name	nvic_vector_table_set
Function prototype	void nvic_vector_table_set(uint32_t base, uint32_t offset)
Function description	Set NVIC interruupt vector table base address and offset address
Input parameter 1	Base: base address of interrupt vector table The base address can be set in RAM or FLASH
Input parameter 2	Offset: offset address of interrupt vector table This parameter defines the start address of interrupt vector table, so it must be set to a multiple of 0x20.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

base

base is used to select base address of interrupt vector table, including:
NVIC_VECTTAB_RAM: Interrupt vector table base address is located in RAM
NVIC_VECTTAB_FLASH: Interrupt vector table base address is located in FLASH

Example:

```
/* config vector table offset */
nvic_vector_table_set(NVIC_VECTTAB_FLASH, 0x4000);
```

5.14.6 nvic_lowpower_mode_config function

The table below describes the function nvic_lowpower_mode_config.

Table 333. nvic_lowpower_mode_config function

Name	Description
Function name	nvic_lowpower_mode_config
Function prototype	void nvic_lowpower_mode_config(nvic_lowpower_mode_type lp_mode, confirm_state new_state)
Function description	Configure NVIC low-power mode
Input parameter 1	lp_mode: select low-power modes This parameter can be any enumerated value in the nvic_lowpower_mode_type.
Input parameter 2	new_state: indicates the pre-configured status of battery powered domain This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

lp_mode

lp_mode is used to select low-power modes, including:

NVIC_LP_SEVONPEND:

Send wakeup event upon interrupt suspend (this option is usually used in conjunction with WFE)

NVIC_LP_SLEEPDEEP:

Deepsleep mode control bit (enable or disable core clock)）

NVIC_LP_SLEEPONEXIT: Sleep mode entry when system leaves the lowest-priority interrupt

Example:

```
/* enable sleep-on-exit feature */
nvic_lowpower_mode_config(NVIC_LP_SLEEPONEXIT, TRUE);
```

5.15 Power controller (PWC)

The PWC register structure pwc_type is defined in the "at32f403a_407_pwc.h":

```
/**
  * @brief type define pwc register all
  */
typedef struct
{
      ……
} pwc_type;
```

The table below gives a list of the PWC registers.

Table 334. Summary of PWC registers

Register	Description
ctrl	Power control register
ctrlsts	Power control/status register

The table below gives a list of PWC library functions.

Table 335. Summary of PWC library functions

Function name	Description
pwc_reset	Reset PWC registers to their reset values.
pwc_battery_powered_domain_access	Enable battery powered domain access
pwc_pvm_level_select	Select PVM threshold
pwc_power_voltage_monitor_enable	Enable Voltage monitor
pwc_wakeup_pin_enable	Enable standby-mode wakeup pin
pwc_flag_clear	Clear flag
pwc_flag_get	Get flag status
pwc_sleep_mode_enter	Enter Sleep mode
pwc_deep_sleep_mode_enter	Enter Deepsleep mode
pwc_voltage_regulate_set	Select voltage regulator status in Deepsleep mode
pwc_standby_mode_enter	Enter Standby mode

5.15.1 pwc_reset function

The table below describes the function pwc_reset.

Table 336. pwc_reset function

Name	Description
Function name	pwc_reset
Function prototype	void pwc_reset(void)
Function description	Reset all PWC registers to their reset values.
Input parameter	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	crm_periph_reset()

Example:

```
/* deinitialize pwc */
pwc_reset();
```

5.15.2 pwc_battery_powered_domain_access function

The table below describes the function pwc_battery_powered_domain_access.

Table 337. pwc_battery_powered_domain_access function

Name	Description
Function name	pwc_battery_powered_domain_access
Function prototype	void pwc_battery_powered_domain_access(confirm_state new_state)
Function description	Battery powered domain access enable
Input parameter	new_state: indicates the pre-configured status of battery powered domain This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable the battery-powered domain write operations */
pwc_battery_powered_domain_access(TRUE);
```

Note: Access to battery powered domain (such as, RTC) is allowed only after enabling it through this function.

5.15.3 pwc_pvm_level_select function

The table below describes the function pwc_pvm_level_select.

Table 338. pwc_pvm_level_select function

Name	Description
Function name	pwc_pvm_level_select
Function prototype	void pwc_pvm_level_select(pwc_pvm_voltage_type pvm_voltage)
Function description	Select PVM threshold
Input parameter	pvm_voltage: indicates the selected PVM threshold This parameter can be any enumerated value in the pwc_pvm_voltage_type.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

pvm_voltage

pvm_voltage is used to select PVM threshold, including:

PWC_PVM_VOLTAGE_2V3: PVM threshold is 2.3V
PWC_PVM_VOLTAGE_2V4: PVM threshold is 2.4V
PWC_PVM_VOLTAGE_2V5: PVM threshold is 2.5V
PWC_PVM_VOLTAGE_2V6: PVM threshold is 2.6V
PWC_PVM_VOLTAGE_2V7: PVM threshold is 2.7V
PWC_PVM_VOLTAGE_2V8: PVM threshold is 2.8V
PWC_PVM_VOLTAGE_2V9: PVM threshold is 2.9V

Example:

```
/* set the threshold voltage to 2.9v */
pwc_pvm_level_select(PWC_PVM_VOLTAGE_2V9);
```

5.15.4 pwc_power_voltage_monitor_enable function

The table below describes the function pwc_power_voltage_monitor_enable.

Table 339. pwc_power_voltage_monitor_enable function

Name	Description
Function name	pwc_power_voltage_monitor_enable
Function prototype	void pwc_power_voltage_monitor_enable(confirm_state new_state)
Function description	Enable power voltage monitor (PVM)
Input parameter	new_state: indicates the pre-configured status of PVM This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable power voltage monitor */
pwc_power_voltage_monitor_enable(TRUE);
```

5.15.5 pwc_wakeup_pin_enable function

The table below describes the function pwc_wakeup_pin_enable.

Table 340. pwc_wakeup_pin_enable function

Name	Description
Function name	pwc_wakeup_pin_enable
Function prototype	void pwc_wakeup_pin_enable(uint32_t pin_num, confirm_state new_state)
Function description	Enable Standby wakeup pin
Input parameter 1	pin_num: select a standby wakeup pins This parameter can be any pin that is capable of waking up from Standby mode.
Input parameter 2	new_state: indicates the pre-configured status of Standby wakup pins This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

pin_num

pin_num is used to select Standby-mode wakeup pin, including:

PWC_WAKEUP_PIN_1: Standby wakeup pin 1 (corresponding GPIO is PA0)

Example:

```
/* enable wakeup pin - pa0 */
pwc_wakeup_pin_enable(PWC_WAKEUP_PIN_1, TRUE);
```

5.15.6 pwc_flag_clear function

The table below describes the function pwc_flag_clear.

Table 341. pwc_flag_clear function

Name	Description
Function name	pwc_flag_clear
Function prototype	void pwc_flag_clear(uint32_t pwc_flag)
Function description	Clear flag
Input parameter	pwc_flag: select flag to clear Refer to the “pwc_flag” description below for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

pwc_flag

pwc_flag is used to select flags, including:

PWC_WAKEUP_FLAG: Standby wakeup event

PWC_STANDBY_FLAG: Standby mode entry

PWC_PVM_OUTPUT_FLAG: PVM output (this parameter cannot be cleared by software)

Example:

```
/* wakeup event flag clear */
pwc_flag_clear(PWC_WAKEUP_FLAG);
```

5.15.7 pwc_flag_get function

The table below describes the function pwc_flag_get.

Table 342. pwc_flag_get function

Name	Description
Function name	pwc_flag_get
Function prototype	flag_status pwc_flag_get(uint32_t pwc_flag)
Function description	Get flag status
Input parameter	pwc_flag: select a flag. Refer to *pwc_flag* for details.
Output parameter	NA
Return value	flag_status: indicates flag status Return SET or RESET.
Required preconditions	NA
Called functions	NA

Example:

```
/* check if wakeup event flag is set */
if(pwc_flag_get(PWC_WAKEUP_FLAG) != RESET)
```

5.15.8 pwc_sleep_mode_enter function

The table below describes the function pwc_sleep_mode_enter.

Table 343. pwc_sleep_mode_enter function

Name	Description
Function name	pwc_sleep_mode_enter
Function prototype	void pwc_sleep_mode_enter(pwc_sleep_enter_type pwc_sleep_enter)
Function description	Enter Sleep mode
Input parameter	pwc_sleep_enter: select a command to enter Sleep mode This parameter can be any enumerated value in the pwc_sleep_enter_type
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

pwc_sleep_enter

pwc_sleep_enter is used to select a command to enter Sleep mode, including:

PWC_SLEEP_ENTER_WFI: Enter Sleep mode by WFI

PWC_SLEEP_ENTER_WFE: Enter Sleep mode by WFE

Example:

```
/* enter sleep mode */
pwc_sleep_mode_enter(PWC_SLEEP_ENTER_WFI);
```

5.15.9 pwc_deep_sleep_mode_enter function

The table below describes the function pwc_deep_sleep_mode_enter.

Table 344. pwc_deep_sleep_mode_enter function

Name	Description
Function name	pwc_deep_sleep_mode_enter
Function prototype	void pwc_deep_sleep_mode_enter(pwc_deep_sleep_enter_type pwc_deep_sleep_enter)
Function description	Enter Deepsleep mode
Input parameter	pwc_deep_sleep_enter: select a command to enter Deepsleep mode This parameter can be any enumerated value in the pwc_deep_sleep_enter_type
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

pwc_deep_sleep_enter

pwc_deep_sleep_enter is used to select a command to enter Deepsleep mode, including:

PWC_DEEP_SLEEP_ENTER_WFI: Enter Deepsleep mode by WFI

PWC_DEEP_SLEEP_ENTER_WFE: Enter Deepsleep mode by WFE

Example:

```
/* enter deep sleep mode */
pwc_deep_sleep_mode_enter(PWC_DEEP_SLEEP_ENTER_WFI);
```

5.15.10 pwc_voltage_regulate_set function

The table below describes the function pwc_voltage_regulate_set.

Table 345. pwc_voltage_regulate_set function

Name	Description
Function name	pwc_voltage_regulate_set
Function prototype	void pwc_voltage_regulate_set(pwc_regulator_type pwc_regulator)
Function description	Select PVM status in Deepsleep mode
Input parameter	pwc_regulator: select voltage regulator status This parameter can be any enumerated value in the pwc_regulator_type
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

pwc_regulator

pwc_regulator is used to select the status of voltage regulator, including:

PWC_REGULATOR_ON: Voltage regulator ON in Deepsleep mode

PWC_REGULATOR_LOW_POWER: Voltage regulator low-power mode in Deepsleep mode

Example:

```
/* congfig the voltage regulator mode */
pwc_voltage_regulate_set(PWC_REGULATOR_LOW_POWER);
```

5.15.11 pwc_standby_mode_enter function

The table below describes the function pwc_standby_mode_enter

Table 346. pwc_standby_mode_enter function

Name	Description
Function name	pwc_standby_mode_enter
Function prototype	void pwc_standby_mode_enter(void)
Function description	Enter Standby mode
Input parameter	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enter standby mode */
pwc_standby_mode_enter();
```

5.16 Real-time clock (RTC)

The RTC register structure ertc_type is defined in the "at32f403a_407_rtc.h":

```
/**
  * @brief type define rtc register all
  */
typedef struct
{


} rtc_type;
```

The table below gives a list of the ERTC registers.

Table 347. Summary of RTC registers

Register	Description
ctrlh	RTC control register high
ctrll	RTC control register low
divh	RTC divider register high
divl	RTC divider register low
divcnth	RTC divider counter register high
divcntl	RTC divider counter register low
cnth	RTC counter value register high
cntl	RTC counter value register low
tah	RTC alarm register high
tal	RTC alarm register low

The table below gives a list of RTC library functions.

Table 348. Summary of RTC library functions

Function name	Description
rtc_counter_set	Set RTC counter value
rtc_counter_get	Get RTC counter value
rtc_divider_set	Set RTC divider
rtc_divider_get	Get RTC divider
rtc_alarm_set	Set RTC alarm
rtc_interrupt_enable	Enable RTC interrupts
rtc_flag_get	Get RTC flag
rtc_flag_clear	Clear RTC flag
rtc_wait_config_finish	RTC waits for configuration complete
rtc_wait_update_finish	RTC waits for time update complete

5.16.1 rtc_counter_set function

The table below describes the function rtc_counter_set.

Table 349. rtc_counter_set function

Name	Description
Function name	rtc_counter_set
Function prototype	void rtc_counter_set(uint32_t counter_value);
Function description	Set RTC counter value
Input parameter 1	counter_value: RTC counter value (range 0~0xFFFFFFFF)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
rtc_counter_set(0x00000008);
```

5.16.2 rtc_counter_get function

The table below describes the function rtc_counter_get.

Table 350. rtc_counter_get function

Name	Description
Function name	rtc_counter_get
Function prototype	uint32_t rtc_counter_get(void);
Function description	Get RTC counter value
Input parameter 1	NA
Output parameter	NA
Return value	uint32_t: the current counter value. The value is usually incremented by 1 per second.
Required preconditions	NA
Called functions	NA

Example:

```
value = rtc_counter_get();
```

5.16.3 rtc_divider_set function

The table below describes the function rtc_divider_set.

Table 351. rtc_divider_set function

Name	Description
Function name	rtc_divider_set
Function prototype	void rtc_divider_set(uint32_t div_value);
Function description	Set RTC divider
Input parameter 1	div_value: RTC divider value (range 0~0x000FFFFF)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
rtc_divider_set(32767);
```

5.16.4 rtc_divider_get function

The table below describes the function rtc_divider_get.

Table 352. rtc_divider_get function

Name	Description
Function name	rtc_divider_get
Function prototype	uint32_t rtc_divider_get(void);
Function description	Get RTC divider value
Input parameter 1	NA
Output parameter	NA
Return value	uint32_t: the current divider value
Required preconditions	NA
Called functions	NA

Example:

```
value = rtc_divider_get();
```

5.16.5 rtc_alarm_set function

The table below describes the function rtc_alarm_set.

Table 353. rtc_alarm_set function

Name	Description
Function name	rtc_alarm_set
Function prototype	void rtc_alarm_set(uint32_t alarm_value);
Function description	Set RTC alarm
Input parameter 1	alarm_value: RTC alarm value, range 0~0xFFFFFFFF,. An alarm event occus when RTC counter value equals to the alarm value.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
rtc_alarm_set(0x00000006);
```

5.16.6 rtc_interrupt_enable function

The table below describes the function rtc_interrupt_enable.

Table 354. rtc_interrupt_enable function

Name	Description
Function name	rtc_interrupt_enable
Function prototype	void rtc_interrupt_enable(uint16_t source, confirm_state new_state);
Function description	Enabe interrupts
Input parameter 1	Source: interrupt source. Refer to the “source” descriptions below for details.
Input parameter 2	new_state: indicates the status of interrupt This parameter can be TRUE or FALSE
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Source

Interrupt source selection.

RTC_TS_INT: Second interrupt

RTC_TA_INT: Alarm interrupt

RTC_OVF_INT: Counter overflow interrupt

Example:

```
rtc_interrupt_enable(RTC_TS_INT, TRUE);
```

5.16.7 rtc_flag_get function

The table below describes the function rtc_flag_get.

Table 355. rtc_flag_get function

Name	Description
Function name	rtc_flag_get
Function prototype	flag_status rtc_flag_get(uint16_t flag);
Function description	Get flag status
Input parameter 1	Flag: flag selection Refer to the “flag” descriptions below for details.
Output parameter	NA
Return value	flag_status: flag status Return SET or RESET
Required preconditions	NA
Called functions	NA

flag

This is used for flag selection, including:

RTC_TS_FLAG: Second flag
RTC_TA_FLAG: Alarm flag
RTC_OVF_FLAG: Counter overflow flag
RTC_UPDF_FLAG: Time update flag
RTC_CFGF_FLAG: RTC register configuration complete flag

Example:

```
rtc_flag_get(RTC_TS_FLAG);
```

5.16.8 rtc_interrupt_flag_get

The table below describes the function rtc_interrupt_flag_get.

Table 356. rtc_interrupt_flag_get function

Name	Description
Function name	rtc_interrupt_flag_get
Function prototype	flag_status rtc_interrupt_flag_get(uint16_t flag);
Function description	Get flag status and judge the corresponding interrupt enable bit
Input parameter 1	flag: flag selection. Refer to the “flag” descriptions below for details.
Output parameter	NA
Return value	flag_status: flag status Return SET or RESET
Required preconditions	NA
Called functions	NA

flag

This is used for flag selection, including:

RTC_TS_FLAG: Second flag
RTC_TA_FLAG: Alarm flag
RTC_OVF_FLAG: Counter overflow flag

Example

```
rtc_interrupt_flag_get(RTC_TS_FLAG);
```

5.16.9 rtc_flag_clear function

The table below describes the function rtc_flag_clear.

Table 357. rtc_flag_clear function

Name	Description
Function name	rtc_flag_clear
Function prototype	void rtc_flag_clear(uint16_t flag);
Function description	Clear flag
Input parameter 1	Flag: flag selection Refer to the “flag” descriptions below for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

flag

This is used for flag selection, including:

RTC_TS_FLAG: Second flag

RTC_TA_FLAG: Alarm flag

RTC_OVF_FLAG: Counter overflow flag

RTC_UPDF_FLAG: Time update flag

Example:

```
ertc_init_mode_exit();
```

5.16.10 rtc_wait_config_finish function

The table below describes the function rtc_wait_config_finish.

Table 358. rtc_wait_config_finish function

Name	Description
Function name	rtc_wait_config_finish
Function prototype	void rtc_wait_config_finish(void);
Function description	RTC waits for the completion of configuration
Input parameter 1	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
rtc_wait_config_finish();
```

5.16.11 rtc_wait_update_finish function

The table below describes the function rtc_wait_update_finish.

Table 359. rtc_wait_update_finish function

Name	Description
Function name	rtc_wait_update_finish
Function prototype	void rtc_wait_update_finish(void);
Function description	RTC waits for the completion of time update
Input parameter 1	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
rtc_wait_update_finish();
```

5.17 SDIO interfaces

SDIO register structure crm_type is defined in the “at32f403a_407_sdio.h”:

```
/**
  * @brief type define sdio register all
  */
typedef struct
{
      ...

} sdio_type;
```

The table below gives a list of the SDIO registers.

Table 360. SDIO register description

Register	Description
pwrctrl	Power control register
clkctrl	Clock control register
arg	Argument register
cmd	Command register
rspcmd	Command response register
rsp1	Response register 1
rsp2	Response register 2
rsp3	Response register 3
rsp4	Response register 4
dttmr	Data timer register
dtlen	Data length register
dtctrl	Data control register
dtcntr	Data counter register
sts	Status register
intclr	Clear interrupt register
inten	Interrupt mask register
bufcntr	BUF counter register
buf	Data BUF register

The table below gives a list of SDIO library functions.

Table 361. SDIO library functions

Function name	Description
sdio_reset	Reset SDIO pheripheral register and control status
sdio_power_set	Set controller power stauts
sdio_power_status_get	Get controller power status
sdio_clock_config	Configure clock parameters
sdio_bus_width_config	Set bus width
sdio_clock_bypass	Enable clock bypass mode
sdio_power_saving_mode_enable	Enable controller power saving mode
sdio_flow_control_enable	Enable flow control mode
sdio_clock_enable	Enable clock
sdio_dma_enable	Enable DMA
sdio_interrupt_enable	Enable interrupts
sdio_flag_get	Get flags
sdio_flag_clear	Clear flags
sdio_command_config	Set command parameters
sdio_command_state_machine_enable	Enable command state machine
sdio_command_response_get	Get command response
sdio_response_get	Return command response
sdio_data_config	Set data parameters
sdio_data_state_machine_enable	Enable data state machine
sdio_data_counter_get	Get data counter
sdio_data_read	Read data (word) from receive FIFO
sdio_buffer_counter_get	Get buffer counter
sdio_data_write	Write data (word) to transmit FIFO
sdio_read_wait_mode_set	Set read wait mode
sdio_read_wait_start	Set read wait start
sdio_read_wait_stop	Set read wait stop
sdio_io_function_enable	Enable IO functional mode
sdio_io_suspend_command_set	Enable IO suspend command

5.17.1 sdio_reset function

The table below describes the function sdio_reset.

Table 362. sdio_reset function

Name	Description
Function name	sdio_reset
Function prototype	void sdio_reset(sdio_type *sdio_x);
Function description	Reset SDIO peripheral registers and control status
Input parameter 1	sdio_x: select SDIO peripherals, such as SDIO1
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* reset sdio */
sdio_reset(SDIO1);
```

5.17.2 sdio_power_set function

The table below describes the function sdio_power_set.

Table 363. sdio_power_set function

Name	Description
Function name	sdio_power_set
Function prototype	void sdio_power_set(sdio_type *sdio_x, sdio_power_state_type power_state);
Function description	Set SDIO power status
Input parameter 1	sdio_x: select SDIO peripheral, such as SDIO1
Input parameter 2	power_state: indicates power status
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

power_state

Set power supply status.

SDIO_POWER_ON: Power ON

SDIO_POWER_OFF: Power OFF

Example:

```
/* sdio power on */
sdio_power_set(SDIO1, SDIO_POWER_ON);
```

5.17.3 sdio_power_status_get function

The table below describes the function sdio_power_status_get.

Table 364. sdio_power_status_get function

Name	Description
Function name	sdio_power_status_get
Function prototype	sdio_power_state_type sdio_power_status_get(sdio_type *sdio_x);
Function description	Get SDIO power status
Input parameter 1	sdio_x: select SDIO peripheral, such as SDIO1
Input parameter 2	NA
Output parameter	NA
Return value	sdio_power_state_type: indicates power supply status
Required preconditions	NA
Called functions	NA

Example:

```
/* check power status */
if(sdio_power_status_get(SDIO1) == SDIO_POWER_OFF)
{
  return SD_REQ_NOT_APPLICABLE;
}
```

5.17.4 sdio_clock_config function

The table below describes the function sdio_clock_config.

Table 365. sdio_clock_config function

Name	Description
Function name	sdio_clock_config
Function prototype	void sdio_clock_config(sdio_type *sdio_x, uint16_t clk_div, sdio_edge_phase_type clk_edg);
Function description	Configure clock parameters
Input parameter 1	sdio_x: select SDIO peripheral, such as SDIO1
Input parameter 2	clk_div: set clock division, from 0~0x3FF
Input parameter 3	clk_edg: set clock edge
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

clk_edg

Select clock edge.

SDIO_CLOCK_EDGE_RISING: Rising edge

SDIO_CLOCK_EDGE_FALLING: Falling edge

Example:

```
/* config sdio clock divide and edge phase */
sdio_clock_config(SDIO1, 0x2, SDIO_CLOCK_EDGE_FALLING);
```

5.17.5 sdio_bus_width_config function

The table below describes the function sdio_bus_width_config.

Table 366. sdio_bus_width_config function

Name	Description
Function name	sdio_bus_width_config
Function prototype	void sdio_bus_width_config(sdio_type *sdio_x, sdio_bus_width_type width);
Function description	Configure SDIO bus width
Input parameter 1	sdio_x: select SDIO peripheral, such as SDIO1
Input parameter 2	Width: select SDIO bus width
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

width

Select bus width.

SDIO_BUS_WIDTH_D1: 1-bit

SDIO_BUS_WIDTH_D4: 4-bit

SDIO_BUS_WIDTH_D8: 8-bit

Example:

```
/* config sdio bus width */
sdio_bus_width_config(SDIOx, SDIO_BUS_WIDTH_D1);
```

5.17.6 sdio_clock_bypass function

The table below describes the function sdio_clock_bypass.

Table 367. sdio_clock_bypass function

Name	Description
Function name	sdio_clock_bypass
Function prototype	void sdio_clock_bypass(sdio_type *sdio_x, confirm_state new_state);
Function description	Enable SDIO clock bypass mode
Input parameter 1	sdio_x: select SDIO peripheral, such as SDIO1
Input parameter 2	new_state: Bypass ON (TRUE), Bypass OFF (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* disable clock bypass */
sdio_clock_bypass(SDIO1, FALSE);
```

5.17.7 sdio_power_saving_mode_enable function

The table below describes the function sdio_power_saving_mode_enable.

Table 368. sdio_power_saving_mode_enable function

Name	Description
Function name	sdio_power_saving_mode_enable
Function prototype	void sdio_power_saving_mode_enable(sdio_type *sdio_x, confirm_state new_state);
Function description	Enable SDIO power saving mode
Input parameter 1	sdio_x: select SDIO peripheral, such as SDIO1
Input parameter 2	new_state: ON (TRUE), OFF (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* disable power saving mode */
sdio_power_saving_mode_enable(SDIO1, FALSE);
```

5.17.8 sdio_flow_control_enable function

The table below describes the function sdio_flow_control_enable.

Table 369. sdio_flow_control_enable function

Name	Description
Function name	sdio_flow_control_enable
Function prototype	void sdio_flow_control_enable(sdio_type *sdio_x, confirm_state new_state);
Function description	Enable SDIO flow control mode
Input parameter 1	sdio_x: select SDIO peripheral, such as SDIO1
Input parameter 2	new_state: ON (TRUE), OFF (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* disable flow control */
sdio_flow_control_enable(SDIO1, FALSE);
```

5.17.9 sdio_clock_enable function

The table below describes the function sdio_clock_enable.

Table 370. sdio_clock_enable function

Name	Description
Function name	sdio_clock_enable
Function prototype	void sdio_clock_enable(sdio_type *sdio_x, confirm_state new_state);
Function description	Enable SDIO clock
Input parameter 1	sdio_x: select SDIO peripheral, such as SDIO1
Input parameter 2	new_state: ON (TRUE), OFF (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable to output sdio_ck */
sdio_clock_enable(SDIO1, TRUE);
```

5.17.10 sdio_dma_enable function

The table below describes the function sdio_dma_enable.

Table 371. sdio_dma_enable function

Name	Description
Function name	sdio_dma_enable
Function prototype	void sdio_dma_enable(sdio_type *sdio_x, confirm_state new_state);
Function description	Enable SDIO DMA mode
Input parameter 1	sdio_x: select SDIO peripheral, such as SDIO1
Input parameter 2	new_state: ON (TRUE), OFF (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable sdio dma */
sdio_dma_enable(SDIO1, TRUE);
```

5.17.11 sdio_interrupt_enable function

The table below describes the function sdio_interrupt_enable.

Table 372. sdio_interrupt_enable function

Name	Description
Function name	sdio_interrupt_enable
Function prototype	void sdio_interrupt_enable(sdio_type *sdio_x, uint32_t int_opt, confirm_state new_state);
Function description	Enable SDIO interrupts
Input parameter 1	sdio_x: select SDIO peripheral, such as SDIO1
Input parameter 2	int_opt: select interrupt type
Input parameter 3	new_state: ON (TRUE), OFF (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

int_opt

Select interrupts.

SDIO_CMDFAIL_INT: Command CRC fail interrupt
SDIO_DTFAIL_INT: Data CRC fail interrupt
SDIO_CMDTIMEOUT_INT: Command timeout interrupt
SDIO_DTTIMEOUT_INT: Data timeout interrupt
SDIO_TXERRU_INT: TxBUF underrun error interrupt
SDIO_RXERRO_INT: RxBUF overrun error interrupt
SDIO_CMDRSPCMPL_INT: Command response received interrupt
SDIO_CMDCMPL_INT: Command transfer complete interrupt
SDIO_DTCMP_INT: Data transfer complete interrupt
SDIO_SBITERR_INT: Start bit error interrupt
SDIO_DTBLKCMPL_INT: Data block transfer complete interrupt
SDIO_DOCMD_INT: Command transfer in-progress interrupt
SDIO_DOTX_INT: Data transfer in-progress interrupt
SDIO_DORX_INT: Data receive in-progress interrupt
SDIO_TXBUFH_INT: Transmit buffer half-empty
SDIO_RXBUFH_INT: Receive buffer half-empty
SDIO_TXBUFF_INT: Transmit buffer full
SDIO_RXBUFF_INT: Receive buffer full
SDIO_TXBUFE_INT: Transmit buffer empty
SDIO_RXBUFE_INT: Receive buffer empty
SDIO_TXBUF_INT: Data available in transmit BUF
SDIO_RXBUF_INT: Data available in receive BUF
SDIO_SDIOIF_INT: SD I/O receive interrupt

Example:

```
/* disable interrupt */
sdio_interrupt_enable(SDIO1, (SDIO_DTFAIL_INT   | SDIO_DTTIMEOUT_INT | \
                              SDIO_DTCMP_INT | SDIO_TXBUFH_INT | SDIO_RXBUFH_INT      | \
                              SDIO_TXERRU_INT| SDIO_RXERRO_INT | SDIO_SBITERR_INT), FALSE);
```

5.17.12 sdio_flag_get function

The table below describes the function sdio_flag_get.

Table 373. sdio_flag_get function

Name	Description
Function name	sdio_flag_get
Function prototype	flag_status sdio_flag_get(sdio_type *sdio_x, uint32_t flag);
Function description	Get flag status
Input parameter 1	sdio_x: select SDIO peripheral, such as SDIO1
Input parameter 2	Flag: interrupt type
Output parameter	NA
Return value	flag_status: SET or RESET
Required preconditions	NA
Called functions	NA

flag

Select flags.

SDIO_CMDFAIL_FLAG: Command CRC failure
SDIO_DTFAIL_FLAG: Data CRC failure
SDIO_CMDTIMEOUT_FLAG: Command timeout
SDIO_DTTIMEOUT_FLAG: Data timeout
SDIO_TXERRU_FLAG: TxBUF underrun error
SDIO_RXERRO_FLAG: RxBUF overrun error
SDIO_CMDRSPCMPL_FLAG: Command response received
SDIO_CMDCMPL_FLAG: Command transfer complete
SDIO_DTCMP_FLAG: Data transfer complete
SDIO_SBITERR_FLAGv Start bit error
SDIO_DTBLKCMPL_FLAG: Data block transfer complete
SDIO_DOCMD_FLAG: Command transfer in-progress
SDIO_DOTX_FLAG: Data transfer in-progress
SDIO_DORX_FLAG: Data receive in-progress
SDIO_TXBUFH_FLAG: Transmit buffer half-empty
SDIO_RXBUFH_FLAG: Receive buffer half-empty
SDIO_TXBUFF_FLAG: Transmit buffer full
SDIO_RXBUFF_FLAG: Receive buffer full
SDIO_TXBUFE_FLAG: Transmit buffer empty
SDIO_RXBUFE_FLAG: Receive buffer empty
SDIO_TXBUF_FLAG: Data available in transmit BUF
SDIO_RXBUF_FLAG: Data available in receive BUF
SDIO_SDIOIF_FLAG: SD I/O receive

Example:

```
/* check dttimeout flag */
if(sdio_flag_get(SDIOx, SDIO_DTTIMEOUT_FLAG) != RESET)
{
}
```

5.17.13 sdio_interrupt_flag_get function

The table below describes the function sdio_interrupt_flag_get.

Table 374. sdio_interupt_flag_get function

Name	Description
Function name	sdio_interrupt_flag_get
Function prototype	flag_status sdio_interrupt_flag_get(sdio_type *sdio_x, uint32_t flag);
Function description	Check whether the selected interrupt flag is set or not
Input parameter 1	sdio_x: select SDIO peripheral, such as SDIO1
Input parameter 2	flag: interrupt type
Output parameter	NA
Return value	flag_status: SET or RESET
Required preconditions	NA
Called functions	NA

flag

Select a flag

SDIO_CMDFAIL_FLAG: Command CRC failure
SDIO_DTFAIL_FLAG: Data CRC failure
SDIO_CMDTIMEOUT_FLAG: Command timeout
SDIO_DTTIMEOUT_FLAG: Data timeout
SDIO_TXERRU_FLAG: TxBUF underrun error
SDIO_RXERRO_FLAG: RxBUF overrun error
SDIO_CMDRSPCMPL_FLAG: Command response received
SDIO_CMDCMPL_FLAG: Command transfer complete
SDIO_DTCMP_FLAG: Data transfer complete
SDIO_SBITERR_FLAG: Start bit error
SDIO_DTBLKCMPL_FLAG: Data block transfer complete
SDIO_DOCMD_FLAG: Command transfer in-progress
SDIO_DOTX_FLAG: Data transfer in-progress
SDIO_DORX_FLAG: Data receive in-progress
SDIO_TXBUFH_FLAG: Transmit BUF half-empty
SDIO_RXBUFH_FLAG: Receive BUF half-empty
SDIO_TXBUFF_FLAG: Transmit BUF full
SDIO_RXBUFF_FLAG: Receive BUF full
SDIO_TXBUFE_FLAG: Transmit BUF empty
SDIO_RXBUFE_FLAG: Receive BUF empty
SDIO_TXBUF_FLAG: Data available in transmit BUF
SDIO_RXBUF_FLAG: Data available in receive BUF
SDIO_SDIOIF_FLAG: SD I/O receive

Example

```
/* check dttimeout interrupt flag */
if(sdio_interrupt_flag_get(SDIOx, SDIO_DTTIMEOUT_FLAG) != RESET)
{
}
```

5.17.14 sdio_flag_clear function

The table below describes the function sdio_flag_clear.

Table 375. sdio_flag_clear function

Name	Description
Function name	sdio_flag_clear
Function prototype	void sdio_flag_clear(sdio_type *sdio_x, uint32_t flag);
Function description	Clear flags
Input parameter 1	sdio_x: select SDIO peripheral, such as SDIO1
Input parameter 2	Flag: interrupt type
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

flag

Select flags.

SDIO_CMDFAIL_FLAG: Command CRC failure
SDIO_DTFAIL_FLAG: Data CRC failure
SDIO_CMDTIMEOUT_FLAG: Command timeout
SDIO_DTTIMEOUT_FLAG: Data timeout
SDIO_TXERRU_FLAG: TxBUF underrun error
SDIO_RXERRO_FLAG: RxBUF overrun error
SDIO_CMDRSPCMPL_FLAG: Command response received
SDIO_CMDCMPL_FLAG: Command transfer complete
SDIO_DTCMP_FLAG: Data transfer complete
SDIO_SBITERR_FLAGv Start bit error
SDIO_DTBLKCMPL_FLAG: Data block transfer complete
SDIO_SDIOIF_FLAG: SD I/O receive

Example:

```
/* clear flags */
#define SDIO_STATIC_FLAGS                ((uint32_t)0x000005FF)
sdio_flag_clear(SDIO1, SDIO_STATIC_FLAGS);
```

5.17.15 sdio_command_config function

The table below describes the function sdio_command_config.

Table 376. sdio_command_config function

Name	Description
Function name	sdio_command_config
Function prototype	void sdio_command_config(sdio_type *sdio_x, sdio_command_struct_type *command_struct);
Function description	Configure command parameters
Input parameter 1	sdio_x: select SDIO peripheral, such as SDIO1
Input parameter 2	command_struct: sdio_command_struct_type pointer
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

command_struct

sdio_command_struct_type is defined in the at32f403a_407_sdio.h:

```
typedef struct
{
  uint32_t                argument;
  uint8_t                 cmd_index;
  sdio_reponse_type       rsp_type;
  sdio_wait_type          wait_type;
} sdio_command_struct_type;
```

argument

This command parameter is part of command information sent to the card, depending on the type of commands.

cmd_index

Indicates command index sent.

rsp_type

This is a response parameter, depending on the type of commands, including:

SDIO_RESPONSE_NO: No response
SDIO_RESPONSE_SHORT: Short response
SDIO_RESPONSE_LONG: Long response

wait_type

This a wait parameter, depending on the types of commands, including:

SDIO_WAIT_FOR_NO: No response
SDIO_WAIT_FOR_INT: Wait for interrupt requests
SDIO_WAIT_FOR_PEND: Wait for the end of transfer

Example:

```
/* send cmd16, set block length */
sdio_command_struct_type sdio_command_init_struct;
sdio_command_init_struct.argument = (uint32_t)8;
sdio_command_init_struct.cmd_index = SD_CMD_SET_BLOCKLEN;
sdio_command_init_struct.rsp_type = SDIO_RESPONSE_SHORT;
sdio_command_init_struct.wait_type = SDIO_WAIT_FOR_NO;
```

```
/* sdio command config */
sdio_command_config(SDIOx, &sdio_command_init_struct);
```

5.17.16 sdio_command_state_machine_enable function

The table below describes the function sdio_command_state_machine_enable.

Table 377. sdio_command_state_machine_enable function

Name	Description
Function name	sdio_command_state_machine_enable
Function prototype	void sdio_command_state_machine_enable(sdio_type *sdio_x, confirm_state new_state);
Function description	Enable command state machine
Input parameter 1	sdio_x: select SDIO peripheral, such as SDIO1
Input parameter 2	new_state: ON (TRUE), OFF (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable ccsm */
sdio_command_state_machine_enable(SDIO1, TRUE);
```

5.17.17 sdio_command_response_get function

The table below describes the function sdio_command_response_get.

Table 378. sdio_command_response_get function

Name	Description
Function name	sdio_command_response_get
Function prototype	uint8_t sdio_command_response_get(sdio_type *sdio_x);
Function description	Get command index corresponding to the received command reponse
Input parameter 1	sdio_x: select SDIO peripheral, such as SDIO1
Input parameter 2	NA
Output parameter	NA
Return value	uint8_t: return command index corresponding to command response
Required preconditions	NA
Called functions	NA

Example:

```
/* get response of command index */
uint8_t rsp_cmd = 0;
rsp_cmd = sdio_command_response_get(SDIO1);
```

5.17.18 sdio_response_get function

The table below describes the function sdio_response_get.

Table 379. sdio_response_get function

Name	Description
Function name	sdio_response_get
Function prototype	uint32_t sdio_response_get(sdio_type *sdio_x, sdio_rsp_index_type reg_index);
Function description	Get command response
Input parameter 1	sdio_x: select SDIO peripheral, such as SDIO1
Input parameter 2	reg_index: indicate response register index, including 1/2/3/4 response registers
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

reg_div

Response register index.

SDIO_RSP1_INDEX: Response register 1

SDIO_RSP2_INDEX: Response register 2

SDIO_RSP3_INDEX: Response register 3

SDIO_RSP4_INDEX: Response register 4

Example:

```
/* get response register1 */
response = sdio_response_get(SDIO1, SDIO_RSP1_INDEX);
```

5.17.19 sdio_data_config function

The table below describes the function sdio_data_config.

Table 380. sdio_data_config function

Name	Description
Function name	sdio_data_config
Function prototype	void sdio_data_config(sdio_type *sdio_x, sdio_data_struct_type *data_struct);
Function description	Configure SDIO data parameters
Input parameter 1	sdio_x: select SDIO peripheral, such as SDIO1
Input parameter 2	data_struct: sdio_data_struct_type pointer
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

data_struct

The sdio_data_struct_type is defined in the at32f403a_407_sdio.h:

```
typedef struct
{
  uint32_t                  timeout;
  uint32_t                  data_length;
  sdio_block_size_type      block_size;
```

```
  sdio_transfer_mode_type       transfer_mode;
  sdio_transfer_direction_type  transfer_direction;
} sdio_data_struct_type;
```

timeout

Indicates timeout period value for data transfer, counting based on bus clock

data_length

Indicates the length of data to transfer

block_size

Indicates block size, including:

SDIO_DATA_BLOCK_SIZE_1B:	1-bit
SDIO_DATA_BLOCK_SIZE_2B:	2-bit
SDIO_DATA_BLOCK_SIZE_4B:	4-bit
SDIO_DATA_BLOCK_SIZE_8B:	8-bit
SDIO_DATA_BLOCK_SIZE_16B:	16-bit
SDIO_DATA_BLOCK_SIZE_32B:	32-bit
SDIO_DATA_BLOCK_SIZE_64B:	64-bit
SDIO_DATA_BLOCK_SIZE_128B:	128-bit
SDIO_DATA_BLOCK_SIZE_256B:	256-bit
SDIO_DATA_BLOCK_SIZE_512B:	512-bit
SDIO_DATA_BLOCK_SIZE_1024B:	1024-bit
SDIO_DATA_BLOCK_SIZE_2048B:	2048-bit
SDIO_DATA_BLOCK_SIZE_4096B:	4096-bit
SDIO_DATA_BLOCK_SIZE_8192B:	8192-bit
SDIO_DATA_BLOCK_SIZE_16384B:	16384-bit

transfer_mode

Select data transfer mode

SDIO_DATA_BLOCK_TRANSFER:	Data block transfer
SDIO_DATA_STREAM_TRANSFER:	Data stream transfer

transfer_direction

Select data transfer direction.

SDIO_DATA_TRANSFER_TO_CARD:	Controller to card
SDIO_DATA_TRANSFER_TO_CONTROLLER:	Card to controller

Example:

```
sdio_data_struct_type sdio_data_init_struct;
sdio_data_init_struct.block_size = SDIO_DATA_BLOCK_SIZE_512B;
sdio_data_init_struct.data_length = 8 ;
sdio_data_init_struct.timeout = SD_DATATIMEOUT ;
sdio_data_init_struct.transfer_direction = SDIO_DATA_TRANSFER_TO_CARD;
sdio_data_init_struct.transfer_mode = SDIO_DATA_BLOCK_TRANSFER;
/* config sdio data */
sdio_data_config(SDIO1, &sdio_data_init_struct);
```

5.17.20 sdio_data_state_machine_enable function

The table below describes the function sdio_data_state_machine_enable.

Table 381. sdio_data_state_machine_enable function

Name	Description
Function name	sdio_data_state_machine_enable
Function prototype	void sdio_data_state_machine_enable(sdio_type *sdio_x, confirm_state new_state);
Function description	Enable data state machine
Input parameter 1	sdio_x: select SDIO peripheral, such as SDIO1
Input parameter 2	new_state: ON (TRUE), OFF (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable dcsm */
sdio_data_state_machine_enable(SDIO1, TRUE);
```

5.17.21 sdio_data_counter_get function

The table below describes the function sdio_data_counter_get.

Table 382. sdio_data_counter_get function

Name	Description
Function name	sdio_data_counter_get
Function prototype	uint32_t sdio_data_counter_get(sdio_type *sdio_x);
Function description	Get SDIO data counter
Input parameter 1	sdio_x: select SDIO peripheral, such as SDIO1
Input parameter 2	NA
Output parameter	NA
Return value	uint32_t: return the data count to be sent
Required preconditions	NA
Called functions	NA

Example:

```
/* get data counter */
uint32_t count = 0;
count = sdio_data_counter_get (SDIO1);
```

5.17.22 sdio_data_read function

The table below describes the function sdio_data_read.

Table 383. sdio_data_read function

Name	Description
Function name	sdio_data_read
Function prototype	uint32_t sdio_data_read(sdio_type *sdio_x);
Function description	Read a word data from receive FIFO
Input parameter 1	sdio_x: select SDIO peripheral, such as SDIO1
Input parameter 2	NA
Input parameter 3	NA
Output parameter	NA
Return value	uint32_t: read data (word)
Required preconditions	NA
Called functions	NA

Example:

```
/* read data */
uint32_t data = 0;
data = sdio_data_read(SDIO1);
```

5.17.23 sdio_buffer_counter_get function

The table below describes the function sdio_buffer_counter_get.

Table 384. sdio_buffer_counter_get function

Name	Description
Function name	sdio_buffer_counter_get
Function prototype	uint32_t sdio_buffer_counter_get(sdio_type *sdio_x);
Function description	Get the number of data to read/write from/to BUFF
Input parameter 1	sdio_x: select SDIO peripheral, such as SDIO1
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* get buffer count */
uint32_t count = 0;
count = sdio_buffer_counter_get(SDIO1);
```

5.17.24 sdio_data_write function

The table below describes the function sdio_data_write.

Table 385. sdio_data_write function

Name	Description
Function name	sdio_data_write
Function prototype	void sdio_data_write(sdio_type *sdio_x, uint32_t data);
Function description	Write a word data to transmit FIFO
Input parameter 1	sdio_x: select SDIO peripheral, such as SDIO1
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* write data */
uin32_t data = 0x11223344;
sdio_data_write(SDIO1, data);
```

5.17.25 sdio_read_wait_mode_set function

The table below describes the function sdio_read_wait_mode_set.

Table 386. sdio_read_wait_mode_set function

Name	Description
Function name	sdio_read_wait_mode_set
Function prototype	void sdio_read_wait_mode_set(sdio_type *sdio_x, sdio_read_wait_mode_type mode);
Function description	Set read wait mode
Input parameter 1	sdio_x: select SDIO peripheral, such as SDIO1
Input parameter 2	Mode: read wait mode
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

mode

SDIO_READ_WAIT_CONTROLLED_BY_D2: Read wait controlled by DATA Line2

SDIO_READ_WAIT_CONTROLLED_BY_CK: Read wait controlled by clock line

Example:

```
/* config read wait mode */
sdio_read_wait_mode_set(SDIO1, SDIO_READ_WAIT_CONTROLLED_BY_D2);
```

5.17.26 sdio_read_wait_start function

The table below describes the function sdio_read_wait_start.

Table 387. sdio_read_wait_start function

Name	Description
Function name	sdio_read_wait_start
Function prototype	void sdio_read_wait_start(sdio_type *sdio_x, confirm_state new_state);
Function description	Read wait start
Input parameter 1	sdio_x: select SDIO peripheral, such as SDIO1
Input parameter 2	new_state: ON (TRUE), OFF (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Calling this function ON indicates that read wait mode is enabled, OFF no effect.

Example:

```
/* start read wait mode */
sdio_read_wait_start (SDIO1, TRUE);
```

5.17.27 sdio_read_wait_stop function

The table below describes the function sdio_read_wait_stop.

Table 388. sdio_read_wait_stop function

Name	Description
Function name	sdio_read_wait_stop
Function prototype	void sdio_read_wait_stop(sdio_type *sdio_x, confirm_state new_state);
Function description	Read wait stop
Input parameter 1	sdio_x: select SDIO peripheral, such as SDIO1
Input parameter 2	new_state: ON (TRUE), OFF (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Calling this function “ON” indicates that read wait mode is disabled, “OFF” read wait mode continues.

Example:

```
/* stop read wait mode */
sdio_read_wait_stop (SDIO1, TRUE);
```

5.17.28 sdio_io_function_enable function

The table below describes the function sdio_io_function_enable.

Table 389. sdio_io_function_enable function

Name	Description
Function name	sdio_io_function_enable
Function prototype	void sdio_io_function_enable(sdio_type *sdio_x, confirm_state new_state);
Function description	Enable SDIO IO functional mode
Input parameter 1	sdio_x: select SDIO peripheral, such as SDIO1
Input parameter 2	new_state: ON (TRUE), OFF (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable sdio IO mode */
sdio_io_function_enable (SDIO1, TRUE);
```

5.17.29 sdio_io_suspend_command_set function

The table below describes the function sdio_io_suspend_command_set.

Table 390. sdio_io_suspend_command_set function

Name	Description
Function name	sdio_io_suspend_command_set
Function prototype	void sdio_io_suspend_command_set(sdio_type *sdio_x, confirm_state new_state);
Function description	Set SDIO IO suspend command
Input parameter 1	sdio_x: select SDIO peripheral, such as SDIO1
Input parameter 2	new_state: ON (TRUE), OFF (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* send suspend command */
sdio_io_suspend_command_set (SDIO1, TRUE);
```

5.18 Serial peripheral interface (SPI)/ I²S

The SPI register structure spi_type is defined in the "at32f403a_407_spi.h":

```
/**
  * @brief type define spi register all
  */
typedef struct
{
  ...
} spi_type;
```

The table below gives a list of the SPI registers.

Table 391. Summary of SPI registers

Register	Description
ctrl1	SPI control register 1
ctrl2	SPI control register 2
sts	SPI status register
dt	SPI data register
cpoly	SPI CRC register
rcrc	SPI RxCRC register
tcrc	SPI TxCRC register
i2sctrl	SPI_I2S configuration register
i2sclkp	SPI_I2S prescaler register

The table below gives a list of SPI library functions.

Table 392. Summary of SPI library functions

Function name	Description
spi_i2s_reset	Reset SPI/I²S registers to their reset values
spi_default_para_init	Configure the SPI initialization structure with an initial value
spi_init	Initialize SPI
spi_crc_next_transmit	Next data transfer is CRC command
spi_crc_polynomial_set	SPI CRC polynomial configuration
spi_crc_polynomial_get	Get SPI CRC polynomial
spi_crc_enable	Enable SPI CRC
spi_crc_value_get	Get CRC result of SPI receive/transmit
spi_hardware_cs_output_enable	Enable hardware CS output
spi_software_cs_internal_level_set	Set software CS internal level
spi_frame_bit_num_set	Set the number of frame bits
spi_half_duplex_direction_set	Set transfer direction of single-wire bidirectional half-duplex mode
spi_enable	Enable SPI
i2s_default_para_init	Set an initial value for the I²S initialization structure
i2s_init	Initialize I²S
i2s_enable	Enable I²S
spi_i2s_interrupt_enable	Enable SPI/I²S interrupts
spi_i2s_dma_transmitter_enable	Enable SPI/I²S DMA transmit

spi_i2s_dma_receiver_enable	Enable SPI/I²S DMA receive
spi_i2s_data_transmit	SPI/I²S transmits data
spi_i2s_data_receive	SPI/I²S receives data
spi_i2s_flag_get	Get SPI/I²S flags
spi_i2s_flag_clear	Clear SPI/I²S flags

5.18.1 spi_i2s_reset function

The table below describes the function spi_i2s_reset.

Table 393. spi_i2s_reset function

Name	Description
Function name	spi_i2s_reset
Function prototype	void spi_i2s_reset(spi_type *spi_x);
Function description	Reset SPI/I²S registers to their reset values.
Input parameter 1	spi_x: select SPI peripherals This parameter can be SPI1, SPI2, SPI3, SPI4, I2S2EXT, I2S3EXT.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	crm_periph_reset();

Example:

```
spi_i2s_reset (SPI1);
```

5.18.2 spi_default_para_init function

The table below describes the function spi_default_para_init.

Table 394. spi_default_para_init function

Name	Description
Function name	spi_default_para_init
Function prototype	void spi_default_para_init(spi_init_type* spi_init_struct);
Function description	Set an initial value for the SPI initialization structure
Input parameter 1	spi_init_struct: *spi_init_type* pointer
Output parameter	NA
Return value	NA
Required preconditions	It is necessary to define a variable of spi_init_type before starting.
Called functions	NA

Example:

```
spi_init_type spi_init_struct;
spi_default_para_init (&spi_init_struct);
```

5.18.3 spi_init function

The table below describes the function spi_init.

Table 395. spi_init function

Name	Description
Function name	spi_init
Function prototype	void spi_init(spi_type* spi_x, spi_init_type* spi_init_struct);
Function description	Initialize SPI
Input parameter 1	spi_x: select SPI peripherals This parameter can be SPI1, SPI2, SPI3, SPI4.
Input parameter 2	spi_init_struct: *spi_init_type* pointer
Output parameter	NA
Return value	NA
Required preconditions	It is necessary to define a variable of spi_init_type before starting.
Called functions	NA

spi_init_type is defined in the at32f403a_407_spi.h:

```
typedef struct
{
  spi_transmission_mode_type      transmission_mode;
  spi_master_slave_mode_type      master_slave_mode;
  spi_mclk_freq_div_type          mclk_freq_division;
  spi_first_bit_type              first_bit_transmission;
  spi_frame_bit_num_type          frame_bit_num;
  spi_clock_polarity_type         clock_polarity;
  spi_clock_phase_type            clock_phase;
  spi_cs_mode_type                cs_mode_selection;
} spi_init_type;
```

spi_transmission_mode

SPI transmission mode.

SPI_TRANSMIT_FULL_DUPLEX: Two-wire unidirectional full-duplex mode
SPI_TRANSMIT_SIMPLEX_RX: Two-wire unidirectional receive-only mode
SPI_TRANSMIT_HALF_DUPLEX_RX: Single-wire bidirectional receive-only mode
SPI_TRANSMIT_HALF_DUPLEX_TX: Single-wire bidirectional transmit-only mode

master_slave_mode

Master/slave mode selection.

SPI_MODE_SLAVE: Slave mode
SPI_MODE_MASTER: Master mode

mclk_freq_division

Frequency division factor selection.

SPI_MCLK_DIV_2: Divided by 2
SPI_MCLK_DIV_4: Divided by 4
SPI_MCLK_DIV_8: Divided by 8
SPI_MCLK_DIV_16: Divided by 16
SPI_MCLK_DIV_32: Divided by 32
SPI_MCLK_DIV_64: Divided by 64
SPI_MCLK_DIV_128: Divided by 128

SPI_MCLK_DIV_256: Divided by 256
SPI_MCLK_DIV_512: Divided by 512
SPI_MCLK_DIV_1024: Divided by 1024

first_bit_transmission

SPI MSB-first/LSB-first selection
SPI_FIRST_BIT_MSB: MSB-first
SPI_FIRST_BIT_LSB: LSB-first

frame_bit_num

Set the number of bits in a frame
SPI_FRAME_8BIT: 8-bit data in a frame
SPI_FRAME_16BIT: 16-bit data in a frame

clock_polarity

Select Clock polarity.
SPI_CLOCK_POLARITY_LOW: Clock output low in idle state
SPI_CLOCK_POLARITY_HIGH: Clock output high in idle state

clock_phase

Select clock phase.
SPI_CLOCK_PHASE_1EDGE: Sample on the first clock edge
SPI_CLOCK_PHASE_2EDGE: Sample on the second clock edge

cs_mode_selection

Select CS mode.
SPI_CS_HARDWARE_MODE: Hardware CS mode
SPI_CS_SOFTWARE_MODE: Software CS mode

Example:

```
spi_init_type spi_init_struct;
spi_default_para_init(&spi_init_struct);
spi_init_struct.transmission_mode = SPI_TRANSMIT_FULL_DUPLEX;
spi_init_struct.master_slave_mode = SPI_MODE_MASTER;
spi_init_struct.mclk_freq_division = SPI_MCLK_DIV_8;
spi_init_struct.first_bit_transmission = SPI_FIRST_BIT_MSB;
spi_init_struct.frame_bit_num = SPI_FRAME_16BIT;
spi_init_struct.clock_polarity = SPI_CLOCK_POLARITY_LOW;
spi_init_struct.clock_phase = SPI_CLOCK_PHASE_2EDGE;
spi_init_struct.cs_mode_selection = SPI_CS_SOFTWARE_MODE;
spi_init(SPI1, &spi_init_struct);
```

5.18.4 spi_crc_next_transmit function

The table below describes the function spi_crc_next_transmit.

Table 396. spi_crc_next_transmit function

Name	Description
Function name	spi_crc_next_transmit
Function prototype	void spi_crc_next_transmit(spi_type* spi_x);
Function description	The next data to be sent is CRC command
Input parameter 1	spi_x: select SPI peripherals This parameter can be SPI1, SPI2, SPI3, SPI4.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
spi_crc_next_transmit (SPI1);
```

5.18.5 spi_crc_polynomial_set function

The table below describes the function spi_crc_polynomial_set.

Table 397. spi_crc_polynomial_set function

Name	Description
Function name	spi_crc_polynomial_set
Function prototype	void spi_crc_polynomial_set(spi_type* spi_x, uint16_t crc_poly);
Function description	Set SPI CRC polynomial
Input parameter 1	spi_x: select SPI peripherals This parameter can be SPI1, SPI2, SPI3, SPI4.
Input parameter 2	crc_poly: CRC polynomial Value is 0x0000~0xFFFF
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/*set spi crc polynomial value */
spi_crc_polynomial_set (SPI1, 0x07);
```

5.18.6 spi_crc_polynomial_get function

The table below describes the function spi_crc_polynomial_get.

Table 398. spi_crc_polynomial_get function

Name	Description
Function name	spi_crc_polynomial_get
Function prototype	uint16_t spi_crc_polynomial_get(spi_type* spi_x);
Function description	Get SPI CRC polynomial
Input parameter 1	spi_x: select SPI peripheral This parameter can be SPI1, SPI2, SPI3, SPI4.
Output parameter	NA
Return value	CRC polynomial Value is 0x0000~0xFFFF
Required preconditions	NA
Called functions	NA

Example:

```
/*get spi crc polynomial value */
uint16_t crc_poly;
crc_poly = spi_crc_polynomial_get (SPI1);
```

5.18.7 spi_crc_enable function

The table below describes the function spi_crc_enable.

Table 399. spi_crc_enable function

Name	Description
Function name	spi_crc_enable
Function prototype	void spi_crc_enable(spi_type* spi_x, confirm_state new_state);
Function description	Enable SPI CRC
Input parameter 1	spi_x: select SPI peripheral This parameter can be SPI1, SPI2, SPI3, SPI4.
Input parameter 2	new_state: Enable or disable This parameter can be FALSE or TRUE
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* spi crc enable */
spi_crc_enable (SPI1, TRUE);
```

5.18.8 spi_crc_value_get function

The table below describes the function spi_crc_value_get.

Table 400. spi_crc_value_get function

Name	Description
Function name	spi_crc_value_get
Function prototype	uint16_t spi_crc_value_get(spi_type* spi_x, spi_crc_direction_type crc_direction);
Function description	Get SPI receive/transmit CRC result
Input parameter 1	spi_x: select SPI peripheral This parameter can be SPI1, SPI2, SPI3, SPI4.
Input parameter 2	*crc_direction*: Select receive/transmit CRC
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

crc_direction

Select receive/transmit CRC

SPI_CRC_RX: Receive CRC

SPI_CRC_TX: Transmit CRC

Example:

```
/* get spi rx & tx crc enable */
uint16_t spi_rx_crc, spi_tx_crc;
spi_rx_crc = spi_crc_value_get (SPI1, SPI_CRC_RX);
spi_tx_crc = spi_crc_value_get (SPI1, SPI_CRC_TX);
```

5.18.9 spi_hardware_cs_output_enable function

The table below describes the function spi_hardware_cs_output_enable.

Table 401. spi_hardware_cs_output_enable function

Name	Description
Function name	spi_hardware_cs_output_enable
Function prototype	void spi_hardware_cs_output_enable(spi_type* spi_x, confirm_state new_state);
Function description	Enable hardware CS output
Input parameter 1	spi_x: select SPI peripheral This parameter can be SPI1, SPI2, SPI3, SPI4.
Input parameter 2	new_state: Enable or disable This parameter can FALSE or TRUE
Output parameter	NA
Return value	NA
Required preconditions	This setting is applicable to SPI master mode only.
Called functions	NA

Example:

```
/* enable the hardware cs output */
spi_hardware_cs_output_enable (SPI1, TRUE);
```

5.18.10 spi_software_cs_internal_level_set function

The table below describes the function spi_software_cs_internal_level_set.

Table 402. spi_software_cs_internal_level_set function

Name	Description
Function name	spi_software_cs_internal_level_set
Function prototype	void spi_software_cs_internal_level_set(spi_type* spi_x, spi_software_cs_level_type level);
Function description	Set software CS internal level
Input parameter 1	spi_x: select SPI peripheral This parameter can be SPI1, SPI2, SPI3, SPI4.
Input parameter 2	*level*: set software CS internal level
Output parameter	NA
Return value	NA
Required preconditions	1. This setting is applicable to software CS mode only; 2. In master mode, the “level” value must be “SPI_SWCS_INTERNAL_LEVEL_HIGHT”.
Called functions	NA

level

Set software CS internal level

SPI_SWCS_INTERNAL_LEVEL_LOW: Software CS internal low level

SPI_SWCS_INTERNAL_LEVEL_HIGHT: Software CS internal high level

Example:

```
/* set the internal level high */
spi_software_cs_internal_level_set (SPI1, SPI_SWCS_INTERNAL_LEVEL_HIGHT);
```

5.18.11 spi_frame_bit_num_set function

The table below describes the function spi_frame_bit_num_set.

Table 403. spi_frame_bit_num_set function

Name	Description
Function name	spi_frame_bit_num_set
Function prototype	void spi_frame_bit_num_set(spi_type* spi_x, spi_frame_bit_num_type bit_num);
Function description	Set the number of bits in a frame
Input parameter 1	spi_x: select SPI peripheral This parameter can be SPI1, SPI2, SPI3, SPI4.
Input parameter 2	*bit_num*: Set the number of bits in a frame
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

bit_num

Set the number of bits in a frame

SPI_FRAME_8BIT: 8-bit data in a frame

SPI_FRAME_16BIT: 16-bit data in a frame

Example:

```
/* set the data frame bit num as 8 */
spi_frame_bit_num_set (SPI1, SPI_FRAME_8BIT);
```

5.18.12 spi_half_duplex_direction_set function

The table below describes the function spi_half_duplex_direction_set.

Table 404. spi_half_duplex_direction_set function

Name	Description
Function name	spi_half_duplex_direction_set
Function prototype	void spi_half_duplex_direction_set(spi_type* spi_x, spi_half_duplex_direction_type direction);
Function description	Set the transfer direction of single-wire bidirectional half-duplex mode
Input parameter 1	spi_x: select SPI peripheral This parameter can be SPI1, SPI2, SPI3, SPI4.
Input parameter 2	*direction*: transfer direction
Output parameter	NA
Return value	NA
Required preconditions	This setting is applicable to the single-wire bidirectional half-duplex mode only.
Called functions	NA

direction

Transfer direction

SPI_HALF_DUPLEX_DIRECTION_RX: Receive

SPI_HALF_DUPLEX_DIRECTION_TX: Transmit

Example:

```
/* set the data transmission direction as transmit */
```

```
spi_half_duplex_direction_set (SPI1, SPI_HALF_DUPLEX_DIRECTION_TX);
```

5.18.13 spi_enable function

The table below describes the function spi_enable.

Table 405. spi_enable function

Name	Description
Function name	spi_enable
Function prototype	void spi_enable(spi_type* spi_x, confirm_state new_state);
Function description	Enable SPI
Input parameter 1	spi_x: select SPI peripheral This parameter can be SPI1, SPI2, SPI3, SPI4.
Input parameter 2	new_state: Enable or disable This parameter can be FALSE or TRUE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable spi */
spi_enable (SPI1, TRUE);
```

5.18.14 i2s_default_para_init function

The table below describes the function i2s_default_para_init.

Table 406. i2s_default_para_init function

Name	Description
Function name	i2s_default_para_init
Function prototype	void i2s_default_para_init(i2s_init_type* i2s_init_struct);
Function description	Set an initial value for the I²S initialization structure
Input parameter 1	i2s_init_struct: *spi_i2s_flag* pointer
Output parameter	NA
Return value	NA
Required preconditions	It is necessary to define a variable of i2s_init_type before starting.
Called functions	NA

Example:

```
i2s_init_type i2s_init_struct;
i2s_default_para_init (&i2s_init_struct);
```

5.18.15 i2s_init function

The table below describes the function i2s_init.

Table 407. i2s_init function

Name	Description
Function name	i2s_init
Function prototype	void i2s_init(spi_type* spi_x, i2s_init_type* i2s_init_struct);
Function description	Initialize I²S
Input parameter 1	spi_x: select SPI peripheral This parameter can be SPI1, SPI2, SPI3, SPI4, I2S2EXT, I2S3EXT.
Input parameter 2	i2s_init_struct: *spi_i2s_flag* pointer
Output parameter	NA
Return value	NA
Required preconditions	It is necessary to define a variable of i2s_init_type before starting.
Called functions	NA

i2s_init_type is defined in the at32f421_spi.h:

```
typedef struct
{
  i2s_operation_mode_type          operation_mode;
  i2s_audio_protocol_type          audio_protocol;
  i2s_audio_sampling_freq_type     audio_sampling_freq;
  i2s_data_channel_format_type     data_channel_format;
  i2s_clock_polarity_type          clock_polarity;
  confirm_state                    mclk_output_enable;
} i2s_init_type;
```

operation_mode

I²S transfer mode

I2S_MODE_SLAVE_TX: I2S slave transmit
I2S_MODE_SLAVE_RX: I2S slave receive
I2S_MODE_MASTER_TX: I2S master transmit
I2S_MODE_MASTER_RX: I2S master receive

audio_protocol

I²S audio protocol standards

I2S_AUDIO_PROTOCOL_PHILLIPS: Phillips
I2S_AUDIO_PROTOCOL_MSB: MSB aligned (left-aligned)
I2S_AUDIO_PROTOCOL_LSB: LSB aligned (right-aligned)
I2S_AUDIO_PROTOCOL_PCM_SHORT: PCM short frame syncrhonization
I2S_AUDIO_PROTOCOL_PCM_LONG: PCM long frame syncrhonization

audio_sampling_freq

I²S audio sampling frequency.

I2S_AUDIO_FREQUENCY_DEFAULT:
Kept at its reset value (sampling frequency changes with SCLK)
I2S_AUDIO_FREQUENCY_8K: I2S sampling frequency 8K
I2S_AUDIO_FREQUENCY_11_025K: I2S sampling frequency 11.025K
I2S_AUDIO_FREQUENCY_16K: I2S sampling frequency 16K
I2S_AUDIO_FREQUENCY_22_05K: I2S sampling frequency 22.05K
I2S_AUDIO_FREQUENCY_32K: I2S sampling frequency 32K

I2S_AUDIO_FREQUENCY_44_1K: I2S sampling frequency 44.1K
I2S_AUDIO_FREQUENCY_48K: I2S sampling frequency 48K
I2S_AUDIO_FREQUENCY_96K: I2S sampling frequency 96K
I2S_AUDIO_FREQUENCY_192K: I2S sampling frequency 192K

data_channel_format

I²S data/channel bits format

I2S_DATA_16BIT_CHANNEL_16BIT: 16-bit data, 16-bit channel
I2S_DATA_16BIT_CHANNEL_32BIT: 16-bit data, 32-bit channel
I2S_DATA_24BIT_CHANNEL_32BIT: 24-bit data, 32-bit channel
I2S_DATA_32BIT_CHANNEL_32BIT: 32-bit data, 32-bit channel

clock_polarity

I²S clock polarity

I2S_CLOCK_POLARITY_LOW: Clock ouput low in idle state
I2S_CLOCK_POLARITY_HIGH: Clock ouput high in idle state

mclk_output_enable

Enable mclk clock output

This parameter can be FALSE or TURE.

Example:

```
i2s_init_type i2s_init_struct;
i2s_default_para_init(&i2s_init_struct);
i2s_init_struct.audio_protocol = I2S_AUDIO_PROTOCOL_PHILLIPS;
i2s_init_struct.data_channel_format = I2S_DATA_16BIT_CHANNEL_32BIT;
i2s_init_struct.mclk_output_enable = FALSE;
i2s_init_struct.audio_sampling_freq = I2S_AUDIO_FREQUENCY_48K;
i2s_init_struct.clock_polarity = I2S_CLOCK_POLARITY_LOW;
i2s_init_struct.operation_mode = I2S_MODE_MASTER_TX;
i2s_init(SPI2, &i2s_init_struct);
```

5.18.16 i2s_enable function

The table below describes the function i2s_enable.

Table 408. i2s_enable function

Name	Description
Function name	i2s_enable
Function prototype	void i2s_enable(spi_type* spi_x, confirm_state new_state);
Function description	Enable I²S
Input parameter 1	spi_x: select SPI peripheral This parameter can be SPI1, SPI2, SPI3, SPI4.
Input parameter 2	new_state: Enable or disable This parameter can be FALSE or TRUE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable i2s*/
```

```
i2s_enable (SPI1, TRUE);
```

5.18.17 spi_i2s_interrupt_enable function

The table below describes the function spi_i2s_interrupt_enable.

Table 409. spi_i2s_interrupt_enable function

Name	Description
Function name	spi_i2s_interrupt_enable
Function prototype	void spi_i2s_interrupt_enable(spi_type* spi_x, uint32_t spi_i2s_int, confirm_state new_state);
Function description	Enable SPI/I²S interrupts
Input parameter 1	spi_x: select SPI peripheral This parameter can be SPI1, SPI2, SPI3, SPI4, I2S2EXT, I2S3EXT.
Input parameter 2	*spi_i2s_int*: select SPI interrupts
Input parameter 3	new_state: Enable or disable This parameter can be FALSE or TURE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

spi_i2s_int

Select SPI/I²S interrupts.

SPI_I2S_ERROR_INT: SPI/I²S error interrupts (including CRC error, overflow error, underflow error and mode error)

SPI_I2S_RDBF_INT: Receive data buffer full

SPI_I2S_TDBE_INT: Transmit data buffe empty

Example:

```
/* enable the specified spi/i2s interrupts */
spi_i2s_interrupt_enable (SPI1, SPI_I2S_ERROR_INT);
spi_i2s_interrupt_enable (SPI1, SPI_I2S_RDBF_INT);
spi_i2s_interrupt_enable (SPI1, SPI_I2S_TDBE_INT);
```

5.18.18 spi_i2s_dma_transmitter_enable function

The table below describes the function spi_i2s_dma_transmitter_enable.

Table 410. spi_i2s_dma_transmitter_enable function

Name	Description
Function name	spi_i2s_dma_transmitter_enable
Function prototype	void spi_i2s_dma_transmitter_enable(spi_type* spi_x, confirm_state new_state);
Function description	Enable SPI/I²S DMA transmitter
Input parameter 1	spi_x: select SPI peripheral This parameter can be SPI1, SPI2, SPI3, SPI4, I2S2EXT, I2S3EXT.
Input parameter 2	new_state: Enable or disable This parameter can be FALSE or TRUE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable spi transmitter dma */
spi_i2s_dma_transmitter_enable (SPI1, TRUE);
```

5.18.19 spi_i2s_dma_receiver_enable function

The table below describes the function spi_i2s_dma_receiver_enable.

Table 411. spi_i2s_dma_receiver_enable function

Name	Description
Function name	spi_i2s_dma_receiver_enable
Function prototype	void spi_i2s_dma_receiver_enable(spi_type* spi_x, confirm_state new_state);
Function description	Enable SPI/I²S DMA receiver
Input parameter 1	spi_x: select SPI peripheral This parameter can be SPI1, SPI2, SPI3, SPI4, I2S2EXT, I2S3EXT.
Input parameter 2	new_state: Enable or disable This parameter can be FALSE or TRUE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable spi dma transmitter */
spi_i2s_dma_transmitter_enable (SPI1, TRUE);
```

5.18.20 spi_i2s_data_transmit function

The table below describes the function spi_i2s_data_transmit.

Table 412. spi_i2s_data_transmit function

Name	Description
Function name	spi_i2s_data_transmit
Function prototype	void spi_i2s_data_transmit(spi_type* spi_x, uint16_t tx_data);
Function description	SPI/I²S sends data
Input parameter 1	spi_x: select SPI peripheral This parameter can be SPI1, SPI2, SPI3, SPI4, I2S2EXT, I2S3EXT.
Input parameter 2	tx_data: data to send Value range (for 8-bit bit in a frame): 0x00~0xFF Value range (for16-bit in a frame): 0x0000~0xFFFF
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* spi data transmit */
uint16_t tx_data = 0x6666;
spi_i2s_data_transmit (SPI1, tx_data);
```

5.18.21 spi_i2s_data_receive function

The table below describes the function spi_i2s_data_receive.

Table 413. spi_i2s_data_receive function

Name	Description
Function name	spi_i2s_data_receive
Function prototype	uint16_t spi_i2s_data_receive(spi_type* spi_x);
Function description	SPI/I²S receives data
Input parameter 1	spi_x: select SPI peripheral This parameter can be SPI1, SPI2, SPI3, SPI4, I2S2EXT, I2S3EXT.
Output parameter	rx_data: data to receive Value range (for 8-bit bit in a frame): 0x00~0xFF Value range (for16-bit in a frame): 0x0000~0xFFFF
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* spi data receive */
uint16_t rx_data = 0;
rx_data = spi_i2s_data_receive (SPI1);
```

5.18.22 spi_i2s_flag_get function

The table below describes the function spi_i2s_flag_get.

Table 414. spi_i2s_flag_get function

Name	Description
Function name	spi_i2s_flag_get
Function prototype	flag_status spi_i2s_flag_get(spi_type* spi_x, uint32_t spi_i2s_flag);
Function description	Get SPI/I²S flags
Input parameter 1	spi_x: select SPI peripheral This parameter can be SPI1, SPI2, SPI3, SPI4, I2S2EXT, I2S3EXT.
Input parameter 2	spi_i2s_flag: Select flags Refer to the “spi_i2s_flag” description below for details.
Output parameter	NA
Return value	flag_status: flag status This parameter can be SET or RESET.
Required preconditions	NA
Called functions	NA

spi_i2s_flag

SPI/I²S is used to select flags, including:

SPI_I2S_RDBF_FLAG: SPI/I²S receive data buffer full
SPI_I2S_TDBE_FLAG: SPI/I²S transmit data buffer empty
I2S_ACS_FLAG: I2S Audio channel state (indicating left/right channel)
I2S_TUERR_FLAG: I2S transmitter underload error
SPI_CCERR_FLAG: SPI CRC error
SPI_MMERR_FLAG: SPI master mode error
SPI_I2S_ROERR_FLAG: SPI/I²S receive overflow error
SPI_I2S_BF_FLAG: SPI/I²S busy

Example:

```
/* get receive data buffer full flag */
flag_status status;
status = spi_i2s_flag_get(SPI1, SPI_I2S_RDBF_FLAG);
```

5.18.23 spi_i2s_interrupt_flag_get function

The table below describes the function spi_i2s_interrupt_flag_get.

Table 415. spi_i2s_interrupt_flag_get function

Name	Description
Function name	spi_i2s_interrupt_flag_get
Function prototype	flag_status spi_i2s_interrupt_flag_get(spi_type* spi_x, uint32_t spi_i2s_flag);
Function description	Get the selected SPI/I²S interrupt flag
Input parameter 1	spi_x: select SPI peripheral This parameter can be SPI1, SPI2, SPI3, SPI4, I2S2EXT or I2S3EXT.
Input parameter 2	spi_i2s_flag: Select flags Refer to the “spi_i2s_flag” description below for details.
Output parameter	NA
Return value	flag_status: flag status This parameter can be SET or RESET.
Required preconditions	NA
Called functions	NA

spi_i2s_flag

SPI/I²S is used to select flags, including:

SPI_I2S_RDBF_FLAG: SPI/I2S receive data buffer full
SPI_I2S_TDBE_FLAG: SPI/I2S transmit data buffer empty
I2S_TUERR_FLAG: I2S transmitter underload error
SPI_CCERR_FLAG: SPI CRC error
SPI_MMERR_FLAG: SPI master mode error
SPI_I2S_ROERR_FLAG: SPI/I2S receive overflow error

Example

```
/* get receive data buffer full flag */
flag_status status;
status = spi_i2s_interrupt_flag_get(SPI, SPI_I2S_RDBF_FLAG);
```

5.18.24 spi_i2s_flag_clear function

The table below describes the function spi_i2s_flag_clear.

Table 416. spi_i2s_flag_clear function

Name	Description
Function name	spi_i2s_flag_clear
Function prototype	void spi_i2s_flag_clear(spi_type* spi_x, uint32_t spi_i2s_flag)
Function description	Clear SPI/I²S flags
Input parameter 1	spi_x: select SPI peripheral This parameter can be SPI1, SPI2, SPI3, SPI4, I2S2EXT, I2S3EXT.
Input parameter 2	*spi_i2s_flag:* select a flag to clear Refer to the “spi_i2s_flag” description below for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

spi_i2s_flag:

SPI/I²S is used for flag selection, including:

SPI_I2S_RDBF_FLAG: SPI/I²S receive data buffer full

I2S_TUERR_FLAG: I2S transmitter underload error

SPI_CCERR_FLAG: SPI CRC error

SPI_MMERR_FLAG: SPI master mode error

SPI_I2S_ROERR_FLAG: SPI/I²S receive overflow error

*Note: the SPI_I2S_TDBE_FLAG (*SPI/I²S transmit data buffer empty*), the I2S_ACS_FLAG (*Audio channel state*) and the SPI_I2S_BF_FLAG (*SPI/I²S busy*) are all set and cleared by hardware to indicate communication state, without the intervention of software.*

Example:

```
/* clear receive data buffer full flag */
spi_i2s_flag_clear (SPI1, SPI_I2S_RDBF_FLAG);
```

5.19 SysTick

The SysTick register structure SysTick_Type is defined in the "core_cm4.h":

```
typedef struct
{
    ...

} SysTick_Type;
```

The table below gives a list of the SysTick registers.

Table 417. Summary of SysTick registers

Register	Description
ctrl	Controls status register
load	Reload value register
val	Current counter value register
calib	Calibration register

The table below gives a list of SysTick library functions.

Table 418. Summary of SysTick library functions

Function name	Description
systick_clock_source_config	Configure SysTick clock sources
SysTick_Config	Configure SysTick counter reload value and interrupts

5.19.1 systick_clock_source_config function

The table below describes the function systick_clock_source_config.

Table 419. systick_clock_source_config function

Name	Description
Function name	systick_clock_source_config
Function prototype	void systick_clock_source_config(systick_clock_source_type source);
Function description	Configure SysTick clock source
Input parameter 1	Source: systick clock source
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

source

SYSTICK_CLOCK_SOURCE_AHBCLK_DIV8: AHB/8 as SysTick clock

SYSTICK_CLOCK_SOURCE_AHBCLK_NODIV: AHB as SysTick clock

Example:

```
/* config systick clock source */
systick_clock_source_config(SYSTICK_CLOCK_SOURCE_AHBCLK_NODIV);
```

5.19.2 SysTick_Config function

The table below describes the function SysTick_Config.

Table 420. SysTick_Config function

Name	Description
Function name	SysTick_Config
Function prototype	uint32_t SysTick_Config(uint32_t ticks);
Function description	Configure SysTick counter reload value and interrupts
Input parameter 1	Ticks: SysTick counter interrupt reload value
Output parameter	NA
Return value	Return the setting status of this function, success (0) or failure (1)
Required preconditions	NA
Called functions	NA

Example:

```
/* config systick reload value and enable interrupt */
SysTick_Config(1000);
```

5.20 TMR

The TMR register structure tmr_type is defined in the”at32f403a_407_tmr.h”:

```
/**
  * @brief type define tmr register all
  */
typedef struct
{


} tmr_type;
```

The table below gives a list of the TMR registers.

Table 421. Summary of TMR registers

Register	Description
ctrl1	TMR control register 1
ctrl2	TMR control register 2
stctrl	TMR slave timer control register
iden	TMR DMA/ interrupt enable register
ists	TMR interrupt status register
swevt	TMR software event register
cm1	TMR channel mode register 1
cm2	TMR channel mode register 2
cctrl	TMR channel control register
cval	TMR counter value register
div	TMR division register
pr	TMR period register
rpr	TMR repetition period channel
c1dt	TMR channel 1 data register
c2dt	TMR channel 2 data register
c3dt	TMR channel 3 data register
c4dt	TMR channel 4 data register
brk	TMR break register
dmactrl	TMR DMA control register
dmadt	TMR DMA data register

The table below gives a list of TMR library functions.

Table 422. Summary of TMR library functions

Function name	Description
tmr_reset	TMR is reset by CRM reset register
tmr_counter_enable	Enable or disable TMR
tmr_output_default_para_init	Initialize TMR output default parameters
tmr_input_default_para_init	Initialize TMR input default parameters
tmr_brkdt_default_para_init	Initialize TMR brkdt default parameters
tmr_base_init	Initialize TMR period and division
tmr_clock_source_div_set	Set TMR clock source frequency division factor
tmr_cnt_dir_set	Set TMR counter direction
tmr_repetition_counter_set	Set repetition period register
tmr_counter_value_set	Set TMR counter value
tmr_counter_value_get	Get TMR counter value
tmr_div_value_set	Set TMR division value
tmr_div_value_get	Get TMR division value
tmr_output_channel_config	Configure TMR output channels
tmr_output_channel_mode_select	Select TMR output channel mode
tmr_period_value_set	Set TMR period value
tmr_period_value_get	Get TMR period value
tmr_channel_value_set	Set TMR channel value
tmr_channel_value_get	Get TMR channel value
tmr_period_buffer_enable	Enable or disable TMR periodic buffer
tmr_output_channel_buffer_enable	Enable or disable TMR output channel buffer
tmr_output_channel_immediately_set	TMR output channel enable immediately
tmr_output_channel_switch_set	Set TMR output channel switch
tmr_one_cycle_mode_enable	Enable or disable TMR one-cycle mode
tmr_32_bit_function_enable	Enable or disable TMR 32-bit function (plus mode)
tmr_overflow_request_source_set	Select TMR overflow event source
tmr_overflow_event_disable	Enable or disable TMR overflow event generation
tmr_channel_enable	Enable or disable TMR channel
tmr_input_channel_filter_set	Set TMR input channel filter
tmr_pwm_input_config	Configure TMR pwm input
tmr_channel1_input_select	Select TMR channel 1 input
tmr_input_channel_divider_set	Set TMR input channel divider
tmr_primary_mode_select	Select TMR master mode
tmr_sub_mode_select	Select TMR slave timer mode
tmr_channel_dma_select	Select TMR channel DMA request source
tmr_hall_select	Select TMR hall mode
tmr_channel_buffer_enable	Enable or disable TMR channel buffer
tmr_trigger_input_select	Select TMR slave timer trigger input
tmr_sub_sync_mode_set	Set TMR slave timer synchronization mode
tmr_dma_request_enable	Enable or disable TMR DMA request
tmr_interrupt_enable	Enable or disable TMR interrupt
tmr_flag_get	Get TMR flags
tmr_flag_clear	Clear TMR flags

tmr_event_sw_trigger	Software trigger TMR event
tmr_output_enable	Enable or disable TMR output
tmr_internal_clock_set	Set TMR internal clock
tmr_output_channel_polarity_set	Set TMR output channel polarity
tmr_external_clock_config	Set TMR external clock
tmr_external_clock_mode1_config	Set TMR external clock mode 1
tmr_external_clock_mode2_config	Set TMR external clock mode 2
tmr_encoder_mode_config	Set TMR encode mode
tmr_force_output_set	Set TMR forced output
tmr_dma_control_config	Set TMR DMA control
tmr_brkdt_config	Set TMR break mode and dead-time
tmr_event_sw_trigger	Software trigger TMR event

5.20.1 tmr_reset function

The table below describes the function tmr_reset.

Table 423. tmr_reset function

Name	Description
Function name	tmr_reset
Function prototype	void tmr_reset(tmr_type *tmr_x);
Function description	TMR is reset by CRM reset register.
Input parameter	tmr_x: select TMR peripheral, including: TMR1, TMR2, TMR3, TMR4, TMR5, TMR6, TMR7, TMR8, TMR9, TMR10, TMR11, TMR12, TMR13, TMR14
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	crm_periph_reset();

Example:

```
tmr_reset(TMR1);
```

5.20.2 tmr_counter_enable function

The table below describes the function tmr_counter_enable.

Table 424. tmr_counter_enable function

Name	Description
Function name	tmr_counter_enable
Function prototype	void tmr_counter_enable(tmr_type *tmr_x, confirm_state new_state);
Function description	Enable or disable TMR
Input parameter 1	tmr_x: select TMR peripheral, including: TMR1, TMR2, TMR3, TMR4, TMR5, TMR6, TMR7, TMR8, TMR9, TMR10, TMR11, TMR12, TMR13, TMR14
Input parameter 2	new_state: indicates counter status, ON (TRUE) or OFF (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
tmr_counter_enable(TMR1, TRUE);
```

5.20.3 tmr_output_default_para_init function

The table below describes the function tmr_output_default_para_init.

Table 425. tmr_output_default_para_init function

Name	Description
Function name	tmr_output_default_para_init
Function prototype	void tmr_output_default_para_init(tmr_output_config_type *tmr_output_struct);
Function description	Initialize tmr output default parameters
Input parameter	tmr_output_struct: tmr_output_config_type pointer
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

The table below describes the default values of members of the function tmr_output_struct.

Table 426. tmr_output_struct default values

Member	Default values
oc_mode	TMR_OUTPUT_CONTROL_OFF
oc_idle_state	FALSE
occ_idle_state	FALSE
oc_polarity	TMR_OUTPUT_ACTIVE_HIGH
occ_polarity	TMR_OUTPUT_ACTIVE_HIGH
oc_output_state	FALSE
occ_output_state	FALSE

Example:

```
tmr_output_config_type tmr_output_struct;
tmr_output_default_para_init(&tmr_output_struct);
```

5.20.4 tmr_input_default_para_init function

The table below describes the function tmr_input_default_para_init.

Table 427. tmr_input_default_para_init function

Name	Description
Function name	tmr_input_default_para_init
Function prototype	void tmr_input_default_para_init(tmr_input_config_type *tmr_input_struct);
Function description	Initialize TMR input default parameters
Input parameter	tmr_input_struct: tmr_input_config_type pointer
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

The table below describes the default values of members of the function tmr_input_struct.

Table 428. tmr_input_struct default values

Member	Default values
input_channel_select	TMR_SELECT_CHANNEL_1
input_polarity_select	TMR_INPUT_RISING_EDGE
input_mapped_select	TMR_CC_CHANNEL_MAPPED_DIRECT
input_filter_value	0x0

Example:

```
tmr_input_config_type tmr_input_struct;
tmr_input_default_para_init(&tmr_input_struct);
```

5.20.5 tmr_brkdt_default_para_init function

The table below describes the function tmr_brkdt_default_para_init.

Table 429. tmr_brkdt_default_para_init function

Name	Description
Function name	tmr_brkdt_default_para_init
Function prototype	void tmr_brkdt_default_para_init(tmr_brkdt_config_type *tmr_brkdt_struct);
Function description	Initialize TMR brkdt default parameters
Input parameter	tmr_brkdt_struct: tmr_brkdt_config_type pointer
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

The table below describes the default values of members of the function tmr_brkdt_struct.

Table 430. tmr_brkdt_struct default values

Member	Default values
deadtime	0x0
brk_polarity	TMR_BRK_INPUT_ACTIVE_LOW
wp_level	TMR_WP_OFF
auto_output_enable	FALSE
fcsoen_state	FALSE
fcsodis_state	FALSE
brk_enable	FALSE

Example:

```
tmr_brkdt_config_type tmr_brkdt_struct;
tmr_brkdt_default_para_init(&tmr_brkdt_struct);
```

5.20.6 tmr_base_init function

The table below describes the function tmr_base_init.

Table 431. tmr_base_init function

Name	Description
Function name	tmr_base_init
Function prototype	void tmr_base_init(tmr_type* tmr_x, uint32_t tmr_pr, uint32_t tmr_div);
Function description	Initialize TMR period and division
Input parameter 1	tmr_x: TMR peripheral including: TMR1, TMR2, TMR3, TMR4, TMR5, TMR6, TMR7, TMR8, TMR9, TMR10, TMR11, TMR12, TMR13, TMR14
Input parameter 2	tmr_pr: timer period value, for 16-bit timer, it can be 0x0000~0xFFFF; for 32-bit timer, it can be 0x0000_0000~0xFFFF_FFFF
Input parameter 3	tmr_div: timer division value, 0x0000~0xFFFF
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
tmr_base_init(TMR1, 0xFFFF, 0xFFFF);
```

5.20.7 tmr_clock_source_div_set function

The table below describes the function tmr_clock_source_div_set.

Table 432. tmr_clock_source_div_set function

Name	Description
Function name	tmr_clock_source_div_set
Function prototype	void tmr_clock_source_div_set(tmr_type *tmr_x, tmr_clock_division_type tmr_clock_div);
Function description	Set TMR clock source division factor
Input parameter 1	tmr_x: TMR peripheral, including TMR1, TMR2, TMR3, TMR4, TMR5, TMR8, TMR9, TMR10, TMR11, TMR12, TMR13, TMR14
Input parameter 2	tmr_clock_div: timer clock source frequency division factor
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

tmr_clock_div

Select TMR clock source frequency division factor

TMR_CLOCK_DIV1: Divided by 1

TMR_CLOCK_DIV2: Divided by 2

TMR_CLOCK_DIV4: Divided by 4

Example:

```
tmr_clock_source_div_set(TMR1, TMR_CLOCK_DIV4);
```

5.20.8 tmr_cnt_dir_set function

The table below describes the function tmr_cnt_dir_set.

Table 433. tmr_cnt_dir_set function

Name	Description
Function name	tmr_cnt_dir_set
Function prototype	void tmr_cnt_dir_set(tmr_type *tmr_x, tmr_count_mode_type tmr_cnt_dir);
Function description	Set TMR counter direction
Input parameter 1	tmr_x: indicates the selected TMR peripheral, including TMR1, TMR2, TMR3, TMR4, TMR5, TMR8, TMR9, TMR10, TMR11, TMR12, TMR13, TMR14
Input parameter 2	tmr_cnt_dir: timer counting direction; refer to “tmr_cnt_dir” below
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

tmr_cnt_dir

Select timer counting direction.

TMR_COUNT_UP: Up counting

TMR_COUNT_DOWN: Down counting

TMR_COUNT_TWO_WAY_1: Center-aligned mode (up/down counting) 1

TMR_COUNT_TWO_WAY_2: Center-aligned mode (up/down counting) 2

TMR_COUNT_TWO_WAY_3: Center-aligned mode (up/down counting) 3

Example:

```
tmr_cnt_dir_set(TMR1, TMR_COUNT_UP);
```

5.20.9 tmr_repetition_counter_set function

The table below describes the function tmr_repetition_counter_set.

Table 434. tmr_repetition_counter_set function

Name	Description
Function name	tmr_repetition_counter_set
Function prototype	void tmr_repetition_counter_set(tmr_type *tmr_x, uint8_t tmr_rpr_value);
Function description	Set repetition period register (rpr)
Input parameter 1	tmr_x: TMR peripheral This parameter can be TMR1, TMR8.
Input parameter 2	tmr_rpr_value: timer repetition period value, it can be 0x00~0xFF
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
tmr_repetition_counter_set(TMR1, 0x10);
```

5.20.10 tmr_counter_value_set function

The table below describes the function tmr_counter_value_set.

Table 435. tmr_counter_value_set function

Name	Description
Function name	tmr_counter_value_set
Function prototype	void tmr_counter_value_set(tmr_type *tmr_x, uint32_t tmr_cnt_value);
Function description	Set TMR counter value
Input parameter 1	tmr_x: indicates the selected TMR peripheral It can be TMR1, TMR2, TMR3, TMR4, TMR5, TMR6, TMR7, TMR8, TMR9, TMR10, TMR11, TMR12, TMR13, TMR14
Input parameter 2	tmr_cnt_value: timer counter value, for 16-bit timer, it can be 0x0000~0xFFFF; For 32-bit timer, it can be 0x0000_0000~0xFFFF_FFFF
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
tmr_counter_value_set(TMR1, 0xFFFF);
```

5.20.11 tmr_counter_value_get function

The table below describes the function tmr_counter_value_get.

Table 436. tmr_counter_value_get function

Name	Description
Function name	tmr_counter_value_get
Function prototype	uint32_t tmr_counter_value_get(tmr_type *tmr_x);
Function description	Get TMR counter value
Input parameter	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR5, TMR6, TMR7, TMR8, TMR9, TMR10, TMR11, TMR12, TMR13, TMR14
Output parameter	NA
Return value	Timer counter value
Required preconditions	NA
Called functions	NA

Example:

```
uint32_t counter_value;
counter_value = tmr_counter_value_get(TMR1);
```

5.20.12 tmr_div_value_set function

The table below describes the function tmr_div_value_set.

Table 437. tmr_div_value_set function

Name	Description
Function name	tmr_div_value_set
Function prototype	void tmr_div_value_set(tmr_type *tmr_x, uint32_t tmr_div_value);
Function description	Set TMR frequency division value
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR5, TMR6, TMR7, TMR8, TMR9, TMR10, TMR11, TMR12, TMR13, TMR14
Input parameter 2	tmr_div_value: timer frequency division value. For 16-bit timer, it can be0x0000~0xFFFF; for 32-bit timer, it can be 0x0000_0000~0xFFFF_FFFF
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
tmr_div_value_set(TMR1, 0xFFFF);
```

5.20.13 tmr_div_value_get function

The table below describes the function tmr_div_value_get.

Table 438. tmr_div_value_get function

Name	Description
Function name	tmr_div_value_get
Function prototype	uint32_t tmr_div_value_get(tmr_type *tmr_x);
Function description	Get TMR frequency division value
Input parameter	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR5, TMR6, TMR7, TMR8, TMR9, TMR10, TMR11, TMR12, TMR13, TMR14
Output parameter	NA
Return value	Timer frequency division value
Required preconditions	NA
Called functions	NA

Example:

```
uint32_t div_value;
div_value = tmr_div_value_get(TMR1);
```

5.20.14 tmr_output_channel_config function

The table below describes the function tmr_output_channel_config.

Table 439. tmr_output_channel_config function

Name	Description
Function name	tmr_output_channel_config
Function prototype	void tmr_output_channel_config(tmr_type *tmr_x, tmr_channel_select_type tmr_channel, tmr_output_config_type *tmr_output_struct);
Function description	Configure TMR output channels
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR5, TMR8, TMR9, TMR10, TMR11, TMR12, TMR13, TMR14
Input parameter 2	tmr_channel: timer channel. Refer to tmr_channel for details.
Input parameter 3	tmr_output_struct: tmr_output_config_type pointer
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

tmr_channel

The tmr_channel is used to select a TMR channel.

TMR_SELECT_CHANNEL_1: Channel 1

TMR_SELECT_CHANNEL_2: Channel 2

TMR_SELECT_CHANNEL_3: Channel 3

TMR_SELECT_CHANNEL_4: Channel 4

tmr_output_config_type structure

tmr_output_config_type is defined in the at32f403a_407_tmr.h:

```
typedef struct
{
  tmr_output_control_mode_type      oc_mode;
  confirm_state                     oc_idle_state;
  confirm_state                     occ_idle_state;
  tmr_output_polarity_type          oc_polarity;
  tmr_output_polarity_type          occ_polarity;
  confirm_state                     oc_output_state;
  confirm_state                     occ_output_state;
} tmr_output_config_type;
```

oc_mode

Set output channel mode, that is to configure channel original signals (CxORAW).

TMR_OUTPUT_CONTROL_OFF: Disconnect channel output (CxOUT) from CxORAW

TMR_OUTPUT_CONTROL_HIGH: CxORAW high

TMR_OUTPUT_CONTROL_LOW: CxORAW low

TMR_OUTPUT_CONTROL_SWITCH: Switch CxORAW level

TMR_OUTPUT_CONTROL_FORCE_LOW: Forced CxORAW low

TMR_OUTPUT_CONTROL_FORCE_HIGH: Forced CxORAW high

TMR_OUTPUT_CONTROL_PWM_MODE_A: PWM A mode

TMR_OUTPUT_CONTROL_PWM_MODE_B: PWM B mode

oc_idle_state

Set output channel idle state.

FALSE: Output channel idle state is 0

TRUE: Output channel idle state is 1

occ_idle_state

Set complementary output channel idle state.

FALSE: Complementary output channel idle state is 0

TRUE: Complementary output channel idle state is 1

oc_polarity

Set the polarity of output channels.

TMR_OUTPUT_ACTIVE_HIGH: Active high

TMR_OUTPUT_ACTIVE_LOW: Active low

occ_polarity

Set the polarity of complementary output channels.

TMR_OUTPUT_ACTIVE_HIGH: Active high

TMR_OUTPUT_ACTIVE_LOW: Active low

oc_output_state

Set the state of output channels.

FALSE: Output channel OFF

TRUE: Output channel ON

occ_output_state

Set the state of complementary output channels.

FALSE: Complementary output channel OFF

TRUE: Complementary output channel ON

Example:

```
tmr_output_config_type tmr_output_struct;
tmr_output_struct.oc_mode = TMR_OUTPUT_CONTROL_OFF;
tmr_output_struct.oc_output_state = TRUE;
tmr_output_struct.oc_polarity = TMR_OUTPUT_ACTIVE_HIGH;
tmr_output_struct.oc_idle_state = TRUE;
tmr_output_struct.occ_output_state = TRUE;
tmr_output_struct.occ_polarity = TMR_OUTPUT_ACTIVE_HIGH;
tmr_output_struct.occ_idle_state = TRUE;
tmr_output_channel_config(TMR1, TMR_SELECT_CHANNEL_1, &tmr_output_struct);
```

5.20.15 tmr_output_channel_mode_select function

The table below describes the function tmr_output_channel_mode_select.

Table 440. tmr_output_channel_mode_select function

Name	Description
Function name	tmr_output_channel_mode_select
Function prototype	void tmr_output_channel_mode_select(tmr_type *tmr_x, tmr_channel_select_type tmr_channel, tmr_output_control_mode_type oc_mode);
Function description	Select TMR output channel mode
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR5, TMR8, TMR9, TMR10, TMR11, TMR12, TMR13, TMR14
Input parameter 2	tmr_channel: Refer to tmr_channel for details.
Input parameter 3	oc_mode: refer to *oc_mode* for details
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
tmr_output_channel_mode_select(TMR1, TMR_SELECT_CHANNEL_1, TMR_OUTPUT_CONTROL_SWITCH);
```

5.20.16 tmr_period_value_set function

The table below describes the function tmr_period_value_set.

Table 441. tmr_period_value_set function

Name	Description
Function name	tmr_period_value_set
Function prototype	void tmr_period_value_set(tmr_type *tmr_x, uint32_t tmr_pr_value);
Function description	Set TMR period value
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR5, TMR6, TMR7, TMR8, TMR9, TMR10, TMR11, TMR12, TMR13, TMR14
Input parameter 2	tmr_pr_value: timer period value. For 16-bit timer, it can be 0x0000~0xFFF; for 32-bit timer, it can be 0x0000_0000~0xFFFF_FFFF
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
tmr_period_value_set(TMR1, 0xFFFF);
```

5.20.17 tmr_period_value_get function

The table below describes the function tmr_period_value_get.

Table 442. tmr_period_value_get function

Name	Description
Function name	tmr_period_value_get
Function prototype	uint32_t tmr_period_value_get(tmr_type *tmr_x);
Function description	Get TMR period value
Input parameter	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR5, TMR6, TMR7, TMR8, TMR9, TMR10, TMR11, TMR12, TMR13, TMR14
Output parameter	NA
Return value	Timer period value
Required preconditions	NA
Called functions	NA

Example:

```
uint32_t pr_value;
pr_value = tmr_period_value_get(TMR1);
```

5.20.18 tmr_channel_value_set function

The table below describes the function tmr_channel_value_set.

Table 443. tmr_channel_value_set function

Name	Description
Function name	tmr_channel_value_set
Function prototype	void tmr_channel_value_set(tmr_type *tmr_x, tmr_channel_select_type tmr_channel, uint32_t tmr_channel_value);
Function description	Set TMR channel value
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR5, TMR8, TMR9, TMR10, TMR11, TMR12, TMR13, TMR14
Input parameter 2	tmr_channel: timer channel. Refer to tmr_channel for details.
Input parameter 3	tmr_channel_value: timer channel value. For 16-bit timer, it can be 0x0000~0xFFFF; for 32-bit timer, it can be 0x0000_0000~0xFFFF_FFFF
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
tmr_channel_value_set(TMR1, TMR_SELECT_CHANNEL_1, 0xFFFF);
```

5.20.19 tmr_channel_value_get function

The table below describes the function tmr_channel_value_get.

Table 444. tmr_channel_value_get function

Name	Description
Function name	tmr_channel_value_get
Function prototype	uint32_t tmr_channel_value_get(tmr_type *tmr_x, tmr_channel_select_type tmr_channel);
Function description	Get TMR channel value
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR5, TMR8, TMR9, TMR10, TMR11, TMR12, TMR13, TMR14
Input parameter 2	tmr_channel: timer channel. Refer to tmr_channel for details.
Output parameter	Timer channel value
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
uint32_t ch_value;
ch_value = tmr_channel_value_get(TMR1, TMR_SELECT_CHANNEL_1);
```

5.20.20 tmr_period_buffer_enable function

The table below describes the function tmr_period_buffer_enable.

Table 445. tmr_period_buffer_enable function

Name	Description
Function name	tmr_period_buffer_enable
Function prototype	void tmr_period_buffer_enable(tmr_type *tmr_x, confirm_state new_state);
Function description	Enable or disable TMR period buffer
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR5, TMR6, TMR7, TMR8, TMR9, TMR10, TMR11, TMR12, TMR13, TMR14
Input parameter 2	new_state: indicates the status of period buffer. It can be Enable (TRUE) or Disable (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
tmr_period_buffer_enable(TMR1, TRUE);
```

5.20.21 tmr_output_channel_buffer_enable function

The table below describes the function tmr_output_channel_buffer_enable.

Table 446. tmr_output_channel_buffer_enable function

Name	Description
Function name	tmr_output_channel_buffer_enable
Function prototype	void tmr_output_channel_buffer_enable(tmr_type *tmr_x, tmr_channel_select_type tmr_channel, confirm_state new_state);
Function description	Enable or disable TMR output channel buffer
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR5, TMR8, TMR9, TMR10, TMR11, TMR12, TMR13, TMR14
Input parameter 2	tmr_channel: timer channel. Refer to tmr_channel for details.
Input parameter 3	new_state: indicates the status of output channel buffer. It can be Enable (TRUE) or Disable (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
tmr_output_channel_buffer_enable(TMR1, TMR_SELECT_CHANNEL_1, TRUE);
```

5.20.22 tmr_output_channel_immediately_set function

The table below describes the function tmr_output_channel_immediately_set.

Table 447. tmr_output_channel_immediately_set function

Name	Description
Function name	tmr_output_channel_immediately_set
Function prototype	void tmr_output_channel_immediately_set(tmr_type *tmr_x, tmr_channel_select_type tmr_channel, confirm_state new_state);
Function description	Enable TMR output channel immediately
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR5, TMR8, TMR9, TMR10, TMR11, TMR12, TMR13, TMR14
Input parameter 2	tmr_channel: timer channel. Refer to tmr_channel for details.
Input parameter 3	new_state: indicates the status of output channel enable. This parameter can be Enable (TRUE) or Disable (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
tmr_output_channel_immediately_set(TMR1, TMR_SELECT_CHANNEL_1, TRUE);
```

5.20.23 tmr_output_channel_switch_set function

The table below describes the function tmr_output_channel_switch_set.

Table 448. tmr_output_channel_switch_set function

Name	Description
Function name	tmr_output_channel_switch_set
Function prototype	void tmr_output_channel_switch_set(tmr_type *tmr_x, tmr_channel_select_type tmr_channel, confirm_state new_state);
Function description	Set TMR output channel switch
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR5, TMR8, TMR9, TMR10, TMR11, TMR12, TMR13, TMR14
Input parameter 2	tmr_channel: timer channel. Refer to tmr_channel for details.
Input parameter 3	new_state: indicates the status of output channel swith. This parameter can be Enable (TRUE) or Disable (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
tmr_output_channel_switch_set(TMR1, TMR_SELECT_CHANNEL_1, TRUE);
```

5.20.24 tmr_one_cycle_mode_enable function

The table below describes the function tmr_one_cycle_mode_enable.

Table 449. tmr_one_cycle_mode_enable function

Name	Description
Function name	tmr_one_cycle_mode_enable
Function prototype	void tmr_one_cycle_mode_enable(tmr_type *tmr_x, confirm_state new_state);
Function description	Enable or disable TMR one-cycle mode
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR5, TMR6, TMR7, TMR8, TMR9, TMR10, TMR11, TMR12, TMR13, TMR14
Input parameter 2	new_state: indicates the status of one-cycle mode. This parameter can be Enable (TRUE) or Disable (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
tmr_one_cycle_mode_enable(TMR1, TRUE);
```

5.20.25 tmr_32_bit_function_enable function

The table below describes the function tmr_32_bit_function_enable.

Table 450. tmr_32_bit_function_enable function

Name	Description
Function name	tmr_32_bit_function_enable
Function prototype	void tmr_32_bit_function_enable(tmr_type *tmr_x, confirm_state new_state);
Function description	Enable or disable TMR 32-bit feature (plus mode)
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR2, TMR5
Input parameter 2	new_state: the status of 32-bit mode This parameter can be Enable (TRUE) or Disable (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
tmr_32_bit_function_enable(TMR1, TRUE);
```

5.20.26 tmr_overflow_request_source_set function

The table below describes the function tmr_overflow_request_source_set.

Table 451. tmr_overflow_request_source_set function

Name	Description
Function name	tmr_overflow_request_source_set
Function prototype	void tmr_overflow_request_source_set(tmr_type *tmr_x, confirm_state new_state);
Function description	Select TMR overflow event sources
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR5, TMR6, TMR7, TMR8, TMR9, TMR10, TMR11, TMR12, TMR13, TMR14.
Input parameter 2	new_state: indicates the overflow event source.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

new_state

Select an overflow event source.

FALSE: Counter overflow, OVFSWTR being set, overflow event from slave timer controller

TRUE: Counter overflow only.

Example:

```
tmr_overflow_request_source_set(TMR1, TRUE);
```

5.20.27 tmr_overflow_event_disable function

The table below describes the function tmr_overflow_event_disable.

Table 452. tmr_overflow_event_disable function

Name	Description
Function name	tmr_overflow_event_disable
Function prototype	void tmr_overflow_event_disable(tmr_type *tmr_x, confirm_state new_state);
Function description	Enable or disable TMR overflow event generation
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR5, TMR6, TMR7, TMR8, TMR9, TMR10, TMR11, TMR12, TMR13, TMR14.
Input parameter 2	new_state: indicates the status of overflow event generation.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

new_state

Select the status of overflow event generation.

FALSE: Enable overflow event generation, which can be generated from the following:

- Counter overflow
- Set OVFSWTR=1
- Overflow event from slave timer controller

TRUE: Disable overflow event generation

Example:

```
tmr_overflow_event_disable(TMR1, TRUE);
```

5.20.28 tmr_input_channel_init function

The table below describes the function tmr_input_channel_init.

Table 453. tmr_input_channel_init function

Name	Description
Function name	tmr_input_channel_init
Function prototype	void tmr_input_channel_init(tmr_type *tmr_x, tmr_input_config_type *input_struct, tmr_channel_input_divider_type divider_factor);
Function description	Initialize TMR input channels
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR5, TMR8, TMR9, TMR10, TMR11, TMR12, TMR13, TMR14.
Input parameter 2	input_struct: tmr_input_config_type pointer
Input parameter 3	divider_factor: input channel frequency division factor
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

tmr_input_config_type structure

tmr_input_config_type is defined in the at32f403a_407_tmr.h:

typedef struct

```
{
  tmr_channel_select_type          input_channel_select;
  tmr_input_polarity_type          input_polarity_select;
  tmr_input_direction_mapped_type  input_mapped_select;
  uint8_t                          input_filter_value;
} tmr_input_config_type;
```

input_channel_select

Select a TMR input channel. Refer to tmr_channel for details.

input_polarity_select

Select the polarity of input channels.

TMR_INPUT_RISING_EDGE: Rising edge

TMR_INPUT_FALLING_EDGE: Falling edge

TMR_INPUT_BOTH_EDGE: Both edges (Rising edge and Falling edge)

input_mapped_select

Select input channel mapping.

TMR_CC_CHANNEL_MAPPED_DIRECT:

TMR input channel 1,2,3 and 4 is linked to C1IRAW, C2IRAW, C3IRAW and C4IRAW respectively.

TMR_CC_CHANNEL_MAPPED_INDIRECT:

TMR input channel 1,2,3 and 4 is linked to C2IRAW, C1IRAW, C4IRAW and C3IRAW respectively.

TMR_CC_CHANNEL_MAPPED_STI:

TMR input channel is mapped on STI

input_filter_value

Select an input channel filter value, between 0x00~0x0F

divider_factor

Select input channel frequency division factor.

TMR_CHANNEL_INPUT_DIV_1: Divided by 1

TMR_CHANNEL_INPUT_DIV_2: Divided by 2

TMR_CHANNEL_INPUT_DIV_4: Divided by 4

TMR_CHANNEL_INPUT_DIV_8: Divided by 8

Example:

```
tmr_input_config_type tmr_input_config_struct;
tmr_input_config_struct.input_channel_select = TMR_SELECT_CHANNEL_2;
tmr_input_config_struct.input_mapped_select = TMR_CC_CHANNEL_MAPPED_DIRECT;
tmr_input_config_struct.input_polarity_select = TMR_INPUT_RISING_EDGE;
tmr_input_config_struct.input_filter_value = 0x00;
tmr_input_channel_init(TMR1, &tmr_input_config_struct, TMR_CHANNEL_INPUT_DIV_1);
```

5.20.29 tmr_channel_enable function

The table below describes the function tmr_channel_enable.

Table 454. tmr_channel_enable function

Name	Description
Function name	tmr_channel_enable
Function prototype	void tmr_channel_enable(tmr_type *tmr_x, tmr_channel_select_type tmr_channel, confirm_state new_state);
Function description	Enable or disable TMR channels
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR5, TMR8, TMR9, TMR10, TMR11, TMR12, TMR13, TMR14.
Input parameter 2	tmr_channel: timer channel
Input parameter 3	new_state: indicates the status of timer channels. This parameter can be Enable (TRUE) or Disable (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

tmr_channel

Select a TMR channel.

TMR_SELECT_CHANNEL_1: Timer channel 1
TMR_SELECT_CHANNEL_1C: Complementary channel 1
TMR_SELECT_CHANNEL_2: Timer channel 2
TMR_SELECT_CHANNEL_2C: Complementary channel 2
TMR_SELECT_CHANNEL_3: Timer channel 3
TMR_SELECT_CHANNEL_3C: Complementary channel 3
TMR_SELECT_CHANNEL_4: Timer channel 4

Example:

```
tmr_channel_enable(TMR1, TMR_SELECT_CHANNEL_1, TRUE);
```

5.20.30 tmr_input_channel_filter_set function

The table below describes the function tmr_input_channel_filter_set.

Table 455. tmr_input_channel_filter_set function

Name	Description
Function name	tmr_input_channel_filter_set
Function prototype	void tmr_input_channel_filter_set(tmr_type *tmr_x, tmr_channel_select_type tmr_channel, uint16_t filter_value);
Function description	Set TMR input channel filter
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR5, TMR8, TMR9, TMR10, TMR11, TMR12, TMR13, TMR14.
Input parameter 2	tmr_channel: timer channel. Refer to tmr_channel for details.
Input parameter 3	filter_value: set channel filter value, 0x00~0x0F
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

tmr_channel

Select a TMR channel.

TMR_SELECT_CHANNEL_1: Timer channel 1

TMR_SELECT_CHANNEL_2: Timer channel 2

TMR_SELECT_CHANNEL_3: Timer channel 3

TMR_SELECT_CHANNEL_4: Timer channel 4

Example:

```
tmr_input_channel_filter_set(TMR1, TMR_SELECT_CHANNEL_1, 0x0F);
```

5.20.31 tmr_pwm_input_config function

The table below describes the function tmr_pwm_input_config.

Table 456. tmr_pwm_input_config function

Name	Description
Function name	tmr_pwm_input_config
Function prototype	void tmr_pwm_input_config(tmr_type *tmr_x, tmr_input_config_type *input_struct, tmr_channel_input_divider_type divider_factor);
Function description	Configure TMR pwm input
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR5, TMR8, TMR9, TMR10, TMR11, TMR12, TMR13, TMR14.
Input parameter 2	input_struct: tmr_input_config_type pointer
Input parameter 3	divider_factor: input channel frequency division factor
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

input_struct

Point to the tmr_input_config_type, see *tmr_input_config_type* for details.

divider_factor

Select input channel frequency division factor

TMR_CHANNEL_INPUT_DIV_1: Divided by 1

TMR_CHANNEL_INPUT_DIV_2: Divided by 2

TMR_CHANNEL_INPUT_DIV_4: Divided by 4

TMR_CHANNEL_INPUT_DIV_8: Divided by 8

Example:

```
tmr_input_config_type tmr_ic_init_structure;
tmr_ic_init_structure.input_filter_value = 0;
tmr_ic_init_structure.input_channel_select = TMR_SELECT_CHANNEL_2;
tmr_ic_init_structure.input_mapped_select = TMR_CC_CHANNEL_MAPPED_DIRECT;
tmr_ic_init_structure.input_polarity_select = TMR_INPUT_RISING_EDGE;
tmr_pwm_input_config(TMR1, &tmr_ic_init_structure, TMR_CHANNEL_INPUT_DIV_1);
```

5.20.32 tmr_channel1_input_select function

The table below describes the function tmr_channel1_input_select.

Table 457. tmr_channel1_input_select function

Name	Description
Function name	tmr_channel1_input_select
Function prototype	void tmr_channel1_input_select(tmr_type *tmr_x, tmr_channel1_input_connected_type ch1_connect);
Function description	Select TMR channel 1 input
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR5, TMR8.
Input parameter 2	ch1_connect: channel 1 input selection
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

ch1_connect

Select channel 1 input.

TMR_CHANEL1_CONNECTED_C1IRAW: CH1 pin is connected to C1IRAW

TMR_CHANEL1_2_3_CONNECTED_C1IRAW_XOR: Connect the XOR results of CH1, CH2 and CH3 pins to C1IRAW

Example:

```
tmr_channel1_input_select(TMR1, TMR_CHANEL1_2_3_CONNECTED_C1IRAW_XOR);
```

5.20.33 tmr_input_channel_divider_set function

The table below describes the function tmr_input_channel_divider_set.

Table 458. tmr_input_channel_divider_set function

Name	Description
Function name	tmr_input_channel_divider_set
Function prototype	void tmr_input_channel_divider_set(tmr_type *tmr_x, tmr_channel_select_type tmr_channel, tmr_channel_input_divider_type divider_factor);
Function description	Set TMR input channel divider
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR5, TMR8, TMR9, TMR10, TMR11, TMR12, TMR13, TMR14.
Input parameter 2	tmr_channel: timer channel. Refer to tmr_channel for details.
Input parameter 3	divider_factor: input channel frequency division factor
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

divider_factor

Select input channel frequency division factor

TMR_CHANNEL_INPUT_DIV_1: Divided by 1

TMR_CHANNEL_INPUT_DIV_2: Divided by 2

TMR_CHANNEL_INPUT_DIV_4: Divided by 4

TMR_CHANNEL_INPUT_DIV_8: Divided by 8

Example:

```
tmr_input_channel_divider_set(TMR1, TMR_SELECT_CHANNEL_1, TMR_CHANNEL_INPUT_DIV_2);
```

5.20.34 tmr_primary_mode_select function

The table below describes the function tmr_primary_mode_select.

Table 459. tmr_primary_mode_select function

Name	Description
Function name	tmr_primary_mode_select
Function prototype	void tmr_primary_mode_select(tmr_type *tmr_x, tmr_primary_select_type primary_mode);
Function description	Select TMR master mode
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR5, TMR6, TMR7, TMR8.
Input parameter 2	primary_mode: master mode
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

primary_mode

Select master mode, that is, master timer output signal selection.

TMR_PRIMARY_SEL_RESET: Reset

TMR_PRIMARY_SEL_ENABLE: Enable
TMR_PRIMARY_SEL_OVERFLOW: Overflow
TMR_PRIMARY_SEL_COMPARE: Compare
TMR_PRIMARY_SEL_C1ORAW: C1ORAW
TMR_PRIMARY_SEL_C2ORAW: C2ORAW
TMR_PRIMARY_SEL_C3ORAW: C3ORAW
TMR_PRIMARY_SEL_C4ORAW: C4ORAW

Example:

```
tmr_primary_mode_select(TMR1, TMR_PRIMARY_SEL_RESET);
```

5.20.35 tmr_sub_mode_select function

The table below describes the function tmr_sub_mode_select.

Table 460. tmr_sub_mode_select function

Name	Description
Function name	tmr_sub_mode_select
Function prototype	void tmr_sub_mode_select(tmr_type *tmr_x, tmr_sub_mode_select_type sub_mode);
Function description	Select TMR slave timer mode
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR5, TMR8, TMR9, TMR12.
Input parameter 2	sub_mode: slave timer mode
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

primary_mode

primary_mode

Select slave timer modes.

TMR_SUB_MODE_DIABLE: Disable
TMR_SUB_ENCODER_MODE_A: Encoder mode A
TMR_SUB_ENCODER_MODE_B: Encoder mode B
TMR_SUB_ENCODER_MODE_C: Encoder mode C
TMR_SUB_RESET_MODE: Reset
TMR_SUB_HANG_MODE: Suspend
TMR_SUB_TRIGGER_MODE: Trigger
TMR_SUB_EXTERNAL_CLOCK_MODE_A: External clock A

Example:

```
tmr_sub_mode_select(TMR1, TMR_SUB_HANG_MODE);
```

5.20.36 tmr_channel_dma_select function

The table below describes the function tmr_channel_dma_select.

Table 461. tmr_channel_dma_select function

Name	Description
Function name	tmr_channel_dma_select
Function prototype	void tmr_channel_dma_select(tmr_type *tmr_x, tmr_dma_request_source_type cc_dma_select);
Function description	Select TMR channel DMA request source
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR5, TMR8, TMR9, TMR12.
Input parameter 2	cc_dma_select: TMR channel DMA request source
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

cc_dma_select

Select DMA request source for TMR channels.

TMR_DMA_REQUEST_BY_CHANNEL: DMA request upon a channel event (CxIF = 1)

TMR_DMA_REQUEST_BY_OVERFLOW: DMA request upon an overflow event (OVFIF = 1)

Example:

```
tmr_channel_dma_select(TMR1, TMR_DMA_REQUEST_BY_OVERFLOW);
```

5.20.37 tmr_hall_select function

The table below describes the function tmr_hall_select.

Table 462. tmr_hall_select function

Name	Description
Function name	tmr_hall_select
Function prototype	void tmr_hall_select(tmr_type *tmr_x, confirm_state new_state);
Function description	Select TMR hall mode
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR8.
Input parameter 2	new_state: indicates the status of TMR hall mode
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

new_state

Select the status of TMR hall mode in order to refresh channel control bit.

FALSE: Refresh channel control bit through HALL

TRUE: Refresh channel control bit through HALL or the rising edge of TRGIN

Example:

```
tmr_hall_select(TMR1, TRUE);
```

5.20.38 tmr_channel_buffer_enable function

The table below describes the function tmr_channel_buffer_enable.

Table 463. tmr_channel_buffer_enable function

Name	Description
Function name	tmr_channel_buffer_enable
Function prototype	void tmr_channel_buffer_enable(tmr_type *tmr_x, confirm_state new_state);
Function description	Enable or disable TMR channel buffer
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR8.
Input parameter 2	new_state: indicates the status of TMR channel buffer. This parameter can be Enable (TRUE) or Disable (FALSE).
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
tmr_channel_buffer_enable(TMR1, TRUE);
```

5.20.39 tmr_trigger_input_select function

The table below describes the function tmr_trigger_input_select.

Table 464. tmr_trigger_input_select function

Name	Description
Function name	tmr_trigger_input_select
Function prototype	void tmr_trigger_input_select(tmr_type *tmr_x, sub_tmr_input_sel_type trigger_select);
Function description	Select TMR slave timer trigger input
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR5, TMR8, TMR9, TMR12.
Input parameter 2	trigger_select: select TMR slave timer trigger input
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

trigger_select

Select TMR slave timer trigger input.

TMR_SUB_INPUT_SEL_IS: Internal input 0
TMR_SUB_INPUT_SEL_IS1: Internal input 1
TMR_SUB_INPUT_SEL_IS2: Internal input 2
TMR_SUB_INPUT_SEL_IS3: Internal input 3
TMR_SUB_INPUT_SEL_C1INC: C1IRAW input detection
TMR_SUB_INPUT_SEL_C1DF1: Filter input channel 1
TMR_SUB_INPUT_SEL_C2DF2: Filter input channel 2
TMR_SUB_INPUT_SEL_EXTIN: External input channel EXT

Example:

```
tmr_trigger_input_select(TMR1, TMR_SUB_INPUT_SEL_IS0);
```

5.20.40 tmr_sub_sync_mode_set function

The table below describes the function tmr_sub_sync_mode_set.

Table 465. tmr_sub_sync_mode_set function

Name	Description
Function name	tmr_sub_sync_mode_set
Function prototype	void tmr_sub_sync_mode_set(tmr_type *tmr_x, confirm_state new_state);
Function description	Set TMR slave timer synchronization mdoe
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR5, TMR8, TMR9, TMR12.
Input parameter 2	new_state: indicates the status of TMR slave timer synchronization mode This parameter can be Enable (TRUE) or Disable (FALSE).
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
tmr_sub_sync_mode_set(TMR1, TRUE);
```

5.20.41 tmr_dma_request_enable function

The table below describes the function tmr_dma_request_enable.

Table 466. tmr_dma_request_enable function

Name	Description
Function name	tmr_dma_request_enable
Function prototype	void tmr_dma_request_enable(tmr_type *tmr_x, tmr_dma_request_type dma_request, confirm_state new_state);
Function description	Enable or disable TMR DMA request
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR5, TMR6, TMR7, TMR8, TMR9, TMR10, TMR11, TMR12, TMR13, TMR14.
Input parameter 2	dma_request: DMA request
Input parameter 3	new_state: indicates the status of DMA request. This parameter can be Enable (TRUE) or Disable (FALSE).
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

dma_request

Select a DMA request.

TMR_OVERFLOW_DMA_REQUEST: Overflow event DMA request
TMR_C1_DMA_REQUEST: Channel 1 DMA request
TMR_C2_DMA_REQUEST: Channel 2 DMA request
TMR_C3_DMA_REQUEST: Channel 3 DMA request

TMR_C4_DMA_REQUEST: Channel 4 DMA request
TMR_HALL_DMA_REQUEST: HALL event DMA request
TMR_TRIGGER_DMA_REQUEST: Trigger event DMA request

Example:

```
tmr_dma_request_enable(TMR1, TMR_OVERFLOW_DMA_REQUEST, TRUE);
```

5.20.42 tmr_interrupt_enable function

The table below describes the function tmr_interrupt_enable.

Table 467. tmr_interrupt_enable function

Name	Description
Function name	tmr_interrupt_enable
Function prototype	void tmr_interrupt_enable(tmr_type *tmr_x, uint32_t tmr_interrupt, confirm_state new_state);
Function description	Enable or disable TMR interrupts
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR5, TMR6, TMR7, TMR8, TMR9, TMR10, TMR11, TMR12, TMR13, TMR14.
Input parameter 2	tmr_interrupt: TMR interrupts
Input parameter 3	new_state: indicates the status of TMR interrupts. This parameter can be Enable (TRUE) or Disable (FALSE).
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

tmr_interrupt

Select a TMR interrupt.

TMR_OVF_INT: Overflow event interrupt
TMR_C1_INT: Channel 1 event interrupt
TMR_C2_INT: Channel 2 event interrupt
TMR_C3_INT: Channel 3 event interrupt
TMR_C4_INT: Channel 4 event interrupt
TMR_HALL_INT: HALL event interrupt
TMR_TRIGGER_INT: Trigger event interrupt
TMR_BRK_INT: Break event interrupt

Example:

```
tmr_interrupt_enable(TMR1, TMR_OVF_INT, TRUE);
```

5.20.43 tmr_interrupt_flag_get function

The table below describes the function tmr_interrupt_flag_get.

Table 468. tmr_interrupt_flag_get function

Name	Description
Function name	tmr_interrupt_flag_get
Function prototype	flag_status tmr_interrupt_flag_get (tmr_type *tmr_x, uint32_t tmr_flag);
Function description	Get the interrupt flag status
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR5, TMR6, TMR7, TMR8, TMR9, TMR10, TMR11, TMR12, TMR13, TMR14.
Input parameter 2	tmr_flag: Interrupt flag selection Refer to the “tmr_flag” description below for details.
Output parameter	NA
Return value	flag_status: indicates the status of flags Return SET or RESET.
Required preconditions	NA
Called functions	NA

tmr_flag

This is used for flag selection, including:

TMR_OVF_FLAG: Overflow interrupt flag
TMR_C1_FLAG: Channel 1 interrupt flag
TMR_C2_FLAG: Channel 2 interrupt flag
TMR_C3_FLAG: Channel 3 interrupt flag
TMR_C4_FLAG: Channel 4 interrupt flag
TMR_HALL_FLAG: HALL interrupt flag
TMR_TRIGGER_FLAG: Trigger interrupt flag
TMR_BRK_FLAG: Break interrupt flag

Example

```
if(tmr_interrupt_flag_get (TMR1, TMR_OVF_FLAG) != RESET)
```

5.20.44 tmr_flag_get function

The table below describes the function tmr_flag_get.

Table 469. tmr_flag_get function

Name	Description
Function name	tmr_flag_get
Function prototype	flag_status tmr_flag_get(tmr_type *tmr_x, uint32_t tmr_flag);
Function description	Get flag status
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR5, TMR6, TMR7, TMR8, TMR9, TMR10, TMR11, TMR12, TMR13, TMR14.
Input parameter 2	tmr_flag: Flag selection Refer to the “tmr_flag” description below for details.
Output parameter	NA
Return value	flag_status: indicates the status of flags Return SET or RESET.
Required preconditions	NA
Called functions	NA

tmr_flag

This is used for flag selection, including:

TMR_OVF_FLAG:	Overflow interrupt flag
TMR_C1_FLAG:	Channel 1 interrupt flag
TMR_C2_FLAG:	Channel 2 interrupt flag
TMR_C3_FLAG:	Channel 3 interrupt flag
TMR_C4_FLAG:	Channel 4 interrupt flag
TMR_HALL_FLAG:	HALL interrupt flag
TMR_TRIGGER_FLAG:	Trigger interrupt flag
TMR_BRK_FLAG:	Break interrupt flag
TMR_C1_RECAPTURE_FLAG:	Channel 1 recapture flag
TMR_C2_RECAPTURE_FLAG:	Channel 2 recapture flag
TMR_C3_RECAPTURE_FLAG:	Channel 3 recapture flag
TMR_C4_RECAPTURE_FLAG:	Channel 4 recapture flag

Example:

```
if(tmr_flag_get(TMR1, TMR_OVF_FLAG) != RESET)
```

5.20.45 tmr_flag_clear function

The table below describes the function tmr_flag_clear.

Table 470. tmr_flag_clear function

Name	Description
Function name	tmr_flag_clear
Function prototype	void tmr_flag_clear(tmr_type *tmr_x, uint32_t tmr_flag);
Function description	Clear flag
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR5, TMR6, TMR7, TMR8, TMR9, TMR10, TMR11, TMR12, TMR13, TMR14.
Input parameter 2	tmr_flag: flag selection Refer to *tmr_flag* for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
tmr_flag_clear(TMR1, TMR_OVF_FLAG);
```

5.20.46 tmr_event_sw_trigger function

The table below describes the function tmr_event_sw_trigger

Table 471. tmr_event_sw_trigger function

Name	Description
Function name	tmr_event_sw_trigger
Function prototype	void tmr_event_sw_trigger(tmr_type *tmr_x, tmr_event_trigger_type tmr_event);
Function description	Software triggers TMR events
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR5, TMR6, TMR7, TMR8, TMR9, TMR10, TMR11, TMR12, TMR13, TMR14.
Input parameter 2	tmr_event: select a TMR event
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

tmr_event

Set TMR events triggered by software.

TMR_OVERFLOW_SWTRIG: Overflow event
TMR_C1_SWTRIG: Channel 1 event
TMR_C2_SWTRIG: Channel 2 event
TMR_C3_SWTRIG: Channel 3 event
TMR_C4_SWTRIG: Channel 4 event
TMR_HALL_SWTRIG: HALL event
TMR_TRIGGER_SWTRIG: Trigger ecent

TMR_BRK_SWTRIG: Break event

Example:

```
tmr_event_sw_trigger(TMR1, TMR_OVERFLOW_SWTRIG);
```

5.20.47 tmr_output_enable function

The table below describes the function tmr_output_enable.

Table 472. tmr_output_enable function

Name	Description
Function name	tmr_output_enable
Function prototype	void tmr_output_enable(tmr_type *tmr_x, confirm_state new_state);
Function description	Enable or disable TMR output
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR8
Input parameter 2	new_state: TMR output status This parameter can be Enable (TRUE) or Disable (FALSE).
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
tmr_output_enable(TMR1, TRUE);
```

5.20.48 tmr_internal_clock_set function

The table below describes the function tmr_internal_clock_set.

Table 473. tmr_internal_clock_set function

Name	Description
Function name	tmr_internal_clock_set
Function prototype	void tmr_internal_clock_set(tmr_type *tmr_x);
Function description	Set TMR internal clock
Input parameter	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR5, TMR8, TMR9, TMR12.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
tmr_internal_clock_set(TMR1);
```

5.20.49 tmr_output_channel_polarity_set function

The table below describes the function tmr_output_channel_polarity_set.

Table 474. tmr_output_channel_polarity_set function

Name	Description
Function name	tmr_output_channel_polarity_set
Function prototype	void tmr_output_channel_polarity_set(tmr_type *tmr_x, tmr_channel_select_type tmr_channel, tmr_polarity_active_type oc_polarity);
Function description	Set TMR output channel polarity
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR5, TMR8, TMR9, TMR10, TMR11, TMR12, TMR13, TMR14.
Input parameter 2	tmr_channel: Timer channel. Refer to tmr_channel for details.
Input parameter 3	oc_polarity: output channel polarity
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

oc_polarity

Select TMR channel polarity.

TMR_POLARITY_ACTIVE_HIGH: Active high

TMR_POLARITY_ACTIVE_LOW: Active low

Example:

```
tmr_output_channel_polarity_set(TMR1, TMR_SELECT_CHANNEL_1, TMR_POLARITY_ACTIVE_HIGH);
```

5.20.50 tmr_external_clock_config function

The table below describes the function tmr_external_clock_config.

Table 475. tmr_external_clock_config function

Name	Description
Function name	tmr_external_clock_config
Function prototype	void tmr_external_clock_config(tmr_type *tmr_x, tmr_external_signal_divider_type es_divide, tmr_external_signal_polarity_type es_polarity, uint16_t es_filter);
Function description	Configure TMR external clock
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR5, TMR8.
Input parameter 2	es_divide: external signal frequency division factor
Input parameter 3	es_polarity: external signal polarity
Input parameter 4	es_filter: external signal filter value, 0x00~0x0F
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

es_divide

Set TMR external signal frequency division factor.

TMR_ES_FREQUENCY_DIV_1: Divided by 1

TMR_ES_FREQUENCY_DIV_2: Divided by 2
TMR_ES_FREQUENCY_DIV_4: Divided by 4
TMR_ES_FREQUENCY_DIV_8: Divided by 8

es_polarity

Select TMR external signal polarity.

TMR_ES_POLARITY_NON_INVERTED: Hiht or risging edge
TMR_ES_POLARITY_INVERTED: Low or falling edge

Example:

```
tmr_external_clock_config(TMR1, TMR_ES_FREQUENCY_DIV_1, TMR_ES_POLARITY_INVERTED, 0x0F);
```

5.20.51 tmr_external_clock_mode1_config function

The table below describes the function tmr_external_clock_mode1_config.

Table 476. tmr_external_clock_mode1_config function

Name	Description
Function name	tmr_external_clock_mode1_config
Function prototype	void tmr_external_clock_mode1_config(tmr_type *tmr_x, tmr_external_signal_divider_type es_divide, tmr_external_signal_polarity_type es_polarity, uint16_t es_filter);
Function description	Configure TMR external clock mode 1 (corresponding to external mode A in the reference manual)
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR5, TMR8.
Input parameter 2	es_divide: external signal frequency division factor
Input parameter 3	es_polarity: external signal polarity
Input parameter 4	es_filter: external signal filter value, 0x00~0x0F
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

es_divide

Set TMR external signal frequency division factor, refer to *es_divide* for details.

es_polarity

Set TMR external signal polarity, refer to *es_polarity* for details.

Example:

```
tmr_external_clock_mode1_config(TMR1, TMR_ES_FREQUENCY_DIV_1, TMR_ES_POLARITY_INVERTED,
0x0F);
```

5.20.52 tmr_external_clock_mode2_config function

The table below describes the function tmr_external_clock_mode2_config.

Table 477. tmr_external_clock_mode2_config function

Name	Description
Function name	tmr_external_clock_mode2_config
Function prototype	void tmr_external_clock_mode2_config(tmr_type *tmr_x, tmr_external_signal_divider_type es_divide, tmr_external_signal_polarity_type es_polarity, uint16_t es_filter);
Function description	Configure TMR external clock mode 2 (corresponding to external mode B in the reference manual)
Input parameter 1	tmr_x : indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR5, TMR8.
Input parameter 2	es_divide: external signal frequency division factor
Input parameter 3	es_polarity: external signal polarity
Input parameter 4	es_filter: external signal filter value,0x00~0x0F
Output parameter	NA
Return value	NA
Input parameter 2	es_divide: external signal frequency division factor
Input parameter 3	es_polarity: external signal polarity

es_divide

Set TMR external signal frequency division factor, refer to *es_divide* for details.

es_polarity

Set TMR external signal polarity, refer to ***es_polarity*** for details.

Example:

```
tmr_external_clock_mode2_config(TMR1, TMR_ES_FREQUENCY_DIV_1, TMR_ES_POLARITY_INVERTED, 0x0F);
```

5.20.53 tmr_encoder_mode_config function

The table below describes the function tmr_encoder_mode_config.

Table 478. tmr_encoder_mode_config function

Name	Description
Function name	tmr_encoder_mode_config
Function prototype	void tmr_encoder_mode_config(tmr_type *tmr_x, tmr_encoder_mode_type encoder_mode, tmr_input_polarity_type ic1_polarity, tmr_input_polarity_type ic2_polarity);
Function description	Configure TMR encoder mode
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR5, TMR8.
Input parameter 2	encoder_mode: encoder mode
Input parameter 3	ic1_polarity: input channel 1 polarity
Input parameter 4	Ic2_polarity: input channel 2 polarity
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

encoder_mode

Select a TMR encoder mode.

TMR_ENCODER_MODE_A: Encoder mode A

TMR_ENCODER_MODE_B: Encoder mode B

TMR_ENCODER_MODE_C: Encoder mode C

ic1_polarity

Select TMR input channel 1 polarity.

TMR_INPUT_RISING_EDGE: Rising edge

TMR_INPUT_FALLING_EDGE: Falling edge

TMR_INPUT_BOTH_EDGE: Both edges (Rising edge and Falling edge)

ic2_polarity

Select TMR input channel 2 polarity.

TMR_INPUT_RISING_EDGE: Rising edge

TMR_INPUT_FALLING_EDGE Falling edge

TMR_INPUT_BOTH_EDGE: Both edges (Rising edge and Falling edge)

Example:

```
tmr_encoder_mode_config(TMR1, TMR_ENCODER_MODE_A, TMR_INPUT_RISING_EDGE,
TMR_INPUT_RISING_EDGE);
```

5.20.54 tmr_force_output_set function

The table below describes the function tmr_force_output_set.

Table 479. tmr_force_output_set function

Name	Description
Function name	tmr_force_output_set
Function prototype	void tmr_force_output_set(tmr_type *tmr_x, tmr_channel_select_type tmr_channel, tmr_force_output_type force_output);
Function description	Set TMR forced output
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR5, TMR8, TMR9, TMR10, TMR11, TMR12, TMR13, TMR14.
Input parameter 2	tmr_channel: timer channel. Refer to tmr_channel for details.
Input parameter 3	force_output: forced output level
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

force_output

Forced output level of output channels.

TMR_FORCE_OUTPUT_HIGH: CxORAW forced high

TMR_FORCE_OUTPUT_LOW: CxORAW forced low

Example:

```
tmr_force_output_set(TMR1, TMR_SELECT_CHANNEL_1, TMR_FORCE_OUTPUT_HIGH);
```

5.20.55 tmr_dma_control_config function

The table below describes the function tmr_dma_control_config.

Table 480. tmr_dma_control_config function

Name	Description
Function name	tmr_dma_control_config
Function prototype	void tmr_dma_control_config(tmr_type *tmr_x, tmr_dma_transfer_length_type dma_length, tmr_dma_address_type dma_base_address);
Function description	Configure TMR DMA control
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR5, TMR8.
Input parameter 2	dma_length: DMA transfer length
Input parameter 3	dma_base_address: DMA transfer offset address
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

dma_length

Set DAM transfer bytes, including:

TMR_DMA_TRANSFER_1BYTE: 1 byte

TMR_DMA_TRANSFER_2BYTES: 2 bytes

TMR_DMA_TRANSFER_3BYTES: 3 bytes
…
TMR_DMA_TRANSFER_17BYTES: 17 bytes
TMR_DMA_TRANSFER_18BYTES: 18 bytes

dma_base_address

Set DMA transfer offset address, starting from TMR control register 1, including:
TMR_CTRL1_ADDRESS
TMR_CTRL2_ADDRESS
TMR_STCTRL_ADDRESS
TMR_IDEN_ADDRESS
TMR_ISTS_ADDRESS
TMR_SWEVT_ADDRESS
TMR_CM1_ADDRESS
TMR_CM2_ADDRESS
TMR_CCTRL_ADDRESS
TMR_CVAL_ADDRESS
TMR_DIV_ADDRESS
TMR_PR_ADDRESS
TMR_RPR_ADDRESS
TMR_C1DT_ADDRESS
TMR_C2DT_ADDRESS
TMR_C3DT_ADDRESS
TMR_C4DT_ADDRESS
TMR_BRK_ADDRESS
TMR_DMACTRL_ADDRESS

Example:

```
tmr_dma_control_config(TMR1, TMR_DMA_TRANSFER_8BYTES, TMR_CTRL1_ADDRESS);
```

5.20.56 tmr_brkdt_config function

The table below describes the function tmr_brkdt_config.

Table 481. tmr_brkdt_config function

Name	Description
Function name	tmr_brkdt_config
Function prototype	void tmr_brkdt_config(tmr_type *tmr_x, tmr_brkdt_config_type *brkdt_struct);
Function description	Configure TMR break mode and dead time
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR8.
Input parameter 2	brkdt_struct: tmr_brkdt_config_type pointer
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

tmr_brkdt_config_type structure

The tmr_brkdt_config_type is defined in the at32f403a_407_tmr.h:
typedef struct

```
{
  uint8_t                deadtime;
  tmr_brk_polarity_type  brk_polarity;
  tmr_wp_level_type      wp_level;
  confirm_state          auto_output_enable;
  confirm_state          fcsoen_state;
  confirm_state          fcsodis_state;
  confirm_state          brk_enable;
} tmr_brkdt_config_type;
```

deadtime

Set dead time, between 0x00~0xFF

brk_polarity

Select break input polarity

TMR_BRK_INPUT_ACTIVE_LOW: Active low

TMR_BRK_INPUT_ACTIVE_HIGH: Active high

wp_level

Set write protection level.

TMR_WP_OFF: Write protection OFF

TMR_WP_LEVEL_3:

Level 3 write protection, protecting the bits below:

- TMRx_BRK: DTC, BRKEN, BRKV and AOEN
- TMRx_CTRL2: CxIOS and CxCIOS

TMR_WP_LEVEL_2:

Level 2 write protection, protecting the bits below in addition to level-3 protected bits:

- TMRx_CCTRL: CxP and CxCP
- TMRx_BRK: FCSODIS and FCSOEN

TMR_WP_LEVEL_1:

Level 1 write protection, protecting the bits below in addition to level-2 protected bits:

- TMRx_CMx: CxOCTRL and CxOBEN

auto_output_enable

Enable auto output, Enable (TRUE) or disable (FALSE)

fcsoen_state

Indicates the frozen status when main output is ON. It is used to configure the status of complementary output channels when timer is OFF and output is enabled (OEN=1).

FALSE: Disable CxOUT/CxCOUT output

TRUE: Enable CxOUT/CxCOUT output, inactive level

fcsodis_state

Indicates the frozen status when main output is OFF. It is used to configure the status of complementary output channels when timer is OFF and output is disabled (OEN=0).

FALSE: Disable CxOUT/CxCOUT output

TRUE: Enable CxOUT/CxCOUT output, idle level

brk_enable

Enable break feature, Enable (TRUE) or disable (FALSE).

Example

```
tmr_brkdt_config_type tmr_brkdt_config_struct;
tmr_brkdt_config_struct.brk_enable = TRUE;
tmr_brkdt_config_struct.auto_output_enable = TRUE;
tmr_brkdt_config_struct.deadtime = 0;
tmr_brkdt_config_struct.fcsodis_state = TRUE;
tmr_brkdt_config_struct.fcsoen_state = TRUE;
tmr_brkdt_config_struct.brk_polarity = TMR_BRK_INPUT_ACTIVE_HIGH;
tmr_brkdt_config_struct.wp_level = TMR_WP_OFF;
tmr_brkdt_config(TMR1, &tmr_brkdt_config_struct);
```

5.21 Universal synchronous/asynchronous receiver/transmitter (USART)

The USART register structure usart_type is defined in the "at32f403a_407_usart.h":

```
/**
  * @brief type define usart register all
  */
typedef struct
{
      ...

} usart_type;
```

The table below gives a list of the USART registers.

Table 482. Summary of USART registers

Register	Description
sts	Status register
dt	Data register
baudr	Baud rate register
ctrl1	Control register 1
ctrl2	Control register 2
ctrl3	Control register 3
gdiv	Guard time and divider Control register 1

The table below gives a list of USART library functions.

Table 483. Summary of USART library functions

Function name	Description
usart_reset	Reset USART peripheral registers
usart_init	Set baud rate, data bits and stop bits.
usart_parity_selection_config	Parity selection
usart_enable	Enable USART peripherals
usart_transmitter_enable	Enable USART transmitter
usart_receiver_enable	Enable USART receiver
usart_clock_config	Set clock polarity and phases for synchronization
usart_clock_enable	Set clock output for synchronization
usart_interrupt_enable	Enable interrupts
usart_dma_transmitter_enable	Enable DMA transmitter
usart_dma_receiver_enable	Enable DMA receiver
usart_wakeup_id_set	Set wakeup ID
usart_wakeup_mode_set	Set wakeup mode
usart_receiver_mute_enable	Enable receiver mute mode
usart_break_bit_num_set	Set break frame length
usart_lin_mode_enable	Enable LIN mode
usart_data_transmit	Data transmit
usart_data_receive	Data receive

usart_break_send	Send break frame
usart_smartcard_guard_time_set	Set smartcard guard time
usart_irda_smartcard_division_set	Set infrared and smartcard division
usart_smartcard_mode_enable	Enable smartcard mode
usart_smartcard_nack_set	Enable smartcard NACK
usart_single_line_halfduplex_select	Enable single-wire half-duplex mode
usart_irda_mode_enable	Enable infrared mode
usart_irda_low_power_enable	Enable infrared low-power mode
usart_hardware_flow_control_set	Enable hardware flow control
usart_flag_get	Get flag
usart_flag_clear	Clear flag

5.21.1 usart_reset function

The table below describes the function usart_reset.

Table 484. usart_reset function

Name	Description
Function name	usart_reset
Function prototype	void usart_reset(usart_type* usart_x);
Function description	Reset USART peripheral registers
Input parameter 1	usart_x: indicates the selected peripherals, it can be USART1, USART2, and USART3…
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	crm_periph_reset

Example:

```
/* reset usart1 */
usart_reset(USART1);
```

5.21.2 usart_init function

The table below describes the function usart_init.

Table 485. usart_init function

Name	Description
Function name	usart_init
Function prototype	void usart_init(usart_type* usart_x, uint32_t baud_rate, usart_data_bit_num_type data_bit, usart_stop_bit_num_type stop_bit);
Function description	Set baud rate, data bits and stop bit.
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2, and USART3…
Input parameter 2	baud_rate: baud rate for serial interfaces
Input parameter 3	data_bit: data bit width for serial interfaces
Input parameter 4	stop_bit: stop bit width for serial interfaces
Output parameter	NA
Return value	NA
Required preconditions	This operation can be allowed only when external low-speed clock is disabled.
Called functions	NA

data_bit

Select data bit size for seripal interface communication.

USART_DATA_8BITS: 8-bit

USART_DATA_9BITS: 9-bit

stop_bit

Select stop bit size for seripal interface communication.

USART_STOP_1_BIT: 1 bit

USART_STOP_0_5_BIT: 0.5 bit

USART_STOP_2_BIT: 2 bit

USART_STOP_1_5_BIT: 1.5 bit

Example:

```
/* configure uart param */
usart_init(USART1, 115200, USART_DATA_8BITS, USART_STOP_1_BIT);
```

5.21.3 usart_parity_selection_config function

The table below describes the function usart_parity_selection_config.

Table 486. usart_parity_selection_config function

Name	Description
Function name	usart_parity_selection_config
Function prototype	void usart_parity_selection_config(usart_type* usart_x, usart_parity_selection_type parity);
Function description	Parity selection
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2, and USART3…
Input parameter 2	Parity: parity mode for serial interface communication
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

parity

Select parity mode for serial interface communication.

USART_PARITY_NONE: No parity

USART_PARITY_EVEN: Even

USART_PARITY_ODD: Odd

Example:

```
/* config usart even parity */
usart_parity_selection_config(USART1, USART_PARITY_EVEN);
```

5.21.4 usart_enable function

The table below describes the function usart_enable.

Table 487. usart_enable function

Name	Description
Function name	usart_enable
Function prototype	void usart_enable(usart_type* usart_x, confirm_state new_state);
Function description	Enable USART
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2, and USART3…
Input parameter 2	new_state: Enable (TRUE) and disable (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable usart1 */
usart_enable(USART1, TRUE);
```

5.21.5 usart_transmitter_enable function

The table below describes the function usart_transmitter_enable.

Table 488. usart_transmitter_enable function

Name	Description
Function name	usart_transmitter_enable
Function prototype	void usart_transmitter_enable(usart_type* usart_x, confirm_state new_state);
Function description	Enable USART transmitter
Input parameter 1	usart_x: indicates the selected peripheral it can be USART1, USART2, and USART3…
Input parameter 2	new_state: Enable (TRUE) and disable (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable usart1 transmitter */
usart_transmitter_enable(USART1, TRUE);
```

5.21.6 usart_receiver_enable function

The table below describes the function usart_receiver_enable.

Table 489. usart_receiver_enable function

Name	Description
Function name	usart_receiver_enable
Function prototype	void usart_receiver_enable(usart_type* usart_x, confirm_state new_state);
Function description	Enable USART receiver
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2, and USART3…
Input parameter 2	new_state: Enable (TRUE) and disable (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable usart1 receiver */
usart_ receiver _enable(USART1, TRUE);
```

5.21.7 usart_clock_config function

The table below describes the function usart_clock_config.

Table 490. usart_clock_config function

Name	Description
Function name	usart_clock_config
Function prototype	void usart_clock_config(usart_type* usart_x, usart_clock_polarity_type clk_pol, usart_clock_phase_type clk_pha, usart_lbcp_type clk_lb);
Function description	Configure clock polarity and phase for synchronization feature
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2, and USART3…
Input parameter 2	clk_pol: clock polarity for synchronization
Input parameter 3	clk_pha: clock phase for synchronization
Input parameter 4	clk_lb: selects whether to output clock on the last bit (upper bit) of data sent through synchronization feature
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

clk_pol

Clock polarity selection.

USART_CLOCK_POLARITY_LOW: Low

USART_CLOCK_POLARITY_HIGH: High

clk_pha

Clock phase selection.

USART_CLOCK_PHASE_1EDGE: 1st edge

USART_CLOCK_PHASE_2EDGE: 2nd edge

clk_lb

Select whether to output clock on the last bit of data.

USART_CLOCK_LAST_BIT_NONE: No clock output

USART_CLOCK_LAST_BIT_OUTPUT: Clock output

Example:

```
/* config synchronous mode */
usart_clock_config(USART1, USART_CLOCK_POLARITY_HIGH, USART_CLOCK_PHASE_2EDGE,
USART_CLOCK_LAST_BIT_OUTPUT);
```

5.21.8 usart_clock_enable function

The table below describes the function usart_clock_enable.

Table 491. usart_clock_enable function

Name	Description
Function name	usart_clock_enable
Function prototype	void usart_clock_enable(usart_type* usart_x, confirm_state new_state);
Function description	Enable clock output
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2, and USART3…
Input parameter 2	new_state: Enable (TRUE) or disable (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable clock */
usart_clock_enable(USART1, TRUE);
```

5.21.9 usart_interrupt_enable function

The table below describes the function usart_interrupt_enable.

Table 492. usart_interrupt_enable function

Name	Description
Function name	usart_interrupt_enable
Function prototype	void usart_interrupt_enable(usart_type* usart_x, uint32_t usart_int, confirm_state new_state);
Function description	Enable interrupts
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2, and USART3…
Input parameter 2	usart_int: interrupt type
Input parameter 3	new_state: Enable (TRUE) or disable (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

usart_int

Defines a peripheral interrupt.

USART_IDLE_INT: Bus idle
USART_RDBF_INT: Receive data buffer full
USART_TDC_INT: Transmit data complete
USART_TDBE_INT: Transmit data buffer empty
USART_PERR_INT: Parity error
USART_BF_INT: Break frame receive
USART_ERR_INT: Error

USART_CTSCF_INT: CTS (Clear To Send) change

Example:

```
/* enable usart1 transmit complete interrupt */
usart_interrupt_enable (USART1, USART_TDC_INT, TRUE);
```

5.21.10 usart_dma_transmitter_enable function

The table below describes the function usart_dma_transmitter_enable.

Table 493. usart_dma_transmitter_enable function

Name	Description
Function name	usart_dma_transmitter_enable
Function prototype	void usart_dma_transmitter_enable(usart_type* usart_x, confirm_state new_state);
Function description	Enable DMA transmitter
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2, and USART3…
Input parameter 2	new_state: Enable (TRUE) or disable (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable dma transmitter */
usart_dma_transmitter_enable (USART1, TRUE);
```

5.21.11 usart_dma_receiver_enable function

The table below describes the function usart_dma_receiver_enable.

Table 494. usart_dma_receiver_enable function

Name	Description
Function name	usart_dma_receiver_enable
Function prototype	void usart_dma_receiver_enable(usart_type* usart_x, confirm_state new_state);
Function description	Enable DMA receiver
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2, and USART3…
Input parameter 2	new_state: Enable (TRUE) or disable (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable dma receiver */
usart_dma_receiver_enable (USART1, TRUE);
```

5.21.12 usart_wakeup_id_set function

The table below describes the function usart_wakeup_id_set.

Table 495. usart_wakeup_id_set function

Name	Description
Function name	usart_wakeup_id_set
Function prototype	void usart_wakeup_id_set(usart_type* usart_x, uint8_t usart_id);
Function description	Set wakeup ID
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2, and USART3…
Input parameter 2	usart_id: wakeup ID
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* config wakeup id */
usart_wakeup_id_set (USART1, 0x88);
```

5.21.13 usart_wakeup_mode_set function

The table below describes the function usart_wakeup_mode_set.

Table 496. usart_wakeup_mode_set function

Name	Description
Function name	usart_wakeup_mode_set
Function prototype	void usart_wakeup_mode_set(usart_type* usart_x, usart_wakeup_mode_type wakeup_mode);
Function description	Set wakeup mode
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2, and USART3
Input parameter 2	wakeup_mode: wakeup mode
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

wakeup_mode

Set wakeup mode to wake up from silent state.

USART_WAKEUP_BY_IDLE_FRAME: Woke up by idle frame

USART_WAKEUP_BY_MATCHING_ID: Woke up by ID matching

Example:

```
/* config usart1 wakeup mode */
usart_wakeup_mode_set (USART1, USART_WAKEUP_BY_MATCHING_ID);
```

5.21.14 usart_receiver_mute_enable function

The table below describes the function usart_receiver_mute_enable.

Table 497. usart_receiver_mute_enable function

Name	Description
Function name	usart_receiver_mute_enable
Function prototype	void usart_receiver_mute_enable(usart_type* usart_x, confirm_state new_state);
Function description	Enable USART receiver mute mode
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2, and USART3
Input parameter 2	new_state: Enable (TRUE) or disable (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* config receiver mute */
usart_receiver_mute_enable (USART1, TRUE);
```

5.21.15 usart_break_bit_num_set function

The table below describes the function usart_break_bit_num_set.

Table 498. usart_break_bit_num_set function

Name	Description
Function name	usart_break_bit_num_set
Function prototype	void usart_break_bit_num_set(usart_type* usart_x, usart_break_bit_num_type break_bit);
Function description	Set USART break frame length
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2, and USART3
Input parameter 2	break_bit: break frame length type
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

break_bit

Set break frame length.

USART_BREAK_10BITS: 10 bits

USART_BREAK_11BITS: 11 bits

Example:

```
/* config break frame length 10bits */
usart_break_bit_num_set (USART1, USART_BREAK_10BITS);
```

5.21.16 usart_lin_mode_enable function

The table below describes the function usart_lin_mode_enable.

Table 499. usart_lin_mode_enable function

Name	Description
Function name	usart_lin_mode_enable
Function prototype	void usart_lin_mode_enable(usart_type* usart_x, confirm_state new_state);
Function description	Enable LIN mode
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2, and USART3
Input parameter 2	new_state: Enable (TRUE) or disable (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable usart1 lin mode */
usart_lin_mode_enable (USART1, TRUE);
```

5.21.17 usart_data_transmit function

The table below describes the function usart_data_transmit.

Table 500. usart_data_transmit function

Name	Description
Function name	usart_data_transmit
Function prototype	void usart_data_transmit(usart_type* usart_x, uint16_t data);
Function description	Transmit data
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2, and USART3
Input parameter 2	Data: data to send
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* transmit data */
uint16_t data = 0x88;
usart_data_transmit (USART1, data);
```

5.21.18 usart_data_receive function

The table below describes the function usart_data_receive.

Table 501. usart_data_receive function

Name	Description
Function name	usart_data_receive
Function prototype	uint16_t usart_data_receive(usart_type* usart_x);
Function description	Receives data
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2, and USART3
Input parameter 2	NA
Output parameter	NA
Return value	uint16_t: return the received data
Required preconditions	NA
Called functions	NA

Example:

```
/* receive data */
uint16_t data = 0;
data = usart_data_receive (USART1);
```

5.21.19 usart_break_send function

The table below describes the function usart_break_send.

Table 502. usart_break_send function

Name	Description
Function name	usart_break_send
Function prototype	void usart_break_send(usart_type* usart_x);
Function description	Sends break frame
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2, and USART3
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* send break frame */
usart_break_send (USART1);
```

5.21.20 usart_smartcard_guard_time_set function

The table below describes the function usart_smartcard_guard_time_set.

Table 503. usart_smartcard_guard_time_set function

Name	Description
Function name	usart_smartcard_guard_time_set
Function prototype	void usart_smartcard_guard_time_set(usart_type* usart_x, uint8_t guard_time_val);
Function description	Set smartcard guard time
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2, and USART3
Input parameter 2	guard_time_val: guard time, 0x00~0xFF
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* usart guard time set to 2 bit */
usart_smartcard_guard_time_set(USART1, 0x2);
```

5.21.21 usart_irda_smartcard_division_set function

The table below describes the function usart_irda_smartcard_division_set.

Table 504. usart_irda_smartcard_division_set function

Name	Description
Function name	usart_irda_smartcard_division_set
Function prototype	void usart_irda_smartcard_division_set(usart_type* usart_x, uint8_t div_val);
Function description	Infrared and smartcard frequency division settings
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2 and USART3
Input parameter 2	div_val: division value
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* usart clock set to (apbclk / (2 * 20)) */
usart_irda_smartcard_division_set(USART1, 20);
```

5.21.22 usart_smartcard_mode_enable function

The table below describes the function usart_smartcard_mode_enable.

Table 505. usart_smartcard_mode_enable function

Name	Description
Function name	usart_smartcard_mode_enable
Function prototype	void usart_smartcard_mode_enable(usart_type* usart_x, confirm_state new_state);
Function description	Enable smartcode mode
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2 and USART3
Input parameter 2	new_state: Enable (TRUE), Disable (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable the smartcard mode */
usart_smartcard_mode_enable(USART1, TRUE);
```

5.21.23 usart_smartcard_nack_set function

The table below describes the function usart_smartcard_nack_set.

Table 506. usart_smartcard_nack_set function

Name	Description
Function name	usart_smartcard_nack_set
Function prototype	void usart_smartcard_nack_set(usart_type* usart_x, confirm_state new_state);
Function description	Enable smartcard NACK
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2 and USART3
Input parameter 2	new_state: Enable (TRUE), Disable (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable the nack transmission */
usart_smartcard_nack_set(USART1, TRUE);
```

5.21.24 usart_single_line_halfduplex_select function

The table below describes the function usart_single_line_halfduplex_select.

Table 507. usart_single_line_halfduplex_select function

Name	Description
Function name	usart_single_line_halfduplex_select
Function prototype	void usart_single_line_halfduplex_select(usart_type* usart_x, confirm_state new_state);
Function description	Enable single-wire half-duplex mode
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2 and USART3
Input parameter 2	new_state: Enable (TRUE), Disable (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable halfduplex */
usart_single_line_halfduplex_select(USART1, TRUE);
```

5.21.25 usart_irda_mode_enable function

The table below describes the function usart_irda_mode_enable.

Table 508. usart_irda_mode_enable function

Name	Description
Function name	usart_irda_mode_enable
Function prototype	void usart_irda_mode_enable(usart_type* usart_x, confirm_state new_state);
Function description	Enable infrared mode
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2 and USART3
Input parameter 2	new_state: Enable (TRUE), Disable (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable irda mode */
usart_irda_mode_enable(USART1, TRUE);
```

5.21.26 usart_irda_low_power_enable function

The table below describes the function usart_irda_low_power_enable.

Table 509. usart_irda_low_power_enable function

Name	Description
Function name	usart_irda_low_power_enable
Function prototype	void usart_irda_low_power_enable(usart_type* usart_x, confirm_state new_state);
Function description	Enable infrared low-power mode
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2 and USART3
Input parameter 2	new_state: Enable (TRUE), Disable (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable irda lowpower mode */
usart_irda_low_power_enable (USART1, TRUE);
```

5.21.27 usart_hardware_flow_control_set function

The table below describes the function usart_hardware_flow_control_set.

Table 510. usart_hardware_flow_control_set function

Name	Description
Function name	usart_hardware_flow_control_set
Function prototype	void usart_hardware_flow_control_set(usart_type* usart_x,usart_hardware_flow_control_type flow_state);
Function description	Set peripheral flow control
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2 and USART3
Input parameter 2	flow_state: flow control type
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

flow_state

USART_HARDWARE_FLOW_NONE: No hardware flow control
USART_HARDWARE_FLOW_RTS: RTS
USART_HARDWARE_FLOW_CTS: CTS
USART_HARDWARE_FLOW_RTS_CTS: RTS and CTS

Example:

```
/* hardware flow set none */
usart_hardware_flow_control_set (USART1, USART_HARDWARE_FLOW_NONE);
```

5.21.28 usart_flag_get function

The table below describes the function usart_flag_get.

Table 511. usart_flag_get function

Name	Description
Function name	usart_flag_get
Function prototype	flag_status usart_flag_get(usart_type* usart_x, uint32_t flag);
Function description	Get flag status
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2 and USART3
Input parameter 2	Flag: flag selection
Output parameter	NA
Return value	flag_status: SET or RESET
Required preconditions	NA
Called functions	NA

flag

USART_CTSCF_FLAG: CTS (Clear To Send) change flag
USART_BFF_FLAG: Break frame receive flag
USART_TDBE_FLAG: Transmit buffer empty flag
USART_TDC_FLAG: Transmit complete flag
USART_RDBF_FLAG: Receive data buffer full flag
USART_IDLEF_FLAG: Idle frame flag
USART_ROERR_FLAG: Receive overflow flag
USART_NERR_FLAG: Noise error flag
USART_FERR_FLAG: Frame error flag
USART_PERR_FLAG: Parity error flag

Example:

```
/* wait data transmit complete flag */
while(usart_flag_get (USART1, USART_TDC_FLAG) == RESET);
```

5.21.29 usart_interrupt_flag_get function

The table below describes the function usart_interrupt_flag_get.

Table 512. usart_interrupt_flag_get function

Name	Description
Function name	usart_interrupt_flag_get
Function prototype	flag_status usart_interrupt_flag_get(usart_type* usart_x, uint32_t flag);
Function description	Check whether the selected flag is set or not
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2…
Input parameter 2	flag: flag selection
Output parameter	NA
Return value	flag_status: SET or RESET
Required preconditions	NA
Called functions	NA

flag

USART_CTSCF_FLAG: CTS (Clear To Send) change flag
USART_BFF_FLAG: Break frame receive flag
USART_TDBE_FLAG: Transmit buff empty flag
USART_TDC_FLAG: Transmit complete flag
USART_RDBF_FLAG: Receive data buffer full flag
USART_IDLEF_FLAG: Idle frame flag
USART_ROERR_FLAG: Receive overflow flag
USART_NERR_FLAG: Noise error flag
USART_FERR_FLAG: Frame error flag
USART_PERR_FLAG: Parity error flag

Example

```
/* check received data flag */
if(usart_interrupt_flag_get(USART1, USART_RDBF_FLAG) != RESET)
{
}
```

5.21.30 usart_flag_clear function

The table below describes the function usart_flag_clear.

Table 513. usart_flag_clear function

Name	Description
Function name	usart_flag_clear
Function prototype	void usart_flag_clear(usart_type* usart_x, uint32_t flag);
Function description	Clear flag
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2 and USART3
Input parameter 2	Flag: clear the selected flag
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

flag

USART_CTSCF_FLAG: CTS (Clear To Send) change flag

USART_BFF_FLAG: Break frame receive flag

USART_TDC_FLAG: Transmit complete flag

USART_RDBF_FLAG: Receive data buffer full flag

Example:

```
/* clear data transmit complete flag */
usart_flag_clear (USART1, USART_TDC_FLAG );
```

5.22 Watchdog timer (WDT)

The WDT register structure wdt_type is defined in the "at32f403a_407_wdt.h":

```
/**
  * @brief type define wdt register all
  */
typedef struct
{


} wdt_type;
```

The table below gives a list of the WDT registers.

Table 514. Summary of WDT registers

Register	Description
cmd	Command register
div	Divider register
rld	Reload register
sts	Status register

The table below gives a list of WDT library functions.

Table 515. Summary of WDT library functions

Function name	Description
wdt_enable	Enable watchdog
wdt_counter_reload	Reload counter
wdt_reload_value_set	Set reload value
wdt_divider_set	Set division value
wdt_register_write_enable	Unlock WDT_DIV and WDT_RLD register write protection
wdt_flag_get	Get falg

5.22.1 wdt_enable function

The table below describes the function wdt_enable.

Table 516. wdt_enable function

Name	Description
Function name	wdt_enable
Function prototype	void wdt_enable(void);
Function description	Enable watchdog
Input parameter	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
wdt_enable();
```

5.22.2 wdt_counter_reload function

The table below describes the function wdt_counter_reload.

Table 517. wdt_counter_reload function

Name	Description
Function name	wdt_counter_reload
Function prototype	void wdt_counter_reload(void);
Function description	Reload counter
Input parameter	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
wdt_counter_reload();
```

5.22.3 wdt_reload_value_set function

The table below describes the function wdt_reload_value_set.

Table 518. wdt_reload_value_set function

Name	Description
Function name	wdt_reload_value_set
Function prototype	void wdt_reload_value_set(uint16_t reload_value);
Function description	Set reload value
Input parameter	reload_value: reload value, 0x000~0xFFF
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
wdt_reload_value_set(0xFFF);
```

5.22.4 wdt_divider_set function

The table below describes the function wdt_divider_set.

Table 519. wdt_divider_set function

Name	Description
Function name	wdt_divider_set
Function prototype	void wdt_divider_set(wdt_division_type division);
Function description	Set division value
Input parameter	Division: watchdog division value Refer to the “division” description below for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

division

Select watchdog division value.

WDT_CLK_DIV_4: Divided by 4

WDT_CLK_DIV_8: Divided by 8

WDT_CLK_DIV_16: Divided by 16

WDT_CLK_DIV_32: Divided by 32

WDT_CLK_DIV_64: Divided by 64

WDT_CLK_DIV_128: Divided by128

WDT_CLK_DIV_256: Divided by 256

Example:

```
wdt_divider_set(WDT_CLK_DIV_4);
```

5.22.5 wdt_register_write_enable function

The table below describes the function wdt_register_write_enable.

Table 520. wdt_register_write_enable function

Name	Description
Function name	wdt_register_write_enable
Function prototype	void wdt_register_write_enable(confirm_state new_state);
Function description	Unlock WDT_DIV and WDT_RLD write protection
Input parameter	new_state: unlock register write protection This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
wdt_register_write_enable(TRUE);
```

5.22.6 wdt_flag_get function

The table below describes the function wdt_flag_get.

Table 521. wdt_flag_get function

Name	Description
Function name	wdt_flag_get
Function prototype	flag_status wdt_flag_get(uint16_t wdt_flag);
Function description	Get flag
Input parameter	Flag: flag selection Refer to the “flag” description below for details.
Output parameter	NA
Return value	flag_status: flag status This parameter can be SET or RESET.
Required preconditions	NA
Called functions	NA

flag

This is used for flag selection, including:

WDT_DIVF_UPDATE_FLAG: Division value update complete

WDT_RLDF_UPDATE_FLAG:Reload value update complete

Example:

```
wdt_flag_get(WDT_DIVF_UPDATE_FLAG);
```

5.23 Window watchdog timer (WWDT)

The WWDT register structure wwdt_type is defined in the "at32f403a_407_wwdt.h":

```
/**
  * @brief type define wwdt register all
  */
typedef struct
{


} wwdt_type;
```

The table below gives a list of the WWDT registers.

Table 522. Summary of WWDT registers

Register	Description
ctrl	Control register
cfg	Configuration register
sts	Status register

The table below gives a list of WWDT library functions.

Table 523. Summary of WWDT library functions

Function name	Description
wwdt_reset	Reset window watchdog registers
wwdt_divider_set	Set divider
wwdt_flag_clear	Clear reload counter interrupt flag
wwdt_enable	Enable WWDT
wwdt_interrupt_enable	Enable reload counter interrupt
wwdt_flag_get	Get flag
wwdt_counter_set	Set counter value
wwdt_window_counter_set	Set window value

5.23.1 wwdt_reset function

The table below describes the function wwdt_reset.

Table 524. wwdt_reset function

Name	Description
Function name	wwdt_reset
Function prototype	void wwdt_reset(void);
Function description	Reset window watchdog registers to their initial values.
Input parameter	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	void crm_periph_reset(crm_periph_reset_type value, confirm_state new_state);

Example:

```
wwdt_reset();
```

5.23.2 wwdt_divider_set function

The table below describes the function wwdt_divider_set.

Table 525. wwdt_divider_set function

Name	Description
Function name	wwdt_divider_set
Function prototype	void wwdt_divider_set(wwdt_division_type division);
Function description	Set divider
Input parameter	Division: WWDT division value Refer to the “division” description below for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

division

Select WWDT division value.

WWDT_PCLK1_DIV_4096: Divided by 4096

WWDT_PCLK1_DIV_8192: Divided by 8192

WWDT_PCLK1_DIV_16384: Divided by 16384

WWDT_PCLK1_DIV_32768: Divided by 32768

Example:

```
wwdt_divider_set(WWDT_PCLK1_DIV_4096);
```

5.23.3 wwdt_enable function

The table below describes the function wwdt_enable.

Table 526. wwdt_enable function

Name	Description
Function name	wwdt_enable
Function prototype	void wwdt_enable(uint8_t wwdt_cnt);
Function description	Enable WWDT
Input parameter	wwdt_cnt: WWDT counter initial value, 0x40~0x7F
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
wwdt_enable(0x7F);
```

5.23.4 wwdt_interrupt_enable function

The table below describes the function wwdt_interrupt_enable.

Table 527. wwdt_interrupt_enable function

Name	Description
Function name	wwdt_interrupt_enable
Function prototype	void wwdt_interrupt_enable(void);
Function description	Enable reload counter interrupt
Input parameter	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
wwdt_interrupt_enable();
```

5.23.5 wwdt_counter_set function

The table below describes the function wwdt_counter_set.

Table 528. wwdt_counter_set function

Name	Description
Function name	wwdt_counter_set
Function prototype	void wwdt_counter_set(uint8_t wwdt_cnt);
Function description	Set counter value
Input parameter	wwdt_cnt: WWDT counter value, 0x40~0x7F
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
wwdt_counter_set(0x7F);
```

5.23.6 wwdt_window_counter_set function

The table below describes the function wwdt_window_counter_set.

Table 529. wwdt_window_counter_set function

Name	Description
Function name	wwdt_window_counter_set
Function prototype	void wwdt_window_counter_set(uint8_t window_cnt);
Function description	Set window value
Input parameter	wwdt_cnt: WWDT window value, 0x40~0x7F
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
wwdt_window_counter_set(0x6F);
```

5.23.7 wwdt_flag_get function

The table below describes the function wwdt_flag_get.

Table 530. wwdt_flag_get function

Name	Description
Function name	wwdt_flag_get
Function prototype	flag_status wwdt_flag_get(void);
Function description	Get reload counter interrupt flag
Input parameter	NA
Output parameter	NA
Return value	flag_status: flag status Return SET or RESET
Required preconditions	NA
Called functions	NA

Example:

```
wwdt_flag_get();
```

5.23.8 wwdt_interrupt_flag_get function

The table below describes the function wwdt_interrupt_flag_clear.

Table 531. wwdt_interrupt_flag_clear function

Name	Description
Function name	wwdt_interrupt_flag_get
Function prototype	flag_status wwdt_ interrupt_flag_get(void);
Function description	Get reload counter interrupt flag and judge the corresponding interrupt enable bit
Input parameter 1	NA
Output parameter	NA
Return value	flag_status: flag status Return SET or RESET.
Required preconditions	NA
Called functions	NA

Example

```
wwdt_ interrupt_flag_get();
```

5.23.9 wwdt_flag_clear function

The table below describes the function wwdt_flag_clear.

Table 532. wwdt_flag_clear function

Name	Description
Function name	wwdt_flag_clear
Function prototype	void wwdt_flag_clear(void);
Function description	Clear reload counter interrupt flag
Input parameter	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
wwdt_flag_clear();
```

5.24 External memory controller (XMC)

For XMC bank1 chip-select control register and timing register, their register structure xmc_bank1_ctrl_tmg_reg_type is defined in the *"at32f403a_407_xmc.h"*:

```
/**
  * @brief type define xmc bank1 ctrl and tmg register
  */
typedef struct
{
      …

} xmc_bank1_ctrl_tmg_reg_type;
```

XMC bank1 write timing register structure xmc_bank1_tmgwr_reg_type is defined in the *"at32f403a_407_xmc.h"*:

```
/**
  * @brief type define xmc bank1 tmgwr register
  */
typedef struct
{
      …

} xmc_bank1_tmgwr_reg_type;
```

XMC bank1 register structure xmc_bank1_type is defined in the *"at32f403a_407_xmc.h"*:

```
/**
  * @brief xmc bank1 registers
  */
typedef struct
{
      …

} xmc_bank1_type;
```

XMC bank2 register structure xmc_bank2_type is defined in the *"at32f403a_407_xmc.h"*:

```
/**
  * @brief xmc bank2 registers
  */
typedef struct
{
      …

} xmc_bank2_type;
```

The table below gives a list of the XMC registers.

Table 533. Summary of XMC registers

Register	Description
xmc_bk1ctrl1	SRAM/NOR Flash chip select control register 1
xmc_bk1tmg1	SRAM/NOR Flash chip select timing register 1
xmc_bk1ctrl4	SRAM/NOR Flash chip select control register 4
xmc_bk1tmg4	SRAM/NOR Flash chip select timing register 4
xmc_bk2ctrl	Nand Flash control register 2
xmc_bk2is	Interrupt enable and FIFO status register 2
xmc_bk2tmgrg	Regular memory timing register 2
xmc_bk2tmgsp	Special memory timing register 2
xmc_bk2ecc	ECC value register2
xmc_bk1tmgwr1	SRAM/NOR Flash write timing register 1
xmc_bk1tmgwr4	SRAM/NOR Flash write timing register 4
xmc_ext1	SRAM/NOR Flash extra timing register 1
xmc_ext4	SRAM/NOR Flash extra timing register 4

The table below gives a list of XMC library functions.

Table 534. Summary of XMC library functions

Function name	Description
xmc_nor_sram_reset	Reset nor/sram controller
xmc_nor_sram_init	Initialize nor/sram controller
xmc_nor_sram_timing_config	Configure nor/sram controller timing
xmc_norsram_default_para_init	Initialize the parameters of the xmc_nor_sram_init_struct
xmc_norsram_timing_default_para_init	Initialize the parameters of the xmc_rw_timing_struct and xmc_w_timing_struct
xmc_nor_sram_enable	Enable nor/sram controller
xmc_ext_timing_config	Configure nor/sram extension controller
xmc_nand_reset	Reset nand controller
xmc_nand_init	Initialize nand controller
xmc_nand_timing_config	Configure nand controller timing
xmc_nand_default_para_init	Initialize the parameters of the xmc_nand_init_struct
xmc_nand_timing_default_para_init	Initialize the parameters of the xmc_common_spacetiming_struct and xmc_attribute_spacetiming_struct
xmc_nand_enable	Enable nand controller
xmc_nand_ecc_enable	Enable nand controller ECC function
xmc_ecc_get	Get ECC value
xmc_interrupt_enable	Enable XMC interrupts
xmc_flag_status_get	Get XMC interrupt flags
xmc_flag_clear	Clear XMC interrupt flags

5.24.1 xmc_nor_sram_reset function

The table below describes the function xmc_nor_sram_reset.

Table 535. xmc_nor_sram_reset function

Name	Description
Function name	xmc_nor_sram_reset
Function prototype	void xmc_nor_sram_reset(xmc_nor_sram_subbank_type xmc_subbank);
Function description	Reset nor/sram controller
Input parameter	xmc_subbank: subbank selection
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

xmc_subbank

Select a subbank to reset.

XMC_BANK1_NOR_SRAM1: XMC subbank1

XMC_BANK1_NOR_SRAM4: XMC subbank4

Example:

```
/* reset nor/sram subbank1 */
xmc_nor_sram_reset(XMC_BANK1_NOR_SRAM1);
```

5.24.2 xmc_nor_sram_init function

The table below describes the function xmc_nor_sram_init.

Table 536. xmc_nor_sram_init function

Name	Description
Function name	xmc_nor_sram_init
Function prototype	void xmc_nor_sram_init(xmc_norsram_init_type* xmc_norsram_init_struct);
Function description	Initialize nor/sram controller
Input parameter 1	xmc_norsram_init_struct: xmc_norsram_init_type pointer
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

xmc_norsram_init_type struct

xmc_norsram_init_type is defined in the at32f403a_407_xmc.h:

```
typedef struct
{
  xmc_nor_sram_subbank_type    subbank;
  xmc_data_addr_mux_type       data_addr_multiplex;
  xmc_memory_type              device;
  xmc_data_width_type          bus_type;
  xmc_burst_access_mode_type   burst_mode_enable;
  xmc_asyn_wait_type           asynwait_enable;
  xmc_wait_signal_polarity_type wait_signal_lv;
```

```
  xmc_wrap_mode_type        wrapped_mode_enable;
  xmc_wait_timing_type      wait_signal_config;
  xmc_write_operation_type  write_enable;
  xmc_wait_signal_type      wait_signal_enable;
  xmc_extended_mode_type    write_timing_enable;
  xmc_write_burst_type      write_burst_syn;
} xmc_norsram_init_type;
```

subbank

Select a subbank to reset.

XMC_BANK1_NOR_SRAM1: XMC subbank1

XMC_BANK1_NOR_SRAM4: XMC subbank4

data_addr_multiplex

Define whether the lower 16 bits of the xmc address line is multiplexed with data line or not.

XMC_DATA_ADDR_MUX_DISABLE: Not multiplexed

XMC_DATA_ADDR_MUX_ENABLE: Multiplexed

Device

Select an external memory device.

XMC_DEVICE_SRAM: sram

XMC_DEVICE_PSRAM: psram

XMC_DEVICE_NOR: nor

bus_type

xmc data bus bit width.

XMC_BUSTYPE_8_BITS: 8-bit data bus

XMC_BUSTYPE_16_BITS: 1-6bit data bus

burst_mode_enable

Enable or disable burst mode.

XMC_BURST_MODE_DISABLE: Burst mode disabled

XMC_BURST_MODE_ENABLE: Burst mode enabled

asynwait_enable

Enable or disable wait signals during asynchronous transmission.

XMC_ASYN_WAIT_DISABLE: Disabled

XMC_ASYN_WAIT_ENABLE: Enabled

wait_signal_lv

Select the polarity of wait signals. This bit is used to set the polarity of NWAIT sigal in synchronous mode.

XMC_WAIT_SIGNAL_LEVEL_LOW: Active low

XMC_WAIT_SIGNAL_LEVEL_HIGH: Active high

wrapped_mode_enable

In synchronous mode, this bit is used to define whether the XMC controoler will or not split an not-aligned AHB access intro two accesses.

XMC_WRAPPED_MODE_DISABLE: Direct wrapped burst mode is disabled

XMC_WRAPPED_MODE_ENABLE: Direct wrapped burst mode is enabled

wait_signal_config

Wait timing configuration. Valid only in synchronous mode.

XMC_WAIT_SIGNAL_SYN_BEFORE: NWAIT signal is active one data cycle before wait state

XMC_WAIT_SIGNAL_SYN_DURING: NWAIT signal is active during wait state

write_enable

Write enable bit.

XMC_WRITE_OPERATION_DISABLE: Write operation is disabled

XMC_WRITE_OPERATION_ENABLE: Write operation is enabled

wait_signal_enable

Wait signal enable bit during synchronous transmission.

XMC_WAIT_SIGNAL_DISABLE: NWAIT signal disabled

XMC_WAIT_SIGNAL_ENABLE: NWAIT signal enabled

write_timing_enable

Wait timing enable bit. Read memory and write memory operate at different timings. In other words, SRAM/NOR write timing register (XMC_BKxTMGWR) is released.

XMC_WRITE_TIMING_DISABLE: Read and write timings are the same

XMC_WRITE_TIMING_ENABLE: Read and write timings are different

write_burst_syn

Write burst enabe bit

XMC_WRITE_BURST_SYN_DISABLE: Write operation is performed in asynchronous mode

XMC_WRITE_BURST_SYN_ENABLE: Write operation is performed in synchronous mode

Example:

```
xmc_norsram_init_type xmc_norsram_init_struct;
xmc_norsram_init_struct.subbank                = XMC_BANK1_NOR_SRAM1;
xmc_norsram_init_struct.data_addr_multiplex    = XMC_DATA_ADDR_MUX_ENABLE;
xmc_norsram_init_struct.device                 = XMC_DEVICE_PSRAM;
xmc_norsram_init_struct.bus_type               = XMC_BUSTYPE_16_BITS;
xmc_norsram_init_struct.burst_mode_enable      = XMC_BURST_MODE_DISABLE;
xmc_norsram_init_struct.asynwait_enable        = XMC_ASYN_WAIT_DISABLE;
xmc_norsram_init_struct.wait_signal_lv         = XMC_WAIT_SIGNAL_LEVEL_LOW;
xmc_norsram_init_struct.wrapped_mode_enable    = XMC_WRAPPED_MODE_DISABLE;
xmc_norsram_init_struct.wait_signal_config     = XMC_WAIT_SIGNAL_SYN_BEFORE;
xmc_norsram_init_struct.write_enable           = XMC_WRITE_OPERATION_ENABLE;
xmc_norsram_init_struct.wait_signal_enable     = XMC_WAIT_SIGNAL_DISABLE;
xmc_norsram_init_struct.write_burst_syn        = XMC_WRITE_BURST_SYN_DISABLE;
xmc_norsram_init_struct.write_timing_enable    = XMC_WRITE_TIMING_DISABLE;
xmc_nor_sram_init(&xmc_norsram_init_struct);
```

5.24.3 xmc_nor_sram_timing_config function

Table 51describes the function xmc_nor_sram_timing_config.

Table 537. scfg_mem_map_get function

Name	Description
Function name	xmc_nor_sram_timing_config
Function prototype	void xmc_nor_sram_timing_config(xmc_norsram_timing_init_type* xmc_rw_timing_struct, xmc_norsram_timing_init_type* xmc_w_timing_struct);
Function description	Configure the nor/sram controller timing
Input parameter 1	xmc_rw_timing_struct: xmc_norsram_timing_init_type pointer It is used to configure read/write timing when the write timing is not enabled independently.
Input parameter 2	xmc_w_timing_struct: xmc_norsram_timing_init_type pointer It is used to configure write timing when the write timing is enabled independently.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

xmc_norsram_timing_init_type struct

xmc_norsram_timing_init_type is defined in the at32f403a_407_xmc.h:

```
typedef struct
{
  xmc_nor_sram_subbank_type    subbank;
  xmc_extended_mode_type       write_timing_enable;
  uint32_t                     addr_setup_time;
  uint32_t                     addr_hold_time;
  uint32_t                     data_setup_time;
  uint32_t                     bus_latency_time;
  uint32_t                     clk_psc;
  uint32_t                     data_latency_time;
   xmc_access_mode_type        mode;
} xmc_norsram_timing_init_type;
```

subbank

Select a subbank to reset.

XMC_BANK1_NOR_SRAM1: XMC subbank1

XMC_BANK1_NOR_SRAM4: XMC subbank4

write_timing_enable

Wait timing enable bit. Read memory and write memory operate at different timings. In other words, SRAM/NOR write timing register (XMC_BKxTMGWR) is released.

XMC_WRITE_TIMING_DISABLE: Read and write timings are the same

XMC_WRITE_TIMING_ENABLE: Read and write timings are different

addr_setup_time

Address setup time.

0000: 0 HCLK clock cycle added
0001: 1 HCLK clock cycle added
……
1111: 15 HCLK clock cycles added

addr_hold_time

Address hold time.
0000: 0 HCLK clock cycle added
0001: 1 HCLK clock cycles added
……
1111: 15 HCLK clock cycles added

data_setup_time

Data setup time
0000: 0 HCLK clock cycle added
0001: 1 HCLK clock cycle added
……
1111: 15 HCLK clock cycles added

bus_latency_time

Bus latency time. In order to avoid conflicts on data bus, in multiplexed mode or synchronous mode, the XMC controller inserts latency time into data bus after a single read operation.
0000: 1 HCLK clock cycle added
0001: 2 HCLK clock cycles added
……
1111: 16 HCLK clock cycles added

clk_psc

Clock frequency division factor. Valid only in synchronous mode. It is used to define the clock frequency of the XMC_CLK.
0000: Reserved
0001: XMC_CLK cycles= 2 x HCLK clock cycles
0010: XMC_CLK cycls= 3 x HCLK clock cycles
……
1111:XMC_CLK cycles= 16 x HCLK clock cycles

data_latency_time

Data latency time. Valid only in synchronous mode.
0000: 0 HCLK clock cycle added
0001: 1 HCLK clock cycle added
……
1111: 15 HCLK clock cycles added

Mode

Asynchronous access mode selection bit. Valid only when the RWTD bit is enabled.
00: Mode A
01: Mode B
10: Mode C
11: Mode D

Example:

```
xmc_norsram_timing_init_type    rw_timing_struct, w_timing_struct;
```

```
rw_timing_struct.subbank                = XMC_BANK1_NOR_SRAM1;
rw_timing_struct.mode                   = XMC_ACCESS_MODE_A;
rw_timing_struct.write_timing_enable    = XMC_WRITE_TIMING_DISABLE;
rw_timing_struct.addr_hold_time         = 0x08;
rw_timing_struct.addr_setup_time        = 0x09;
rw_timing_struct.data_setup_time        = 0x0F;
rw_timing_struct.data_latency_time      = 0x0;
rw_timing_struct.bus_latency_time       = 0x0;
rw_timing_struct.clk_psc                = 0x0;

w_timing_struct.subbank                 = XMC_BANK1_NOR_SRAM1;
w_timing_struct.mode                    = XMC_ACCESS_MODE_A;
w_timing_struct.write_timing_enable     = XMC_WRITE_TIMING_DISABLE;
w_timing_struct.addr_hold_time          = 0x08;
w_timing_struct.addr_setup_time         = 0x09;
w_timing_struct.data_setup_time         = 0x0F;
w_timing_struct.data_latency_time       = 0x0;
w_timing_struct.bus_latency_time        = 0x0;
w_timing_struct.clk_psc                 = 0x0;

xmc_nor_sram_timing_config(&rw_timing_struct, &w_timing_struct);
```

5.24.4 xmc_norsram_default_para_init function

The table below describes the function xmc_ norsram_default_para_init.

Table 538. xmc_ norsram_default_para_init function

Name	Description
Function name	xmc_norsram_default_para_init
Function prototype	void xmc_norsram_default_para_init(xmc_norsram_init_type* xmc_nor_sram_init_struct);
Function description	Initialize the parameters of the xmc_nor_sram_init_struct
Input parameter	xmc_nor_sram_init_struct: xmc_norsram_init_type pointer
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Table 521 shows the values of members in the xmc_nor_sram_init_struct:

Table 539. xmc_nor_sram_init_struct default values

Member	Default values
subbank	XMC_BANK1_NOR_SRAM1
data_addr_multiplex	XMC_DATA_ADDR_MUX_ENABLE
device	XMC_DEVICE_SRAM
bus_type	XMC_BUSTYPE_8_BITS
burst_mode_enable	XMC_BURST_MODE_DISABLE
asynwait_enable	XMC_ASYN_WAIT_DISABLE
wait_signal_lv	XMC_WAIT_SIGNAL_LEVEL_LOW
wrapped_mode_enable	XMC_WRAPPED_MODE_DISABLE
wait_signal_config	XMC_WAIT_SIGNAL_SYN_BEFORE
write_enable	XMC_WRITE_OPERATION_ENABLE
wait_signal_enable	XMC_WAIT_SIGNAL_ENABLE
write_timing_enable	XMC_WRITE_TIMING_DISABLE
write_burst_syn	XMC_WRITE_BURST_SYN_DISABLE

Example:

```
xmc_norsram_init_type xmc_norsram_init_struct;
/* fill each xmc_nor_sram_init_struct member with its default value */
xmc_norsram_default_para_init(&xmc_norsram_init_struct);
```

5.24.5 xmc_norsram_timing_default_para_init function

The table below describes the function xmc_norsram_timing_default_para_init.

Table 540. xmc_norsram_timing_default_para_init function

Name	Description
Function name	xmc_norsram_timing_default_para_init
Function prototype	void xmc_norsram_timing_default_para_init(xmc_norsram_timing_init_type* xmc_rw_timing_struct, xmc_norsram_timing_init_type* xmc_w_timing_struct);
Function description	Initialize the parameters of the xmc_rw_timing_struct and xmc_w_timing_struct
Input parameter	xmc_rw_timing_struct: xmc_norsram_timing_init_type pointer xmc_w_timing_struct: xmc_norsram_timing_init_type pointer
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

The table below shows the values of members in the xmc_rw_timing_struct and xmc_w_timing_struct:

Table 541. xmc_rw_timing_struct and xmc_w_timing_struct

Member	Default values
subbank	XMC_BANK1_NOR_SRAM1
write_timing_enable	XMC_WRITE_TIMING_DISABLE
addr_setup_time	0xF
addr_hold_time	0xF
data_setup_time	0xFF
bus_latency_time	0xF
clk_psc	0xF
data_latency_time	0xF
mode	XMC_ACCESS_MODE_A

Example:

```
xmc_norsram_timing_init_type  rw_timing_struct, w_timing_struct;
/* fill each xmc_rw_timing_struct and xmc_w_timing_struct member with its default value */
xmc_norsram_timing_default_para_init (&xmc_rw_timing_struct, &xmc_w_timing_struct);
```

5.24.6 xmc_nor_sram_enable function

The table below describes the function xmc_nor_sram_enable.

Table 542. xmc_nor_sram_enable function

Name	Description
Function name	xmc_nor_sram_enable
Function prototype	void xmc_nor_sram_enable(xmc_nor_sram_subbank_type xmc_subbank, confirm_state new_state);
Function description	Enable nor/sram controller
Input parameter 1	xmc_subbank: subbank selection
Input parameter 2	new_state: enable or disable
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

xmc_subbank

Select a subbank.

XMC_BANK1_NOR_SRAM1: xmc subbank1

XMC_BANK1_NOR_SRAM4: xmc subbank4

new_state

FALSE: Disable controller

TRUE: Enable controller

Example:

```
/* enable xmc bank1_sram bank */
xmc_nor_sram_enable(XMC_BANK1_NOR_SRAM1, TRUE);
```

5.24.7 xmc_ext_timing_config function

The table below describes the function xmc_ext_timing_config.

Table 543. xmc_ext_timing_configfunction

Name	Description
Function name	xmc_ext_timing_config
Function prototype	void xmc_ext_timing_config(xmc_nor_sram_subbank_type xmc_sub_bank, uint16_t w2w_timing, uint16_t r2r_timing);
Function description	Configure nor/sram extension controller
Input parameter 1	xmc_subbank: subbank selection
Input parameter 2	w2w_timing: bus turnaround phase duration between two consecutive write operations.
Input parameter 3	r2r_timing: bus turnaround phase duration between two consecutive read operations.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

xmc_subbank

Select a subbank to reset.

XMC_BANK1_NOR_SRAM1: xmc subbank1

XMC_BANK1_NOR_SRAM4: xmc subbank4

w2w_timing

This is used to define the bus turnaround phase duration between two consecutive write operations.

00000000: 1 HCLK clock cycle inserted between two consecutive write operations

00000001: 2 HCLK clock cycles inserted between two consecutive write operations

……

00001000: 9 HCLK clock cycles (default value) inserted between two consecutive read operations

……

11111111: 256 HCLK clock cycles inserted between two consecutive read operations

r2r_timing

This is used to define the bus turnaround phase duration between two consecutive read operations.

00000000: 1 HCLK clock cycle inserted between two consecutive write operations

00000001: 2 HCLK clock cycles inserted between two consecutive write operations

……

00001000: 9 HCLK clock cycles (default value) inserted between two consecutive read operations

……

11111111: 256 HCLK clock cycles inserted between two consecutive write operations

Example:

```
/* bus turnaround phase for consecutive read duration and consecutive write duration */
xmc_ext_timing_config(XMC_BANK1_NOR_SRAM1, 0x08, 0x08);
```

5.24.8 xmc_nand_reset function

The table below describes the function xmc_nand_reset.

Table 544. xmc_nand_reset function

Name	Description
Function name	xmc_nand_reset
Function prototype	void xmc_nand_reset(xmc_class_bank_type xmc_bank);
Function description	Reset nand controller
Input parameter	xmc_bank: define nand controller, it can be XMC_BANK2_NAND
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
  /* reset nand flash */
xmc_nand_reset(XMC_BANK2_NAND);
```

5.24.9 xmc_nand_init function

The table below describes the function xmc_nand_init.

Table 545. xmc_nand_init function

Name	Description
Function name	xmc_nand_init
Function prototype	void xmc_nand_init(xmc_nand_init_type* xmc_nand_init_struct);
Function description	Initialize nand controller
Input parameter	xmc_nand_init_struct: xmc_nand_init_type pointer
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

xmc_nand_init_type struct

xmc_nand_init_type is defined in the at32f403a_407_xmc.h:

```
typedef struct
{
  xmc_class_bank_type      nand_bank;
  xmc_nand_wait_type       wait_enable;
  xmc_data_width_type      bus_type;
  xmc_ecc_enable_type      ecc_enable;
  xmc_ecc_pagesize_type    ecc_pagesize;
  uint32_t                 delay_time_cycle;
  uint32_t                 delay_time_ar;
} xmc_nand_init_type;
```

Nand_bank

Select a nand_bank to initialize.

XMC_BANK2_NAND: Initialize bank2 nand controller

wait_enable

Wait enable bit. This is to enable the wait feature for the NAND Flash memory bank.

XMC_WAIT_OPERATION_DISABLE: Disabled

XMC_WAIT_OPERATION_ENABLE: Enabled

bus_type

External memory data width. This is to define the width of the external NAND Flash data bus.

XMC_BUSTYPE_8_BITS: 8-bit data bus

XMC_BUSTYPE_16_BITS: 16-bit data bus

ecc_enable

xmc data bus bit width.

XMC_BUSTYPE_8_BITS: 8-bit data bus

XMC_BUSTYPE_16_BITS: 16-bit data bus

ecc_pagesize

ECC page size

000: 256 bytes

001: 512 bytes

010: 1024 bytes

011: 2048 bytes

100: 4096 bytes

101: 8192 bytes

delay_time_cycle

The delay time from CLE to RE, that is, from the CLE falling edge to the falling edge of RE.

0000:1 HCLK clock cycle

……

1111:16 HCLK clock cycles

delay_time_ar

The delay time from ALE to RE, that is, from the ALE falling edge to the falling edge of RE.

0000:1 HCLK clock cycle

……

1111:16 HCLK clock cycles

Example:

```
  xmc_nand_init_type nand_init_struct;
nand_init_struct.nand_bank = XMC_BANK2_NAND;
nand_init_struct.wait_enable = XMC_WAIT_OPERATION_DISABLE;
nand_init_struct.bus_type = XMC_BUSTYPE_8_BITS;
nand_init_struct.ecc_enable = XMC_ECC_OPERATION_DISABLE;
nand_init_struct.ecc_pagesize = XMC_ECC_PAGESIZE_2048_BYTES;
nand_init_struct.delay_time_cycle = 0x10;
nand_init_struct.delay_time_ar = 0x10;
/* xmc nand flash configuration */
xmc_nand_init(&nand_init_struct);
```

5.24.10 xmc_nand_timing_config function

The table below describes the function xmc_nand_timing_config.

Table 546. xmc_nand_timing_config function

Name	Description
Function name	xmc_nand_timing_config
Function prototype	void xmc_nand_timing_config(xmc_nand_timinginit_type* xmc_common_spacetiming_struct, xmc_nand_timinginit_type* xmc_attribute_spacetiming_struct);
Function description	Configure nand controller timings
Input parameter 1	xmc_common_spacetiming_struct: xmc_nand_timinginit_type pointer, indicating common space timing parameters
Input parameter 2	xmc_ attribute _spacetiming_struct: xmc_nand_timinginit_type pointer, indicating the space timing parameters in a special space
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

xmc_nand_timinginit_type struct

xmc_nand_timinginit_type is defined in the at32f403a_407_xmc.h:

```
typedef struct
{
  xmc_class_bank_type      class_bank;
  uint32_t                 mem_setup_time;
  uint32_t                 mem_waite_time;
  uint32_t                 mem_hold_time;
  uint32_t                 mem_hiz_time;
} xmc_nand_timinginit_type;
```

class_bank

nand flash bank selection.

XMC_BANK2_NAND

mem_setup_time

For NAND Flash write accesess to common memory space, this bit is used to define the number of HCLK clock cycles to set up the address.

00000000: 0 HCLK clock cycle inserted between two consecutive write operations

00000001: 1 HCLK clock cycle inserted between two consecutive write operations

……

11111111: 255 HCLK clock cycles inserted between two consecutive write operations

mem_waite_time

For NAND Flash write accesses to common memory space, this is used to define the number of HCLK clock cycles to be waited during which the XMC_NWE and XMC_NOE are active low.

00000000: 0 HCLK clock cycle inserted between two consecutive write operations

00000001: 1 HCLK clock cycle inserted between two consecutive write operations

……

11111111: 255 HCLK clock cycles inserted between two consecutive write operations

mem_hold_time

For NAND Flash write accesses to the common memory space, this is used to define the number of HCLK clock cycles during which the data are held

00000000: Reserved

00000001: 1 HCLK clock cycle inserted between two consecutive write operations

……

11111111: 255 HCLK clock cycles inserted between two consecutive write operations

mem_hiz_time

This is used to define the number of HCLK clock cycles during which the data bus is kept in Hi-Z after the start of a NAND Flash write access to common memory space.

00000000: 0 HCLK clock cycle inserted between two consecutive write operations

00000001: 1 HCLK clock cycle inserted between two consecutive write operations

……

11111111: 255 HCLK clock cycles inserted between two consecutive write operations

Example:

```
xmc_regular_spacetimingstruct.class_bank = XMC_BANK2_NAND;
xmc_regular_spacetimingstruct.mem_setup_time = 254;
xmc_regular_spacetimingstruct.mem_hiz_time = 254;
xmc_regular_spacetimingstruct.mem_hold_time = 254;
xmc_regular_spacetimingstruct.mem_waite_time = 254;
xmc_nand_timing_config(&xmc_regular_spacetimingstruct, &xmc_regular_spacetimingstruct);
```

5.24.11 xmc_nand_default_para_init function

The table below describes the function xmc_nand_default_para_init.

Table 547. xmc_nand_default_para_init

Name	Description
Function name	xmc_nand_default_para_init
Function prototype	void xmc_nand_default_para_init(xmc_nand_init_type* xmc_nand_init_struct);
Function description	Initialize the parameters of the xmc_nand_init_struct
Input parameter	xmc_nand_init_struct: xmc_nand_init_type pointer
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

The table below shows the default values of members in the xmc_nand_init_struct.

Table 548. xmc_nand_init_struct

Member	Default values
nand_bank	XMC_BANK2_NAND
wait_enable	XMC_WAIT_OPERATION_DISABLE
bus_type	XMC_BUSTYPE_8_BITS
ecc_enable	XMC_ECC_OPERATION_DISABLE
ecc_pagesize	XMC_ECC_PAGESIZE_256_BYTES
delay_time_cycle	0x0
delay_time_ar	0x0

Example:

```
/* fill each nand_init_struct member with its default value */
xmc_nand_default_para_init(&nand_init_struct);
```

5.24.12 xmc_nand_timing_default_para_init function

The table below describes the function xmc_nand_timing_default_para_init.

Table 549. xmc_nand_timing_default_para_init function

Name	Description
Function name	xmc_nand_timing_default_para_init
Function prototype	void xmc_nand_timing_default_para_init(xmc_nand_timinginit_type* xmc_common_spacetiming_struct, xmc_nand_timinginit_type* xmc_attribute_spacetiming_struct);
Function description	Initialize the parameters of the xmc_common_spacetiming_struct and xmc_attribute_spacetiming_struct
Input parameter	xmc_common_spacetiming_struct: xmc_nand_timinginit_type pointer xmc_attribute_spacetiming_struct: mc_nand_timinginit_type pointer
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

The table below shows the default values of members in the xmc_rw_timing_struc and xmc_w_timing_struct.

Table 550. xmc_common_spacetiming_struct and xmc_attribute_spacetiming_struct

Member	Default values
class_bank	XMC_BANK2_NAND
mem_hold_time	0xFC
mem_waite_time	0xFC
mem_setup_time	0xFC
mem_hiz_time	0xFC

Example:

```
xmc_nand_timinginit_type  xmc_regular_spacetimingstruct;
/* fill each xmc_regular_spacetimingstruct member with its default value */
xmc_nand_timing_default_para_init(&xmc_regular_spacetimingstruct, &xmc_regular_spacetimingstruct);
```

5.24.13 xmc_nand_enable function

The table below describes the function xmc_nand_enable.

Table 551. xmc_nand_enable function

Name	Description
Function name	xmc_nand_enable
Function prototype	void xmc_nand_enable(xmc_class_bank_type xmc_bank, confirm_state new_state);
Function description	Enable nand flash controller
Input parameter 1	xmc_bank: bank selection
Input parameter 2	new_state: enable or disable
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

xmc_bank

Bank selection

XMC_BANK2_NAND

new_state

FALSE: Disabled

TRUE: Enabled

Example:

```
/* xmc nand bank enable*/
xmc_nand_enable(XMC_BANK2_NAND, TRUE);
```

5.24.14 xmc_nand_ecc_enable function

The table below describes the function xmc_nand_ecc_enable.

Table 552. xmc_nand_ecc_enable function

Name	Description
Function name	xmc_nand_ecc_enable
Function prototype	void xmc_nand_ecc_enable(xmc_class_bank_type xmc_bank, confirm_state new_state);
Function description	Enable nand flash controller ECC feature
Input parameter 1	xmc_bank: bank selection
Input parameter 2	new_state: enable or disable ECC feature.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

xmc_bank

Select a bank to ECC enable.

XMC_BANK2_NAND

new_state

FALSE: Disable ECC feature

TRUE: Enable ECC feature

Example:

```
/* calculate ecc value while transmiting */
```

```
xmc_nand_ecc_enable(XMC_BANK2_NAND, TRUE);
```

5.24.15 xmc_ecc_get function

The table below describes the function xmc_ecc_get.

Table 553. xmc_ecc_get function

Name	Description
Function name	xmc_ecc_get
Function prototype	uint32_t xmc_ecc_get(xmc_class_bank_type xmc_bank);
Function description	Get nand flash ECC value
Input parameter 1	xmc_bank: bank selection
Output parameter	NA
Return value	ECC value
Required preconditions	NA
Called functions	NA

xmc_bank

Select a bank to enable ECC.

XMC_BANK2_NAND

Example:

```
/* get ecc value */
xmc_ecc_get(XMC_BANK2_NAND, TRUE);
```

5.24.16 xmc_interrupt_enable function

The table below describes the function xmc_interrupt_enable.

Table 554. xmc_interrupt_enable function

Name	Description
Function name	xmc_interrupt_enable
Function prototype	void xmc_interrupt_enable(xmc_class_bank_type xmc_bank, xmc_interrupt_sources_type xmc_int, confirm_state new_state);
Function description	Enable xmc interrupts
Input parameter 1	xmc_bank: bank selection
Input parameter 2	xmc_int: interrupt selection
Input parameter 3	new_state: enable or disable interrupts
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

xmc_bank

Bank selection.

XMC_BANK2_NAND

xmc_int

XMC_INT_RISING_EDGE: XMC rising edge detection interrupt

XMC_INT_LEVEL: XMC high level detection interrupt

XMC_INT_FALLING_EDGE: XMC falling edge detection interrupt

new_state

FALSE: Interrupt disabled

TRUE: Interrupt enabled

Example:

```
/* xmc rising edge detection interrupt enable */
xmc_interrupt_enable(XMC_BANK2_NAND, XMC_INT_RISING_EDGE, TRUE);
```

5.24.17 xmc_flag_status_get function

The table below describes the function xmc_flag_status_get.

Table 555. xmc_flag_status_get function

Name	Description
Function name	xmc_flag_status_get
Function prototype	flag_status xmc_flag_status_get(xmc_class_bank_type xmc_bank, xmc_interrupt_flag_type xmc_flag);
Function description	Get XMC flag status
Input parameter 1	xmc_bank: xmc bank selection
Input parameter 2	xmc_flag: flag selection
Output parameter	NA
Return value	flag_status: check if the selected flag is set or not.
Required preconditions	NA
Called functions	NA

xmc_bank

XMC bank selection.

xmc_flag

xmc_flag is used for flag selection to get its status, including:

XMC_RISINGEDGE_FLAG: Rising edge status

XMC_LEVEL_FLAG: High level status

XMC_FALLINGEDGE_FLAG: Falling edge status

XMC_FEMPT_FLAG: FIFO empty status

flag_status

RESET: Flag is not set

SET: Flag is set

Example:

```
if(xmc_flag_status_get (XMC_BANK2_NAND, XMC_RISINGEDGE_FLAG) != RESET)
{
  ……
}
```

5.24.18 xmc_interrupt_flag_clear function

The table below describes the function xmc_interrupt_flag_clear.

Table 556. xmc_interrupt_flag_clear function

Name	Description
Function name	xmc_interrupt_flag_status_get
Function prototype	flag_status xmc_interrupt_flag_status_get(xmc_class_bank_type xmc_bank, xmc_interrupt_flag_type xmc_flag)
Function description	Get XMC interrupt flag status
Input parameter 1	xmc_bank: xmc bank selection
Input parameter 2	xmc_flag: flag selection
Output parameter	NA
Return value	flag_status: check if the selected flag is set or not.
Required preconditions	NA
Called functions	NA

xmc_bank

XMC bank selection.

xmc_flag

xmc_flag is used for flag selection to get its status, including:

XMC_RISINGEDGE_FLAG: Rising edge status

XMC_LEVEL_FLAG: High level status

XMC_FALLINGEDGE_FLAG: Falling edge status

flag_status

RESET: Flag is not set

SET: Flag is set

Example

```
if(xmc_interrupt_flag_status_get (XMC_BANK2_NAND, XMC_RISINGEDGE_FLAG) != RESET)
{
  ……
}
```

5.24.19 xmc_flag_clear function

The table below describes the function xmc_flag_clear.

Table 557. xmc_flag_clear function

Name	Description
Function name	xmc_flag_clear
Function prototype	void xmc_flag_clear(xmc_class_bank_type xmc_bank, xmc_interrupt_flag_type xmc_flag);
Function description	Clear flag
Input parameter 1	xmc_bank: xmc bank selection
Input parameter 2	xmc_flag: flag selection
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

xmc_bank

XMC bank selection.

xmc_flag

xmc_flag is used for flag selection to get its status, including:

XMC_RISINGEDGE_FLAG: Rising edge status

XMC_LEVEL_FLAG: High level status

XMC_FALLINGEDGE_FLAG: Falling edge status

XMC_FEMPT_FLAG: FIFO empty status

Example:

```
if(xmc_flag_status_get (XMC_BANK2_NAND, XMC_RISINGEDGE_FLAG) != RESET)
{
  ……
  xmc_flag_clear(XMC_BANK2_NAND, XMC_RISINGEDGE_FLAG);
}
```

6 Precautions

6.1 Device model replacement

While replacing the device model in an existing project or demo with another one, if necessary, the macro definition is also required to be changed with the device model. Therefore, it is necessary to check and find out the macro definition corresponding to the device model defined in *Table 1* before replacing the device model. The subsequent sections give a detailed description of how to replace a device model in KEIL and IAR environment (Just taking the at32f403avgt7 as an example as other devices share similar operations).

There are two steps to get this happen:

1. By changing device
2. By changing macro definition

6.1.1 KEIL environment

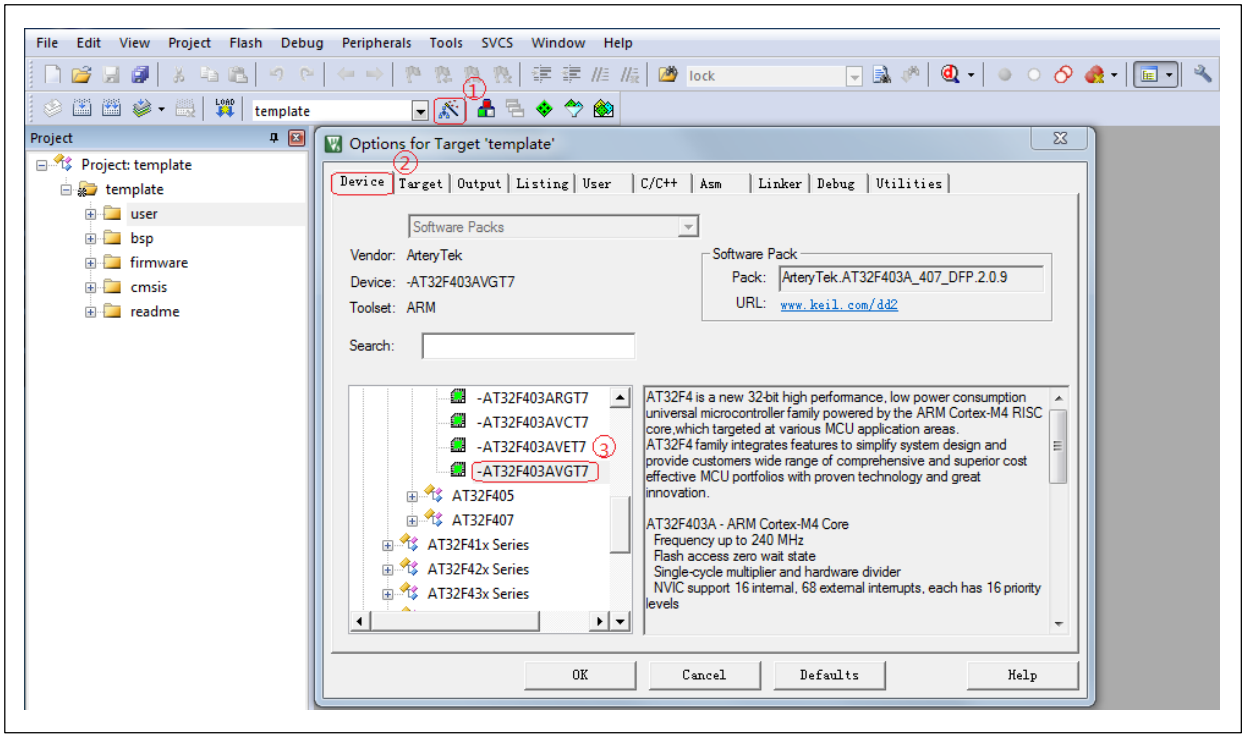
Follow the steps and illustration below for device replacement in Keil environment:

① Click on magic stick “Options for Target”

② Click on “Device”

③ Select the desired device part number

Figure 29. Change device part number in Keil



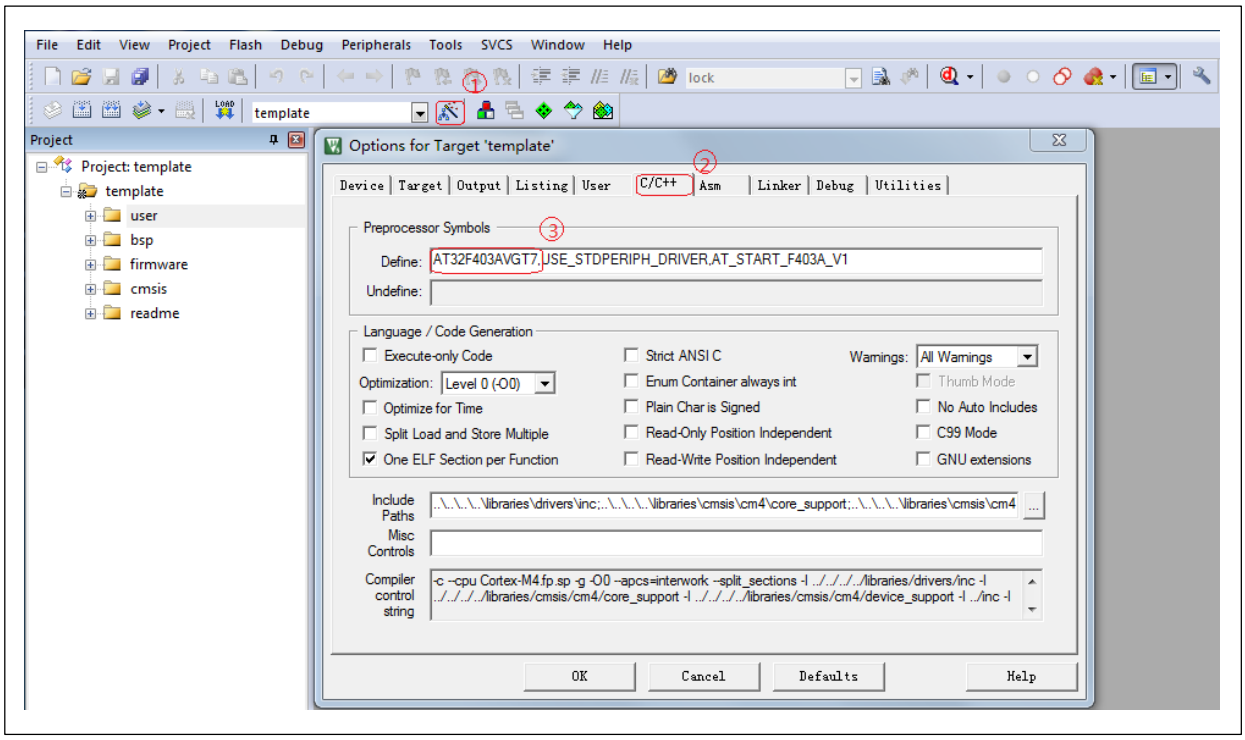
Follow the steps and illustration below for changing macro definition.

① Click on magic stick “Options for Target”.

② Click on “C/C++”.

③ Delete the original macro definition in “Define” box, and write the desired macro definition corresponding to the selected device part number based on *Table 1*.

Figure 30. Change macro definition in Keil



6.1.2 IAR environment

Follow the steps and illustration below for device replacement in IAR environment.

① Right click on the file name, and select “Options…”

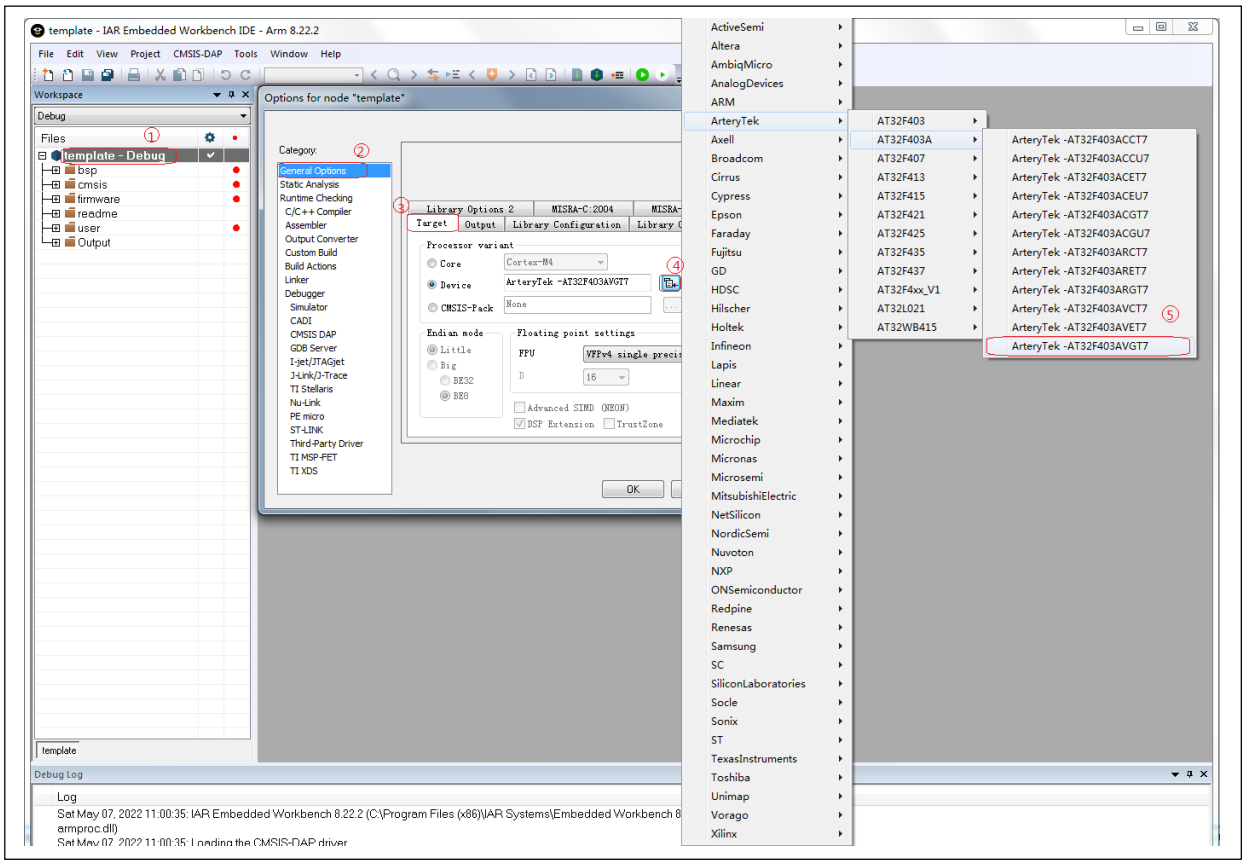
② Select “General Options”

③ Select “Target”

④ Click on check box

⑤ Select the desired device part number.

Figure 31. Change device part number in IAR



Follow the steps and illustration below for changing macro definition in IAR environment.

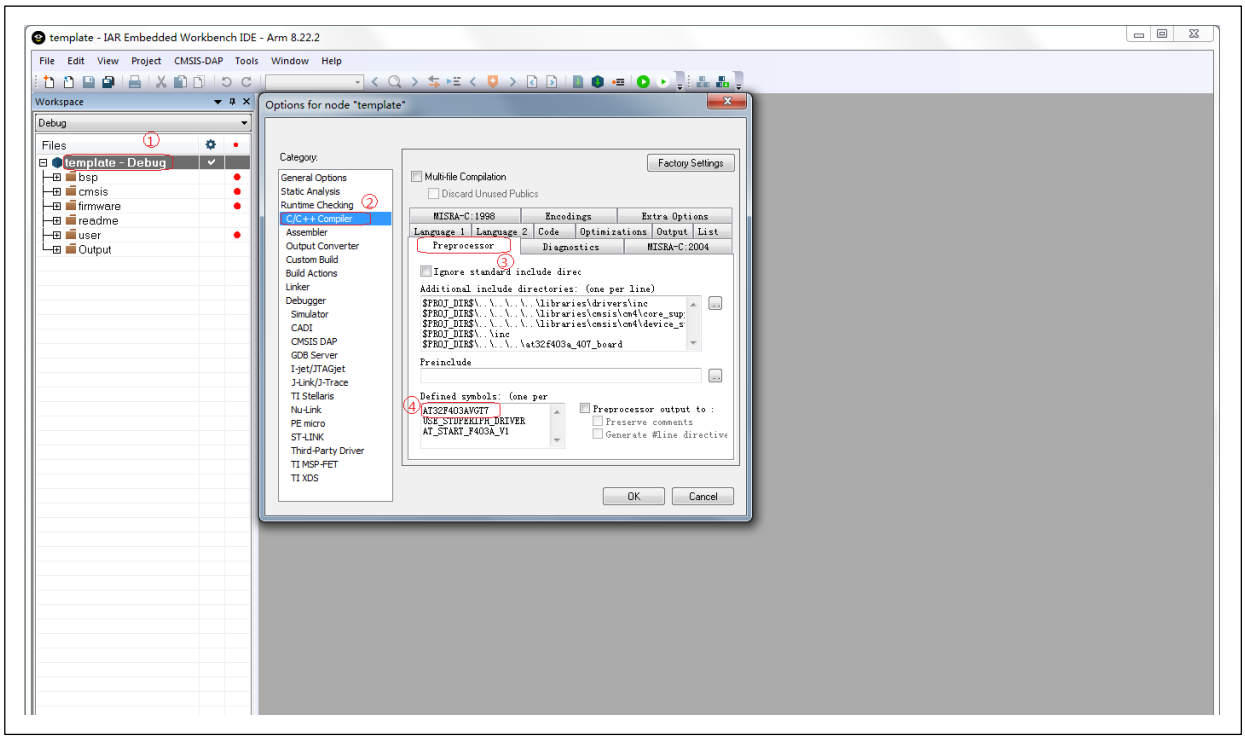
① Right click on the file name, and select “Options…”

② Select “C/C++ Compiler”

③ Click on “Preprocessor”

④ Delete the original macro definition in “Defined symbols” column, and write the desired macro definition corresponding to the selected device part number based on *Table 1*.

Figure 32. Change macro definition in IAR



6.2 Unable to identify IC by JLink software in Keil

In special circumstances, the Keil project compiled by an engineer is unknown to the J-Link software even if it can be compiled by other engineers and identified by ICP software. For example, some warnings like below will be displayed.

Figure 33. Error warning 1

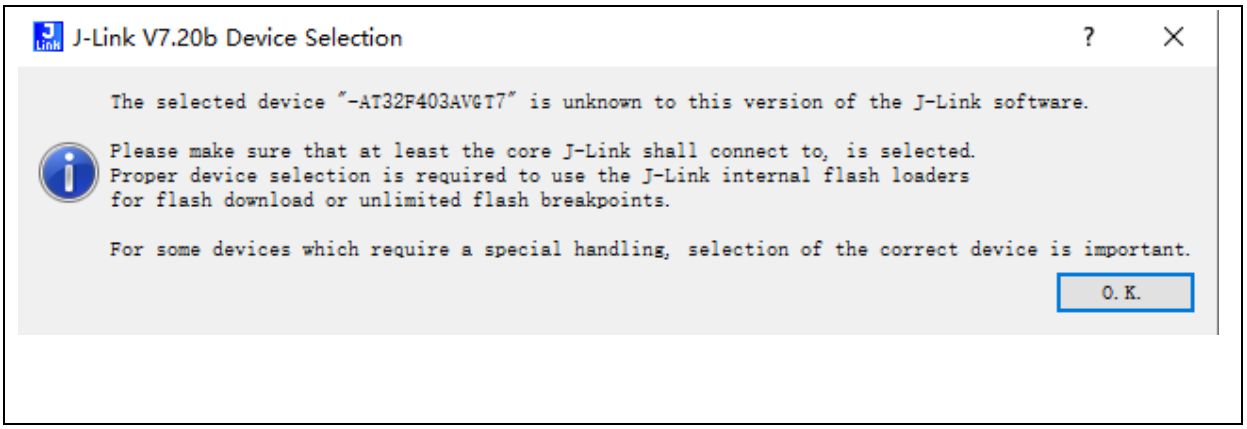


Figure 34. Error warning 2

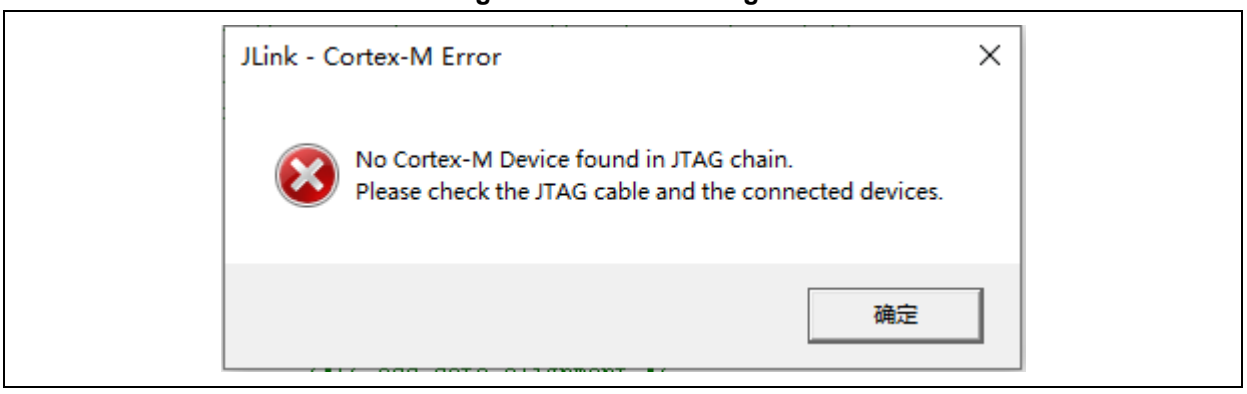
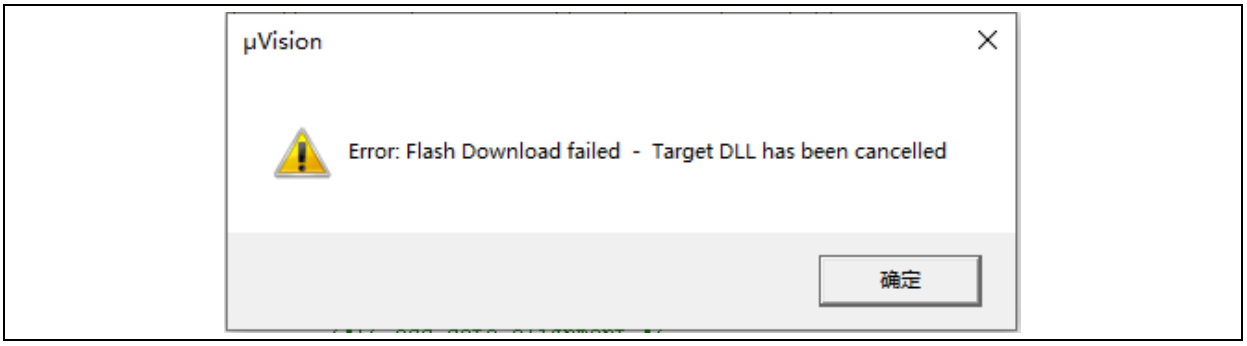


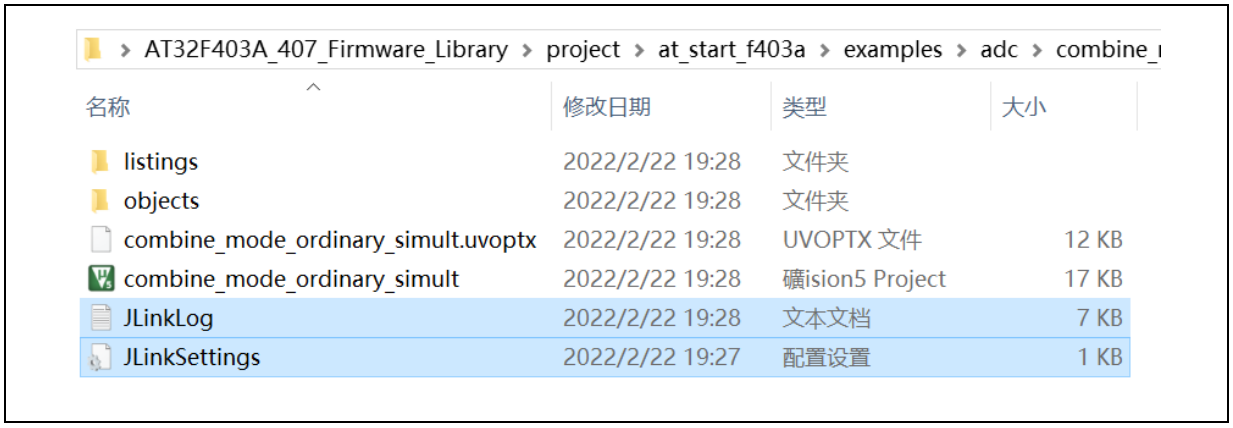
Figure 35. Error warning 3



How to solve this problem?

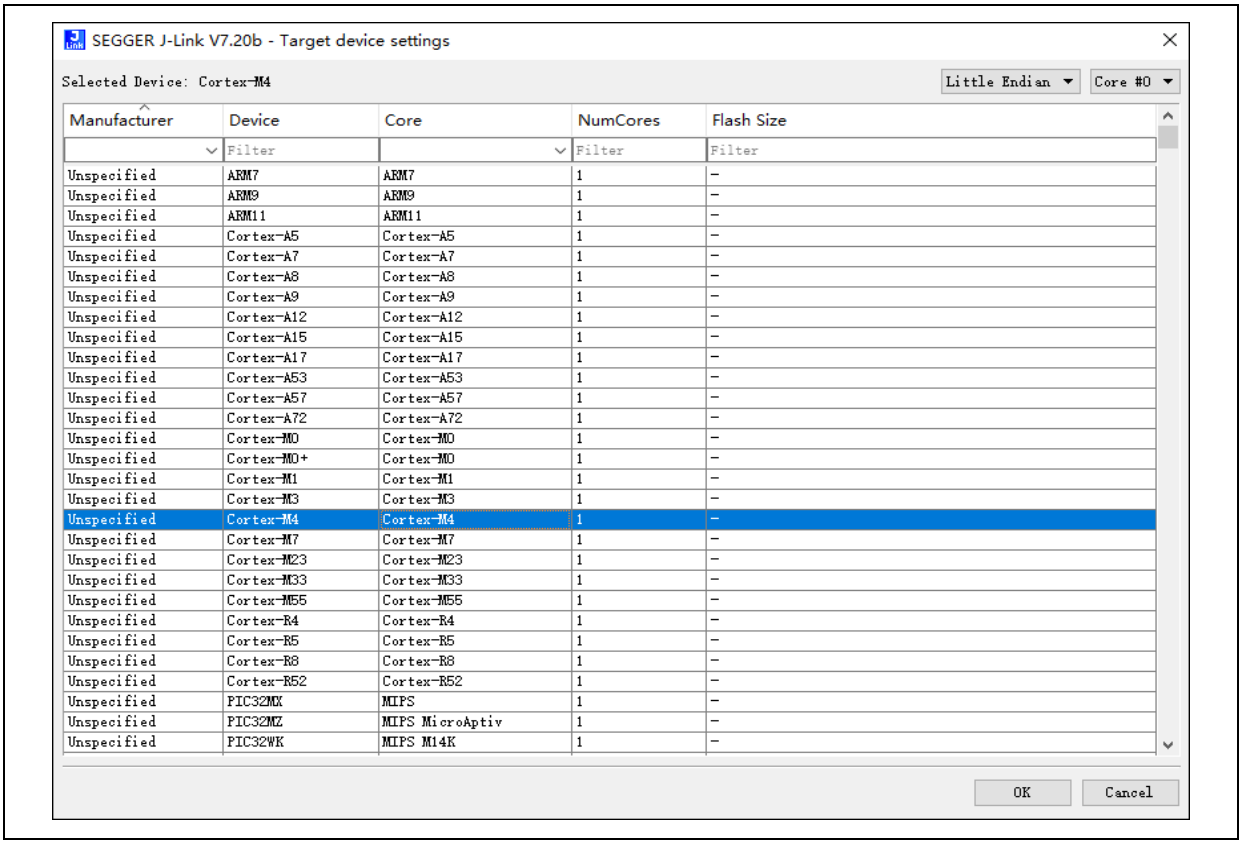
Step 1: Find “JLinkLog” and “JLinkSettings” files according to project path, and delete them.

Figure 36. JLinkLog and JLinkSettings



Step 2: Click on magic wand, go to “Debug”, select “Unspecified Cortex–M4”

Figure 37. Unspecified Cortex–M4



6.3 How to change HEXT crystal

All examples used in BSP implements frequency multiplication based on 8 MHz external high-speed crystal oscillator on the evaluation board. If a non-8 MHz external crystal is used in actual scenarios, it is necessary to modify clock configuration in BSP to allow for accurate and stable clock frequency.

Therefore, the “AT32_New_Clock_Configuration” tool is specially developed by Artery to generate the desired BSP system clock code file, including external clock source, frequency division factor, frequency multiplication factor, clock source selection and other parameters, marked in red in Figure 38. After the completion of parameter configuration, it is ready to generate code file, avoiding complicated operations involved in code modification.

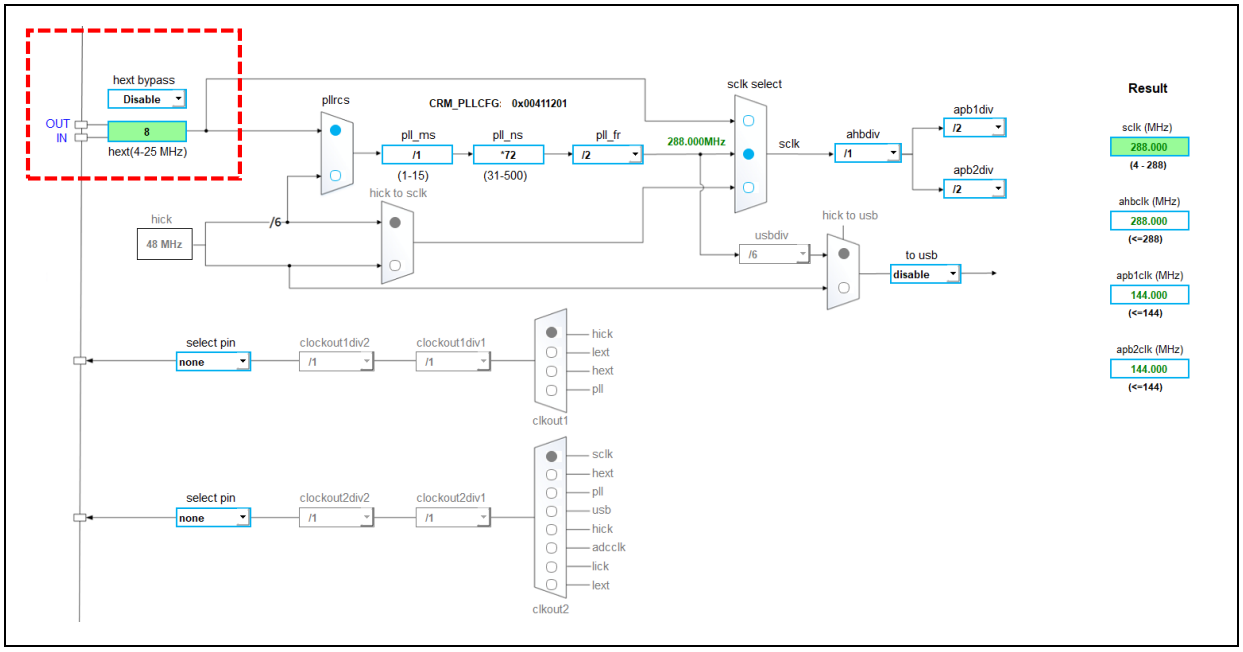
The users simply need to replace the original one in BSP demo with the newly generated clock code file (at32f4xx_clock.c/ at32f4xx_clock.h/ at32f4xx_conf.h), and call the function system_clock_config in main function.

Also, it is necessary to replace the macro definition HEXT_VALUE in the at32f4xx_conf.h. Taking the AT32F403A as an example, the HEXT_VALUE of the at32f403a_407_conf.h is defined as:

```
#define HEXT_VALUE        ((uint32_t)8000000) /*!< value of the high speed external crystal in hz */
```

Figure 38 shows the window of AT32_New_Clock_Configuration tool.

Figure 38. AT32_New_Clock_Configuration window



For more information on the AT32_New_Clock_Configuration, please refer to the corresponding Application Note shown in Table 539, which are all available from the official website of Artery.

Table 558. Clock configuration guideline

Part number	Application note
AT32F403A/407 clock configuration	AN0082
AT32F435/437 clock configuration	AN0084
AT32F421 clock configuration	AN0116
AT32F415 clock configuration	AN0117
AT32F413 clock configuration	AN0118
AT32F425 clock configuration	AN0121

7 Revision history

Table 559. Document revision history

Date	Revision	Changes
2021.05.25	2.0.0	Initial release
2021.11.19	2.0.1	Updated figures and description in *Section 2 How to install Pack*
2022.05.09	2.0.2	Added *Section 6.1 Device model replacement*
2022.05.19	2.0.3	Added the descriptions of peripherals functions.
2022.06.10	2.0.4	Updated document format and fixed some typos.
2022.11.15	2.0.5	Modified abbreviations of peripherals in I2C section.
2023.07.18	2.0.6	Added CRC related registers and functions.
2023.10.26	2.0.7	Added interrupt_flag_get function for each IP.
2024.03.01	2.0.8	Changed “PWC” to “NVIC” in the titles of Table 325/326 of *Section 5.14*
2025.04.25	2.0.9	Added gpio_bits_toggle function for GPIO.

IMPORTANT NOTICE – PLEASE READ CAREFULLY

Purchasers are solely responsible for the selection and use of ARTERY's products and services, and ARTERY assumes no liability whatsoever relating to the choice, selection or use of the ARTERY products and services described herein.

No license, express or implied, to any intellectual property rights is granted under this document. If any part of this document deals with any third party products or services, it shall not be deemed a license grant by ARTERY for the use of such third party products or services, or any intellectual property contained therein, or considered as a warranty regarding the use in any manner whatsoever of such third party products or services or any intellectual property contained therein.

Unless otherwise specified in ARTERY's terms and conditions of sale, ARTERY provides no warranties, express or implied, regarding the use and/or sale of ARTERY products, including but not limited to any implied warranties of merchantability, fitness for a particular purpose (and their equivalents under the laws of any jurisdiction), or infringement of any patent, copyright or other intellectual property right.

Purchasers hereby agree that ARTERY's products are not designed or authorized for use in: (A) any application with special requirements of safety such as life support and active implantable device, or system with functional safety requirements; (B) any aircraft application; (C) any aerospace application or environment; (D) any weapon application, and/or (E) or other uses where the failure of the device or product could result in personal injury, death, property damage. Purchasers' unauthorized use of them in the aforementioned applications, even if with a written notice, is solely at purchasers' risk, and Purchasers are solely responsible for meeting all legal and regulatory requirements in such use.

Resale of ARTERY products with provisions different from the statements and/or technical features stated in this document shall immediately void any warranty grant by ARTERY for ARTERY products or services described herein and shall not create or expand in any manner whatsoever, any liability of ARTERY.

© 2025 ARTERY Technology - All rights reserved